Krylov Subspace Methods for Linear Systems

全书翻译进度稿

本文件只收录已完成的译文单元. 尚未翻译的引用目标列于文末源文定位表, 不表示相应内容已完成.

序言

在科学计算和数据科学的许多领域, 我们经常面临求解形如 $A\boldsymbol{x} = \boldsymbol{b}$ 的大型稀疏线性方程组的问题, 这是整个计算过程中最耗时的部分之一. 因此, 许多研究者致力于开发求解线性方程组的高效数值算法. 如今, Krylov 子空间方法是广泛使用的数值算法, 被认为是 20 世纪十大算法之一, 其他算法包括快速 Fourier 变换和快速排序 [39]. 尽管基本理论在 20 世纪就已建立, 数学家, 工程师, 物理学家和许多其他领域的研究者仍不断发展着 Krylov 子空间方法. 关于 Krylov 子空间方法, 已有许多优秀著作, 包括下列作者的著作:

- Owe Axelsson, 1994 [10],
- Richard Barrett et al., 1994 [18],
- Wolfgang Hackbusch, 1994 [92],
- Are Magnus Bruaset, 1995 [29],
- Rüdiger Weiss, 1996 [203],
- Anne Greenbaum, 1997 [81],
- Yousef Saad, 2003 [151],
- Henk A. van der Vorst, 2003 [196],
- Jörg Liesen 和 Zdeněk Strakoš, 2012 [122],
- Gérard Meurant 和 Jurjen Duintjer Tebbens, 2020 [129].

文献 [129] 详细记述了 Krylov 子空间方法的历史, 是一部关于非 Hermitian 线性方程组的 Krylov 子空间方法的巨著 (约 700 页). 本书的特点如下:

(1) 线性方程组在计算科学和数据科学中的许多应用;

(2) 复对称线性方程组的 Krylov 子空间方法, 如 COCG 方法和 COCR 方法;

(3) 非 Hermitian 线性方程组的 Krylov 子空间方法, 如 BiCR 方法, GPBiCG 方法和 (块) IDR(s) 方法;

(4) 移位线性方程组的 Krylov 子空间方法, 如移位 IDR(s) 方法;

(5) 作为移位线性方程组应用的矩阵函数.

特点 (1) 对应于 Chapter 2, 其中从各种应用中导出线性方程组: 偏微分方程 (有限差分离散方法和有限元方法); 计算物理: 凝聚态物理 (Green 函数的计算) 和格点量子色动力学 (Wilson 费米子矩阵); 机器学习 (最小二乘问题); 矩阵方程 (Sylvester 型矩阵方程); 最优化 (使用张量计算记号表示 Euclidean 空间和 Riemannian 流形上的 Hessian 矩阵). 特点 (2) 和 (3) 对应于 Chapter 3. 该章将 Krylov 子空间方法分成三类: 针对 Hermitian 线性方程组, 复对称线性方程组和非 Hermitian 线性方程组的方法. 对于 Hermitian 线性方程组, 从 Lanczos 过程的矩阵形式导出 CG 方法, 从 CG 方法导出 CR 方法, 并从 Lanczos 过程导出 MINRES 方法. 对于复对称线性方程组, 分别将 COCG 方法, COCR 方法和 QMR_SYM 方法作为 CG 方法, CR 方法和 MINRES 方法的推广加以介绍. 对于非 Hermitian 线性方程组, 分别将 BiCG 方法, BiCR 方法和 QMR 方法作为 CG 方法, CR 方法和 MINRES 方法的推广加以介绍. 该章还详细推导了 GPBiCG 方法和 (块) IDR(s) 方法, 这是本书的特点之一. 此外, 还简要介绍了一些预条件技术. 特点 (4) 对应于 Chapter 4. 该章将移位线性方程组的 Krylov 子空间方法分成三类: 针对 Hermitian, 复对称和非 Hermitian 线性方程组的方法, 并系统地给出这些算法的详细推导. 特点 (5) 对应于 Chapter 5. 如果需要计算大型矩阵函数, 那么 Krylov 子空间方法以及移位线性方程组的 Krylov 子空间方法就是可选的方法, 因为这些算法能够计算矩阵函数中的任意元素. 该章还介绍矩阵函数的定义和一些著名的矩阵函数算法. 本书的另一个特点是, 除了为进一步理解 Krylov 子空间方法的收敛行为而给出的一些典型数值例子以外, 没有数值实验. Krylov 子空间方法的收敛性高度依赖于系数矩阵, 系数矩阵改变时, 最佳算法也会随之改变. 因此, 如果读者需要求解线性方程组, 我建议对自己的问题尝试若干种 Krylov 子空间方法 (包括 BiCGSTAB 方法和 GMRES 方法), 并从中选出最佳方法. 本书适合任何学过线性代数并且需要求解大型稀疏线性方程组的读者. 我希望本书能够帮助读者理解 Krylov 子空间方法的原理和性质, 并正确地使用 Krylov 子空间方法求解自己的问题.

日本名古屋　　Tomohiro Sogabe

2022 年 9 月

致谢

感谢大连理工大学的 Prof. Dr. Lei Du 慷慨地向我提供他的块 IDR(s) 方法的 LaTeX 代码. 感谢 Masayuki Nakamura 的工作. 他及时的提醒以及亲切的节日问候, 使我能够提高撰写本书的优先级. 感谢 Lokeshwaran Manickavasagam 和 Poojitha Ravichandran 在本书制作过程中提供的帮助. 感谢三位匿名审稿人仔细阅读书稿, 并提出极具建设性的意见, 提高了书稿质量. 我衷心感谢 Springer Series in Computational Mathematics (SSCM) 编委会的三位编辑: Prof. Dr. Randolph E. Bank, Prof. Dr. Wolfgang Hackbusch, Prof. Dr. Josef Stoer 和 Prof. Dr. Harry Yserentant, 他们以广博而杰出的专业视角为作者提供了富有成效的意见. 最后, 感谢 Prof. Dr. Shao-Liang Zhang, 大约 20 年前我在东京大学攻读博士学位时, 是他引领我进入 Krylov 子空间方法领域. 我还要感谢我亲爱的妻子 Yuan Li, 感谢她日复一日的支持与关爱. 她建议我利用周末撰写本书, 同时照料两个活泼的儿子 Kazuma 和 Takuma, 以及她年迈的父亲.

1 线性方程组数值解法导论

摘要 求解线性方程组的数值方法分为两类: 直接方法和迭代方法. 直接方法通过有限次算术运算求解线性方程组, 其中最著名的是 LU 分解. 迭代方法生成一个近似解序列, 大致分为定常迭代方法和 Krylov 子空间方法. 多重网格方法也是一类重要的迭代方法. 本章旨在介绍直接方法和定常迭代方法的原理, 并简要介绍 Krylov 子空间方法的理论. 此外, 还将简要说明多重网格方法.

1.1 线性方程组

本书考虑下列形式的线性方程组:

$$A\boldsymbol{x} = \boldsymbol{b}, \tag{1.1}$$

其中 $A \in \mathbb{C}^{N\times N}$, $\boldsymbol{b} \in \mathbb{C}$, 并假设系数矩阵 A 非奇异, 即 A 的逆存在. 设 $\tilde{\boldsymbol{x}}$ 为线性方程组 (1.1) 的一个近似解. 定量了解解 $\boldsymbol{x}$ 与近似解 $\tilde{\boldsymbol{x}}$ 之间距离的一种方法, 是计算 $\boldsymbol{x} - \tilde{\boldsymbol{x}}$ 的向量范数. 类似地, 两个矩阵 A, B 之间的距离可由 $A - B$ 的矩阵范数定量衡量. 向量范数和矩阵范数的定义将在下面两个小节中介绍. 这里的说明以 [170] 为基础; 关于数值线性代数的详细内容, 参见 [45], [79], [107], [191].

1.1.1 向量范数

向量 $\boldsymbol{x} := (x_1, x_2)^\top \in \mathbb{R}^2$ 的长度为 $(x_1^2 + x_2^2)^{1/2}$. 范数是长度概念的推广. 如果对所有 $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n, \alpha \in \mathbb{R}$, 映射 $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ 满足下列规则, 就称它为向量范数:[1]

(Nv1) $\|\boldsymbol{x}\| \geq 0$; $\|\boldsymbol{x}\| = 0 \iff \boldsymbol{x} = \boldsymbol{0}$. (正性)

(Nv2) $\|\alpha\boldsymbol{x}\| = |\alpha|\|\boldsymbol{x}\|$. (齐次性)

(Nv3) $\|\boldsymbol{x} + \boldsymbol{y}\| \leq \|\boldsymbol{x}\| + \|\boldsymbol{y}\|$. (三角不等式)

例如, p-范数 ($p \geq 1$) 定义为

$$\|\boldsymbol{x}\|_p := (|x_1|^p + |x_2|^p + \cdots + |x_n|^p)^{\frac{1}{p}}.$$

当 $p = 1, 2, \infty$ 时, 向量 p-范数写为

- $\|\boldsymbol{x}\|_1 = \sum_{i=1}^{n} |x_i|$,
- $\|\boldsymbol{x}\|_2 = (\sum_{i=1}^{n} |x_i|^2)^{\frac{1}{2}}$ $(= \sqrt{\boldsymbol{x}^\top \boldsymbol{x}})$,
- $\|\boldsymbol{x}\|_\infty = \max_{1\leq i\leq n} |x_i|$.

设 A 为对称正定矩阵, 即 A 对称且其全部特征值均为正. 则称 $\|\boldsymbol{x}\|_A := \boldsymbol{x}^\top A\boldsymbol{x}$ 为 $\boldsymbol{x}$ 的 A-范数; Section 3.1.1 将使用这一范数. 设 $\tilde{\boldsymbol{x}}$ 为线性方程组 (1.1) 的一个近似解. 则 $\|\boldsymbol{x} - \tilde{\boldsymbol{x}}\|_p$ 是 (1.1) 的精确解与近似解之间的距离. 本书中除非另有说明, 符号 $\|\cdot\|$ 都表示 2-范数, 即 $\|\cdot\| = \|\cdot\|_2$.

[1] 一般地, 给定向量空间 V, 如果映射 $\|\cdot\| : V \to \mathbb{R}$ 对所有 $\boldsymbol{x}, \boldsymbol{y} \in V, \alpha \in \mathbb{K}$ ($\mathbb{K} = \mathbb{R}$ 或 $\mathbb{C}$) 满足 (Nv1)–(Nv3), 就称它为范数.

1.1.2 矩阵范数

类似于向量范数的定义, 矩阵范数定义如下: 如果映射 $\|\cdot\|:\mathbb{R}^{m\times n}\to\mathbb{R}$ 对所有 $A,B\in\mathbb{R}^{m\times n},\alpha\in\mathbb{R}$ 满足下列三项, 就称它为矩阵范数:

(Nm1) $\|A\|\geq 0;\quad \|A\|=0\iff A=O,$ (O: 零矩阵) (正性)

(Nm2) $\|\alpha A\|=|\alpha|\|A\|,$ (齐次性)

(Nm3) $\|A+B\|\leq\|A\|+\|B\|.$ (三角不等式)

此外, 若矩阵范数满足

(Nm4) $\|AB\|\leq\|A\|\|B\|,\quad A\in\mathbb{R}^{m\times n},B\in\mathbb{R}^{n\times q},$

则称该矩阵范数具有次乘性. 注意, 有些著作也将满足 (Nm1)–(Nm4) 的映射称为矩阵范数. 作为具有次乘性的矩阵范数的一个例子, 下列从属矩阵范数被广泛使用:[2]

$$\|A\|_p:=\sup_{\boldsymbol{x}\neq\boldsymbol{0}}\frac{\|A\boldsymbol{x}\|_p}{\|\boldsymbol{x}\|_p}. \tag{1.2}$$

当 $p=1,2,\infty$ 时, $\|A\|_p$ 如下:

- $\|A\|_1=\max\limits_j\sum\limits_{i=1}^{n}|a_{ij}|,$
- $\|A\|_2=\sqrt{\text{maximum eigenvalue of } A^\top A},$
- $\|A\|_\infty=\max\limits_i\sum\limits_{j=1}^{n}|a_{ij}|.$

Frobenius 范数也具有次乘性:

- $\|A\|_{\mathrm{F}}:=\sqrt{\sum\limits_{i=1}^{m}\sum\limits_{j=1}^{n}|a_{ij}|^2}.$

设 $\boldsymbol{a}_1,\ldots,\boldsymbol{a}_n$ 为矩阵 $A\in\mathbb{R}^{m\times n}$ 的列向量, 并令 $\boldsymbol{a}=[\boldsymbol{a}_1^\top,\ldots,\boldsymbol{a}_n^\top]^\top$, 即将全部列首尾相接得到一个长向量 $\boldsymbol{a}$. 则 A 的 Frobenius 范数等于 $\boldsymbol{a}$ 的 2-范数, 即 $\|A\|_{\mathrm{F}}=\|\boldsymbol{a}\|_2$. 下列形式的最大值范数

$$\|A\|_{\max}:=\max_{i,j}|a_{ij}|$$

不具有次乘性. 例如, 考虑所有元素均为 1 的 2 行 2 列矩阵 A. 则 $\|AA\|_{\max}=\|A^2\|_{\max}=2$, 而 $\|A\|_{\max}\|A\|_{\max}=1$. 因此, 最大值范数不满足 (Nm4), 因为 $\|AA\|_{\max}>\|A\|_{\max}\|A\|_{\max}$. 下面给出矩阵范数的一些性质.

Theorem 1.1:

下列性质成立:

(1) $\|A\boldsymbol{x}\|_p\leq\|A\|_p\|\boldsymbol{x}\|_p,\quad \|A\boldsymbol{x}\|_2\leq\|A\|_{\mathrm{F}}\|\boldsymbol{x}\|_2,$

[2]Sup 可以替换为 max.

(2) $\|AB\|_p \le \|A\|_p\|B\|_p, \quad \|AB\|_{\mathrm{F}} \le \|A\|_{\mathrm{F}}\|B\|_{\mathrm{F}},$

(3) $\|QA\tilde{Q}\|_2 = \|A\|_2, \quad \|QA\tilde{Q}\|_{\mathrm{F}} = \|A\|_{\mathrm{F}}.$

这里, Q 和 $\tilde{Q}$ 是正交矩阵.[a]

[a]若实方阵 Q 满足 $Q^\top Q = I$, 则称其为正交矩阵, 其中 I 是单位矩阵.

Theorem 1.2:

设 A 为 $n \times n$ 矩阵. 则

(1) $n^{-1/2}\|A\|_2 \le \|A\|_1 \le n^{1/2}\|A\|_2,$

(2) $n^{-1/2}\|A\|_2 \le \|A\|_\infty \le n^{1/2}\|A\|_2,$

(3) $n^{-1}\|A\|_\infty \le \|A\|_1 \le n\|A\|_\infty,$

(4) $\|A\|_1 \le \|A\|_{\mathrm{F}} \le n^{1/2}\|A\|_2.$

1.2 条件数

考虑 $A\boldsymbol{x} = \boldsymbol{b}$; 如果矩阵 A 和右端向量 $\boldsymbol{b}$ 发生微小变化, 解会改变多少? 为回答这一问题, 设 ΔA 和 $\Delta\boldsymbol{b}$ 分别为 A 和 $\boldsymbol{b}$ 的扰动. 以 $\Delta\boldsymbol{x}$ 表示的解的变化满足下列关系:

$$(A + \Delta A)(\boldsymbol{x} + \Delta\boldsymbol{x}) = \boldsymbol{b} + \Delta\boldsymbol{b}.$$

利用 $A\boldsymbol{x} = \boldsymbol{b}$ 得到

$$\Delta\boldsymbol{x} = A^{-1}[-\Delta A(\boldsymbol{x} + \Delta\boldsymbol{x}) + \Delta\boldsymbol{b}].$$

利用 Theorem 1.1-(1) 和三角不等式 (Nv3), 可由下式估计 $\boldsymbol{x}$ 的 p-范数:

$$\|\Delta\boldsymbol{x}\|_p \le \|A^{-1}\|_p(\|\Delta A\|_p\|\boldsymbol{x} + \Delta\boldsymbol{x}\|_p + \|\Delta\boldsymbol{b}\|_p).$$

因此, 有

$$\frac{\|\Delta\boldsymbol{x}\|_p}{\|\boldsymbol{x} + \Delta\boldsymbol{x}\|_p} \le \|A\|_p\|A^{-1}\|_p \left(\frac{\|\Delta A\|_p}{\|A\|_p} + \frac{\|\Delta\boldsymbol{b}\|_p}{\|A\|_p\|\boldsymbol{x} + \Delta\boldsymbol{x}\|_p}\right). \tag{1.3}$$

这里,

$$\kappa_p(A) := \|A\|_p\|A^{-1}\|_p \tag{1.4}$$

称为 p-范数条件数. 不等式 (1.3) 表明, 如果条件数很大, 计算得到的近似解可能远离精确解. 注意, 这一事实与求解线性方程组所使用的算法无关. 如果条件数很大, 则称该线性方程组是病态的. 我们将在 Chapter 3 看到, 条件数也会影响 Krylov 子空间方法的收敛速度. 本书经常使用 2-范数条件数 $\kappa_2(A)$, 并将其简记为 κ 或 $\mathrm{cond}(A)$.

1.3 直接方法

为简化叙述, 仅考虑 $A\boldsymbol{x} = \boldsymbol{b}$, 其中 $A \in \mathbb{R}^{n\times n}$, $\boldsymbol{b} \in \mathbb{R}^n$. Section 1.3.1 介绍用 LU 分解求解线性方程组, 随后的 Section 1.3.2 说明带选主元技术的 LU 分解, 这种方法数值稳定, 在实践中广泛使用. Section 1.3.3 介绍提高近似解精度的一种方法.

1.3.1 LU 分解

设 A 为方阵. 将 A 分解为下三角矩阵 L 与上三角矩阵 U 的乘积, 就得到 A 的 LU 分解, 其形式为

$$A = LU, \tag{1.5}$$

其中

$$L = \begin{pmatrix} l_{11} & & \\ \vdots & \ddots & \\ l_{n1} & \cdots & l_{nn} \end{pmatrix}, \quad U = \begin{pmatrix} u_{11} & \cdots & u_{1n} \\ & \ddots & \vdots \\ & & u_{nn} \end{pmatrix}.$$

矩阵 L 和 U 中的空白表示相应元素均为零. 若 L 或 U 的对角元素均为 1, 则 LU 分解唯一确定. 对所有 i 都有 $l_{ii} = 1$ 的 LU 分解称为 Doolittle 方法, 对所有 i 都有 $u_{ii} = 1$ 的 LU 分解称为 Crout 方法. 一旦得到 A 的 LU 分解, 就容易通过下列过程得到线性方程组的解 $\boldsymbol{x}$:

$$A = LU, \quad L\boldsymbol{y} = \boldsymbol{b}, \quad U\boldsymbol{x} = \boldsymbol{y}. \tag{1.6}$$

具体地, 前代过程

$$y_i = \frac{1}{l_{ii}} \left(b_i - \sum_{j=1}^{i-1} l_{ij} y_j \right) \quad (i = 1, \ldots, n)$$

给出 $\boldsymbol{y}$, 随后的回代过程

$$x_i = \frac{1}{u_{ii}} \left(y_i - \sum_{j=i+1}^{n} u_{ij} x_j \right) \quad (i = n, \ldots, 1)$$

给出解 $\boldsymbol{x}$, 其中 $y_1 = b_1/l_{11}$, $x_n = y_n/u_{nn}$. LU 分解存在的充要条件是全部顺序主子式[3] 均不为零. 下面列出 LU 分解存在的几种情形.

1. 严格对角占优矩阵: $|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad (1 \leq i \leq n)$.
2. M-矩阵: $a_{ij} \leq 0$ $(i \neq j)$, 且 A^{-1} 的全部元素均非负.
3. 矩阵 $A \in \mathbb{R}^{n \times n}$ 的对称部分正定. 对称部分指 $(A + A^\top)/2$, 正定指对任意非零向量 $\boldsymbol{v}$, 均有 $\boldsymbol{v}^\top A \boldsymbol{v} > 0$. 对于任意方阵 A, 矩阵 A 都可写成对称部分 $(A + A^\top)/2$ 与斜对称部分 $(A - A^\top)/2$ 之和.

LU 分解 (Doolittle 方法) 的算法见 Algorithm 1.1, 它生成满足 $A = LU$ 的 L 和 U.

[3]使用 MATLAB 记号, A 的全部顺序主子式定义为 $\det(A(1:1,1:1))$, $\det(A(1:2,1:2))$, ..., $\det(A(1:n,1:n))$.

Algorithm 1.1 LU 分解

Input: 一个 $n \times n$ 矩阵 A

Output: 单位下三角矩阵 L 和上三角矩阵 U

1: **for** $i = 1, \ldots, n$ **do**
2: $\quad a_{ji} = a_{ji}/a_{ii} \quad (j = i+1, \ldots, n)$
3: $\quad$ **for** $j = i+1, \ldots, n$ **do**
4: $\quad\quad a_{jk} = a_{jk} - a_{ji} \times a_{ik} \quad (k = i+1, \ldots, n)$
5: $\quad$ **end for**
6: **end for**
7: $l_{ii} = 1 \quad (i = 1, \ldots, n)$
8: $l_{ij} = a_{ij} \quad (i > j)$
9: $u_{ij} = a_{ij} \quad (i \leq j)$

1.3.2 选主元的 LU 分解

对于某些 A, LU 分解可能不存在. 例如, 考虑

$$A = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}.$$

容易看出, LU 分解 (Algorithm 1.1) 会因 $a_{11} = 0$ 而发生零除并失败. 另一方面, 若交换第一行和第二行, 即

$$PA = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix} = \begin{pmatrix} 2 & 3 \\ 0 & 1 \end{pmatrix},$$

则 LU 分解不会失败, 并得到下列形式的 LU 分解

$$PA = LU,$$

其中

$$L = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 3 \\ 0 & 1 \end{pmatrix}.$$

这一技术称为选主元. 部分选主元的 LU 分解见 Algorithm 1.2. 这里, 部分选主元一词对应于 Algorithm 1.2 的第 2–6 行.

Algorithm 1.2 部分选主元的 LU 分解

Input: 一个 $n \times n$ 矩阵 A 和 $n \times n$ 单位矩阵 $P(= I)$

Output: 单位下三角矩阵 L 和上三角矩阵 U

Output: 置换矩阵 P

1: **for** $i = 1, \ldots, n$ **do**
2: 从 $\{|a_{ii}|, \ldots, |a_{ni}|\}$ 中找出最大元素 $a_{p_i i}$.
3: 对 $j = 1, \ldots, n$, 交换 a_{ij} 和 $a_{p_i j}$.
4: **if** $i \neq n$ **then**
5: 交换矩阵 P 的第 i 行和第 p_i 行.
6: **end if**
7: $a_{ji} = a_{ji}/a_{ii} \quad (j = i+1, \ldots, n)$
8: **for** $j = i+1, \ldots, n$ **do**
9: $a_{jk} = a_{jk} - a_{ji} \times a_{ik} \quad (k = i+1, \ldots, n)$
10: **end for**
11: **end for**
12: $l_{ii} = 1 \quad (i = 1, \ldots, n)$
13: $l_{ij} = a_{ij} \quad (i > j)$
14: $u_{ij} = a_{ij} \quad (i \leq j)$

Algorithm 1.2 生成 L, U 和置换矩阵 P, 使得

$$PA = LU.$$

与 LU 分解不同, 部分选主元的 LU 分解不会发生中断 (零除). 接下来, 完全选主元的 LU 分解见 Algorithm 1.3. 完全选主元一词对应于 Algorithm 1.3 的第 2–8 行. 注意, 在第 i 步, 部分选主元从 $\{|a_{k,i}| : k \in \{i, \ldots, n\}\}$ 中找最大值, 而完全选主元从 $\{|a_{k,l}| : k, l \in \{i, \ldots, n\}\}$ 中找最大值. Algorithm 1.3 生成分解因子矩阵 L 和 U 以及置换矩阵 P 和 Q, 使得

$$PAQ = LU.$$

通常使用部分选主元的 LU 分解; 如果分解精度不令人满意, 则可使用完全选主元.

Algorithm 1.3 完全选主元的 LU 分解

Input: 一个 $n \times n$ 矩阵 A 和 $n \times n$ 单位矩阵 $P(=I), Q(=I)$

Output: 单位下三角矩阵 L 和上三角矩阵 U

Output: 置换矩阵 P, Q

for $i = 1, \ldots, n$ **do**
  从 $\{|a_{kl}| : i \le k \le n, i \le l \le n\}$ 中找出最大元素 $a_{p_i q_i}$.
  对 $j = 1, \ldots, n$, 交换 a_{ij} 和 $a_{p_i j}$.
  对 $j = 1, \ldots, n$, 交换 a_{ji} 和 $a_{j q_i}$.
  if $i \neq n$ **then**
    交换矩阵 P 的第 i 行和第 p_i 行.
    交换矩阵 P 的第 i 列和第 q_i 行.
  end if
  $a_{ji} = a_{ji}/a_{ii} \quad (j = i+1, \ldots, n)$
  for $j = i+1, \ldots, n$ **do**
    $a_{jk} = a_{jk} - a_{ji} \times a_{ik} \quad (k = i+1, \ldots, n)$
  end for
end for
$l_{ii} = 1 \quad (i = 1, \ldots, n)$
$l_{ij} = a_{ij} \quad (i > j)$
$u_{ij} = a_{ij} \quad (i \le j)$

如果需要求解下列一组线性方程组

$$A\boldsymbol{x}_i = \boldsymbol{b}_i, \quad i = 1, 2, \ldots, m,$$

那么 LU 分解很有吸引力, 因为一旦得到 L 和 U, 就只需进行前代和回代, 参见 (1.6) 之后的说明.

1.3.3 迭代改进

本小节考虑用 LU 分解得到的线性方程组 (1.1) 的近似解 $\tilde{\boldsymbol{x}}$ 精度不足的情形. 此时, 如果知道使得 $\boldsymbol{x} = \tilde{\boldsymbol{x}} + \Delta\boldsymbol{x}$ 的修正向量 $\Delta\boldsymbol{x}$, 就能得到精确解 $\boldsymbol{x}$. 容易看出, 修正向量 $\Delta\boldsymbol{x}$ 对应于下列线性方程组的解:

$$A\Delta\boldsymbol{x} = \boldsymbol{r}, \tag{1.7}$$

其中 $\boldsymbol{r} := \boldsymbol{b} - A\tilde{\boldsymbol{x}}$. 求解 (1.7), 得到 (1.7) 的近似解 $\Delta\tilde{\boldsymbol{x}}$ 后, 预期修正后的近似解 $\tilde{\boldsymbol{x}} + \Delta\tilde{\boldsymbol{x}}$ 比 $\tilde{\boldsymbol{x}}$ 更精确. 这一过程可重复进行, 直到达到所需精度, 称为迭代改进. 迭代改进算法见 Algorithm 1.4.

Algorithm 1.4 $A\boldsymbol{x}=\boldsymbol{b}$ 的迭代改进

Input: $A, \boldsymbol{b}$, $A\boldsymbol{x}=\boldsymbol{b}$ 的近似解 $\tilde{\boldsymbol{x}}$

Output: 改进后的近似解 $\tilde{\boldsymbol{x}}$

1: 使用双倍精度运算计算 $\boldsymbol{r}=\boldsymbol{b}-A\tilde{\boldsymbol{x}}$.

2: 求解 $A\Delta\boldsymbol{x}=\boldsymbol{r}$, 得到近似解 $\Delta\tilde{\boldsymbol{x}}$.

3: 更新 $\tilde{\boldsymbol{x}}=\tilde{\boldsymbol{x}}+\Delta\tilde{\boldsymbol{x}}$.

4: 重复步骤 1–3, 直到达到所需精度.

若使用 LU 分解求解 $A\boldsymbol{x}=\boldsymbol{b}$, 则很容易得到近似解 $\Delta\tilde{\boldsymbol{x}}$, 因为可以重用分解因子 L 和 U, 因而只需进行前代和回代. 注意, Algorithm 1.4 第 1 步中的双倍精度运算是指: 若使用单精度运算, 则用双精度运算计算 $\boldsymbol{r}=\boldsymbol{b}-A\tilde{\boldsymbol{x}}$; 若使用双精度运算, 则用四倍精度运算计算它. 令 $\boldsymbol{f}(\boldsymbol{x}):=\boldsymbol{b}-A\boldsymbol{x}$, 并对 $\boldsymbol{f}(\boldsymbol{x})=\boldsymbol{0}$ 应用 Newton 方法, 就可将 Algorithm 1.4 解释为 Newton 方法.

1.4 对称线性方程组的直接方法

当矩阵 A 对称时, Cholesky 分解和 $LDL^{\top}$ 分解是 LU 分解的两种变形, 都是高效的直接方法. 作为准备, 先定义正定的概念:

1. 正定

   如果对所有 $\boldsymbol{x}\neq\boldsymbol{0}$ 都有 $\boldsymbol{x}^{\top}A\boldsymbol{x}>0$, 则称矩阵 A 正定.

2. 对称正定

   如果正定矩阵 A 对称, 则称矩阵 A 对称正定.

已知矩阵 A 对称正定, 当且仅当对称矩阵 A 的全部特征值均为正.

1.4.1 Cholesky 分解

如果矩阵 A 对称正定, 则存在下三角矩阵 L, 使得

$$A=LL^{\top}.$$

这一分解称为 Cholesky 分解. Cholesky 分解的算法见 Algorithm 1.5.

Algorithm 1.5 Cholesky 分解

Input: A

Output: L

1: **for** $i=1,\ldots,n$ **do**

2: $\quad l_{ii}=(a_{ii}-\sum_{k=1}^{i-1}l_{ik}^2)^{1/2}$

3: $\quad$ **for** $j=i+1,\ldots,n$ **do**

4: $\quad\quad l_{ji}=(a_{ji}-\sum_{k=1}^{i-1}l_{jk}\times l_{ik})/l_{ii}$

5: $\quad$ **end for**

6: **end for**

由于利用了对称性, Cholesky 分解的计算量约为 LU 分解的一半. 已知由 A 的正定性可知, Cholesky 分解不会发生中断, 也不需要 Section 1.3.2 所述的选主元技术.

1.4.2 $LDL^\top$ 分解

当对称矩阵 A 非正定时, 不能使用 Cholesky 分解. 另一方面, 如果全部顺序主子式均非负, 则存在对角矩阵 D (即全部非对角元素均为零) 和单位下三角矩阵 L (即全部对角元素均为 1 的下三角矩阵), 使得

$$A = LDL^\top.$$

这一分解称为 $LDL^\top$ 分解 (或修正 Cholesky 分解), 算法见 Algorithm 1.6.

Algorithm 1.6 $LDL^\top$ 分解

Input: A

Output: L, D

1: **for** $i = 2, \ldots, n$ **do**
2: **for** $j = 1, \ldots, i-1$ **do**
3: $l_{ij} = (a_{ij} - \sum_{k=1}^{j-1} l_{ik} \times l_{jk} \times d_{kk})/d_{jj}$
4: **end for**
5: $d_{ii} = a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2 \times d_{kk}$
6: **end for**

Algorithm 1.6 生成对角矩阵 D 的对角元素, 以及对所有 i 均满足 $l_{ii} = 1$ 的下三角矩阵 L 的元素 l_{ij}. 与 Cholesky 分解不同, $LDL^\top$ 分解中不出现平方根. 令 S 为非奇异矩阵, 则由 Sylvester 惯性定律可知, 对称矩阵 A 的正/零/负特征值个数, 分别与 $SAS^\top$ 的正/零/负特征值个数相同. 这说明 $LDL^\top$ 分解不仅可用于求解线性方程组, 还可以给出对称矩阵 A 的特征值的粗略分布. 因此, 若有 $A = LDL^\top$, 则正/零/负对角元素的个数就给出了 A 的正/零/负特征值的个数. 进一步, 通过计算 $A - \sigma_1 I = L_1 D_1 L_1^\top$ 和 $A - \sigma_2 I = L_2 D_2 L_2^\top$ 的 $LDL^\top$ 分解, 就可知道给定闭区间 $[\sigma_1, \sigma_2]$ 上的特征值个数. 具体地, 设 $n_1 (\geq 0)$ 为 D_1 的非负对角元素个数, $n_2^{(>0)}$ 为 D_2 的正对角元素个数. 则闭区间 $[\sigma_1, \sigma_2]$ 上的特征值个数为 $n_1^{(\geq 0)} - n_2^{(>0)}$. 这一事实可用于第 k 个 (广义) 特征值问题:

$$A\boldsymbol{x} = \lambda B\boldsymbol{x}, \tag{1.8}$$

其中 A 对称, B 对称正定. 已知全部特征值均为实数, 因而可写为 $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_k \leq \cdots \leq \lambda_n$. 第 k 个 (广义) 特征值问题就是求 λ_k 及 (1.8) 的相应特征向量. 广义特征值问题 (1.8) 等价于下列移位问题:

$$(A - \sigma B)\boldsymbol{x} = (\lambda - \sigma)B\boldsymbol{x}. \tag{1.9}$$

由于 B 对称正定, B 有 Cholesky 分解, 即 $B = L_B L_B^\top$. 因此由 (1.9) 得到

$$L_B^{-1}(A - \sigma B)L_B^{-\top}\tilde{\boldsymbol{x}} = (\lambda - \sigma)\tilde{\boldsymbol{x}},$$

其中 $\tilde{\boldsymbol{x}} = L_B^\top \boldsymbol{x}$. 现在设 D 为 $(A - \sigma B)$ 的 $LDL^\top$ 分解中的对角矩阵. 则

$$L_B^{-1} L D L^\top L_B^{-\top}\tilde{\boldsymbol{x}} = (\lambda - \sigma)\tilde{\boldsymbol{x}} \iff GDG^\top\tilde{\boldsymbol{x}} = (\lambda - \sigma)\tilde{\boldsymbol{x}},$$

其中 $G = L_B^{-1}L$. 这表明, 根据 Sylvester 惯性定律, D 的正 (或负) 对角元素个数, 等于大于 (或小于) σ 的特征值个数. 因此, 若 D 的负对角元素个数例如为 10, 则可知道满足条件 $\lambda \leq \sigma$ 且最接近 σ 的特征值 λ 是第 10 个特征值, 即 $\lambda = \lambda_{10}$. 文献 [120] 基于这一思想并结合若干加速技术, 提出了一种求 (广义) 特征值问题的第 k 个特征值和特征向量的算法. 关于第 k 个特征值问题, 另见 [156].

1.5 大型稀疏线性方程组的直接方法

考虑下列矩阵:

$$A = \begin{pmatrix} 2 & -1 & -1 & -1 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}. \tag{1.10}$$

如果矩阵中的零元素相对较多, 则称该矩阵稀疏. 反之, 如果矩阵中的零元素相对较少, 则称其稠密. 对 (1.10) 应用 LU 分解 (Algorithm 1.1), 得到

$$A = LU = \begin{pmatrix} 1 & & & \\ 1/2 & 1 & & \\ 1/2 & 1/3 & 1 & \\ 1/2 & 1/3 & 1/4 & 1 \end{pmatrix} \begin{pmatrix} 2 & -1 & -1 & -1 \\ & 3/2 & 1/2 & 1/2 \\ & & 4/3 & 1/3 \\ & & & 5/4 \end{pmatrix}.$$

对于 A 的下三角部分, 可见 $a_{32} = a_{42} = a_{43} = 0$, 但在 L 的相应下三角部分却有 $l_{32} = l_{42} = l_{43} \neq 0$. 类似地, 在 A 的上三角部分有 $a_{23} = a_{24} = a_{34} = 0$, 而在 U 的上三角部分却有 $u_{23} = u_{24} = u_{34} \neq 0$. 如果原矩阵 A 中的元素为零, 而 L 和 U 中相应的元素不为零, 则称 L 和 U 中的这些非零元素为填充元. 当矩阵 A 大型且稀疏时, 填充元可能成为计算 LU 分解的瓶颈. 对于大型稀疏矩阵, 通常只使用非零元素, 不在内存中存储零元素. 然而, LU 分解后, L 和 U 可能变得稠密, 即最坏情况下需要在内存中存储 n^2 个元素. 这说明, 若矩阵规模 n 为 $n = 1,000,000$, 并使用实数双精度运算, 则需要 $n^2 \times 8 \approx 8$ TB 内存. 若 $n = 10,000,000$, 则需要一台具有 800 TB 内存的计算机. 正如 Chapter 2 将展示的, 科学计算中产生的矩阵通常大型且稀疏. 因此, 用 LU 分解求解大型稀疏线性方程组时, 减少填充元至关重要. 下面介绍减少填充元的核心思想. 核心思想是对稀疏矩阵 A 的某些行和列作置换, 以减少填充元, 然后对置换后的矩阵作 LU 分解. 具体地, 对 PAQ 作 LU 分解, 其中 P 和 Q 是预先指定的置换矩阵. 随后求解

$$PAQ\boldsymbol{y} = P\boldsymbol{b}$$

从而得到解 $\boldsymbol{x} = Q\boldsymbol{y}$. 这里选择 P 和 Q, 使 L 和 U 中的填充元尽可能少. 下面给出减少 (1.10) 中 A 的填充元的例子. 考虑

$$P = Q = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}.$$

则由 (1.10) 得到

$$PAQ = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ -1 & -1 & -1 & 2 \end{pmatrix}.$$

对 PAQ 作 LU 分解, 得到

$$PAQ = LU = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 5 \end{pmatrix}.$$

幸运的是, 这种情形下没有产生填充元. 因而只需在内存中存储非零元素, 可有效利用内存. 虽然很难选择使填充元个数最少的最优置换矩阵 P 和 Q, 但已有近似优化方法. 著名的方法包括最小度方法和逆 Cuthill–McKee 方法. 关于这些方法的参考文献, 见 Section 3.5.5. 关于稀疏直接方法的详细内容, 例如可参见 [44].

1.6 定常迭代方法

定常迭代方法通过下列递推生成近似解:

$$\boldsymbol{x}_{k+1} = \boldsymbol{f}(\boldsymbol{x}_k), \quad k = 0, 1, \ldots \tag{1.11}$$

其中 $\boldsymbol{x}_0$ 是为 $A\boldsymbol{x} = \boldsymbol{b}$ 预先给定的初始猜测. 这里, 向量值函数 $\boldsymbol{f}$ 定义为

$$\boldsymbol{f}(\boldsymbol{y}) := M^{-1}N\boldsymbol{y} + M^{-1}\boldsymbol{b}, \tag{1.12}$$

其中 M 为非奇异矩阵, 矩阵 N 使 (1.1) 中的矩阵 A 满足 $A = M - N$. 解 $\boldsymbol{x}$ 对应于 $\boldsymbol{f}$ 的不动点, 即 $\boldsymbol{x} = \boldsymbol{f}(\boldsymbol{x})$. 因此, 定常迭代方法可看作通过 (1.11) 中的递推寻找 $\boldsymbol{f}$ 的不动点的迭代方法. 由于 $\boldsymbol{f}$ 不依赖于迭代次数 k, 所以称这些迭代方法为定常的. 由 (1.11) 和 (1.12), 迭代向量 $\boldsymbol{x}_{k+1}$ 可写为

$$\boldsymbol{x}_{k+1} = M^{-1}N\boldsymbol{x}_k + M^{-1}\boldsymbol{b} \tag{1.13}$$

其中 $k = 0, 1, \ldots$. 定常迭代方法可用于求解大型稀疏线性方程组, 因为与 LU 分解等直接方法不同, 计算近似解时不必考虑填充元, 只需在内存中存储系数矩阵的非零元素和一些工作向量. 下面介绍最著名的定常迭代方法, 即 Jacobi 方法, Gauss–Seidel (GS) 方法和逐次超松弛 (SOR) 方法. 设 $A = (a_{ij})$ 为 $A\boldsymbol{x} = \boldsymbol{b}$ 的系数矩阵, 并设

$$\begin{aligned} L &= (a_{ij}), \quad a_{ij} = 0 \quad (i \le j), \\ D &= (a_{ij}), \quad a_{ij} = 0 \quad (i \ne j), \\ U &= (a_{ij}), \quad a_{ij} = 0 \quad (i \ge j). \end{aligned}$$

则有

$$A = L + D + U.$$

L, D 和 U 分别称为严格下三角矩阵, 对角矩阵和严格上三角矩阵. 用 L, D 和 U 选择 M 与 N, 就得到 Jacobi, GS 和 SOR 方法. Table 1.1 列出了这些代表性定常迭代方法的选择.

表 1.1: 三种定常迭代方法的分类

	M	N
Jacobi 方法	D	$-(L+U)$
Gauss–Seidel 方法	$D+L$	$-U$
SOR 方法	$(D+\omega L)/\omega$	$N=[(1-\omega)D-\omega U]/\omega$

1.6.1 Jacobi 方法

由 (1.13) 和 Table 1.1, Jacobi 方法的迭代向量 $\boldsymbol{x}_k$ 如下:

$$\boldsymbol{x}_{k+1}=-D^{-1}(L+U)\boldsymbol{x}_k+D^{-1}\boldsymbol{b}.$$

实际计算中, $\boldsymbol{x}_{k+1}$ 由下列递推关系计算:

$$x_i^{(k+1)}=\frac{1}{a_{ii}}\left(b_i-\sum_{j<i}a_{ij}x_j^{(k)}-\sum_{j>i}a_{ij}x_j^{(k)}\right) \tag{1.14}$$

其中 $i=1,2,\ldots,n$, $x_i^{(k)}$ 和 b_i 分别为 $\boldsymbol{x}_k$ 和 $\boldsymbol{b}$ 的第 i 个元素, 符号 $\sum_{j<i}$ 和 $\sum_{j>i}$ 分别表示 $\sum_{j=1}^{i-1}$ 和 $\sum_{j=i+1}^{n}$.

1.6.2 Gauss–Seidel 方法

由 (1.13) 和 Table 1.1, Gauss–Seidel (GS) 方法的迭代向量 $\boldsymbol{x}_k$ 如下:

$$\boldsymbol{x}_{k+1}=-(D+L)^{-1}U\boldsymbol{x}_k+(D+L)^{-1}\boldsymbol{b}.$$

下面推导 GS 方法. 重新考虑 Jacobi 方法. 用 (1.14) 计算 Jacobi 方法的 $x_i^{(k+1)}$ 时, $x_1^{(k+1)},\ldots,x_{i-1}^{(k+1)}$ 已经计算出来, 可看作新的信息. 因此, 若用新信息 $\sum_{j<i}a_{ij}x_j^{(k+1)}$ 代替旧信息 $\sum_{j<i}a_{ij}x_j^{(k)}$, 就可期望更快的收敛. 这正是 Gauss–Seidel 方法的核心思想, 其形式如下:

$$x_i^{(k+1)}=\frac{1}{a_{ii}}\left(b_i-\sum_{j<i}a_{ij}x_j^{(k+1)}-\sum_{j>i}a_{ij}x_j^{(k)}\right) \tag{1.15}$$

其中 $i=1,2,\ldots,n$.

1.6.3 SOR 方法

由 (1.13) 和 Table 1.1, SOR 方法的迭代向量 $\boldsymbol{x}_k$ 如下:

$$\begin{aligned}\boldsymbol{x}_{k+1}&=\left[\frac{1}{\omega}(D+\omega L)\right]^{-1}\left\{\frac{1}{\omega}[(1-\omega)D-\omega U]\right\}\boldsymbol{x}_k+\left[\frac{1}{\omega}(D+\omega L)\right]^{-1}\boldsymbol{b}\\&=(D+\omega L)^{-1}[(1-\omega)D-\omega U]\boldsymbol{x}_k+(D+\omega L)^{-1}\omega\boldsymbol{b},\end{aligned}$$

等价地, SOR 方法的 $\boldsymbol{x}_{k+1}$ 可通过求解下列线性方程组得到:

$$(D+\omega L)\boldsymbol{x}_{k+1}=[(1-\omega)D-\omega U]\boldsymbol{x}_k+\omega\boldsymbol{b}.$$

SOR 方法逐元素的迭代如下:

$$\tilde{x}_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\right),$$
$$x_i^{(k+1)} = x_i^{(k)} + \omega(\tilde{x}_i^{(k+1)} - x_i^{(k)})$$

其中 $i = 1, 2, \ldots, n$. 由此容易看出, SOR 方法是 Gauss–Seidel 方法的单参数推广, 因为第一个等式的右端与 Gauss–Seidel 方法 (1.15) 相同. 若 $\omega = 1$, SOR 方法就退化为 Gauss–Seidel 方法. 已知收敛的必要条件是 $0 < \omega < 2$. 当 A 对称时, 对称 SOR (SSOR) 方法也很著名, 因为它可用作求解对称线性方程组的 Krylov 子空间方法的预条件子. 关于 SSOR 预条件子, 见 Section 3.5.4. SSOR 方法的迭代如下:

$$(D + \omega L)\tilde{\boldsymbol{x}}_k = [(1-\omega)D - \omega U]\boldsymbol{x}_k + \omega\boldsymbol{b},$$
$$(D + \omega U)\boldsymbol{x}_k = [(1-\omega)D - \omega L]\tilde{\boldsymbol{x}}_k + \omega\boldsymbol{b}.$$

对于 SSOR 方法, (1.13) 中的 M 和 N 为

$$M_{\mathrm{SSOR}} = \frac{1}{\omega(2-\omega)}(D + \omega L)D^{-1}(D + \omega U), \tag{1.16}$$

$$N_{\mathrm{SSOR}} = \frac{1}{\omega(2-\omega)}[(1-\omega)D - \omega L]D^{-1}[(1-\omega)D - \omega U]. \tag{1.17}$$

由于 A 对称, 有 $U = L^\top$. 因此, 由 (1.16) 和 (1.17) 可知, M_{SSOR} 和 N_{SSOR} 对称. 文献 [93] 总结了包括 SSOR 方法在内的许多 SOR 方法基本结果. 关于 SOR 方法的近期发展, 例如可参见 [128], [130], [131] 及其中的参考文献. 文献 [131] 基于 SOR 方法与 [130] 中梯度系统的离散梯度方法之间的一种新联系, 提出了自适应 SOR 方法, 从而给出了 SOR 方法的几何视角.

1.6.4 定常迭代方法的收敛性

定常迭代方法的一个显著性质是, 可通过计算 $\|M^{-1}N\|$ 来估计误差的衰减率, 其中 $\|\cdot\|$ 是从属矩阵范数 (1.2). 为说明这一点, 由 (1.13) 以及关系 $A\boldsymbol{x} = \boldsymbol{b} \iff \boldsymbol{x} = M^{-1}N\boldsymbol{x} + M^{-1}\boldsymbol{b}$, 误差为

$$\begin{aligned}\boldsymbol{e}_{k+1} &:= \boldsymbol{x} - \boldsymbol{x}_{k+1}\\ &= (M^{-1}N\boldsymbol{x} + M^{-1}\boldsymbol{b}) - (M^{-1}N\boldsymbol{x}_k + M^{-1}\boldsymbol{b})\\ &= M^{-1}N(\boldsymbol{x} - \boldsymbol{x}_k)\\ &= M^{-1}N\boldsymbol{e}_k.\end{aligned} \tag{1.18}$$

因此, 误差的范数满足

$$\|\boldsymbol{e}_{k+1}\| \le \|M^{-1}N\|\|\boldsymbol{e}_k\|.$$

误差的衰减率为 $\|M^{-1}N\|$, 这表明它线性收敛. 设 λ_i $(i = 1, \ldots, n)$ 为一个 n 行 n 列矩阵 G 的特征值, 并令 $\rho(G) = \max\{|\lambda_1|, \ldots, |\lambda_n|\}$. 则称 $\rho(G)$ 为 G 的谱半径. 由 (1.18) 和 $M^{-1}N$ 的 Jordan 分解容易看出, 对任意初始猜测均收敛的充要条件是

$$\rho(M^{-1}N) < 1. \tag{1.19}$$

下面介绍保证 (1.19) 成立的一个最著名的事实. **Definition 1.3 (正则分裂):** 若 $M^{-1} \ge O$, $N \ge O$, 即 M^{-1} 和 N 的全部元素都大于或等于零, 则称 $A = M - N$ 为正则分裂.

Theorem 1.4:
若 $A = M - N$ 为正则分裂, 则

$$\rho(M^{-1}N) < 1 \iff A^{-1} \geq O.$$

Proof: 先证明 $\rho(M^{-1}N) < 1 \implies A^{-1} \geq O$. 令 $G := M^{-1}N$, λ_i 为 G 的特征值. 则 $I - G$ 的特征值均非零 (即非奇异), 因为 $I - G$ 的特征值为 $1 - \lambda_i$, 而由假设 $\rho(G) < 1$ 可知 $|\lambda_i| < 1$. 由于 M 和 $I - G$ 非奇异, $A = M(I - G)$ 也非奇异, 即 A^{-1} 存在. 由于 $\rho(G) < 1$, 有

$$A^{-1} = (I - G)^{-1}M^{-1} = (I + G + G^2 + \cdots)M^{-1}.$$

由于假设 $A = M - N$ 为正则分裂, 所以 $G \geq O$, $M^{-1} \geq O$. 因此 $A^{-1} \geq O$. 接下来证明逆命题. 由于 $A = M - N$ 为正则分裂, 有 $M^{-1} \geq O$, $N \geq O$. 因此 $G \geq O$. 由 Perron–Frobenius 定理可知, 存在非负特征向量 $\boldsymbol{x} \geq \mathbf{0}$, 使得 $G\boldsymbol{x} = \rho(G)\boldsymbol{x}$. 容易看出, $A^{-1}N = (I - G)^{-1}G$, 因而有

$$A^{-1}N\boldsymbol{x} = (I - G)^{-1}G\boldsymbol{x} = \frac{\rho(G)}{1 - \rho(G)}\boldsymbol{x}.$$

由于 $A^{-1} \geq O$, $N \geq O$, 有 $A^{-1}N\boldsymbol{x} \geq \mathbf{0}$. 这意味着 $\rho(G)/(1 - \rho(G)) \geq 0$, 即 $(0 \leq)\rho(G) \leq 1$. 由于 $(I - G)^{-1}$ 存在, 所以 $\rho(G) \neq 1$.

■

关于定常迭代方法的更多收敛定理, 例如可参见 Varga 的优秀著作 [200]. 定常迭代方法还可用作 Section 3.5.4 中 Krylov 子空间方法的预条件子.

1.7 多重网格方法

多重网格方法是一类为求解椭圆型偏微分方程所产生的线性方程组而发展起来的迭代方法. 它利用较小线性方程组 (来自分层的较粗网格) 的若干粗略近似解, 以及最小线性方程组 (来自最粗网格) 的精确近似解所提供的信息, 求得原线性方程组的良好近似解. 为得到粗略近似解, 常使用称为光滑子的定常迭代方法 (尤其是 Gauss–Seidel 方法). 光滑子这一名称来自这样一个事实: Gauss–Seidel 方法能够快速衰减线性方程组的误差或残差中的高频分量, 而多重网格方法充分利用了这一性质. 多重网格方法的显著特点是收敛率不依赖于网格尺寸, 而定常迭代方法和 Krylov 子空间方法的收敛率依赖于网格尺寸. 后来, 多重网格方法被推广到求解其他线性方程组, 不仅包括椭圆型问题产生的方程组, 也包括其他模型产生的方程组. 多重网格方法是一个研究活跃的课题, 已有许多优秀著作, 其中一些作者为:

- Hackbusch [91],
- Wesseling [204],
- Bramble [27],
- Briggs et al. [28],
- Trottenberg et al. [193],
- Lottes [125].

Wolfgang Hackbusch 的著作不仅借助简单的一维微分方程介绍多重网格方法的基本概念, 还给出算法, 程序代码以及详细的收敛分析. James Lottes 的著作则主要讨论代数多重网格方法.

1.8 Krylov 子空间方法

Krylov 子空间方法有着悠久的历史, 可追溯至 20 世纪中叶, 其中最著名的是 Hestenes 和 Stiefel 提出的共轭梯度 (CG) 方法.[4]

Krylov 子空间方法是利用如下定义的 Krylov 子空间来求解线性方程组 (1.1) 的迭代方法.

Definition 1.5 (Krylov 子空间): 设向量 $\boldsymbol{v}, A\boldsymbol{v}, \ldots, A^{n-1}\boldsymbol{v} \in \mathbb{C}^N$ 线性无关, 则

$$\begin{aligned}\mathcal{K}_n(A, \boldsymbol{v}) &= \{c_0\boldsymbol{v} + c_1 A\boldsymbol{v} + \cdots + c_{n-1}A^{n-1}\boldsymbol{v} : c_0, c_1, \ldots, c_{n-1} \in \mathbb{C}\} \\ &\quad (= \operatorname{span}\{\boldsymbol{v}, A\boldsymbol{v}, \ldots, A^{n-1}\boldsymbol{v}\})\end{aligned} \tag{1.20}$$

称为一个 (n 维) Krylov 子空间.

$\mathcal{K}_n(A, \boldsymbol{v})$ 的维数依赖于 $\boldsymbol{v}$, 对于给定的 $\boldsymbol{v}$, 其最大维数称为 Krylov 子空间的阶, 定义如下.

Definition 1.6 (Krylov 子空间的阶): 使得

$$\dim(\mathcal{K}_n(A, \boldsymbol{v})) = \dim(\mathcal{K}_{n+1}(A, \boldsymbol{v})) \tag{1.21}$$

成立的最小数 n, 称为 A 关于 $\boldsymbol{v}$ 的阶.

考虑 $A\boldsymbol{x} = \boldsymbol{b}$, 设 m 为 A 关于 $\boldsymbol{r}_0 := \boldsymbol{b} - A\boldsymbol{x}_0$ 的阶, 其中 $\boldsymbol{x}_0$ 是 $A\boldsymbol{x} = \boldsymbol{b}$ 的解的一个初始猜测. 则

$$\dim(\mathcal{K}_m(A, \boldsymbol{r}_0)) = \dim(\mathcal{K}_{m+1}(A, \boldsymbol{r}_0)).$$

这意味着存在 $c_0, c_1, \ldots, c_{m-1}$, 使得

$$A^m\boldsymbol{r}_0 = c_0\boldsymbol{r}_0 + c_1 A\boldsymbol{r}_0 + \cdots + c_{m-1}A^{m-1}\boldsymbol{r}_0.$$

将等式乘以 A^{-1}, 得到

$$A^{m-1}\boldsymbol{r}_0 = c_0 A^{-1}\boldsymbol{r}_0 + c_1\boldsymbol{r}_0 + c_2 A\boldsymbol{r}_0 + \cdots + c_{m-1}A^{m-2}\boldsymbol{r}_0.$$

由于 $A^{-1}\boldsymbol{r}_0 = A^{-1}(\boldsymbol{b} - A\boldsymbol{x}_0) = \boldsymbol{x} - \boldsymbol{x}_0$, 当 $c_0 \neq 0$ 时, 解可以写为

$$\boldsymbol{x} = \boldsymbol{x}_0 + \frac{1}{c_0}(-c_1\boldsymbol{r}_0 - c_2 A\boldsymbol{r}_0 - \cdots - c_{m-1}A^{m-2}\boldsymbol{r}_0 + A^{m-1}\boldsymbol{r}_0)$$

令 $\boldsymbol{z}_m := (-c_1\boldsymbol{r}_0 - \cdots - c_{m-1}A^{m-2}\boldsymbol{r}_0 + A^{m-1}\boldsymbol{r}_0)/c_0$. 则 $\boldsymbol{z}_m \in \mathcal{K}_m(A, \boldsymbol{r}_0)$, 这意味着解 $\boldsymbol{x}$ 满足

$$\boldsymbol{x} = \boldsymbol{x}_0 + \boldsymbol{z}_m, \quad \boldsymbol{z}_m \in \mathcal{K}_m(A, \boldsymbol{r}_0).$$

由此可见, 解 $\boldsymbol{x}$ 属于仿射空间 $\boldsymbol{x}_0 + \mathcal{K}_m(A, \boldsymbol{r}_0)$. 这一事实自然引出在仿射空间 $\boldsymbol{x}_n \in \boldsymbol{x}_0 + \mathcal{K}_n(A, \boldsymbol{r}_0)$ 中计算近似解 $\boldsymbol{x}_n$ 的想法, 或等价地, 对于 $n = 1, 2, \ldots$,

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + \boldsymbol{z}_n, \quad \boldsymbol{z}_n \in \mathcal{K}_n(A, \boldsymbol{r}_0)$$

[4]在 CG 方法提出之后, Stiefel 于 1955 年提出了共轭残差 (CR) 方法. 他还因 Stiefel 流形而闻名, 该流形是单位圆的推广.

若恰当地确定 $\boldsymbol{z}_n$, 则这一方法经过 m 步迭代就能得到精确解, 这正是 Krylov 子空间方法的基础.

下面描述 Krylov 子空间方法的框架. 设 $\boldsymbol{x}_0 \in \mathbb{C}^N$ 为初始猜测, $\boldsymbol{r}_0 := \boldsymbol{b} - A\boldsymbol{x}_0$ 为相应的残差向量. 则 Krylov 子空间方法在下列仿射空间中生成第 n 个近似解:

Krylov 子空间条件

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + \boldsymbol{z}_n, \quad \boldsymbol{z}_n \in \mathcal{K}_n(A, \boldsymbol{r}_0). \tag{1.22}$$

相应的残差向量 $\boldsymbol{r}_n$ 由下式给出:

$$\boldsymbol{r}_n := \boldsymbol{b} - A\boldsymbol{x}_n = \boldsymbol{r}_0 - A\boldsymbol{z}_n, \quad \boldsymbol{r}_n \in \mathcal{K}_{n+1}(A, \boldsymbol{r}_0). \tag{1.23}$$

由于仅利用上述条件 (1.22) 无法唯一确定 $\boldsymbol{x}_n$, 为得到唯一的近似解, 还需要施加下列条件之一:

Ritz–Galerkin 途径

$$\boldsymbol{r}_n \perp \mathcal{K}_n(A, \boldsymbol{r}_0). \tag{1.24}$$

Petrov–Galerkin 途径

$$\boldsymbol{r}_n \perp W_n \subset \mathbb{C}^N, \quad \dim(W_n) = n. \tag{1.25}$$

最小残差途径

$$\min_{\boldsymbol{x}_n \in \boldsymbol{x}_0 + \mathcal{K}_n(A, \boldsymbol{r}_0)} \|\boldsymbol{b} - A\boldsymbol{x}_n\|. \tag{1.26}$$

Petrov–Galerkin 途径包含 Ritz–Galerkin 途径和最小残差途径, 如下所述.

Proposition 1.7: 下列陈述成立:

- Ritz–Galerkin 途径 (1.24) 就是在 Petrov–Galerkin 途径 (1.25) 中选择

$$W_n = \mathcal{K}_n(A, \boldsymbol{r}_0).$$

- 最小残差途径 (1.26) 就是在 Petrov–Galerkin 途径 (1.25) 中选择

$$W_n = A\mathcal{K}_n(A, \boldsymbol{r}_0), \tag{1.27}$$

其中 $A\mathcal{K}_n(A, \boldsymbol{r}_0) := \{\sum_{k=1}^{n} c_k A\boldsymbol{v}_k : c_1, c_2, \ldots, c_n \in \mathbb{C}\}$, 而 $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$ 是 $\mathcal{K}_n(A, \boldsymbol{r}_0)$ 的基向量.

Proof: 对于 Ritz–Galerkin 途径, 结论是显然的, 因此只证明 (1.26) 与 (1.27) 的等价性. 设 $V_n = [\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n]$ 是一个 N 行 n 列的矩阵, 其列向量 $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$ 是 $\mathcal{K}_n(A, \boldsymbol{r}_0)$ 的基向量. 则 (1.22) 中的 $\boldsymbol{z}_n \in \mathcal{K}_n(A, \boldsymbol{r}_0)$ 可写为 $\mathcal{K}_n(A, \boldsymbol{r}_0) \ni \boldsymbol{z}_n = \sum_{k=1}^{n} c_k \boldsymbol{v}_k = V_n \boldsymbol{c}_n$, 其中 $\boldsymbol{c}_n = [c_1, c_2, \ldots, c_n]^\top \in \mathbb{C}^n$. 将 $\boldsymbol{z}_n = V_n \boldsymbol{c}_n$ 代入 (1.23), 得到

$$\boldsymbol{r}_n = \boldsymbol{r}_0 - AV_n \boldsymbol{c}_n. \tag{1.28}$$

注意, 若 $\boldsymbol{r}_n \perp A\boldsymbol{v}_k$ $(k = 1, 2, \ldots, n)$, 则有 $\boldsymbol{r}_n \perp A\mathcal{K}_n(A, \boldsymbol{r}_0)$, 因为容易看出, 对所有 $c_i \in \mathbb{C}$, 都有 $\boldsymbol{r}_n \perp \sum_{k=1}^{n} c_k A\boldsymbol{v}_k$. 条件 $\boldsymbol{r}_n \perp A\boldsymbol{v}_k$ $(k = 1, 2, \ldots, n)$ 等价于 $(AV_n)^{\mathrm{H}} \boldsymbol{r}_n = \boldsymbol{0}$ $(:= [0, \ldots, 0]^\top)$, 因此由 (1.28) 可得 $\boldsymbol{0} = (AV_n)^{\mathrm{H}} \boldsymbol{r}_n = (AV_n)^{\mathrm{H}} \boldsymbol{r}_0 - (AV_n)^{\mathrm{H}} AV_n \boldsymbol{c}_n$. 从而向量 $\boldsymbol{c}$ 由下式确定:

$$\boldsymbol{c}_n = [(AV_n)^{\mathrm{H}} AV_n]^{-1} (AV_n)^{\mathrm{H}} \boldsymbol{r}_0. \tag{1.29}$$

现在证明 $\boldsymbol{c}_n$ 是极小化问题 (1.26) 的解. 由于使 $\|\boldsymbol{b}-A\boldsymbol{x}_n\|^2$ 取最小值的解与 (1.26) 的解相同, 我们考虑

$$\begin{aligned}\min_{\boldsymbol{x}_n\in\boldsymbol{x}_0+\mathcal{K}_n(A,\boldsymbol{r}_0)}\|\boldsymbol{b}-A\boldsymbol{x}_n\|^2 &= \min_{\boldsymbol{c}_n\in\mathbb{C}^n}\|\boldsymbol{r}_0-AV_n\boldsymbol{c}_n\|^2\\ &= \min_{\boldsymbol{c}_n\in\mathbb{C}^n}(\boldsymbol{r}_0-AV_n\boldsymbol{c}_n)^{\mathrm{H}}(\boldsymbol{r}_0-AV_n\boldsymbol{c}_n).\end{aligned}\tag{1.30}$$

下面的分解有助于求得 (1.30) 的解:

$$\|\boldsymbol{d}-M\boldsymbol{x}\|^2=(\boldsymbol{d}-M\boldsymbol{x})^{\mathrm{H}}(\boldsymbol{d}-M\boldsymbol{x})=f(\boldsymbol{x})+c,\tag{1.31}$$

其中

$$\begin{aligned}f(\boldsymbol{x})&=(M^{\mathrm{H}}M\boldsymbol{x}-M^{\mathrm{H}}\boldsymbol{d})^{\mathrm{H}}(M^{\mathrm{H}}M)^{-1}(M^{\mathrm{H}}M\boldsymbol{x}-M^{\mathrm{H}}\boldsymbol{d}),\\ c&=-\boldsymbol{d}^{\mathrm{H}}M(M^{\mathrm{H}}M)^{-1}M^{\mathrm{H}}\boldsymbol{d}+\boldsymbol{d}^{\mathrm{H}}\boldsymbol{d}.\end{aligned}$$

若 M 列满秩, 则 $M^{\mathrm{H}}M$ 是 Hermitian 正定的,[5] 从而 $f(\boldsymbol{x})\geq 0$. 这意味着 (1.31) 的极小化可通过极小化 $f(\boldsymbol{x})$ 实现. 容易看出,

$$\boldsymbol{x}=(M^{\mathrm{H}}M)^{-1}M^{\mathrm{H}}\boldsymbol{d}\tag{1.32}$$

使得 $f(\boldsymbol{x})=0$, 因而它是 (1.31) 所对应的极小化问题的解. 令 $\boldsymbol{x}=(M^{\mathrm{H}}M)^{-1}M^{\mathrm{H}}\boldsymbol{d}$, 并取 $M=AV_n$ 和 $\boldsymbol{d}=\boldsymbol{r}_0$, 就得到与 (1.29) 相同的结果.

■

Remark 1.8: 等式 (1.31) 也可用于求解下列极小化问题:

$$\min_{\omega\in\mathbb{C}}\|\boldsymbol{a}-\omega\boldsymbol{b}\|\tag{1.33}$$

因为在 (1.32) 中取 $\boldsymbol{d}=\boldsymbol{a}$, $M=\boldsymbol{b}$, $\boldsymbol{x}=\omega$, 就得到 (1.33) 的解如下:

$$\omega=\frac{\boldsymbol{b}^{\mathrm{H}}\boldsymbol{a}}{\boldsymbol{b}^{\mathrm{H}}\boldsymbol{b}}.\tag{1.34}$$

这一结果将在 Chapter 3 的 Krylov 子空间方法中使用.

现在, 代表性的 Krylov 子空间方法与三种途径 (1.24)–(1.26) 之间的关系见 Table 1.2 和 Table 1.3.

表 1.2: Hermitian 线性方程组的 Krylov 子空间方法

Ritz–Galerkin 途径	最小残差途径
CG	MINRES, CR

表 1.3: 非 Hermitian 线性方程组的一些 Krylov 子空间方法

Ritz–Galerkin 途径	Petrov–Galerkin 途径	最小残差途径
FOM	BiCG, BiCR	GMRES, GCR

针对 Hermitian 和非 Hermitian 线性方程组的 Krylov 子空间方法之间的关系如下: BiCG 方法和 FOM 是 CG 方法的推广; GMRES 方法是 MINRES 方法的推广; BiCR 方法和 GCR 方法是 CR 方法的推广. 这些 Krylov 子空间方法的详细内容见 Chapter 3.

[5]若矩阵 $G\in\mathbb{C}^{N\times N}$ 是 Hermitian 的, 且对所有 $\boldsymbol{v}(\neq\boldsymbol{0})\in\mathbb{C}^N$ 均有 $\boldsymbol{v}^{\mathrm{H}}G\boldsymbol{v}>0$, 则称 G 为 Hermitian 正定矩阵. 等价地, Hermitian 矩阵 G 的所有特征值均为正. 由此可知, 若 G 是 Hermitian 正定的, 则 G^{-1} 也是 Hermitian 正定的. 关于正定性, 另见 Section 1.4.

1.9 Krylov 子空间的正交化方法

为满足 Section 1.8 中的 Ritz–Galerkin, Petrov–Galerkin 或最小残差途径, 需要构造 Krylov 子空间的基向量. 对于非 Hermitian 矩阵, 使用 Arnoldi 过程和双 Lanczos 过程构造 Krylov 子空间的基向量. 对于复对称矩阵, 使用复对称 Lanczos 过程; 对于 Hermitian 矩阵, 使用 Lanczos 过程. Arnoldi 过程生成 Krylov 子空间 $\mathcal{K}_n(A,\boldsymbol{v})$ 的标准正交基向量. Arnoldi 过程不会发生中断 (即零除), 但计算量随 n 增大而线性增长. 双 Lanczos 过程生成 Krylov 子空间的双正交基向量. 其计算量不随 n 增大而线性增长, 因而小于 Arnoldi 方法的计算量. 然而, 它可能发生中断, 接近中断时会导致基向量的正交性丧失. 复对称 Lanczos 过程和 Lanczos 过程都是双 Lanczos 过程的特例. 特别地, 由于 Lanczos 过程也是 Arnoldi 过程的特例, Lanczos 过程兼具 Arnoldi 过程和双 Lanczos 过程的优点, 即 Lanczos 过程生成标准正交基向量, 且计算量不随 n 增大而线性增长. 下面各小节将介绍这些方法, 并给出其算法以及将在推导 Krylov 子空间方法时使用的矩阵表示.

1.9.1 Arnoldi 过程

Arnoldi 过程生成 Krylov 子空间 $\mathcal{K}_n(A,\boldsymbol{v})$ 的正交化基向量, 即基向量 $\boldsymbol{v}_1,\boldsymbol{v}_2,\dots,\boldsymbol{v}_n$ 满足

$$(\boldsymbol{v}_i,\boldsymbol{v}_j)=\delta_{i,j},$$

其中 $(\boldsymbol{x},\boldsymbol{y}):=\boldsymbol{x}^{\mathrm{H}}\boldsymbol{y}=\sum_{i=1}^{N}\bar{x}_i y_i$ 为标准点积, $\delta_{i,j}$ 为 Kronecker 符号, 即 $i=j$ 时 $\delta_{i,j}=1$, $i\neq j$ 时 $\delta_{i,j}=0$. 为简化叙述, 考虑 $n=3$ 时生成 $\mathcal{K}_n(A,\boldsymbol{v})$ 的标准正交向量, 即由 $\operatorname{span}\{\boldsymbol{v},A\boldsymbol{v},A^2\boldsymbol{v}\}$ 生成 $\boldsymbol{v}_1,\boldsymbol{v}_2,\boldsymbol{v}_3$. 从 $\operatorname{span}\{\boldsymbol{v},A\boldsymbol{v},A^2\boldsymbol{v}\}$ 获得标准正交向量的一种方法是使用 Gram–Schmidt 过程, 按下式计算 $\boldsymbol{v}_1,\boldsymbol{v}_2$ 和 $\boldsymbol{v}_3$:

$$\begin{aligned}\boldsymbol{v}_1&=z_1\boldsymbol{v},\\ \boldsymbol{v}_2&=z_2(A\boldsymbol{v}-c_1\boldsymbol{v}_1),\\ \boldsymbol{v}_3&=z_3(A^2\boldsymbol{v}-d_1\boldsymbol{v}_1-d_2\boldsymbol{v}_2).\end{aligned}$$

这里 z_i $(i=1,2,3)$ 是使归一化条件 $\|\boldsymbol{v}_i\|=1$ 成立的标量. 确定标量 c_1, 使 $\boldsymbol{v}_1$ 和 $\boldsymbol{v}_2$ 正交, 即 $(\boldsymbol{v}_1,\boldsymbol{v}_2)=0$. 类似地, 确定标量 d_1 和 d_2, 使 $\boldsymbol{v}_3\perp\boldsymbol{v}_2$, $\boldsymbol{v}_3\perp\boldsymbol{v}_1$, 即 $(\boldsymbol{v}_1,\boldsymbol{v}_3)=(\boldsymbol{v}_2,\boldsymbol{v}_3)=0$. 然而, 已知这种途径数值不稳定, 因为当 n 较大时, $A^n\boldsymbol{v}$ 和 $A^{n-1}\boldsymbol{v}$ 之间的夹角可能接近零, 从而导致严重的相消. 实际上, 在适当条件下, $A^n\boldsymbol{v}$ 收敛于 A 的一个特征向量, 其对应特征值的模在 A 的全部特征值中最大. Arnoldi 过程对 (经典) Gram-Schmidt 过程作如下小幅修改:

$$\boldsymbol{v}_1=z_1\boldsymbol{v}, \tag{1.35}$$

$$\boldsymbol{v}_2=z_2(A\boldsymbol{v}_1-c_1\boldsymbol{v}_1), \tag{1.36}$$

$$\boldsymbol{v}_3=z_3(A\boldsymbol{v}_2-d_1\boldsymbol{v}_1-d_2\boldsymbol{v}_2). \tag{1.37}$$

与 Gram–Schmidt 过程不同, 这里不计算 $A^2\boldsymbol{v}$. 下面确定各个标量. 令 $z_1=1/(\boldsymbol{v},\boldsymbol{v})^{1/2}\in\mathbb{R}$, 等价地为 $1/\|\boldsymbol{v}\|$. 则 $\|\boldsymbol{v}_1\|=1$, 因为 $\|\boldsymbol{v}_1\|=(z_1\boldsymbol{v},z_1\boldsymbol{v})^{1/2}=(z_1^2(\boldsymbol{v},\boldsymbol{v}))^{1/2}=z_1(\boldsymbol{v},\boldsymbol{v})^{1/2}=1$. 再确定 c_1, 使 $(\boldsymbol{v}_1,\boldsymbol{v}_2)=0$, 即

$$\begin{aligned}0&=(\boldsymbol{v}_1,\boldsymbol{v}_2)\\ &=(\boldsymbol{v}_1,z_2(A\boldsymbol{v}_1-c_1\boldsymbol{v}_1))\\ &=z_2((\boldsymbol{v}_1,A\boldsymbol{v}_1)-c_1(\boldsymbol{v}_1,\boldsymbol{v}_1))=z_2((\boldsymbol{v}_1,A\boldsymbol{v}_1)-c_1).\end{aligned}$$

因此 $c_1 = (\boldsymbol{v}_1, A\boldsymbol{v}_1)$, 而 z_2 由 $\|\boldsymbol{v}_2\| = 1$ 确定, 即 $z_2 = 1/\|A\boldsymbol{v}_1 - c_1\boldsymbol{v}_1\|$. 类似地, (1.37) 中的标量 d_1, d_2 由 $(\boldsymbol{v}_1, \boldsymbol{v}_3) = (\boldsymbol{v}_2, \boldsymbol{v}_3) = 0$ 确定, z_3 由 $\|\boldsymbol{v}_3\| = 1$ 确定. 经典 Gram–Schmidt 型 Arnoldi 过程见 Algorithm 1.7; 更精确的改进 Gram–Schmidt 型 Arnoldi 过程见 Algorithm 1.8.

Algorithm 1.7 经典 Gram–Schmidt 型 Arnoldi 过程

Input: 非 Hermitian 矩阵 $A \in \mathbb{C}^{N\times N}$, 并选择满足 $\|\boldsymbol{v}_1\| = 1$ 的 $\boldsymbol{v}_1$

Output: $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N$

for $n = 1, 2, \ldots, N-1$ **do**
  $h_{i,n} = (\boldsymbol{v}_i, A\boldsymbol{v}_n), \quad i = 1, 2, \ldots, n$
  $\tilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \sum_{i=1}^{n} h_{i,n}\boldsymbol{v}_i$
  $h_{n+1,n} = \|\tilde{\boldsymbol{v}}_{n+1}\|$
  $\boldsymbol{v}_{n+1} = \tilde{\boldsymbol{v}}_{n+1}/h_{n+1,n}$
end for

Algorithm 1.8 改进 Gram–Schmidt 型 Arnoldi 过程

Input: 非 Hermitian 矩阵 $A \in \mathbb{C}^{N\times N}$, 并选择满足 $\|\boldsymbol{v}_1\| = 1$ 的 $\boldsymbol{v}_1$

Output: $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N$

for $n = 1, 2, \ldots, N-1$ **do**
  $\boldsymbol{t} = A\boldsymbol{v}_n$
  for $i = 1, 2, \ldots, n$ **do**
    $h_{i,n} = (\boldsymbol{v}_i, \boldsymbol{t})$
    $\boldsymbol{t} = \boldsymbol{t} - h_{i,n}\boldsymbol{v}_i$
  end for
  $h_{n+1,n} = \|\boldsymbol{t}\|$
  $\boldsymbol{v}_{n+1} = \boldsymbol{t}/h_{n+1,n}$
end for

Arnoldi 过程可表示为矩阵形式. 例如, 考虑递推式 (1.36) 和 (1.37) 的矩阵形式. 由 (1.36) 和 (1.37) 有

$$A\boldsymbol{v}_1 = z_2^{-1}\boldsymbol{v}_2 + c_1\boldsymbol{v}_1, \quad A\boldsymbol{v}_2 = z_3^{-1}\boldsymbol{v}_3 + d_1\boldsymbol{v}_1 + d_2\boldsymbol{v}_2,$$

或等价地

$$A[\boldsymbol{v}_1, \boldsymbol{v}_2] = [\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3] \begin{bmatrix} c_1 & d_1 \\ z_2^{-1} & d_2 \\ 0 & z_3^{-1} \end{bmatrix}.$$

注意, $A[\boldsymbol{v}_1, \boldsymbol{v}_2] = [A\boldsymbol{v}_1, A\boldsymbol{v}_2]$. 类似地, 可按下面的方式得到 Algorithm 1.7 (或 Algorithm 1.8) 的矩阵形式. 设 V_n 为 $N \times n$ 矩阵, 其列是 Algorithm 1.7 (或 Algorithm 1.8) 给出的前 n 个标准正交向量, 即

$$V_n = [\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n], \quad V_n^{\mathrm{H}} V_n = I_n,$$

设 $H_{n+1,n}$ 为 $(n+1)\times n$ Hessenberg 矩阵, 其元素在 $i>j+1$ 时满足 $h_{i,j}=0$, 并设 H_n 为 $n\times n$ Hessenberg 矩阵, 如下:

$$H_{n+1,n}=\begin{bmatrix} h_{1,1} & h_{1,2} & \cdots & h_{1,n-1} & h_{1,n} \\ h_{2,1} & h_{2,2} & \cdots & h_{2,n-1} & h_{2,n} \\ & \ddots & \ddots & \vdots & \vdots \\ & & \ddots & \ddots & \vdots \\ & & & h_{n,n-1} & h_{n,n} \\ & & & & h_{n+1,n} \end{bmatrix}$$

以及

$$H_n=\begin{bmatrix} h_{1,1} & h_{1,2} & \cdots & h_{1,n-1} & h_{1,n} \\ h_{2,1} & h_{2,2} & \cdots & h_{2,n-1} & h_{2,n} \\ & \ddots & \ddots & \vdots & \vdots \\ & & \ddots & \ddots & \vdots \\ & & & h_{n,n-1} & h_{n,n} \end{bmatrix},$$

其中 $h_{i,j}$ 是 Algorithm 1.7 (或 Algorithm 1.8) 生成的标量. 则有

$$AV_n=V_{n+1}H_{n+1,n}=V_nH_n+h_{n+1,n}\boldsymbol{v}_{n+1}\boldsymbol{e}_n^{\top}, \tag{1.38}$$

其中 $\boldsymbol{e}_n=(0,0,\ldots,0,1)^{\top}\in\mathbb{R}^n$. 由上述矩阵形式以及 $V_n^{\mathrm{H}}V_n=I_n$, 得到下列形式的关系

$$H_n=V_n^{\mathrm{H}}AV_n. \tag{1.39}$$

如果 A 是 Hermitian 的, 则由 $H_n^{\mathrm{H}}=(V_n^{\mathrm{H}}AV_n)^{\mathrm{H}}=V_n^{\mathrm{H}}A^{\mathrm{H}}V_n=V_n^{\mathrm{H}}AV_n=H_n$ 可知, Hessenberg 矩阵 H_n 是 Hermitian 的. 因此, 此时 H_n 为三对角矩阵. 这意味着, 若 A 是 Hermitian 的, Arnoldi 过程就有短递推关系, 从而得到 Lanczos 过程. Lanczos 过程的详细内容将在 Section 1.9.4 介绍.

1.9.2 双 Lanczos 过程

双 Lanczos 过程是一种利用短递推获得 $\mathcal{K}_n(A,\boldsymbol{v}_0)$ 的双正交基的算法. 尽管该过程不生成标准正交基向量, 它在以较低内存需求求解线性方程组方面发挥着重要作用. 算法见 Algorithm 1.9. 如果不发生中断, Algorithm 1.9 生成两个 Krylov 子空间的双正交基向量, 满足

$$(\boldsymbol{w}_i,\boldsymbol{v}_j)=\delta_{i,j},$$

其中 $\mathcal{K}_n(A,\boldsymbol{v}_1)=\mathrm{span}\{\boldsymbol{v}_1,\ldots,\boldsymbol{v}_n\}$, $\mathcal{K}_n(A^{\mathrm{H}},\boldsymbol{w}_1)=\mathrm{span}\{\boldsymbol{w}_1,\ldots,\boldsymbol{w}_n\}$.

Algorithm 1.9 双 Lanczos 过程

Input: 非 Hermitian 矩阵 $A \in \mathbb{C}^{N\times N}$, 并选择满足 $(\boldsymbol{v}_0, \boldsymbol{w}_0) \neq 0$ 的 $\boldsymbol{v}_0$ 和 $\boldsymbol{w}_0$

Input: $\boldsymbol{v}_1 = \boldsymbol{v}_0/\|\boldsymbol{v}_0\|, \quad \boldsymbol{w}_1 = \boldsymbol{w}_0/\overline{(\boldsymbol{w}_0, \boldsymbol{v}_1)}$

Output: $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N$ 和 $\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_N$

1: **for** $n = 1, 2, \ldots, N-1$ **do**

2: $\quad \alpha_n = (\boldsymbol{w}_n, A\boldsymbol{v}_n)$,

3: $\quad \tilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \alpha_n \boldsymbol{v}_n - \beta_{n-1}\boldsymbol{v}_{n-1}$

4: $\quad \tilde{\boldsymbol{w}}_{n+1} = A^{\mathrm{H}}\boldsymbol{w}_n - \bar{\alpha}_n \boldsymbol{w}_n - \bar{\gamma}_{n-1}\boldsymbol{w}_{n-1}$

5: $\quad \gamma_n = \|\tilde{\boldsymbol{v}}_{n+1}\|$

6: $\quad \boldsymbol{v}_{n+1} = \tilde{\boldsymbol{v}}_{n+1}/\gamma_n$

7: $\quad \beta_n = (\tilde{\boldsymbol{w}}_{n+1}, \boldsymbol{v}_{n+1})$

8: $\quad \boldsymbol{w}_{n+1} = \tilde{\boldsymbol{w}}_{n+1}/\bar{\beta}_n$

9: **end for**

与 Arnoldi 过程类似, 双 Lanczos 过程也可写成矩阵形式. 设 $T_{n+1,n}$ 为 $(n+1)\times n$ 三对角矩阵, 其元素为双 Lanczos 过程的递推系数, 即

$$T_{n+1,n} = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \gamma_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \gamma_{n-1} & \alpha_n \\ & & & \gamma_n \end{bmatrix}, \quad T_n = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \gamma_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \gamma_{n-1} & \alpha_n \end{bmatrix}.$$

则由双 Lanczos 过程有

$$AV_n = V_{n+1}T_{n+1,n} = V_nT_n + \gamma_n \boldsymbol{v}_{n+1}\boldsymbol{e}_n^{\top}, \tag{1.40}$$

$$A^{\mathrm{H}}W_n = W_{n+1}T_{n+1,n}^{\mathrm{H}} = W_nT_n^{\mathrm{H}} + \bar{\beta}_n \boldsymbol{w}_{n+1}\boldsymbol{e}_n^{\top}.$$

由上式以及 $W_n^{\mathrm{H}}V_n = I_n$, 容易得到公式 $T_n = W_n^{\mathrm{H}}AV_n$.

1.9.3 复对称 Lanczos 过程

当系数矩阵为复对称矩阵, 即 $A = A^{\top} \neq A^{\mathrm{H}}$ 时, 复对称 Lanczos 过程是双 Lanczos 过程 (Algorithm 1.9) 的特例. 若 A 复对称, 则令 $\boldsymbol{w}_0 = \bar{\boldsymbol{v}}_0$, 就容易从 Algorithm 1.9 导出该过程, 得到的算法见 Algorithm 1.10. 如果不发生中断, Algorithm 1.10 生成 Krylov 子空间的共轭正交基向量, 满足

$$(\bar{\boldsymbol{v}}_i, \boldsymbol{v}_j) = \delta_{ij},$$

其中 $\mathcal{K}_n(A, \boldsymbol{v}_1) = \mathrm{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$.

Algorithm 1.10 复对称 Lanczos 过程 (注意: $(\bar{\boldsymbol{a}}, \boldsymbol{b}) = \boldsymbol{a}^{\top}\boldsymbol{b}$)

Input: 复对称矩阵 $A \in \mathbb{C}^{N\times N}$, 并选择满足 $(\bar{\boldsymbol{v}}_1, \boldsymbol{v}_1) \neq 0$ 的 $\boldsymbol{v}_1$

Input: $\boldsymbol{v}_1 = \boldsymbol{v}_1/(\bar{\boldsymbol{v}}_1, \boldsymbol{v}_1)^{1/2}$

Output: $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N$

1: **for** $n = 1, 2, \ldots, N-1$ **do**

2: $\quad \alpha_n = (\bar{\boldsymbol{v}}_n, A\boldsymbol{v}_n)$

3: $\quad \tilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \alpha_n\boldsymbol{v}_n - \beta_{n-1}\boldsymbol{v}_{n-1}$

4: $\quad \beta_n = (\bar{\tilde{\boldsymbol{v}}}_{n+1}, \tilde{\boldsymbol{v}}_{n+1})^{1/2}$

5: $\quad \boldsymbol{v}_{n+1} = \tilde{\boldsymbol{v}}_{n+1}/\beta_n$

6: **end for**

与 Arnoldi 过程和双 Lanczos 过程类似, Algorithm 1.10 也可写成矩阵形式. 设 $T_{n+1,n}$ 为 $(n+1)\times n$ 三对角矩阵, 其元素为复对称 Lanczos 过程的递推系数, 即

$$T_{n+1,n} = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \beta_{n-1} & \alpha_n \\ & & & \beta_n \end{bmatrix}, \quad T_n = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \beta_{n-1} & \alpha_n \end{bmatrix}.$$

则由 Algorithm 1.10 有

$$AV_n = V_{n+1}T_{n+1,n} = V_nT_n + \beta_n\boldsymbol{v}_{n+1}\boldsymbol{e}_n^{\top}. \tag{1.41}$$

由上式可知, T_n 也复对称, 即 $T_n = T_n^{\top} \neq T_n^{\mathrm{H}}$.

1.9.4 Lanczos 过程

Lanczos 过程是 Arnoldi 过程在 A 为 Hermitian 矩阵时的特例. 由于 H_n 既是 Hermitian 矩阵又是 Hessenberg 矩阵, 它是三对角矩阵. 这意味着, 在 Arnoldi 过程 (Algorithm 1.7) 的内循环中, 从 1 到 n 的求和可替换为从 $n-1$ 到 n 的求和. 因此, Lanczos 过程在第 n 步只需要三项递推, 而不需要 $(n+1)$ 项递推. 由于这一出色的性质, Lanczos 过程的每一步都比 Arnoldi 过程或双 Lanczos 过程的相应步骤简洁得多. Lanczos 过程见 Algorithm 1.11.

Algorithm 1.11 Lanczos 过程

Input: Hermitian 矩阵 $A \in \mathbb{C}^{N\times N}$, 并选择满足 $\|\boldsymbol{v}_1\| = 1$ 的 $\boldsymbol{v}_1$

Input: $\beta_0 = 0, \boldsymbol{v}_0 = \boldsymbol{0}$

Output: $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N$

1: **for** $n = 1, 2, \ldots, N-1$ **do**

2: $\quad \alpha_n = (\boldsymbol{v}_n, A\boldsymbol{v}_n)$

3: $\quad \tilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \alpha_n\boldsymbol{v}_n - \beta_{n-1}\boldsymbol{v}_{n-1}$

4: $\quad \beta_n = (\tilde{\boldsymbol{v}}_{n+1}, \tilde{\boldsymbol{v}}_{n+1})^{1/2}$

5: $\quad \boldsymbol{v}_{n+1} = \tilde{\boldsymbol{v}}_{n+1}/\beta_n$

6: **end for**

令 $\boldsymbol{w}_0 = \boldsymbol{v}_0$, 就容易从 Arnoldi 过程或双 Lanczos 过程导出 Lanczos 过程. 若不发生中断, 上述算法生成 $\mathcal{K}_n(A, \boldsymbol{v}_1)$ 的标准正交基, 满足

$$(\boldsymbol{v}_i, \boldsymbol{v}_j) = \delta_{ij},$$

其中 $\mathcal{K}_n(A, \boldsymbol{v}_1) = \mathrm{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$. 与 Arnoldi 过程和双 Lanczos 过程类似, Lanczos 过程也可写成矩阵形式. 设 $T_{n+1,n}$ 为 $(n+1) \times n$ 三对角矩阵, 其元素为 Lanczos 过程的递推系数, 即

$$T_{n+1,n} = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \beta_{n-1} & \alpha_n \\ & & & \beta_n \end{bmatrix}, \quad T_n := \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \beta_{n-1} & \alpha_n \end{bmatrix}.$$

则由 Lanczos 过程有

$$AV_n = V_{n+1}T_{n+1,n} = V_nT_n + \beta_n\boldsymbol{v}_{n+1}\boldsymbol{e}_n^{\top}. \tag{1.42}$$

由上式以及 $V_n^{\mathrm{H}}V_n = I_n$, 容易得到下列公式:

$$T_n = V_n^{\mathrm{H}}AV_n. \tag{1.43}$$

可见 T_n 也是 Hermitian 的 (更确切地说, 是实对称的). Lanczos 过程用于求解 Hermitian 线性方程组.

2 计算科学和数据科学中的若干应用

摘要 形如 $A\boldsymbol{x}=\boldsymbol{b}$ 的线性方程组出现在计算科学和数据科学等多种多样的应用中. 在计算科学中, 物理现象常用偏微分方程描述. 有限差分方法和有限元方法广泛用于求解偏微分方程, 由此产生求解线性方程组的问题. 凝聚态物理中的大规模电子结构计算和粒子物理中的格点量子色动力学都需要求解大规模线性方程组. 在数据科学中, 回归和分类是基本任务, 同样需要求解线性方程组. 求函数的极小值或极大值是最重要的优化问题之一, 使用 Newton 方法时需要求解线性方程组. 本章将通过简单而具体的例子, 说明这些应用中如何产生线性方程组.

2.1 偏微分方程

本节通过若干偏微分方程介绍有限差分方法和有限元方法, 并将说明: 所需的解越精确, 就需要求解规模越大的线性方程组.

2.1.1 有限差分方法

本小节通过几个简单的偏微分方程介绍有限差分方法. 还将说明张量积的概念, 它便于描述偏微分方程经有限差分离散后所产生线性方程组的系数矩阵. 首先给出一些数学预备知识.

2.1.1.1 预备知识 本小节介绍几个公式, 利用 $f(x), f(x-h)$ 和 $f(x+h)$, 在 $0<h<1$ 时近似光滑函数 $f(x)$ 在 x 处的导数 $f'(x), f''(x)$. 考虑如何近似 $f'(x)$. $f(x)$ 的 Taylor 级数为

$$f(x+h)=f(x)+f'(x)h+\frac{1}{2!}f''(x)h^2+\frac{1}{3!}f'''(x)h^3+\cdots. \tag{2.1}$$

于是, $f'(x)$ 可写成

$$\begin{aligned} f'(x)&=\frac{f(x+h)-f(x)}{h}-\frac{1}{2!}f''(x)h-\frac{1}{3!}f'''(x)h^2-\cdots\\ &=\frac{f(x+h)-f(x)}{h}+O(h). \end{aligned}$$

其中 $O(\cdot)$ 是大 O 记号. 粗略地说, 此处 $O(h)$ 表示当 $h\to 0$ 时, 对适当的常数值 c, $-\frac{1}{2!}f''(x)h-\frac{1}{3!}f'''(x)h^2-\cdots$ 可写成 ch. 用递推关系

$$f'(x)\approx\frac{f(x+h)-f(x)}{h} \tag{2.2}$$

来近似 $f'(x)$, 称为向前差分. 向前差分的精度为 h 阶. 类似的近似

$$f'(x)\approx\frac{f(x)-f(x-h)}{h} \tag{2.3}$$

称为向后差分. 向后差分的精度同样为 h 阶. 为得到更精确的公式, 由 (2.1) 有

$$\frac{f(x+h)-f(x-h)}{2h}=f'(x)+O(h^2),$$

由此可得

$$f'(x)\approx\frac{f(x+h)-f(x-h)}{2h}, \tag{2.4}$$

这称为 $f(x)$ 的一阶导数的中心差分, 精度为 h^2 阶. 现在考虑近似二阶导数 $f''(x)$. 由 $f(x+h)$ 和 $f(x-h)$ 的 Taylor 级数 (2.1) 有

$$f(x+h) - 2f(x) + f(x-h) = f''(x)h^2 + O(h^4).$$

因此

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + O(h^2).$$

下面的近似

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} \tag{2.5}$$

称为 $f(x)$ 的二阶导数的中心差分, 精度为 h^2 阶.

2.1.1.2 Example 1 (对称矩阵)

考虑如下形式的 Laplace 方程

$$\begin{aligned} u_{xx} + u_{yy} &= 0, \qquad (x,y) \in (0,1) \times (0,1), \\ u(x,y) &= g(x,y), \quad (x,y) \in \Gamma, \end{aligned}$$

其中 Γ 是 $(0,1) \times (0,1)$ 的边界. 具体地, 边界条件为 $u(0,y) = g(0,y)$, $u(1,y) = g(1,y)$, $u(x,0) = g(x,0)$ 和 $u(x,1) = g(x,1)$. 现在用中心差分 (2.5) 近似求解 Laplace 方程. 为此, 在 $(0,1) \times (0,1)$ 内采用含 $N \times N$ 个点的等距网格. $N = 3$ ($h = 1/4$) 的情形见 Fig. 2.1. 下面使

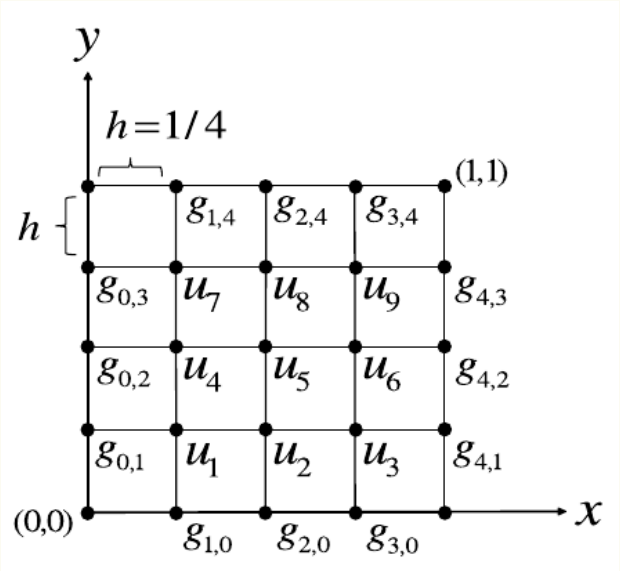


图 2.1: Example 1 (Laplace 方程) 的区域网格

用 Fig. 2.1 中的网格, 说明如何得到线性方程组. 近似 u_{xx} 和 u_{yy} 的中心差分 (2.5) 为

$$u_{xx}(x,y) \approx \frac{1}{h^2}(u(x+h,y) - 2u(x,y) + u(x-h,y)), \tag{2.6}$$

$$u_{yy}(x,y) \approx \frac{1}{h^2}(u(x,y+h) - 2u(x,y) + u(x,y-h)). \tag{2.7}$$

引入简写记号 $u_{i,j} = u(x_i, y_j)$ 很方便. 于是得到下面的五点公式:

$$u_{xx}(x_i,y_j) + u_{yy}(x_i,y_j) \approx \frac{1}{h^2}(u_{i+1,j} + u_{i-1,j} - 4u_{i,j} + u_{i,j+1} + u_{i,j-1}), \tag{2.8}$$

其中在边界上 $u(x_i, y_j) = g(x_i, y_j)$. 令 $u_1 := u(x_1, y_1)$, $u_2 := u(x_1, y_2)$,..., $u_9 := u(x_3, y_3)$, 并在边界上令 $g_{i,j} := u(x_i, y_j)$. 于是由 Fig. 2.1 和 (2.8), 未知量 $u_1, u_2, \ldots, u_9$ 满足以下关系:

$$
\begin{aligned}
u_2 + g_{0,1} - 4u_1 + u_4 + g_{1,0} &= 0,\\
u_3 + u_1 - 4u_2 + u_5 + g_{2,0} &= 0,\\
g_{4,1} + u_2 - 4u_3 + u_6 + g_{3,0} &= 0,\\
u_5 + g_{0,2} - 4u_4 + u_7 + u_1 &= 0,\\
&\vdots\\
g_{4,3} + u_8 - 4u_9 + g_{3,4} + u_6 &= 0.
\end{aligned}
$$

相应的矩阵形式为

$$
\underbrace{\left[\begin{array}{ccc|ccc|ccc}
-4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0\\
1 & -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0\\
0 & 1 & -4 & 0 & 0 & 1 & 0 & 0 & 0\\
\hline
1 & 0 & 0 & -4 & 1 & 0 & 1 & 0 & 0\\
0 & 1 & 0 & 1 & -4 & 1 & 0 & 1 & 0\\
0 & 0 & 1 & 0 & 1 & -4 & 0 & 0 & 1\\
\hline
0 & 0 & 0 & 1 & 0 & 0 & -4 & 1 & 0\\
0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 & 1\\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4
\end{array}\right]}_{A}
\underbrace{\begin{bmatrix} u_1\\ u_2\\ u_3\\ u_4\\ u_5\\ u_6\\ u_7\\ u_8\\ u_9 \end{bmatrix}}_{\boldsymbol{x}}
= -\underbrace{\begin{bmatrix} g_{1,0}\\ g_{2,0}\\ g_{3,0}\\ 0\\ 0\\ 0\\ 0\\ 0\\ 0 \end{bmatrix}}_{\boldsymbol{b}}
- \begin{bmatrix} 0\\ 0\\ 0\\ 0\\ 0\\ 0\\ g_{1,4}\\ g_{2,4}\\ g_{3,4} \end{bmatrix}
- \begin{bmatrix} g_{0,1}\\ 0\\ 0\\ g_{0,2}\\ 0\\ 0\\ g_{0,3}\\ 0\\ 0 \end{bmatrix}
- \begin{bmatrix} 0\\ 0\\ g_{4,1}\\ 0\\ 0\\ g_{4,2}\\ 0\\ 0\\ g_{4,3} \end{bmatrix}. \tag{2.9}
$$

求解这一线性方程组, 即可得到网格上的近似解 $u_1, \ldots, u_9$. 线性方程组 (2.9) 的矩阵具有特殊的非零结构, 可利用 Kronecker 积将其改写为简单形式. 设 $A(= \{a_{ij}\})$ 和 B 为矩阵. 则符号 $A \otimes B$ 称为 Kronecker 积 (或张量积), 定义为

$$
A \otimes B := \begin{bmatrix} a_{11}B & \cdots & a_{1m}B\\ \vdots & \ddots & \vdots\\ a_{n1}B & \cdots & a_{nm}B \end{bmatrix}. \tag{2.10}
$$

下面给出 A, B 为二阶矩阵时的例子:

$$
A \otimes B = \left[\begin{array}{c|c} a_{11}B & a_{12}B\\ \hline a_{21}B & a_{22}B \end{array}\right] = \left[\begin{array}{cc|cc} a_{11}b_{11} & a_{11}b_{12} & a_{12}b_{11} & a_{12}b_{12}\\ a_{11}b_{21} & a_{11}b_{22} & a_{12}b_{21} & a_{12}b_{22}\\ \hline a_{21}b_{11} & a_{21}b_{12} & a_{22}b_{11} & a_{22}b_{12}\\ a_{21}b_{21} & a_{21}b_{22} & a_{22}b_{21} & a_{22}b_{22} \end{array}\right].
$$

现在令

$$
M = \begin{bmatrix} -2 & 1 & 0\\ 1 & -2 & 1\\ 0 & 1 & -2 \end{bmatrix}, \quad I = \begin{bmatrix} 1 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & 1 \end{bmatrix}.
$$

则 (2.9) 中的系数矩阵 A 可写成

$$
A = I \otimes M + M \otimes I. \tag{2.11}
$$

矩阵 A 的规模为 $3^2 \times 3^2$. 一般而言, $N \times N$ 网格产生一个 $N^2 \times N^2$ 矩阵, 它可能非常大, 并且是对称的. 对称线性方程组的 Krylov 子空间方法将在 Section 3.1 中介绍. 最后, 下面列出 Kronecker 积的一些性质.

Theorem 2.1:

Kronecker 积具有以下性质:

1. 对 $A \in \mathbb{C}^{m\times n}, B \in \mathbb{C}^{p\times q}, C \in \mathbb{C}^{r\times s}$,

$$(A \otimes B) \otimes C = A \otimes (B \otimes C).$$

2. 对 $A, B \in \mathbb{C}^{m\times n}, C \in \mathbb{C}^{p\times q}$,

$$(A + B) \otimes C = A \otimes C + B \otimes C.$$

3. 对 $A \in \mathbb{C}^{m\times n}, B, C \in \mathbb{C}^{p\times q}$,

$$A \otimes (B + C) = A \otimes B + A \otimes C.$$

4. 对 $c \in \mathbb{C}, A \in \mathbb{C}^{m\times n}, B \in \mathbb{C}^{p\times q}$,

$$(cA) \otimes B = A \otimes (cB) = c(A \otimes B).$$

5. 对 $A \in \mathbb{C}^{m\times n}, B \in \mathbb{C}^{p\times q}$,

$$(A \otimes B)^{\mathrm{H}} = A^{\mathrm{H}} \otimes B^{\mathrm{H}}.$$

6. 对 $A \in \mathbb{C}^{m\times n}, B \in \mathbb{C}^{p\times q}, C \in \mathbb{C}^{n\times r}, D \in \mathbb{C}^{q\times s}$,

$$(A \otimes B)(C \otimes D) = (AC) \otimes (BD).$$

Proof: 只证明第 (6) 项. $A \otimes B$ 的 (i,k) 块元素为 $a_{i,k}B$, $C \otimes D$ 的 (k,j) 块元素为 $c_{k,j}D$. 因此, $(A \otimes B)(C \otimes D)$ 的 (i,j) 块元素为

$$\begin{aligned}((A \otimes B)(C \otimes D))_{i,j} &= \sum_{k=1}^{n} (a_{i,k}B)(c_{k,j}D) = \left(\sum_{k=1}^{n} a_{i,k}c_{k,j}\right) BD \\ &= (AC)_{i,j}BD = ((AC) \otimes (BD))_{i,j},\end{aligned}$$

证明完毕.

■

注意, (i,j) "块" 元素不是标量, 而是矩阵. 例如, 考虑下列形式的二阶分块矩阵

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}.$$

于是 A 的 $(1,1)$ 块元素是矩阵 A_{11}, B 的 $(1,2)$ 块元素是矩阵 B_{12}. 此外, 当矩阵乘法 $A_{i,j}B_{j,k}$ 有定义时, 可以把子矩阵 $A_{i,j}$ 和 $B_{i,j}$ 如同标量一样计算 AB. 例如, AB 的 $(1,1)$ 块元素为

$A_{11}B_{11} + A_{12}B_{21}$. 由 Theorem 2.1 的第 (6) 项, 下列性质成立. **Corollary 2.2:** 设 A 和 B 为可逆矩阵. 则

$$(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}.$$

Proof: 由 Theorem 2.1 的第 (6) 项, 有 $(A^{-1} \otimes B^{-1})(A \otimes B) = (A^{-1}A) \otimes (B^{-1}B) = I \otimes I$, 即两个单位矩阵的 Kronecker 积. 容易看出, 单位矩阵的 Kronecker 积仍是单位矩阵.

■

Corollary 2.3: 设 $\boldsymbol{p}_i^{(A)}$ 和 $\boldsymbol{p}_j^{(B)}$ 分别为 A 和 B 的特征向量, 对应特征值为 $\lambda_i^{(A)}$ 和 $\lambda_j^{(B)}$. 则 $A \otimes B$ 的一个特征向量为 $\boldsymbol{p}_i^{(A)} \otimes \boldsymbol{p}_j^{(B)}$, 对应特征值为 $\lambda_i^{(A)}\lambda_j^{(B)}$, 即

$$(A \otimes B)\left(\boldsymbol{p}_i^{(A)} \otimes \boldsymbol{p}_j^{(B)}\right) = \lambda_i^{(A)}\lambda_j^{(B)}\left(\boldsymbol{p}_i^{(A)} \otimes \boldsymbol{p}_j^{(B)}\right).$$

Proof: 由 Theorem 2.1 的第 (4) 和 (6) 项, 有

$$(A \otimes B)\left(\boldsymbol{p}_i^{(A)} \otimes \boldsymbol{p}_j^{(B)}\right) = \left(A\boldsymbol{p}_i^{(A)}\right) \otimes \left(B\boldsymbol{p}_j^{(B)}\right) = \lambda_i\lambda_j\left(\boldsymbol{p}_i^{(A)} \otimes \boldsymbol{p}_j^{(B)}\right),$$

证明完毕.

■

Corollary 2.4: 沿用 Corollary 2.3 中的记号, 有

$$(A \otimes I + I \otimes B)\left(\boldsymbol{p}_i^{(A)} \otimes \boldsymbol{p}_j^{(B)}\right) = \left(\lambda_i^{(A)} + \lambda_j^{(B)}\right)\left(\boldsymbol{p}_i^{(A)} \otimes \boldsymbol{p}_j^{(B)}\right),$$

其中 I 为单位矩阵.

矩阵 $A \otimes I + I \otimes B$ 称为 Kronecker 和 (或张量和). 由 Corollary 2.4 可知, (2.11) 中矩阵 A 的特征向量和特征值可由 $(\boldsymbol{p}_i \otimes \boldsymbol{p}_j, \lambda_i + \lambda_j)$ 给出, 其中 $\boldsymbol{p}_i$ 和 λ_i 为 M 的第 i 个特征向量和特征值. 这表明, (2.11) 中的矩阵 A 不可逆当且仅当存在 i, j 使得 $\lambda_i = -\lambda_j$. 因而, 规模比 A 小得多的矩阵 M 的特征值分布决定了矩阵 A 是否可逆. 一般而言, Kronecker 积不满足交换律, 即

$$A \otimes B \neq B \otimes A,$$

其中 $A \in \mathbb{C}^{m\times n}$, $B \in \mathbb{C}^{p\times q}$. 但是, 存在置换矩阵 $P_{mp} \in \mathbb{R}^{mp\times mp}$ 和 $P_{nq} \in \mathbb{R}^{nq\times nq}$ 使得

$$P_{mp}^{\top}(A \otimes B)P_{nq} = B \otimes A.$$

下面详细说明这一事实.

Theorem 2.5:

设 $P_{mn} \in \mathbb{R}^{mn\times mn}$ 为如下形式的矩阵

$$P_{mn} = \begin{bmatrix} (E_{11})^{\top} & (E_{12})^{\top} & \cdots & (E_{1n})^{\top} \\ (E_{21})^{\top} & (E_{22})^{\top} & \cdots & (E_{2n})^{\top} \\ \vdots & \vdots & \ddots & \vdots \\ (E_{m1})^{\top} & (E_{m2})^{\top} & \cdots & (E_{mn})^{\top} \end{bmatrix}, \tag{2.12}$$

其中 $E_{i,j} \in \mathbb{R}^{m\times n}$ 的 (i, j) 元素为一, 其余元素为零. 则

$$P_{mn} = P_{nm}^{\top} = P_{nm}^{-1}$$

并且对 $A \in \mathbb{C}^{m\times n}, B \in \mathbb{C}^{p\times q}$,

$$\underbrace{P_{mp}^{\top}}_{mp\times mp}\ \underbrace{(A\otimes B)}_{mp\times nq}\ \underbrace{P_{nq}}_{nq\times nq} = B\otimes A. \tag{2.13}$$

若 A 和 B 为方阵 (即 $m=n$, $p=q$),

$$P_{mp}^{\top}(A\otimes B)P_{mp} = B\otimes A.$$

Proof: 下面证明 (2.13). (2.12) 中 $P_{m,n}$ 的 (i,j) 块元素为 $E_{ij}^{\top}$, 而 E_{ij} 可分解为

$$E_{ij} = \boldsymbol{e}_i\boldsymbol{e}_j^{\top},$$

其中 $\boldsymbol{e}_i$ 为第 i 个单位向量, 即第 i 个元素为一, 其余元素为零. 设 a_{ij} 和 b_{ij} 分别为 A 和 B 的 (i,j) 元素. 于是, 由 (2.12), $P_{mp}^{\top}(A\otimes B)P_{nq}$ 的 (i,j) 块元素计算如下

$$\begin{aligned}
\left(P_{mp}^{\top}(A\otimes B)P_{nq}\right)_{ij} &= \sum_{k,\ell} E_{ki}(a_{k\ell}B)E_{\ell j}^{\top} = \sum_{k,\ell} a_{k\ell}\left(E_{ki}BE_{j\ell}\right)\\
&= \sum_{k,\ell} a_{k\ell}\left(\boldsymbol{e}_k\boldsymbol{e}_i^{\top}B\boldsymbol{e}_j\boldsymbol{e}_{\ell}^{\top}\right) = \sum_{k,\ell} a_{k\ell}\left(\boldsymbol{e}_k b_{ij}\boldsymbol{e}_{\ell}^{\top}\right)\\
&= b_{ij}\sum_{k,\ell} a_{k\ell}\left(\boldsymbol{e}_k\boldsymbol{e}_{\ell}^{\top}\right) = b_{ij}\left(\sum_{k,\ell} a_{k\ell}E_{k\ell}\right) = b_{ij}A\\
&= (B\otimes A)_{ij},
\end{aligned}$$

证明完毕.

■

2.1.1.3 Example 2 (复对称矩阵)

考虑下列边值问题:

$$\begin{aligned}
u_{xx}+u_{yy}+\sigma^2 u &= 0, && (x,y)\in(0,\pi)\times(0,\pi),\\
u_x|_{x=0} &= i\sqrt{\sigma^2-\tfrac{1}{4}}\cos\tfrac{y}{2}, && \text{Neumann Condition (1)},\\
u_x|_{x=\pi} - i\sqrt{\sigma^2-\tfrac{1}{4}}u|_{x=\pi} &= 0, && \text{Radiation Condition},\\
u_y|_{y=0} &= 0, && \text{Neumann Condition (2)},\\
u|_{y=\pi} &= 0, && \text{Dirichlet Condition},
\end{aligned}$$

这一椭圆型偏微分方程称为 Helmholtz 方程, 例如见 [19]. 为简单起见, 使用 Figs. 2.2 和 2.3 所示的三乘三网格. 与上一小节的 Example 1 类似, 将 (2.6)–(2.8) 应用于 Helmholtz 方程, 得

$$-u_{i+1,j}-u_{i-1,j}+(4-h^2\sigma^2)u_{i,j}-u_{i,j+1}-u_{i,j-1}=0. \tag{2.14}$$

在 Fig. 2.3 给出的网格点上考虑方程 (2.14), 得到以下六个方程:

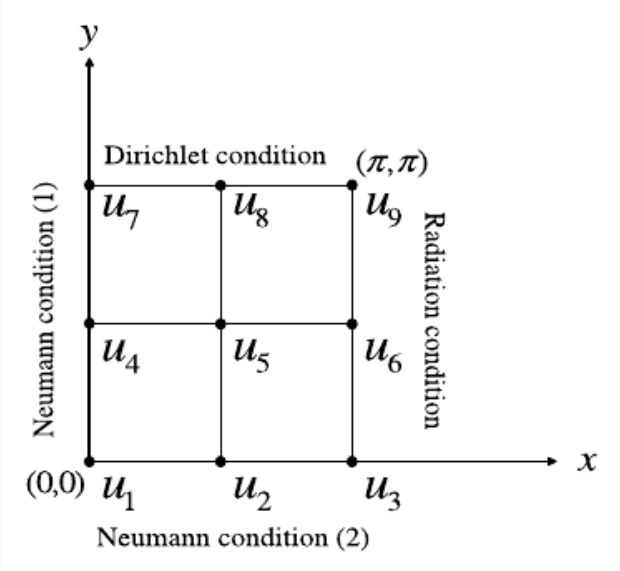


图 2.2: 区域上的网格点

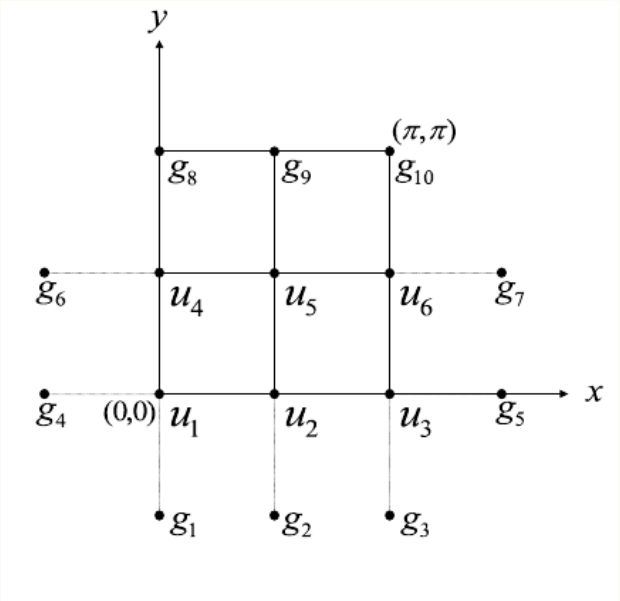


图 2.3: 满足边界条件的区域网格点

$$\begin{cases} -u_2 - g_4 - u_4 - g_1 + (4 - h^2\sigma^2)u_1 = 0, \\ -u_3 - u_1 - u_5 - g_2 + (4 - h^2\sigma^2)u_2 = 0, \\ -g_5 - u_2 - u_6 - g_3 + (4 - h^2\sigma^2)u_3 = 0, \\ -u_5 - g_6 - g_8 - u_1 + (4 - h^2\sigma^2)u_4 = 0, \\ -u_6 - u_4 - g_9 - u_2 + (4 - h^2\sigma^2)u_5 = 0, \\ -g_7 - u_5 - g_{10} - u_3 + (4 - h^2\sigma^2)u_3 = 0, \end{cases} \tag{2.15}$$

其中 $u_1 := u_{1,1} = u(0,0)$, $u_2 := u_{2,1} = u(h,0)$, $u_3 := u_{3,1} = u(2h,0)$, $u_4 := u_{1,2} = u(0,h)$, $u_5 := u_{2,2} = u(h,h)$, $u_6 := u_{3,2} = u(2h,h)$, 且 $h = \pi/2$. Fig. 2.3 中 $k = 1, \ldots, 10$ 对应的 g_k 可通过下述四个边界条件计算: 首先, 由 Neumann Condition (2) 得

$$\frac{u_4 - g_1}{2h} = \frac{u_5 - g_2}{2h} = \frac{u_6 - g_3}{2h} = 0,$$

因此

$$g_1 = u_4, \quad g_2 = u_5, \quad g_3 = u_6.$$

其次, 由 Neumann Condition (1) 有

$$u_x(0,0) \approx \frac{u_2 - g_4}{2h} = id\cos\left(\frac{0}{2}\right), \quad u_x(0,h) \approx \frac{u_5 - g_6}{2h} = id\cos\left(\frac{1}{2} \times \frac{h}{2}\right).$$

或者等价地,

$$g_4 = u_2 - 2idh, \quad g_6 = u_5 - 2idh\cos\left(\frac{h}{4}\right),$$

其中 $d = \sqrt{\sigma^2 - 1/4}$. 第三, 由辐射条件得

$$\frac{g_5 - u_2}{2h} - idu_3 = \frac{g_7 - u_5}{2h} - idu_6 = 0$$

或者等价地,

$$g_5 = u_2 + 2idhu_3, \quad g_7 = u_5 + 2idhu_6.$$

最后, 由 Dirichlet 条件可知, g_8, g_9 和 g_{10} 满足

$$g_8 = g_9 = g_{10} = 0.$$

将 $g_1, g_2, \ldots, g_{10}$ 代入六个方程 (2.15), 得到下列线性方程组:

$$\begin{bmatrix} a & -2 & 0 & -2 & 0 & 0 \\ -1 & a & -1 & 0 & -2 & 0 \\ 0 & -2 & a - idh & 0 & 0 & -2 \\ -1 & 0 & 0 & a & -2 & 0 \\ 0 & -1 & 0 & -1 & a & -1 \\ 0 & 0 & -1 & 0 & -2 & a - idh \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \end{bmatrix} = \begin{bmatrix} f_1 \\ 0 \\ 0 \\ f_4 \\ 0 \\ 0 \end{bmatrix}, \tag{2.16}$$

其中 $a=\sqrt{\sigma^2-1/4}$, $f_1=-2idh$, $f_4=-2idh\cos(h/4)$. 线性方程组 (2.16) 的系数矩阵不是对称的. 对每个方程作尺度变换, 得到如下形式的对称线性方程组

$$\begin{bmatrix} a/4 & -1/2 & 0 & -1/2 & 0 & 0 \\ -1/2 & a/2 & -1/2 & 0 & -1 & 0 \\ 0 & -1/2 & (a-idh)/4 & 0 & 0 & -1/2 \\ -1/2 & 0 & 0 & a/2 & -1 & 0 \\ 0 & -1 & 0 & -1 & a & -1 \\ 0 & 0 & -1/2 & 0 & -1 & (a-idh)/2 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \end{bmatrix} = \begin{bmatrix} f_1/4 \\ 0 \\ 0 \\ f_4/2 \\ 0 \\ 0 \end{bmatrix}. \tag{2.17}$$

(2.17) 的系数矩阵不是 Hermitian 的, 但却是对称的, 即复对称的. 求解复对称线性方程组的 Krylov 子空间方法将在 Section 3.2 中介绍.

2.1.1.4 Example 3 (非对称矩阵)

考虑下列三维偏微分方程:

$$\begin{aligned} [\boldsymbol{a}\cdot(\nabla * \nabla)+\boldsymbol{b}\cdot\nabla+c]u(x,y,z) &= g(x,y,z), \quad \text{in } \Omega, \\ u(x,y,z) &= 0, \qquad\qquad \text{on } \partial\Omega. \end{aligned}$$

这里 Ω 为立方体区域, 即 $(0,1)\times(0,1)\times(0,1)$, $\partial\Omega$ 为其边界. 符号 "$*$" 表示逐元素乘法. 等价地, 算子 $\boldsymbol{a}\cdot(\nabla * \nabla)+\boldsymbol{b}\cdot\nabla+c$ 可改写为

$$a_1\frac{\partial^2}{\partial x^2}+a_2\frac{\partial^2}{\partial y^2}+a_3\frac{\partial^2}{\partial z^2}+b_1\frac{\partial}{\partial x}+b_2\frac{\partial}{\partial y}+b_3\frac{\partial}{\partial z}+c$$

其中 $\boldsymbol{a}=[a_1,a_2,a_3]^\top$, $\boldsymbol{b}=[b_1,b_2,b_3]^\top$, $\boldsymbol{c}=[c_1,c_2,c_3]^\top$. 对 u 的二阶导数使用中心差分 (2.5), 得

$$\begin{aligned} a_1u_{xx}(x,y,z) &\approx \frac{a_1}{h^2}(u(x+h,y,z)-2u(x,y,z)+u(x-h,y,z)), \\ a_2u_{yy}(x,y,z) &\approx \frac{a_2}{h^2}(u(x,y+h,z)-2u(x,y,z)+u(x,y-h,z)), \\ a_3u_{zz}(x,y,z) &\approx \frac{a_3}{h^2}(u(x,y,z+h)-2u(x,y,z)+u(x,y,z-h)). \end{aligned}$$

类似地, 对 u 的一阶导数使用中心差分 (2.4), 得

$$\begin{aligned} b_1u_x(x,y,z) &\approx \frac{b_1}{2h}(u(x+h,y,z)-u(x-h,y,z)), \\ b_2u_y(x,y,z) &\approx \frac{b_2}{2h}(u(x,y+h,z)-u(x,y-h,z)), \\ b_3u_z(x,y,z) &\approx \frac{b_3}{2h}(u(x,y,z+h)-u(x,y,z-h)). \end{aligned}$$

考虑一个 $(N+1)\times(N+1)\times(N+1)$ 网格, 结合上述中心差分, 得到

$$\begin{aligned} &[\boldsymbol{a}\cdot(\nabla * \nabla)+\boldsymbol{b}\cdot\nabla+c]u(x,y,z) \\ &\quad\approx \frac{a_1}{h^2}(u_{i+1,j,k}-2u_{i,j,k}+u_{i-1,j,k})+\frac{b_1}{2h}(u_{i+1,j,k}-u_{i-1,j,k})+cu_{i,j,k} \\ &\qquad+\frac{a_2}{h^2}(u_{i,j+1,k}-2u_{i,j,k}+u_{i,j-1,k})+\frac{b_2}{2h}(u_{i,j+1,k}-u_{i,j-1,k}) \\ &\qquad+\frac{a_3}{h^2}(u_{i,j,k+1}-2u_{i,j,k}+u_{i,j,k-1})+\frac{b_3}{2h}(u_{i,j,k+1}-u_{i,j,k-1}), \end{aligned}$$

其中 $h = 1/(N+1)$. 由于边界上 $u = 0$, 沿用 Section 2.1.1.2 中的方法, 得到

$$(I_N \otimes I_N \otimes A + I_N \otimes B \otimes I_N + C \otimes I_N \otimes I_N)\boldsymbol{u} = \boldsymbol{g}, \tag{2.18}$$

其中 I_N 为 N 行 N 列的单位矩阵, $\boldsymbol{u}$ 和 $\boldsymbol{g}$ 的每个元素分别对应于 $u(x_i, y_i, z_i)$ 和 $g(x_i, y_i, z_i)$, 且

$$\begin{aligned} A &:= \frac{a_1}{h^2}M_1 + \frac{b_1}{2h}M_2 + cI_N, \\ B &:= \frac{a_2}{h^2}M_1 + \frac{b_2}{2h}M_2, \\ C &:= \frac{a_3}{h^2}M_1 + \frac{b_3}{2h}M_2. \end{aligned}$$

这里 M_1 和 M_2 定义为

$$M_1 := \begin{bmatrix} -2 & 1 & & & \\ 1 & -2 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & -2 & 1 \\ & & & 1 & -2 \end{bmatrix}, \quad M_2 := \begin{bmatrix} 0 & 1 & & & \\ -1 & 0 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 0 & 1 \\ & & & -1 & 0 \end{bmatrix}.$$

其中 M_1 和 M_2 为 N 行 N 列的 (三对角 Toeplitz) 矩阵, 矩阵中的空白位置为零. 线性方程组 (2.18) 的规模为 $N^3 \times N^3$. 若 $N = 100$, 即 $h \approx 0.01$ 的等距网格, 则矩阵的阶数为一百万. 若 $N = 1000$, 阶数就达到十亿! 因此, 要较好地近似求解这一偏微分方程, 就需要高效的数值求解器来求解非常大 (且稀疏) 的线性方程组. 当 $\boldsymbol{b} \neq 0$ 时, (2.18) 中的矩阵是 (实) 非对称的. 非对称 (非 Hermitian) 线性方程组的 Krylov 子空间方法将在 Section 3.3 中介绍.

2.1.2 有限元方法

上一小节已说明, 对 (偏) 导数作有限差分近似会得到线性递推关系, 利用这些递推关系即可建立线性方程组. 近似解对应于给定网格点上的值, 这些值就是线性方程组解向量的元素. 本小节介绍有限元方法 (下文简称 FEM), 并通过一个简单例子说明如何得到线性方程组. FEM 相对于有限差分方法的优点是: (i) 不一定需要区域上的正方形 (或矩形) 网格点, 因而它不仅适用于正方形或矩形区域, 也适用于其他各种区域; (ii) 基于泛函分析, 已有许多关于误差分析的数学结果. 本小节考虑将 FEM 应用于下列 Poisson 方程:

$$-\Delta u(x, y) = f, \quad (x, y) \in \Omega, \tag{2.19}$$

$$\cdot\ u(x, y) = g, \quad (x, y) \in \Gamma_1, \quad \text{Dirichlet Condition}, \tag{2.20}$$

$$\cdot\ \frac{\partial}{\partial n}u(x, y) = 0, \quad (x, y) \in \Gamma_2, \quad \text{Neumann Condition}, \tag{2.21}$$

其中 $\Delta := \partial^2/\partial x^2 + \partial^2/\partial y^2$. 区域以及带有边界条件的边界见 Fig. 2.4. 以下说明基于 [118], [170].

2.1.3 弱形式

本小节从 Poisson 方程导出称为弱形式的积分方程. 考虑将 (2.19) 乘以任意函数后在区域 Ω 上积分:

$$-\int_\Omega v\Delta u \,\mathrm{d}\Omega = \int_\Omega vf \,\mathrm{d}\Omega. \tag{2.22}$$

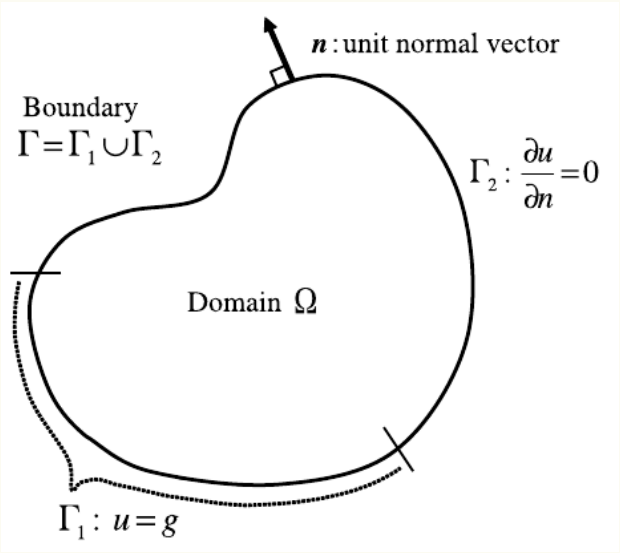


图 2.4: Poisson 方程 (2.19)–(2.21) 的区域与边界

若能找到一个对所有 v 都满足 (2.22) 的函数 u, 则 u 就是 (2.19) 的解. FEM 试图寻找满足 (2.22) 的解 u, 而不是直接求解 (2.19), 这与有限差分方法不同. 这里函数 v 称为检验函数或权函数. 现在, 为简化数学公式, 令 v 为任意光滑函数, 但在 Γ_1 上满足 $v=0$. 对 (2.22) 左端应用 Gauss–Green 定理, 分部积分得

$$\begin{aligned}\int_\Omega v\Delta u\,\mathrm{d}\Omega &= \int_\Gamma v\frac{\partial u}{\partial n}\,\mathrm{d}\Gamma - \int_\Omega \nabla v\cdot\nabla u\,\mathrm{d}\Omega \quad \text{(the Gauss–Green theorem)}\\ &= \int_{\Gamma_1} v\frac{\partial u}{\partial n}\,\mathrm{d}\Gamma_1 + \int_{\Gamma_2} v\frac{\partial u}{\partial n}\,\mathrm{d}\Gamma_2 - \int_\Omega \nabla v\cdot\nabla u\,\mathrm{d}\Omega \qquad (2.23)\\ &= -\int_\Omega \nabla v\cdot\nabla u\,\mathrm{d}\Omega.\end{aligned}$$

这里使用了 Γ_1 上 $v=0$ 的假设以及 Γ_2 上的 Neumann 条件 (2.21), 即 Γ_2 上 $\dfrac{\partial u}{\partial n}=0$. 将 (2.23) 代入 (2.22), 得

$$\int_\Omega \nabla v\cdot\nabla u\,\mathrm{d}\Omega = \int_\Omega vf\,\mathrm{d}\Omega.$$

因此只需寻找 u 使得

$$\iint_\Omega \left(\frac{\partial v}{\partial x}\frac{\partial u}{\partial x}+\frac{\partial v}{\partial y}\frac{\partial u}{\partial y}\right)\mathrm{d}x\mathrm{d}y = \iint_\Omega vf\,\mathrm{d}x\mathrm{d}y. \qquad (2.24)$$

这个方程不含二阶导数, 而原方程 (2.19) 含有二阶导数. 换言之, (2.24) 对 u 的可微性要求看来较弱. 因此, (2.24) 称为弱形式. 由 (2.24) 可见, 弱形式是在整个区域上积分. 现在考虑将整个区域划分成有限元 e_i, 即 $\Omega=\bigcup_{i=1}^{n} e_i$, 且当 $i\neq j$ 时 $e_i\cap e_j=\emptyset$. 于是有

$$\iint_{e_i} \left(\frac{\partial v}{\partial x}\frac{\partial u}{\partial x}+\frac{\partial v}{\partial y}\frac{\partial u}{\partial y}\right)\mathrm{d}x\mathrm{d}y = \iint_{e_i} vf\,\mathrm{d}x\mathrm{d}y \qquad (2.25)$$

其中 $i=1,2,\ldots,n$. 用三角形单元划分区域的例子见 Fig. 2.5. 下一小节将说明如何从 (2.25) 得到线性方程组.

2.1.4 线性方程组的推导

令 (2.25) 中的有限元 e_i 为三角形, 见 Fig. 2.5. 三角形单元 e_i 的每个节点都指定位置和编号, 三角形的节点按顺时针方向编号, 见 Fig. 2.6. 如 Fig. 2.6 所示, 节点 i 的位置为 (x_i, y_i), 其中 $i = 1, 2, 3$. 现在引入下列向量值函数:

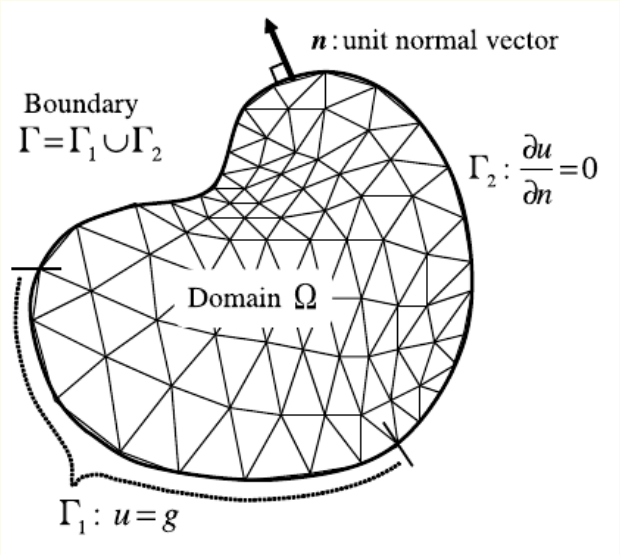


图 2.5: 用三角形有限元划分区域

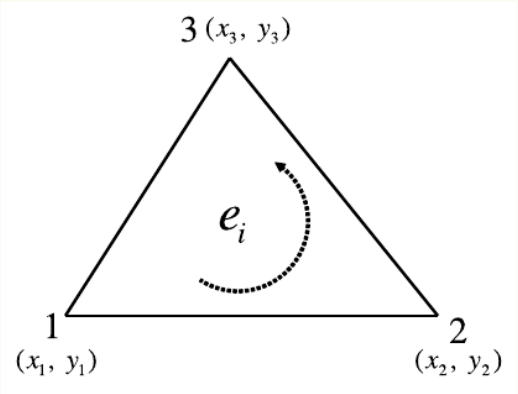


图 2.6: 三角形有限元的编号

$$\boldsymbol{h}(x,y) = \begin{pmatrix} h_1(x,y) \\ h_2(x,y) \\ h_3(x,y) \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 \\ x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{pmatrix}^{-1} \begin{pmatrix} 1 \\ x \\ y \end{pmatrix}, \tag{2.26}$$

并设 $\widetilde{u}$ 是三角形区域上解函数 u 的近似函数, $\widetilde{u}$ 是 x, y 的线性函数, 且对 $i = 1, 2, 3$ 满足 $\widetilde{u}(x_i, y_i) = u_i$. 则 $\widetilde{u}$ 为

$$\widetilde{u} = \boldsymbol{h}^\top \boldsymbol{u}_e = (1 \ x \ y) \begin{pmatrix} 1 & 1 & 1 \\ x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{pmatrix}^{-\top} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix}. \tag{2.27}$$

等式 $\widetilde{u}(x_i, y_i) = u_i$ $(1 \le i \le 3)$ 的验证留给读者作为练习. 现在将 $\widetilde{u}$ 代入 (2.25) 左侧的被积函数, 并选择与 $\widetilde{u}$ 具有相同形式的 v, 即 $v = \boldsymbol{h}^\top \boldsymbol{v}_e$, 其中 $\boldsymbol{v}_e := (v_1, v_2, v_3)^\top$, 得

$$\begin{aligned}\frac{\partial v}{\partial x}\frac{\partial \widetilde{u}}{\partial x} + \frac{\partial v}{\partial y}\frac{\partial \widetilde{u}}{\partial y} &= \frac{\partial}{\partial x}\boldsymbol{v}_e^\top \boldsymbol{h}\frac{\partial}{\partial x}\boldsymbol{h}^\top \boldsymbol{u}_e + \frac{\partial}{\partial y}\boldsymbol{v}_e^\top \boldsymbol{h}\frac{\partial}{\partial y}\boldsymbol{h}^\top \boldsymbol{u}_e \\ &= \boldsymbol{v}_e^\top \left(\frac{\partial}{\partial x}\boldsymbol{h}\frac{\partial}{\partial x}\boldsymbol{h}^\top + \frac{\partial}{\partial y}\boldsymbol{h}\frac{\partial}{\partial y}\boldsymbol{h}^\top\right)\boldsymbol{u}_e,\end{aligned}$$

其中使用了关系 $\boldsymbol{h}^\top \boldsymbol{v}_e = \boldsymbol{v}_e^\top \boldsymbol{h}$. 由 (2.26) 可得

$$\boldsymbol{h}_x := \frac{\partial \boldsymbol{h}}{\partial x} = \begin{pmatrix} 1 & 1 & 1 \\ x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{pmatrix}^{-1} \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} = \frac{1}{D}\begin{pmatrix} y_2 - y_3 \\ y_3 - y_1 \\ y_1 - y_2 \end{pmatrix} := \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}, \tag{2.28}$$

$$\boldsymbol{h}_y := \frac{\partial \boldsymbol{h}}{\partial y} = \begin{pmatrix} 1 & 1 & 1 \\ x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{pmatrix}^{-1} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \frac{1}{D}\begin{pmatrix} x_3 - x_2 \\ x_1 - x_3 \\ x_2 - x_1 \end{pmatrix} := \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix}, \tag{2.29}$$

其中 D 定义为

$$D := \det \begin{pmatrix} 1 & 1 & 1 \\ x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{pmatrix}.$$

其中 D 是三角形面积的两倍. 由于 $\boldsymbol{h}_x$ 和 $\boldsymbol{h}_y$ 不依赖于 x, y, (2.25) 的左端计算如下:

$$\begin{aligned}\iint_{e_i} \left(\frac{\partial v}{\partial x}\frac{\partial \widetilde{u}}{\partial x} + \frac{\partial v}{\partial y}\frac{\partial \widetilde{u}}{\partial y}\right)\mathrm{d}x\mathrm{d}y &= \boldsymbol{v}_e^\top(\boldsymbol{h}_x\boldsymbol{h}_x^\top + \boldsymbol{h}_y\boldsymbol{h}_y^\top)\boldsymbol{u}_e \iint_{e_i} \mathrm{d}x\mathrm{d}y \\ &= \boldsymbol{v}_e^\top \frac{(\boldsymbol{h}_x\boldsymbol{h}_x^\top + \boldsymbol{h}_y\boldsymbol{h}_y^\top)D}{2}\boldsymbol{u}_e \\ &= \boldsymbol{v}_e^\top A_e \boldsymbol{u}_e.\end{aligned}$$

这里 $\iint_{e_i} \mathrm{d}x\mathrm{d}y$ 为三角形 e_i 的面积, 即 $D/2$. 矩阵 A_e 是一个 3×3 对称矩阵, 其 (i, j) 元素 $a_{ij}^{(e)}$ 由下式给出

$$a_{ij}^{(e)} = \frac{D}{2}(b_i b_j + c_i c_j), \tag{2.30}$$

这是由 (2.28) 和 (2.29) 得到的. (2.25) 的右端可写成

$$\boldsymbol{f}_e := \begin{pmatrix} f_1^{(e)} \\ f_2^{(e)} \\ f_3^{(e)} \end{pmatrix} = \begin{pmatrix} \iint_{e_i} f_0 h_1 \,\mathrm{d}x\mathrm{d}y \\ \iint_{e_i} f_0 h_2 \,\mathrm{d}x\mathrm{d}y \\ \iint_{e_i} f_0 h_3 \,\mathrm{d}x\mathrm{d}y \end{pmatrix} = \frac{f_0 D}{6}\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \tag{2.31}$$

其中为简单起见, 使用了 $f(x, y) = f_0$ (常函数). 于是有

$$\iint_{e_i} v f \,\mathrm{d}x\mathrm{d}y = \boldsymbol{v}_e^\top \boldsymbol{f}_e.$$

计算 (2.31) 的右端时, 被积函数 h_i $(1 \le i \le 3)$ 为高为 1, 底面积为 $D/2$ 的三棱锥. 因此三棱锥的体积为 $(D/2) \cdot 1 \cdot 1/3 = D/6$. 由此, (2.25) 的左端和右端分别近似为

$$\boldsymbol{v}_e^\top A_e \boldsymbol{u}_e, \quad \boldsymbol{v}_e^\top \boldsymbol{f}_e. \tag{2.32}$$

上述过程是对小区域 e_i 推导 (2.32). 将所有 e_i 的线性方程组与边界条件一同考虑, 就得到大型线性方程组. 求解该大型线性方程组, 即可得到各节点上的近似解 u_i. 下一小节将说明如何得到大型线性方程组.

2.1.5 Example

作为上一小节的例子, 考虑 Fig. 2.7 中 $(0,1) \times (0,1)$ 正方形区域 Ω 上的 Poisson 方程 (2.19), 并说明如何得到线性方程组. 首先把 Fig. 2.7 中的正方形区域划分成 Fig. 2.8 中的八个三角形. 接着, 对 Fig. 2.8 中每个三角形单元的 A_e 和 $\boldsymbol{f}_e$ 计算 (2.32), 最后将所有信息组合起来, 得到线性方程组. 下面对 A_e 和 $\boldsymbol{f}_e$ 计算 (2.32). 由 Fig. 2.8 可见, 有两类三角形: $e_1^{\mathrm{I}}, e_3^{\mathrm{I}}, e_5^{\mathrm{I}}, e_7^{\mathrm{I}}$ 为 I 型, $e_2^{\mathrm{II}}, e_4^{\mathrm{II}}, e_6^{\mathrm{II}}, e_8^{\mathrm{II}}$ 为 II 型, 从而产生与 (2.8) 对应的两类线性方程组. 两类三角形的详情见 Fig. 2.9. 对 I 型三角形, 由 (2.28), (2.29), 并回顾 D 为三角形面积的两倍 (即 $D = h^2$), 有

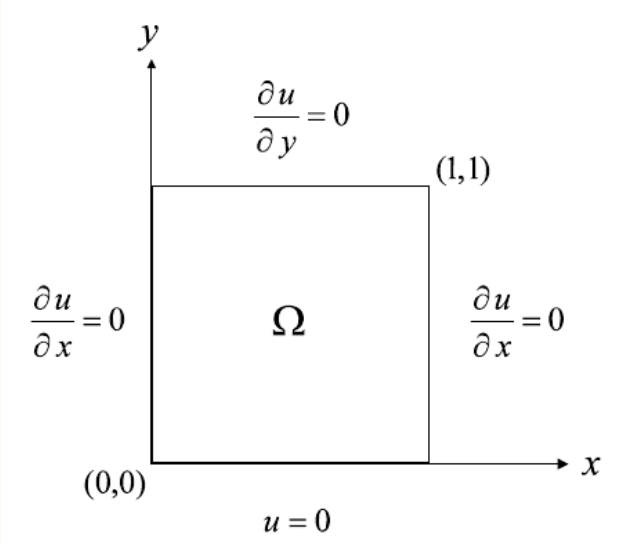


图 2.7: 正方形区域 $(0,1) \times (0,1)$

$$\begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = \frac{1}{h^2} \begin{pmatrix} -h \\ h \\ 0 \end{pmatrix}, \quad \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \frac{1}{h^2} \begin{pmatrix} -h \\ 0 \\ h \end{pmatrix}.$$

于是由 (2.30) 和 (2.31) 得

$$A_{e^{\mathrm{I}}} = \frac{1}{2} \begin{pmatrix} 2 & -1 & -1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}, \quad \boldsymbol{f}_{e^{\mathrm{I}}} = \frac{f_0 h^2}{6} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \tag{2.33}$$

它们对应于 (2.32). 类似地, 对 II 型三角形有

$$\begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = \frac{1}{h^2} \begin{pmatrix} 0 \\ h \\ -h \end{pmatrix}, \quad \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \frac{1}{h^2} \begin{pmatrix} -h \\ h \\ 0 \end{pmatrix},$$

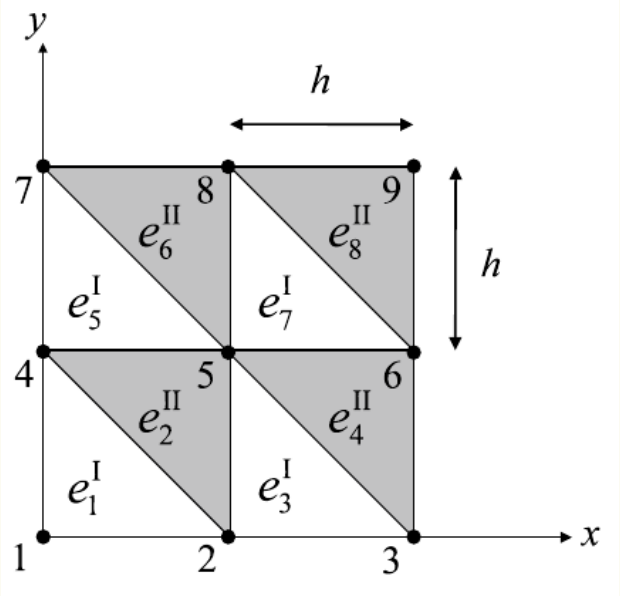


图 2.8: 三角形网格

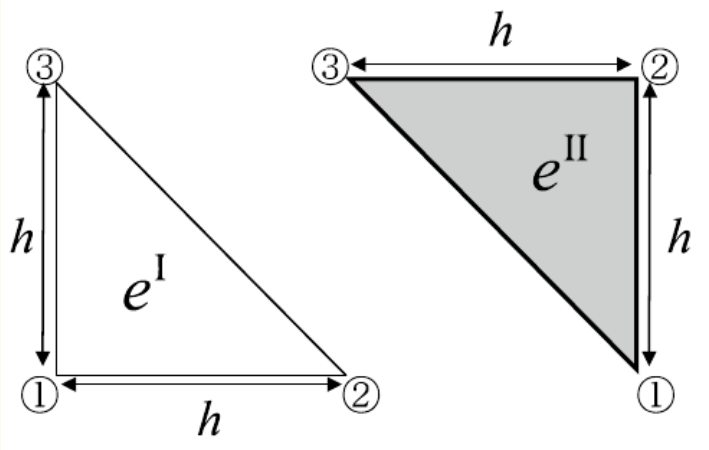


图 2.9: 两类三角形

因而

$$A_{e^{\mathrm{II}}}=\frac{1}{2}\begin{pmatrix}1&-1&0\\-1&2&-1\\0&-1&1\end{pmatrix},\quad \boldsymbol{f}_{e^{\mathrm{II}}}=\frac{f_0h^2}{6}\begin{pmatrix}1\\1\\1\end{pmatrix}. \tag{2.34}$$

下面说明如何组合所有信息来得到线性方程组. 为此, 需要将 Fig. 2.8 中的全局节点编号 1–9

表 2.1: 局部节点编号与全局节点编号的关系.

单元	e_1^{I}	e_2^{II}	e_3^{I}	e_4^{II}	e_5^{I}	e_6^{II}	e_7^{I}	e_8^{II}
三角形单元节点编号	整体节点编号							
1	1	2	2	3	4	5	5	6
2	2	5	3	6	5	8	6	9
3	4	4	5	5	7	7	8	8

与 Fig. 2.9 中的局部节点编号 1, 2, 3 对应起来, 所得结果见 Table 2.1. 最后按如下方式导出线性方程组. 首先考虑三角形单元 e_1^{I}. 由 (2.33) 和 Table 2.1 有

$$A_{e_1^{\mathrm{I}}}=\frac{1}{2}\begin{pmatrix}2&-1&-1\\-1&1&0\\-1&0&1\end{pmatrix}\begin{matrix}1\\2\\4\end{matrix},\quad \boldsymbol{f}_{e^{\mathrm{I}}}=\frac{f_0h^2}{6}\begin{pmatrix}1\\1\\1\end{pmatrix}\begin{matrix}1\\2\\4\end{matrix}$$
$$\begin{matrix}1&2&4\end{matrix}$$

利用这些信息, 构造如下矩阵和右端项:

$$\frac{1}{2}\begin{pmatrix}\mathbf{2}&\mathbf{-1}&0&\mathbf{-1}&0&0&0&0&0\\\mathbf{-1}&\mathbf{1}&0&\mathbf{0}&0&0&0&0&0\\0&0&0&0&0&0&0&0&0\\\mathbf{-1}&\mathbf{0}&0&\mathbf{1}&0&0&0&0&0\\0&0&0&0&0&0&0&0&0\\0&0&0&0&0&0&0&0&0\\0&0&0&0&0&0&0&0&0\\0&0&0&0&0&0&0&0&0\\0&0&0&0&0&0&0&0&0\end{pmatrix}\begin{matrix}\mathbf{1}\\\mathbf{2}\\3\\\mathbf{4}\\5\\6\\7\\8\\9\end{matrix},\quad \frac{f_0h^2}{6}\begin{pmatrix}\mathbf{1}\\\mathbf{1}\\0\\\mathbf{1}\\0\\0\\0\\0\\0\end{pmatrix}\begin{matrix}\mathbf{1}\\\mathbf{2}\\3\\\mathbf{4}\\5\\6\\7\\8\\9\end{matrix}$$
$$\begin{matrix}\mathbf{1}&\mathbf{2}&3&\mathbf{4}&5&6&7&8&9\end{matrix}$$

作为线性方程组的矩阵和右端项. 接着考虑三角形单元 e_2^{II}. 由 (2.34) 和 Table 2.1 有

$$A_{e_2^{\mathrm{II}}}=\frac{1}{2}\begin{pmatrix}1&-1&0\\-1&2&-1\\0&-1&1\end{pmatrix}\begin{matrix}2\\5\\4\end{matrix},\quad \boldsymbol{f}_{e^{\mathrm{II}}}=\frac{f_0h^2}{6}\begin{pmatrix}1\\1\\1\end{pmatrix}\begin{matrix}2\\5\\4\end{matrix}$$
$$\begin{matrix}2&5&4\end{matrix}$$

将这些信息加到前述矩阵和右端项上, 得到

$$\frac{1}{2}\begin{pmatrix} 2 & -1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & \mathbf{2} & 0 & \mathbf{0} & \mathbf{-1} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & \mathbf{0} & 0 & \mathbf{2} & \mathbf{-1} & 0 & 0 & 0 & 0 \\ 0 & \mathbf{-1} & 0 & \mathbf{-1} & \mathbf{2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}\begin{matrix} 1 \\ \mathbf{2} \\ 3 \\ \mathbf{4} \\ \mathbf{5} \\ 6 \\ 7 \\ 8 \\ 9 \end{matrix}, \quad \frac{f_0h^2}{6}\begin{pmatrix} 1 \\ \mathbf{2} \\ 0 \\ \mathbf{2} \\ \mathbf{1} \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}\begin{matrix} 1 \\ \mathbf{2} \\ 3 \\ \mathbf{4} \\ \mathbf{5} \\ 6 \\ 7 \\ 8 \\ 9 \end{matrix}$$
$$\begin{matrix} 1 & \mathbf{2} & 3 & \mathbf{4} & \mathbf{5} & 6 & 7 & 8 & 9 \end{matrix}$$

对其余单元 $e_3^{\mathrm{I}}, e_4^{\mathrm{II}}, \ldots, e_8^{\mathrm{II}}$ 作类似处理, 最终得到

$$\frac{1}{2}\begin{pmatrix} 2 & -1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 4 & -1 & 0 & -2 & 0 & 0 & 0 & 0 \\ 0 & -1 & 2 & 0 & 0 & -1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 4 & -2 & 0 & -1 & 0 & 0 \\ 0 & -2 & 0 & -2 & 8 & -2 & 0 & -2 & 0 \\ 0 & 0 & -1 & 0 & -2 & 4 & 0 & 0 & -1 \\ 0 & 0 & 0 & -1 & 0 & 0 & 2 & -1 & 0 \\ 0 & 0 & 0 & 0 & -2 & 0 & -1 & 4 & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & -1 & 2 \end{pmatrix}\begin{pmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ u_8 \\ u_9 \end{pmatrix} = \frac{f_0h^2}{6}\begin{pmatrix} 1 \\ 3 \\ 2 \\ 3 \\ 6 \\ 3 \\ 2 \\ 3 \\ 1 \end{pmatrix}.$$

这一线性方程组尚未包含 Dirichlet 条件 (对 $x \in [0,1]$, 有 $u(x,0)=0$) 的信息. 由 Figs. 2.7 和 2.8 可知, 满足 Dirichlet 条件要求

$$u_1 = u_2 = u_3 = 0.$$

因此, 从线性方程组中删去前三行, 并将 $u_1 = u_2 = u_3 = 0$ 代入所得线性方程组, 得到:

$$\frac{1}{2}\begin{pmatrix} 4 & -2 & 0 & -1 & 0 & 0 \\ -2 & 8 & -2 & 0 & -2 & 0 \\ 0 & -2 & 4 & 0 & 0 & -1 \\ -1 & 0 & 0 & 2 & -1 & 0 \\ 0 & -2 & 0 & -1 & 4 & -1 \\ 0 & 0 & -1 & 0 & -1 & 2 \end{pmatrix}\begin{pmatrix} u_4 \\ u_5 \\ u_6 \\ u_7 \\ u_8 \\ u_9 \end{pmatrix} = \frac{f_0h^2}{6}\begin{pmatrix} 3 \\ 6 \\ 3 \\ 2 \\ 3 \\ 1 \end{pmatrix}. \tag{2.35}$$

求解线性方程组, 即可得到 $u_4, u_5, \ldots, u_9$. 最后比较近似解与精确解. Poisson 方程的精确解为 $u(x,y) = f_0 y - \frac{1}{2} f_0 y^2$. 当 $f_0 = 1$ 时, 精确解的 u_4, u_5, u_6 的值为 0.375, u_7, u_8, u_9 的值为 0.5. 取 $f_0 = 1$, $h = \frac{1}{2}$ 求解线性方程组 (2.35), 得 $u_4 \approx 0.381, u_5 \approx 0.375, u_6 \approx 0.369$ 和 $u_7 \approx 0.524, u_8 \approx 0.500, u_9 \approx 0.476$. 若需要更精确的近似解, 应增加节点数. 由上述推导可见, 线性方程组的规模与整体节点数几乎相同.

2.2 计算物理

2.2.1 大规模电子结构计算

利用单体密度矩阵 ρ, 可以高效计算大规模系统的分子动力学模拟, 其依据是任意物理量 $\langle X\rangle$ 均可由下式得到

$$\langle X\rangle = \mathrm{Tr}(\rho X) = \iint \rho(\boldsymbol{r},\boldsymbol{r}')X(\boldsymbol{r}',\boldsymbol{r})\,\mathrm{d}\boldsymbol{r}\mathrm{d}\boldsymbol{r}'$$

相应的离散形式为

$$\sum_{i,j}\rho_{ij}X_{ji}.$$

这里需特别注意, 若 X 是短程算子, 则只需密度矩阵的短程行为. 作为极端例子, 若 X 为对角矩阵 (即非对角元素为零), 则因为 $\sum_{i,j}\rho_{ij}X_{ji} = \sum_i \rho_{ii}X_{ii}$, 只需密度矩阵的对角元素, 从而计算非常高效. 下面详细说明密度矩阵 ρ. 已知密度矩阵 ρ 的 (i,j) 元素为

$$\rho_{ij} = -\frac{1}{\pi}\int_{-\infty}^{\infty}\mathrm{Im}\,G_{ij}(\epsilon)f\left(\frac{\epsilon-\mu}{K_BT}\right)\,\mathrm{d}\epsilon, \tag{2.36}$$

其中 $f(x)$ 是 Fermi 分布函数, μ 是化学势, 可由电子总数等于密度矩阵对角元素之和这一要求确定. 这里 $G_{ij}(\epsilon)$ 是下列矩阵的 (i,j) 元素:

$$G(\epsilon) = [(\epsilon+i\delta)I - H]^{-1}, \tag{2.37}$$

其中 i 是虚数单位, δ 为一个 (无穷) 小的正数 ($\delta\to 0_+$), I 和 H 分别为单位矩阵和 Hamiltonian 矩阵. 矩阵 G 称为 Green 函数. Hamiltonian 矩阵 H 的 (i,j) 元素可由下列实空间积分给出:

$$H_{ij} = \int f_i(\boldsymbol{r})\left(\frac{-\hbar}{2m}\Delta + V(\boldsymbol{r})\right)f_j(\boldsymbol{r})\,\mathrm{d}\boldsymbol{r},$$

其中 m 和 $\hbar$ 分别为电子质量和 (约化) Planck 常数, Δ 是 Laplacian 算子. $f_j(\boldsymbol{r})$ 为给定的线性无关函数. $V(\boldsymbol{r})$ 为描述电子与原子核间相互作用的给定函数. 详情例如见 [184], [190] 及其中的参考文献. 现在说明上述计算如何产生线性方程组. 设 $\boldsymbol{e}_i$ 为第 i 个单位向量. 则由 (2.37), $G(\epsilon)$ 的 (i,j) 元素由下式计算

$$G_{ij}(\epsilon) = \boldsymbol{e}_i^\top[(\epsilon+i\delta)I - H]^{-1}\boldsymbol{e}_j.$$

因此, 令 $\boldsymbol{x}_j = [(\epsilon+i\delta)I - H]^{-1}\boldsymbol{e}_j$, 得

$$[(\epsilon+i\delta)I - H]\boldsymbol{x}_j = \boldsymbol{e}_j,$$

这是一个线性方程组. 在计算物理中, Hamiltonian 矩阵 H 为 Hermitian 矩阵或实对称矩阵. 若 H 为实对称矩阵, 则系数矩阵 $(\epsilon+i\delta)I - H$ 为复对称矩阵. 此外, 计算 (2.36) 中的积分时, 使用适当的数值求积公式, 需要计算 $k = 1,2,\ldots,m$ 时的 $G_{ij}(\epsilon_k)$. 因而, 为计算密度矩阵的 (i,j) 元素 $\rho_{i,j}$, 需要求解下列形式的线性方程组

$$(A+\sigma_k I)\boldsymbol{x}_j^{(k)} = \boldsymbol{b},\quad k = 1,2,\ldots,m,$$

其中 $A := i\delta I - H$, $\boldsymbol{b} := \boldsymbol{e}_j$, $\sigma_k := \epsilon_k$. 这称为移位线性方程组, 利用 Krylov 子空间的移位不变性, 可用 Krylov 子空间方法高效求解. 移位线性方程组的 Krylov 子空间方法将在 Chapter 4 中介绍.

2.2.2 格点量子色动力学

量子色动力学 (QCD) 是描述夸克和胶子之间强相互作用的理论, 而格点 QCD 是求解 QCD 的一种非微扰途径, 将该理论建立在时空格点上. 已知格点 QCD 中最耗时的部分是求解线性方程组

$$A\boldsymbol{x} = \boldsymbol{b},$$

它常写成 $A\psi = \phi$, 所得解用于获得夸克传播子. 关于夸克传播子的详情及格点 QCD 的概述, 例如见 [69]. 下面介绍 Wilson 费米子矩阵 A 的结构. Wilson 费米子矩阵可写成 $A = I - \kappa D$, 其中 I 是单位矩阵, κ 是非负实参数, 称为跃迁参数. 矩阵 D 称为跃迁矩阵. 现在说明跃迁矩阵的结构. 考虑四维超立方格点, 它可看作超立方体的等距网格. 每个网格点可写成向量 $\boldsymbol{x}$, 满足

$$\boldsymbol{x} \in \Omega := \{(x_1, x_2, x_3, x_4) : x_1, x_2, x_3, x_4 \in \{1, 2, \ldots, N\}\}.$$

为简单起见, 假设 x_1, x_2, x_3, x_4 均从 1 取到相同的数 N. 容易看出, Ω 的元素个数为 N^4. 利用 $\boldsymbol{x}, \boldsymbol{y} \in \Omega$, 将跃迁矩阵 D 写成分块矩阵形式

$$D(\boldsymbol{x}; \boldsymbol{y}) = \sum_{\mu=1}^{4} \left\{ [(I_4 - \gamma_\mu) \otimes U_\mu(\boldsymbol{x})] \delta_{\boldsymbol{x}, \boldsymbol{y}-\boldsymbol{e}_\mu} + [(I_4 + \gamma_\mu) \otimes U_\mu^{\mathrm{H}}(\boldsymbol{x} - \boldsymbol{e}_\mu)] \delta_{\boldsymbol{x}, \boldsymbol{y}+\boldsymbol{e}_\mu} \right\}. \tag{2.38}$$

这里 $\delta_{\boldsymbol{x},\boldsymbol{y}}$ 为 Kronecker 符号, 即当 $\boldsymbol{x} = \boldsymbol{y}$ 时 $\delta_{\boldsymbol{x},\boldsymbol{y}} = 1$, 当 $\boldsymbol{x} \neq \boldsymbol{y}$ 时 $\delta_{\boldsymbol{x},\boldsymbol{y}} = 0$. $D(\boldsymbol{x}; \boldsymbol{y})$ 是规模为 12×12 的小矩阵, 跃迁矩阵 D 如下:

$$\begin{aligned} D &= \begin{bmatrix} D_{1,1} & D_{1,2} & \cdots & D_{1,N^4} \\ D_{2,1} & D_{2,2} & \cdots & D_{2,N^4} \\ \vdots & \vdots & \ddots & \vdots \\ D_{N^4,1} & D_{N^4,2} & \cdots & D_{N^4,N^4} \end{bmatrix} \\ &= \begin{bmatrix} D(1,1,1,1;1,1,1,1) & D(1,1,1,1;1,1,1,2) & \cdots & D(1,1,1,1;N,N,N,N) \\ D(1,1,1,2;1,1,1,1) & D(1,1,1,2;1,1,1,2) & \cdots & D(1,1,1,2;N,N,N,N) \\ \vdots & \vdots & \ddots & \vdots \\ D(N,N,N,N;1,1,1,1) & D(N,N,N,N;1,1,1,2) & \cdots & D(N,N,N,N;N,N,N,N) \end{bmatrix}. \end{aligned}$$

因此跃迁矩阵 D 的规模为 $12N^4 \times 12N^4$. 若 $N = 64$, 则线性方程组的未知数个数约为两亿! 这一规模的增长速度, 远快于 Section 2.1.1.4 中三维偏微分方程离散所产生的规模. (2.38) 中各符号详述如下: I_4 是四阶单位矩阵, $\delta_{\boldsymbol{x},\boldsymbol{y}}$ 是一个函数 ($\delta : \mathbb{R}^4 \times \mathbb{R}^4 \to \mathbb{R}$), 当 $\boldsymbol{x} \neq \boldsymbol{y}$ 时 $\delta_{\boldsymbol{x},\boldsymbol{y}} = 0$, 当 $\boldsymbol{x} = \boldsymbol{y}$ 时 $\delta_{\boldsymbol{x},\boldsymbol{y}} = 1$. 符号 $\boldsymbol{e}_\mu$ 为第 μ 个单位向量的转置, 或等价地, 为 4×4 单位矩阵 I_4 的第 μ 行. 对每个 μ, 符号 γ_μ 为下列矩阵

$$\gamma_1 = \sigma_y \otimes \sigma_x, \quad \gamma_2 = \sigma_y \otimes \sigma_y, \quad \gamma_3 = \sigma_y \otimes \sigma_z, \quad \gamma_4 = \sigma_z \otimes I_2,$$

其中 I_2 是二阶单位矩阵, σ_x, σ_y 和 σ_z 是 Pauli 矩阵, 即

$$\sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \sigma_y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad \sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

最后, 对每个 μ 和 $\boldsymbol{x}$, 当 $\mu=1,2,3,4$ 时, 符号 $U_\mu(\boldsymbol{x})(=U_\mu([x_1,x_2,x_3,x_4])$ 属于 SU(3), 即为行列式为一的三阶酉矩阵. $U_\mu(\boldsymbol{x})$ 对应于背景规范场. 若 $U_\mu(\boldsymbol{x})=I_3$, 则称 $U_\mu(\boldsymbol{x})$ 为 "冷"; 若随机选择 $U_\mu(\boldsymbol{x})\in\mathrm{SU}(3)$, 则称其为 "热". 下面给出 $N=2$ 时 $D(\boldsymbol{x},\boldsymbol{y})$ 的若干例子:

$$
\begin{aligned}
D_{1,1}&=D(1,1,1,1;1,1,1,1)=O,\\
D_{1,2}&=D(1,1,1,1;1,1,1,2)=(I_4-\gamma_4)\otimes U_4([1,1,1,1]),\\
D_{1,3}&=D(1,1,1,1;1,1,2,1)=(I_4-\gamma_3)\otimes U_3([1,1,1,1]),\\
D_{1,4}&=D(1,1,1,1;1,1,2,2)=O,\\
D_{1,5}&=D(1,1,1,1;1,2,1,1)=(I_4-\gamma_2)\otimes U_2([1,1,1,1]),\\
D_{1,6}&=D(1,1,1,1;1,2,1,2)=O,\\
D_{1,7}&=D(1,1,1,1;1,2,2,1)=O,\\
D_{1,8}&=D(1,1,1,1;1,2,2,2)=O,\\
D_{1,9}&=D(1,1,1,1;2,1,1,1)=(I_4-\gamma_1)\otimes U_1([1,1,1,1]),\\
&\vdots\\
D_{2,1}&=D(1,1,1,2;1,1,1,1)=(I_4+\gamma_4)\otimes U_4^{\mathrm{H}}([1,1,1,1]),\\
D_{2,2}&=D(1,1,1,2;1,1,1,2)=O,\\
D_{2,3}&=D(1,1,1,2;1,1,2,1)=O,\\
D_{2,4}&=D(1,1,1,2;1,1,2,2)=(I_4-\gamma_3)\otimes U_3([1,1,1,2]),\\
&\vdots\\
D_{16,16}&=D([2,2,2,2;2,2,2,2])=O.
\end{aligned}
$$

注意 $D_{2,1}=(I_4+\gamma_4)\otimes U_4^{\mathrm{H}}([1,1,1,2]-\boldsymbol{e}_4)=(I_4+\gamma_4)\otimes U_4^{\mathrm{H}}([1,1,1,1])$. 矩阵 D 既不对称也不为 Hermitian 矩阵, 但它具有下面所述的隐含对称性. 令

$$\gamma_5=\sigma_x\otimes I_2.$$

则跃迁矩阵 D 具有以下性质:

$$\Gamma_5 D=D^{\mathrm{H}}\Gamma_5, \tag{2.39}$$

其中 $\Gamma_5=I_{N^4}\otimes\gamma_5\otimes I_3$. 这一性质称为 γ_5-对称 (或 γ_5-Hermitian). 由 Theorem 2.1 的第 (5) 项, 矩阵 γ_5 为实对称矩阵, 矩阵 Γ_5 也为实对称矩阵, 因而有 $\Gamma_5=\Gamma_5^{\mathrm{H}}$. 于是可将 (2.39) 改写为

$$\Gamma_5 D=(\Gamma_5 D)^{\mathrm{H}}.$$

这表明 $\Gamma_5 D$ 为 Hermitian 矩阵. 由 Corollary 2.2 有 $\Gamma_5^{-1}=I_{N^4}^{-1}\otimes\gamma_5^{-1}\otimes I_3^{-1}=I_{N^4}\otimes\gamma_5\otimes I_3=\Gamma_5$. 因而得到 (2.39) 的另一种表示:

$$\Gamma_5 D\Gamma_5=D^{\mathrm{H}}. \tag{2.40}$$

现在验证 $N=2$ 时的 (2.40). 令 $P=\gamma_5\otimes I_3$. 则 D 的 (i,j) 块元素为

$$
\begin{aligned}
(\Gamma_5 D\Gamma_5)_{i,j} &= \left(\begin{bmatrix} P & & \\ & \ddots & \\ & & P \end{bmatrix}\begin{bmatrix} D_{1,1} & \cdots & D_{1,16} \\ \vdots & \ddots & \vdots \\ D_{16,1} & \cdots & D_{16,16} \end{bmatrix}\begin{bmatrix} P & & \\ & \ddots & \\ & & P \end{bmatrix}\right)_{i,j} \\
&= \left(\begin{bmatrix} PD_{1,1}P & \cdots & PD_{1,16}P \\ \vdots & \ddots & \vdots \\ PD_{16,1}P & \cdots & PD_{16,16}P \end{bmatrix}\right)_{i,j} \\
&= PD_{i,j}P.
\end{aligned}
$$

例如, 考虑 $(i,j)=(1,2)$. 由 Theorem 2.1 的第 (5) 和 (6) 项, 有

$$
\begin{aligned}
(\Gamma_5 D\Gamma_5)_{1,2} &= PD_{1,2}P \\
&= (\gamma_5\otimes I_3)[(I_4-\gamma_4)\otimes U_4([1,1,1,1])](\gamma_5\otimes I_3) \\
&= \gamma_5(I_4-\gamma_4)\gamma_5\otimes U_4([1,1,1,1]) \\
&= (I_4-\gamma_5\gamma_4\gamma_5)\otimes U_4([1,1,1,1]) \\
&= (I_4+\gamma_4)\otimes U_4([1,1,1,1]) \\
&= [(I_4+\gamma_4)^{\mathrm{H}}\otimes U_4^{\mathrm{H}}([1,1,1,1])]^{\mathrm{H}} \\
&= [(I_4+\gamma_4)\otimes U_4^{\mathrm{H}}([1,1,1,1])]^{\mathrm{H}} \\
&= D_{2,1}^{\mathrm{H}}.
\end{aligned}
$$

类似地, 对其余的 i,j, 可验证 $(\Gamma_5 D\Gamma_5)_{i,j}=D_{j,i}^{\mathrm{H}}$. 因此可见, (2.40) 在 $N=2$ 时成立. 在格点 QCD 中, 需要计算一些模较小的特征值. 此时可使用移位求逆 Lanczos (Arnoldi) 方法. 这一方法每次迭代都需要求解线性方程组. 关于特征值问题的计算算法, 见 [12]. 文献 [108] 针对格点 QCD 中产生的特征值问题提出了一种厚重启 Lanczos 型方法.

2.3 机器学习

机器学习领域也会产生线性方程组. 本节通过最小二乘回归和最小二乘分类两个例子, 说明线性方程组的重要性.

2.3.1 最小二乘回归

本小节考虑最简单的监督学习之一: 最小二乘学习. 给定训练数据集 $(\boldsymbol{x}_i,y_i)$ $(i=1,\dots,n)$ 和函数 $f_{\boldsymbol{\theta}}(\boldsymbol{x})$. 最小二乘学习要学习一个参数 $\boldsymbol{\theta}\in\mathbb{R}^m$, 使下列误差函数达到极小值:

$$
J(\boldsymbol{\theta})=\sum_{i=1}^{n}(y_i-f_{\boldsymbol{\theta}}(\boldsymbol{x}_i))^2. \tag{2.41}
$$

若 $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ 由函数 $f_1(\boldsymbol{x}),\dots,f_m(\boldsymbol{x})$ 的线性组合给出, 形式为

$$
f_{\boldsymbol{\theta}}(\boldsymbol{x})=\theta_1 f_1(\boldsymbol{x})+\theta_2 f_2(\boldsymbol{x})+\theta_m f_m(\boldsymbol{x}), \tag{2.42}
$$

则误差函数 (2.41) 可改写为[1]

$$
J(\boldsymbol{\theta})=\|\boldsymbol{y}-M\boldsymbol{\theta}\|_2^2, \tag{2.43}
$$

[1]本节将 2-范数明确写成 $\|\cdot\|_2$. 机器学习中也使用 1-范数 $\|\cdot\|_1$, 尤其用于得到形如 $\|\boldsymbol{y}-M\boldsymbol{\theta}\|_2^2+\lambda\|\boldsymbol{\theta}\|_1$ 的最小二乘问题的稀疏解.

其中 $\boldsymbol{y} = [y_1, y_2, \ldots, y_n]^\top$, $\boldsymbol{\theta} = [\theta_1, \theta_2, \ldots, \theta_m]^\top$, 且

$$M = \begin{bmatrix} f_1(\boldsymbol{x}_1) & f_2(\boldsymbol{x}_1) & \cdots & f_m(\boldsymbol{x}_1) \\ f_1(\boldsymbol{x}_2) & f_2(\boldsymbol{x}_2) & \cdots & f_m(\boldsymbol{x}_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(\boldsymbol{x}_n) & f_2(\boldsymbol{x}_n) & \cdots & f_m(\boldsymbol{x}_n) \end{bmatrix}.$$

为简单起见, 假设 M 的列向量线性无关. 为使误差函数 (2.43) 达到极小值, 下列分解很有用:

$$\begin{aligned} J(\boldsymbol{\theta}) &= \|\boldsymbol{y} - M\boldsymbol{\theta}\|_2^2 = (\boldsymbol{y} - M\boldsymbol{\theta})^\top (\boldsymbol{y} - M\boldsymbol{\theta}) \\ &= \boldsymbol{v}_{\boldsymbol{\theta}}^\top (M^\top M)^{-1} \boldsymbol{v}_{\boldsymbol{\theta}} - (M^\top \boldsymbol{y})^\top (M^\top M)^{-1} (M^\top \boldsymbol{y}) + \boldsymbol{y}^\top \boldsymbol{y}. \end{aligned} \tag{2.44}$$

这里 $\boldsymbol{v}_{\boldsymbol{\theta}} = M^\top \boldsymbol{y} - M^\top M\boldsymbol{\theta}$. 由于 M 的列线性无关, $M^\top M$ 为对称正定矩阵, 即所有特征值均为正, 或等价地, 对所有非零向量 $\boldsymbol{x}$, 有 $\boldsymbol{x}^\top M^\top M\boldsymbol{x} > 0$. 由此, 若 $\boldsymbol{v}_{\boldsymbol{\theta}} = \boldsymbol{0}$, 即 $M^\top \boldsymbol{y} - M^\top M\boldsymbol{\theta} = \boldsymbol{0}$, 则 $J(\boldsymbol{\theta})$ 达到极小值. 因此需要求解下列线性方程组:

$$M^\top M\boldsymbol{\theta} = M^\top \boldsymbol{y}. \tag{2.45}$$

作为数值例子, 考虑 $y = (1/2)\sin(x) + \epsilon$, 其中 ϵ 为噪声. 我们取 $(x_1, x_2, \ldots, x_{63}) = (0, 0.1, 0.2, \ldots, 6.2)$ 和 $(y_1, y_2, \ldots, y_{63}) = (y(0), y(0.1), y(0.2), \ldots, y(6.2))$ 生成一个数据集 (训练数据集). 数据的可视化结果见 Fig. 2.10. 接着考虑 (2.42). 对基函数 $f_1(x), f_2(x), \ldots, f_m(x)$, 使用

$$f_k(x) = \begin{cases} \sin \dfrac{1}{2} kx & (k : \text{even}), \\ \cos \dfrac{1}{2} kx & (k : \text{odd}). \end{cases}$$

模型函数 (2.42) 的未知向量 $\boldsymbol{\theta}$ 通过求解线性方程组 (2.45) 得到, $m = 5$ 和 $m = 10$ 时 $f_{\boldsymbol{\theta}}(x)$ 的图像见 Fig. 2.11. 由 Fig. 2.11a 可见, $m = 5$ 时函数 $f_{\boldsymbol{\theta}}(x)$ 对噪声具有鲁棒性. 另一方面,

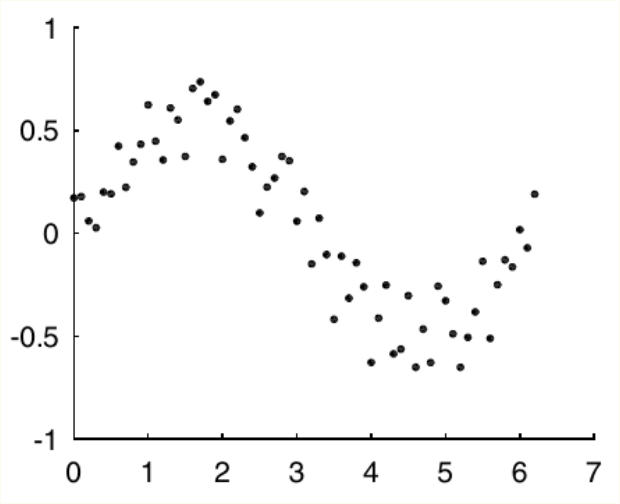


图 2.10: 给定数据集的图像

$m = 10$ 时的函数似乎试图拟合含给定噪声的数据. 这一现象称为过拟合. 下面介绍一种克服过拟合的方法, 它同样需要求解 (参数化的) 线性方程组. 现在考虑下列形式的约束极小化问题

$$\min_{\boldsymbol{\theta}} \|\boldsymbol{y} - M\boldsymbol{\theta}\|_2^2 \quad \text{subject to } \|\boldsymbol{\theta}\|_2^2 \le R. \tag{2.46}$$

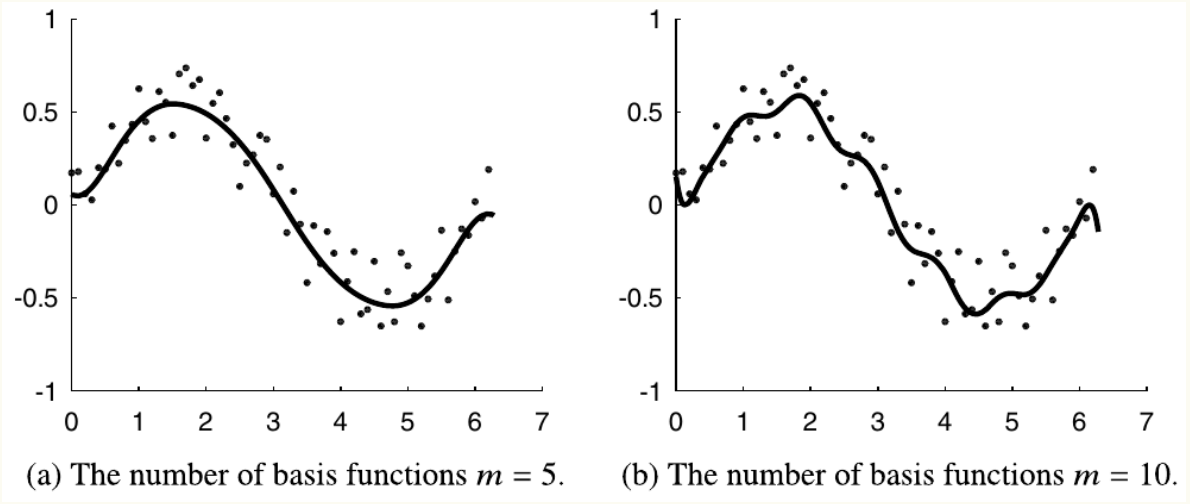


图 2.11: (2.42) 中 $m = 5$ 和 $m = 10$ 时 $f_{\boldsymbol{\theta}}(x)$ 的图像. (a) 基函数个数 $m = 5$. (b) 基函数个数 $m = 10$.

若 R 为无穷大, 则该极小化问题等价于 (2.43). 为求解 (2.46), 对偶问题很有用. 令 $f(\boldsymbol{\theta}) = \|\boldsymbol{y} - M\boldsymbol{\theta}\|_2^2$, $g(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_2^2 - R$, 则 (2.46) 可改写为

$$\min_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) \quad \text{subject to } g(\boldsymbol{\theta}) \leq 0. \tag{2.47}$$

这里对 $\lambda \geq 0$ 引入 Lagrange 函数:

$$L(\boldsymbol{\theta}, \lambda) := f(\boldsymbol{\theta}) + \lambda g(\boldsymbol{\theta}).$$

由于 $\lambda \geq 0$ 且 $g(\boldsymbol{\theta}) \leq 0$, 有

$$\max_{\lambda \geq 0} L(\boldsymbol{\theta}, \lambda) = \begin{cases} f(\boldsymbol{\theta}) & (\text{if } g(\boldsymbol{\theta}) \leq 0), \\ +\infty & (\text{otherwise}). \end{cases}$$

因此 (2.47) 等价于

$$\min_{\boldsymbol{\theta}} \max_{\lambda \geq 0} L(\boldsymbol{\theta}, \lambda).$$

现在考虑下列对偶问题:

$$\max_{\lambda \geq 0} \min_{\boldsymbol{\theta}} L(\boldsymbol{\theta}, \lambda). \tag{2.48}$$

若 f 和 g 是凸的, 并满足一定条件, 则有

$$\max_{\lambda \geq 0} \min_{\boldsymbol{\theta}} L(\boldsymbol{\theta}, \lambda) = \min_{\boldsymbol{\theta}} \max_{\lambda \geq 0} L(\boldsymbol{\theta}, \lambda),$$

这称为强对偶性, 与 Lagrangian 对偶理论有关; 详情见 [26]. 优化问题 (2.48) 为

$$\max_{\lambda \geq 0} \min_{\boldsymbol{\theta}} L(\boldsymbol{\theta}, \lambda) \Leftrightarrow \max_{\lambda \geq 0} \min_{\boldsymbol{\theta}} \|\boldsymbol{y} - M\boldsymbol{\theta}\|_2^2 + \lambda(\|\boldsymbol{\theta}\|_2^2 - R). \tag{2.49}$$

与 (2.44) 类似, 误差函数可分解为

$$\begin{aligned} &\|\boldsymbol{y} - M\boldsymbol{\theta}\|_2^2 + \lambda(\|\boldsymbol{\theta}\|_2^2 - R) \\ &= \boldsymbol{w}_{\boldsymbol{\theta}}^\top (M^\top M + \lambda I)^{-1} \boldsymbol{w}_{\boldsymbol{\theta}} - (M^\top \boldsymbol{y})^\top (M^\top M + \lambda I)^{-1} M^\top \boldsymbol{y} + \boldsymbol{y}^\top \boldsymbol{y} - \lambda R, \end{aligned}$$

其中 $\boldsymbol{w}_{\boldsymbol{\theta}} = M^{\top}\boldsymbol{y} - (M^{\top}M + \lambda I)\boldsymbol{\theta}$. 由于 $\lambda \geq 0$, 矩阵 $M^{\top}M + \lambda I$ 为对称正定矩阵. 因此可通过求解下列线性方程组得到 $\boldsymbol{\theta}$:

$$(M^{\top}M + \lambda I)\boldsymbol{\theta} = M^{\top}\boldsymbol{y}. \tag{2.50}$$

设 $\boldsymbol{\theta}(\lambda)$ 为线性方程组 (2.50) 的解. 则 (2.48) 等价于

$$\max_{\lambda \geq 0} L(\boldsymbol{\theta}(\lambda), \lambda).$$

为近似求解上述极大化问题, 可取 $0 \leq \lambda_1 < \lambda_2 < \cdots < \lambda_m$, 然后计算

$$\max\{L(\boldsymbol{\theta}(\lambda_1), \lambda_1), L(\boldsymbol{\theta}(\lambda_2), \lambda_2), \ldots, L(\boldsymbol{\theta}(\lambda_m), \lambda_m)\}.$$

此时需要 $\boldsymbol{\theta}(\lambda_1), \boldsymbol{\theta}(\lambda_2), \ldots, \boldsymbol{\theta}(\lambda_m)$, 即

$$(M^{\top}M + \lambda_k I)\boldsymbol{\theta}(\lambda_k) = M^{\top}\boldsymbol{y} \quad k = 1, 2, \ldots, m. \tag{2.51}$$

方程 (2.51) 称为移位线性方程组. 求解移位线性方程组的 Krylov 子空间方法将在 Chapter 4 中介绍. 下面给出一个数值例子, 数据与 Fig. 2.10 相同. 给定 λ, 求解线性方程组 (2.50), 得到模型函数 (2.42) 的未知向量 $\boldsymbol{\theta}$. $m = 10$, $\lambda = 1.0$ 时 $f_{\boldsymbol{\theta}}(x)$ 的图像见 Fig. 2.12b. 为便于对照, 在 Fig. 2.12a 中再次给出 Fig. 2.11b. 由 Fig. 2.12b 可见, 结果现在对噪声具有鲁棒性, 因而似乎避免了 Fig. 2.12a 中出现的过拟合.

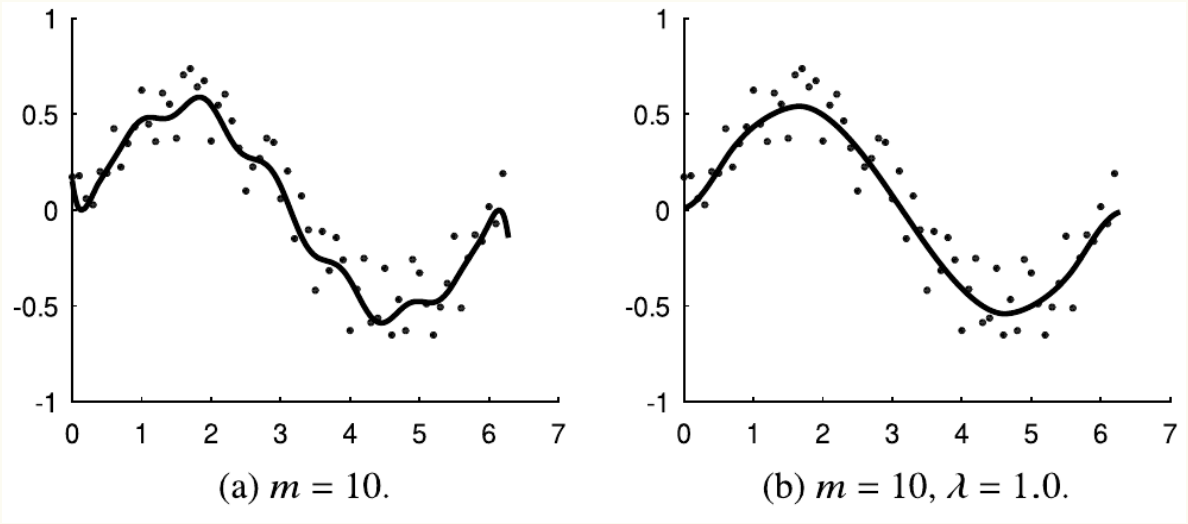


图 2.12: (2.42) 中 $m = 10$ 及 $m = 10$, $\lambda = 1.0$ 时 $f_{\boldsymbol{\theta}}(x)$ 的图像. (a) $m = 10$. (b) $m = 10$, $\lambda = 1.0$.

2.3.2 最小二乘分类

本小节介绍最小二乘概率分类 [178] (概述亦见 [148]). 设 $\mathcal{X} \subset \mathbb{R}^d$, $\mathcal{Y} = \{1, 2, \ldots, c\}$, 其中 c 为类别数. 分类问题是依据给定的数据集 (训练数据集) $(\boldsymbol{x}_i, y_i) \in \mathcal{X} \times \mathcal{Y}$ $(i = 1, 2, \ldots, n)$, 将 $\boldsymbol{x} \in \mathcal{X}$ 分入类别 $y \in \mathcal{Y}$. 这里 $(\boldsymbol{x}_i, y_i)$ 表示数据 $\boldsymbol{x}_i \in \mathcal{X}$ 属于类别 $y_i \in \mathcal{Y}$. 概率模式识别是根据训练数据 $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ 估计类别后验概率 $p(y|\boldsymbol{x})$. 一旦得到 $p(y|\boldsymbol{x})$, 新数据 $\boldsymbol{x}$ 即可按下式分类

$$\widehat{y} = \arg\max_{y \in \{1,2,\ldots,c\}} p(y|\boldsymbol{x}).$$

设 $p(\boldsymbol{x})$ 为 $\boldsymbol{x}$ 的边缘密度, 且对所有 $\boldsymbol{x}$ 满足 $p(\boldsymbol{x})>0$. 则类别后验概率 $p(y|\boldsymbol{x})$ 可写为

$$p(y|\boldsymbol{x})=\frac{p(\boldsymbol{x},y)}{p(\boldsymbol{x})}, \tag{2.52}$$

这是因为 $p(\boldsymbol{x},y)=p(y|\boldsymbol{x})p(\boldsymbol{x})$, 并且假设 $p(\boldsymbol{x})>0$. 最小二乘概率分类对类别后验概率 $p(y|\boldsymbol{x})$ 使用下列线性模型 $q(y|\boldsymbol{x};\boldsymbol{\theta}^{(y)})$:

$$q(y|\boldsymbol{x};\boldsymbol{\theta}^{(y)})=\boldsymbol{\theta}^{(y)\top}\boldsymbol{f}(\boldsymbol{x})=\theta_1^{(y)}f_1(\boldsymbol{x})+\theta_2^{(y)}f_2(\boldsymbol{x})+\cdots+\theta_b^{(y)}f_b(\boldsymbol{x}), \tag{2.53}$$

其中 $\boldsymbol{\theta}^{(y)}=[\theta_1^{(y)},\theta_2^{(y)},\ldots,\theta_b^{(y)}]^\top$, $f_i(\boldsymbol{x})$ $(i=1,\ldots,b)$ 为给定基函数, 例如

$$f_i(\boldsymbol{x})=K(\boldsymbol{x},\boldsymbol{x}_i)=\exp\left(-\frac{\|\boldsymbol{x}-\boldsymbol{x}_i\|_2^2}{2h^2}\right). \tag{2.54}$$

这里 $K(\boldsymbol{x},\boldsymbol{x}_i)$ 称为 Gaussian 核, h 为用户设定的标量参数. 参数 $\boldsymbol{\theta}^{(y)}$ 通过近似极小化下列最小二乘误差确定:

$$J_y(\boldsymbol{\theta}^{(y)})=\frac{1}{2}\int\left(q(y|\boldsymbol{x};\boldsymbol{\theta}^{(y)})-p(y|\boldsymbol{x})\right)^2p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}.$$

$J_y(\boldsymbol{\theta}^{(y)})$ 可改写为

$$\begin{aligned}J_y(\boldsymbol{\theta}^{(y)})&=\frac{1}{2}\int q(y|\boldsymbol{x};\boldsymbol{\theta}^{(y)})^2p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}-\int q(y|\boldsymbol{x};\boldsymbol{\theta}^{(y)})p(y|\boldsymbol{x})p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}+c\\&=\frac{1}{2}\int q(y|\boldsymbol{x};\boldsymbol{\theta}^{(y)})^2p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}-\int q(y|\boldsymbol{x};\boldsymbol{\theta}^{(y)})p(\boldsymbol{x},y)\,\mathrm{d}\boldsymbol{x}+c\\&=\frac{1}{2}\int q(y|\boldsymbol{x};\boldsymbol{\theta}^{(y)})^2p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}-\int q(y|\boldsymbol{x};\boldsymbol{\theta}^{(y)})p(y)p(\boldsymbol{x}|y)\,\mathrm{d}\boldsymbol{x}+c,\end{aligned} \tag{2.55}$$

其中 $c=(1/2)\int p(y|\boldsymbol{x})^2p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}$, 第二个等式使用了 (2.52). 利用训练数据 $(\boldsymbol{x}_1,y_1),(\boldsymbol{x}_2,y_2),\ldots,(\boldsymbol{x}_n,y_n)$ 的平均值, 有如下近似:

$$\begin{aligned}\int q(y|\boldsymbol{x};\boldsymbol{\theta}^{(y)})^2p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}&\approx\frac{1}{n}\sum_{i=1}^n q(y|\boldsymbol{x}_i;\boldsymbol{\theta}^{(y)})^2,\\\int q(y|\boldsymbol{x};\boldsymbol{\theta}^{(y)})p(y)p(\boldsymbol{x}|y)\,\mathrm{d}\boldsymbol{x}&\approx\frac{1}{n_y}\sum_{i:y_i=y}q(y|\boldsymbol{x}_i;\boldsymbol{\theta}^{(y)})p(y)\approx\frac{1}{n}\sum_{i:y_i=y}q(y|\boldsymbol{x}_i;\boldsymbol{\theta}^{(y)}).\end{aligned}$$

最后一个近似使用了 $p(y)\approx n_y/n$, 其中 n_y 为属于类别 $y\in\mathcal{Y}$ 的训练数据个数. 注意, 符号 $\sum_{i:y_i=y}$ 表示对给定类别 $y\in\mathcal{Y}$, 对满足 $y_i=y$ 的 i 求和. 将这一近似用于 (2.55), 并加入正则化项 $(\lambda/2n)\|\boldsymbol{\theta}^{(y)}\|_2^2$, 另见 (2.49), 得到下列新的极小化问题:

$$\widehat{J}_y(\boldsymbol{\theta}^{(y)})=\frac{1}{2n}\sum_{i=1}^n q(y|\boldsymbol{x}_i;\boldsymbol{\theta}^{(y)})^2-\frac{1}{n}\sum_{i:y_i=y}q(y|\boldsymbol{x}_i;\boldsymbol{\theta}^{(y)})+\frac{\lambda}{2n}\|\boldsymbol{\theta}^{(y)}\|_2^2, \tag{2.56}$$

其中省去了常数 c, 因为它不影响极小化问题. 令 $(M)_{i,j}=f_i(\boldsymbol{x}_j)$. 则由 (2.53), 近似误差函数 (2.56) 可改写为

$$\widehat{J}_y(\boldsymbol{\theta}^{(y)})=\frac{1}{2n}\boldsymbol{\theta}^{(y)\top}M^\top M\boldsymbol{\theta}^{(y)}-\frac{1}{n}\boldsymbol{\theta}^{(y)\top}M^\top\boldsymbol{p}^{(y)}+\frac{\lambda}{2n}\boldsymbol{\theta}^{(y)\top}\boldsymbol{\theta}^{(y)}, \tag{2.57}$$

其中 $\boldsymbol{p}^{(y)}$ 是一个向量, 其第 i 个元素 $p_i^{(y)}$ 定义为

$$p_i^{(y)} = \begin{cases} 1 & (\text{if } y_i = y), \\ 0 & (\text{if } y_i \neq y). \end{cases}$$

与上一小节类似, (2.57) 可分解为

$$\begin{aligned} \widehat{J}_y(\boldsymbol{\theta}^{(y)}) = & \frac{1}{2n}\left(M^\top \boldsymbol{p}^{(y)} - G\boldsymbol{\theta}^{(y)}\right)^\top G^{-1}\left(M^\top \boldsymbol{p}^{(y)} - G\boldsymbol{\theta}^{(y)}\right) \\ & - \frac{1}{2n}\boldsymbol{p}^{(y)\top} M G^{-1} M^\top \boldsymbol{p}^{(y)}, \end{aligned}$$

其中 $G = M^\top M + \lambda I$. 若 M 的列线性无关, 则对所有 $\lambda \geq 0$, G 为对称正定矩阵. 因此, 当 $M^\top \boldsymbol{p}^{(y)} - G\boldsymbol{\theta}^{(y)} = \mathbf{0}$ 时, $\widehat{J}_y(\boldsymbol{\theta}^{(y)})$ 达到极小值, 从而得到

$$(M^\top M + \lambda I)\boldsymbol{\theta}^{(y)} = M^\top \boldsymbol{p}^{(y)}. \tag{2.58}$$

求得 $\boldsymbol{\theta}^{(y)}$ 后, 即得到 (2.53). 再作下列归一化, 就得到近似类别概率:

$$\widehat{p}(y|\boldsymbol{x}) = \frac{\max(0, q(y|\boldsymbol{x}; \boldsymbol{\theta}^{(y)}))}{\sum_{\widetilde{y}=1}^{c} \max(0, q(\widetilde{y}|\boldsymbol{x}; \boldsymbol{\theta}^{(\widetilde{y})}))}.$$

下面通过一个简单数值例子说明最小二乘概率分类如何工作. 使用 Fig. 2.13a 给出的训练数据, 其中有两类: $\circ$ 表示类别 $y = 1$, $\times$ 表示类别 $y = 2$. 由 (2.53) 和 (2.54), 使用下列模型:

$$q(y|\boldsymbol{x}; \boldsymbol{\theta}^{(y)}) = \sum_{i:y_i=y} \theta_i^{(y)} K(\boldsymbol{x}, \boldsymbol{x}_i).$$

这意味着, 分别对 $y = 1$ 和 $y = 2$ 求解两个线性方程组 (2.58) 后, 将得到对应于 $y = 1$ 和 $y = 2$ 的两个函数 $q(y|\boldsymbol{x}; \boldsymbol{\theta}^{(y)})$. 最小二乘概率分类的结果见 Fig. 2.13, 其中样本数 $n = 60$, (2.56) 中的参数 λ 和 (2.54) 中的参数 h 分别取为 $\lambda = 0.1$ 和 $h = 1$. Fig. 2.13a 显示前述两类样本的分



(a) Training samples of two classes. (b) The class-posterior probabilities.

图 2.13: 最小二乘概率分类的结果. (a) 两类训练样本. (b) 类别后验概率.

布, Fig. 2.13b 为对应结果. 由 Fig. 2.13b 可见, 点 $x = 0$ 属于类别 1 ($y = 1$) 的概率约为 50%, 即 $p(y = 1|x = 0) \approx 0.5$.

2.4 矩阵方程

考虑下列矩阵方程:

$$AX + XB = C, \tag{2.59}$$

其中 A 和 B 分别为 m 行 m 列和 n 行 n 列的矩阵. 这一矩阵方程称为 Sylvester 方程, 出现在控制理论领域中. 在推导计算矩阵平方根的 Newton 方法时, 也会产生 Sylvester 方程, 见 Section 5.3.1. Sylvester 方程可以转换为形如 $M\boldsymbol{x} = \boldsymbol{c}$ 的线性方程组, 下面说明 Sylvester 方程 (2.59) 与线性方程组 $M\boldsymbol{x} = \boldsymbol{c}$ 之间的联系. 设 G 为 n 行 m 列的矩阵, 其列向量记为 $\boldsymbol{g}_1, \boldsymbol{g}_2, \ldots, \boldsymbol{g}_m$. 则矩阵 G 可写成 $G = [\boldsymbol{g}_1, \boldsymbol{g}_2, \ldots, \boldsymbol{g}_m]$. 现在定义 vec 算子. vec 算子是从 $\mathbb{C}^{n\times m}$ 到 $\mathbb{C}^{nm}$ 的映射, $\operatorname{vec}(G)$ 写成

$$\operatorname{vec}(G) = \begin{bmatrix} \boldsymbol{g}_1 \\ \vdots \\ \boldsymbol{g}_m \end{bmatrix}. \tag{2.60}$$

下面列出 vec 算子的一些性质.

Theorem 2.6:

设 $X \in \mathbb{C}^{m\times n}$, $A_i \in \mathbb{C}^{m\times m}$, $B_i \in \mathbb{C}^{n\times n}$, 且 P_{mn} 是 Theorem 2.5 中给出的置换矩阵. 则

1. $\operatorname{vec}(A_1XB_1 + \cdots + A_kXB_k) = (B_1^\top \otimes A_1 + \cdots + B_k^\top \otimes A_k)\operatorname{vec}(X)$,
2. $\operatorname{vec}(X^\top) = P_{mn}\operatorname{vec}(X)$.

Proof: 先证明第 (2) 项. 由 Theorem 2.5 中 P_{mn} 的定义,

$$P_{mn}\operatorname{vec}(X) = \begin{bmatrix} (E_{11})^\top & \cdots & (E_{1n})^\top \\ \vdots & \ddots & \vdots \\ (E_{m1})^\top & \cdots & (E_{mn})^\top \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_n \end{bmatrix},$$

因此 $P_{mn}\operatorname{vec}(X)$ 的第 k 个块元素为

$$(P_{mn}\operatorname{vec}(X))_k = \sum_{j=1}^{n} E_{kj}^\top \boldsymbol{x}_j = \sum_{j=1}^{n} E_{jk}\boldsymbol{x}_j = \sum_{j=1}^{n} \boldsymbol{e}_j\boldsymbol{e}_k^\top\boldsymbol{x}_j = \sum_{j=1}^{n} \boldsymbol{e}_j x_{kj} = \begin{bmatrix} x_{k1} \\ \vdots \\ x_{kn} \end{bmatrix},$$

这就是 $X^\top$ 的第 k 列. 因此,

$$P_{mn}\operatorname{vec}(X) = \begin{bmatrix} (P_{mn}\operatorname{vec}(X))_1 \\ \vdots \\ (P_{mn}\operatorname{vec}(X))_n \end{bmatrix} = \operatorname{vec}(X^\top).$$

接着证明第 (1) 项. 由于 $\operatorname{vec}(X + Y) = \operatorname{vec}(X) + \operatorname{vec}(Y)$, 只需证明 $\operatorname{vec}(AXB) = (B^\top \otimes A)\operatorname{vec}(X)$. 令 $M = XB$, $M = [\boldsymbol{m}_1, \ldots, \boldsymbol{m}_n]$, 则 $AM = [A\boldsymbol{m}_1, \ldots, A\boldsymbol{m}_n]$. 因此

$$\operatorname{vec}(AXB) = \operatorname{vec}(AM) = \begin{bmatrix} A\boldsymbol{m}_1 \\ \vdots \\ A\boldsymbol{m}_n \end{bmatrix} = (I \otimes A)\begin{bmatrix} \boldsymbol{m}_1 \\ \vdots \\ \boldsymbol{m}_n \end{bmatrix} = (I_n \otimes A)\operatorname{vec}(M), \tag{2.61}$$

而 $\operatorname{vec}(M)$ 计算如下

$$\begin{aligned}\operatorname{vec}(XB) &= \operatorname{vec}((B^\top X^\top)^\top) = P_{nm}\operatorname{vec}(B^\top X^\top) = P_{nm}(I_m \otimes B^\top)\operatorname{vec}(X^\top) \\ &= P_{nm}(I_m \otimes B^\top)P_{mn}\operatorname{vec}(X) = P_{mn}^\top(I_m \otimes B^\top)P_{mn}\operatorname{vec}(X) = (B^\top \otimes I_m)\operatorname{vec}(X).\end{aligned}$$

最后一个等式由 (2.13) 得到, 将此结果与 (2.61) 结合, 得

$$\operatorname{vec}(AXB) = (I_n \otimes A)\operatorname{vec}(XB) = (I_n \otimes A)(B^\top \otimes I_m)\operatorname{vec}(X) = (B^\top \otimes A)\operatorname{vec}(X).$$

最后一个等式由 Theorem 2.1 的第 (6) 项得到. 证明完毕.

■

现在可以说明 Sylvester 方程与线性方程组之间的关系. 对 Sylvester 方程 (2.59) 应用 Theorem 2.6 的第 (1) 项, 得

$$\operatorname{vec}(AX + XB) = \operatorname{vec}(C) \Leftrightarrow \underbrace{[(I_n \otimes A) + (B^\top \otimes I_m)]}_{M}\underbrace{\operatorname{vec}(X)}_{\boldsymbol{x}} = \underbrace{\operatorname{vec}(C)}_{\boldsymbol{c}},$$

其中 I_m 为 m 行 m 列的单位矩阵. 这里考虑稍作修改的如下矩阵方程

$$AX + X^\top B = C.$$

该矩阵称为 T-合同 Sylvester 方程. 应用 vec 算子并使用 Theorem 2.6 的第 (2) 项, 得

$$\begin{aligned}AX + X^\top B = C &\Leftrightarrow \operatorname{vec}(AX + X^\top B) = \operatorname{vec}(C) \\ &\Leftrightarrow \operatorname{vec}(AX) + \operatorname{vec}(X^\top B) = \operatorname{vec}(C) \\ &\Leftrightarrow \operatorname{vec}(I_n \otimes A)\operatorname{vec}(X) + (B^\top \otimes I_m)\operatorname{vec}(X^\top) = \operatorname{vec}(C) \\ &\Leftrightarrow (I_n \otimes A)\operatorname{vec}(X) + (B^\top \otimes I_m)P_{mn}\operatorname{vec}(X) = \operatorname{vec}(C) \\ &\Leftrightarrow \underbrace{[(I_n \otimes A) + (B^\top \otimes I_m)P_{mn}]}_{\widetilde{M}}\underbrace{\operatorname{vec}(X)}_{\boldsymbol{x}} = \underbrace{\operatorname{vec}(C)}_{\boldsymbol{c}}.\end{aligned}$$

因此, T-合同 Sylvester 方程同样等价于形如 $\widetilde{M}\boldsymbol{x} = \boldsymbol{c}$ 的线性方程组.

2.5 优化

考虑下列形式的无约束优化问题

$$\min_{\boldsymbol{x}\in\mathbb{R}^n} f(\boldsymbol{x}), \tag{2.62}$$

其中 $f(\boldsymbol{x})$ 为光滑函数. 除非函数 $f(\boldsymbol{x})$ 具有特殊结构, 例如为凸函数, 否则一般很难找到极小化问题的解. 如通常的微积分所述, 为找到解, 临界点, 即满足 $\nabla f(\boldsymbol{x}_c) = \mathbf{0}$ 的 $\boldsymbol{x}_c$, 通常很重要. 本节介绍 Euclidean 空间和 Riemannian 流形 (Grassmann 流形) 上用于求临界点的 Newton 方法, 并说明 Newton 方法的每次迭代如何产生线性方程组. 作为预备知识, 下一小节先介绍张量记号. 我们将看到, 张量记号能够系统而简洁地描述 Taylor 级数.

2.5.1 张量记号

记为 A 的矩阵是二维数组, 其 (i,j) 元素写作 $(A)_{ij}$ 或 a_{ij}. 记为 $\mathcal{A}$ 的张量是多维数组; 对三维数组, 其 (i,j,k) 元素写作 $(\mathcal{A})_{ijk}$ 或 a_{ijk}. 更一般地, N 阶张量 (或 N 路张量) $\mathcal{A}$ 是一个 N 维数组, 其 $i_1, i_2, \ldots, i_N$ 元素写作 $(\mathcal{A})_{i_1,i_2,\ldots,i_N}$ 或 $a_{i_1,i_2,\ldots,i_N}$. 若所有元素 $a_{i_1,i_2,\ldots,i_N} \in \mathbb{R}$, 且

$i_1, i_2, \ldots, i_N$ 从 $i_1 = i_2 = \cdots = i_N = 1$ 取到 $i_1 = I_1, i_2 = I_2, \ldots, i_N = I_N$, 则 $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$. 给定两个张量 $\mathcal{X}, \mathcal{Y} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$ 和标量值 $\alpha \in \mathbb{R}$, 张量加法和标量乘法定义为

$$(\mathcal{X} + \mathcal{Y})_{i_1,i_2,\ldots,i_N} = x_{i_1,i_2,\ldots,i_N} + y_{i_1,i_2,\ldots,i_N}, \quad (\alpha\mathcal{X})_{i_1,i_2,\ldots,i_N} = \alpha x_{i_1,i_2,\ldots,i_N}.$$

下面定义张量与矩阵的乘法, 或称 n-模乘积. 设 $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$, $U \in \mathbb{R}^{J \times I_n}$. $\mathcal{X}$ 与 U 的 n-模乘积定义为

$$(\mathcal{X} \times_n U)_{i_1,\ldots,i_{n-1},j,i_{n+1},\ldots,i_N} = \sum_{k=1}^{I_n} x_{i_1,i_2,\ldots,i_{n-1},k,i_{n+1},\ldots,i_N} u_{j,k}.$$

对三阶张量,

$$(\mathcal{X} \times_1 U)_{j,i_2,i_3} = \sum_{k=1}^{I_1} x_{k,i_2,i_3} u_{j,k},$$
$$(\mathcal{X} \times_2 U)_{i_1,j,i_3} = \sum_{k=1}^{I_2} x_{i_1,k,i_3} u_{j,k},$$
$$(\mathcal{X} \times_3 U)_{i_1,i_2,j} = \sum_{k=1}^{I_3} x_{i_1,i_2,k} u_{j,k}.$$

由定义容易看出

$$(\mathcal{X} \times_1 A) \times_2 B = (\mathcal{X} \times_2 B) \times_1 A. \tag{2.63}$$

因此可写成

$$\mathcal{X} \times_1 A \times_2 B,$$

更一般地可写成 $\mathcal{X} \times_1 A_1 \times_2 A_2 \times_3 \cdots \times_N A_N$. 若 $\mathcal{X}$ 为矩阵 (改记为 X, 不用 $\mathcal{X}$), 则 (2.63) 等价于

$$(AX)B^\top = A(XB^\top).$$

因此在通常的线性代数中省略括号, 即写成 $AXB^\top$. 矩阵 X 的奇异值分解为 $U\Sigma V^{\mathrm{H}}$, 其中 U 和 V 为酉矩阵, Σ 为对角矩阵, 其对角元素为 $(\Sigma)_{i,i} = \sigma_i \geq 0$. 利用张量记号, X 的奇异值分解可改写为

$$X = U\Sigma V^\top = \Sigma \times_1 U \times_2 V.$$

更一般的乘积是缩并乘积 $\langle \cdot, \cdot \rangle$, 它是下列映射:

$$\langle \cdot, \cdot \rangle_{1,\ldots,m;1,\ldots,m} : \mathbb{R}^{I_1 \times \cdots \times I_m \times J_1 \times \cdots \times J_N} \times \mathbb{R}^{I_1 \times \cdots \times I_m \times K_1 \times \cdots \times K_p} \to \mathbb{R}^{J_1 \times \cdots \times J_N \times K_1 \times \cdots \times K_p},$$
$$\langle \mathcal{X}, \mathcal{Y} \rangle_{1,\ldots,m;1,\ldots,m} := \sum_{i_1=1}^{I_1} \cdots \sum_{i_m=1}^{I_m} x_{i_1,\ldots,i_m,j_1,\ldots,j_N} y_{i_1,\ldots,i_m,k_1,\ldots,k_p}.$$

下面给出两个张量的缩并乘积的若干更一般的例子. 对 $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$ 和 $\mathcal{Y} \in \mathbb{R}^{J_1 \times J_2 \times J_3 \times J_4}$, 下式是两张量缩并乘积的一个例子.

$$\langle \mathcal{X}, \mathcal{Y} \rangle_{1,3:2,3} = \sum_{k=1}^{I_1} \sum_{l=1}^{I_3} x_{k,i_2,l} \times y_{j_1,k,l,j_4},$$

其中假设 $I_1 = J_2$, $I_3 = J_3$. 另一个例子如下:

$$\langle \mathcal{X}, \mathcal{Y} \rangle_{1,2:2,4} = \sum_{k=1}^{I_1} \sum_{l=1}^{I_2} x_{k,l,i_3} \times y_{j_1,k,j_3,l},$$

其中假设 $I_1 = J_2$, $I_2 = J_4$.

2.5.2 Euclidean 空间上的 Newton 方法

对 $\boldsymbol{x} \in \mathbb{R}^n$ 上的光滑函数 $f(\boldsymbol{x})$, 张量与向量 (仅有一列的矩阵) 的 n-模乘积有助于描述 $f(\boldsymbol{x}+\boldsymbol{h})$ 在 $\boldsymbol{x}$ 处的 Taylor 级数:

$$f(\boldsymbol{x}+\boldsymbol{h}) = f(\boldsymbol{x}) + \frac{1}{1!}[f^{(1)}(\boldsymbol{x})] \times_1 \boldsymbol{h} \tag{2.64}$$

$$+ \frac{1}{2!}[f^{(2)}(\boldsymbol{x})] \times_1 \boldsymbol{h} \times_2 \boldsymbol{h} + \frac{1}{3!}[f^{(3)}(\boldsymbol{x})] \times_1 \boldsymbol{h} \times_2 \boldsymbol{h} \times_3 \boldsymbol{h} + \cdots, \tag{2.65}$$

其中

$$\begin{aligned}
[f^{(1)}(\boldsymbol{x})]_{i_1} &= \frac{\partial}{\partial x_{i_1}} f(\boldsymbol{x}), & [f^{(1)}(\boldsymbol{x})] &\in \mathbb{R}^n, \\
[f^{(2)}(\boldsymbol{x})]_{i_1 i_2} &= \frac{\partial^2}{\partial x_{i_1} \partial x_{i_2}} f(\boldsymbol{x}), & [f^{(2)}(\boldsymbol{x})] &\in \mathbb{R}^{n \times n}, \\
[f^{(3)}(\boldsymbol{x})]_{i_1 i_2 i_3} &= \frac{\partial^3}{\partial x_{i_1} \partial x_{i_2} \partial x_{i_3}} f(\boldsymbol{x}), & [f^{(3)}(\boldsymbol{x})] &\in \mathbb{R}^{n \times n \times n}, \\
&\vdots
\end{aligned}$$

这里 $i_1, i_2, i_3, \ldots \in \{1, 2, \ldots, n\}$. 向量 $f^{(1)}(\boldsymbol{x})$ 是 $f(\boldsymbol{x})$ 的梯度, 即 $f^{(1)}(\boldsymbol{x}) = \nabla f(\boldsymbol{x})$. 矩阵 $f^{(2)}(\boldsymbol{x})$ 称为 Hessian 矩阵, 记为 $H(\boldsymbol{x})$. 用这些记号, Taylor 级数 (2.65) 可改写为

$$f(\boldsymbol{x}+\boldsymbol{h}) = f(\boldsymbol{x}) + \boldsymbol{h}^\top \nabla f(\boldsymbol{x}) + \frac{1}{2}\boldsymbol{h}^\top H(\boldsymbol{x})\boldsymbol{h} + \frac{1}{3!}[f^{(3)}(\boldsymbol{x})] \times_1 \boldsymbol{h} \times_2 \boldsymbol{h} \times_3 \boldsymbol{h} + \cdots. \tag{2.66}$$

现在考虑寻找使 $f(\boldsymbol{x})$ 达到局部极小值的局部极小点 $\boldsymbol{x}^*$. 设 $\boldsymbol{x}_n$ 是 $f(\boldsymbol{x})$ 的近似极小点 (即 $\boldsymbol{x}_n \approx \boldsymbol{x}^*$). 则原问题转化为寻找修正向量 $\boldsymbol{h}$, 使 $f(\boldsymbol{x}_k + \boldsymbol{h})$ 达到极小值. Newton 方法寻找修正向量 $\boldsymbol{h}$, 使 $\boldsymbol{x}_k + \boldsymbol{h}$ 极小化的不是 $f(\boldsymbol{x}_k + \boldsymbol{h})$ 本身, 而是 $f(\boldsymbol{x}_k + \boldsymbol{h})$ 的二次近似, 即

$$\min_{\boldsymbol{h} \in \mathbb{R}^n} q(\boldsymbol{h}), \quad q(\boldsymbol{h}) := f(\boldsymbol{x}_k) + \boldsymbol{h}^\top \nabla f(\boldsymbol{x}_k) + \frac{1}{2}\boldsymbol{h}^\top H(\boldsymbol{x}_k)\boldsymbol{h}.$$

若 $H(\boldsymbol{x}_k)$ 正定, 则 $q(\boldsymbol{h})$ 的极小点唯一, 由 $\nabla q(\boldsymbol{h}) = \boldsymbol{0}$ 的解给出. 由于 $\nabla q(\boldsymbol{h}) = \nabla f(\boldsymbol{x}_k) + H(\boldsymbol{x}_k)\boldsymbol{h}$, 第 k 次迭代的修正向量 $\boldsymbol{h}$, 记为 $\boldsymbol{h}_k$, 可由求解下列形式的线性方程组得到

$$H(\boldsymbol{x}_k)\boldsymbol{h}_k = -\nabla f(\boldsymbol{x}_k). \tag{2.67}$$

求解 (2.67) 后, 得到新的近似极小点 $\boldsymbol{x}_{k+1} (= \boldsymbol{x}_k + \boldsymbol{h}_k)$. 每次迭代 k 都需要求解线性方程组 (2.67).

2.5.3 Riemannian 流形上的 Newton 方法

与上一小节类似, 考虑 Riemannian 流形上的 Newton 方法时, Riemannian 流形上的极小化问题也会产生线性方程组. 作为例子, 考虑 Grassmann 流形上的优化问题, 它是 Riemannian 流形的一个例子. 考虑下列极小化问题

$$\min_{X \in \mathbb{R}^{n \times p}} f(X), \tag{2.68}$$

其中假设 f 为光滑函数. 利用 vec 算子 (2.60), (2.68) 中的函数等价于 $f(\widetilde{\boldsymbol{x}})$, 其中 $\widetilde{\boldsymbol{x}} = \mathrm{vec}(X)$. 因而极小化问题 (2.68) 本质上与 (2.62) 相同. 若 X 满足条件 $X^\top X = I_p$, 则极小化问题 (2.68) 可写成

$$\min_{X \in \mathrm{St}(p,n)} f(X), \tag{2.69}$$

其中 $\mathrm{St}(p,n) = \{X \in \mathbb{R}^{n\times p} : X^\top X = I_p\}$ $(p \le n)$, $\mathrm{St}(p,n)$ 称为 Stiefel 流形. 条件 $X^\top X = I_p$ 表示 X 的所有列向量 $\boldsymbol{x}_i$ $(i = 1, \ldots, p)$ 都是标准正交的, 即当 $i \ne j$ 时 $\boldsymbol{x}_i^\top \boldsymbol{x}_j = 0$, 当 $i = j$ 时 $\boldsymbol{x}_i^\top \boldsymbol{x}_j = 1$. 现在进一步假设函数 f 具有以下性质:

$$f(XQ) = f(X) \tag{2.70}$$

其中 $Q \in \mathbb{R}^{p\times p}$ 为任意正交矩阵. 这一性质会使极小化问题的求解变得困难. 具体而言, 设 X_k 为 $f(X)$ 的近似极小点, X_{k+1} 为对某个正交矩阵 Q_k 由 X_kQ_k 给出的下一迭代值. 则由于目标函数值没有改变, 迭代值 X_{k+1} 并不是得到改进的近似解. 为避免这一情形, 一种有希望的方法是: 对给定的 $X_k \in \mathrm{St}(p,n)$, 将集合 $[X_k] := \{X_kQ : Q^\top Q = I_p\}$ 视为一个元素. 这样, 下一迭代值 $[X_{k+1}](\ne [X_k])$ 就有意义, 因为 X_{k+1} 可以避免由 $X_{k+1} = X_kQ_k$ 导致的 $f(X_k) = f(X_{k+1})$ 情形. 一般地, 集合 $\mathrm{Grass}(p,n) := \{[X] : X \in \mathrm{St}(p,n)\}$ 称为 Grassmann 流形. 上述方法对应于考虑下列极小化问题:

$$\min_{[Y] \in \mathrm{Grass}(p,n)} f(Y), \tag{2.71}$$

而不是考虑具有 (2.70) 的 (2.69). 数值计算中产生下列近似解: $X_0 \in [Y_0], X_1 \in [Y_1], \ldots$, 即 $k = 0, 1, \ldots$ 时等价类 $[Y_k]$ 的代表元. 极小化问题 (2.71) 可看作 Grassmann 流形上的无约束极小化问题. 与 (2.67) 类似, Grassmann 流形上的 Newton 方法要求求解下列线性方程组 (Newton-Grassmann 方程):

$$Y_{k\perp}^\top \partial^2 f(Y_k)[Y_{k\perp}C_k] - C_kY_k^\top \partial f(Y_k) = \underbrace{-Y_{k\perp}^\top \partial f(Y_k)}_{(n-p)\times p}, \tag{2.72}$$

其中 $Y_k \in \mathrm{St}(p,n)$, $Y_{k\perp} \in \mathrm{St}(n-p,n)$, 后者的列向量与 Y_k 的列向量正交, 待求未知矩阵为 $C_k \in \mathbb{R}^{(n-p)\times p}$. 这里 $\partial f(Y) \in \mathbb{R}^{n\times p}$ $(Y \in \mathbb{R}^{n\times p})$ 和 $\partial^2 f(Y)[Z] \in \mathbb{R}^{n\times p}$ $(Z \in \mathbb{R}^{n\times p})$ 的 (i,j) 元素为

$$(\partial f(Y))_{ij} = \frac{\partial}{\partial y_{ij}} f(Y),$$

$$(\partial^2 f(Y)[Z])_{ij} = \sum_{k=1}^{n}\sum_{l=1}^{p} \frac{\partial^2}{\partial y_{ij}\partial y_{kl}} f(Y) \times z_{kl}.$$

现在使用四阶张量 $\mathcal{G} \in \mathbb{R}^{n\times p\times n\times p}$, 其 (i,j,k,l) 元素定义为

$$(\mathcal{G})_{ijkl} = \frac{\partial^2}{\partial y_{ij}\partial y_{kl}} f(Y).$$

于是, $\partial^2 f(Y_k)[Z]$ 可看作 $\mathcal{G}$ 和 Z 的下列缩并乘积:

$$\partial^2 f(Y_k)[Y_{k\perp}C_k] = \langle \mathcal{G}, Z\rangle_{3,4;1,2}.$$

关于缩并乘积, 见 Section 2.5.1. 求解 (2.72) 得到 C_k 后, 可能会认为下一迭代值为 $Y_{k+1} = Y_{k\perp}C_k$. 但一般而言, $Y_{k\perp}C_k$ 不属于 Grassmann 流形. 因此需要将 $Y_{k\perp}C_k$ 映射到 Grassmann 流形上, 几何上适当的映射称为回缩. 下一迭代值 Y_{k+1} 是利用某个回缩, 从 $Y_{k\perp}C_k$ 得到的 Grassmann 流形上的矩阵. 关于矩阵流形上优化的详情, 见 [3], [56], [155].

2.5.3.1 Example 作为例子, 考虑在 Grassman 流形上极小化下列 Rayleigh 商:

$$\min_{[Y]\in \mathrm{Grass}(p,n)} f(Y), \quad f(Y) := \frac{1}{2}\operatorname{tr}(Y^\top AY),$$

其中 $Y \in \mathbb{R}^{n\times p}$, $A \in \mathbb{R}^{n\times n}$. 设 $Q \in \mathbb{R}^{p\times p}$ 为正交矩阵. 则 $f(YQ) = \operatorname{tr}((YQ)^\top AYQ)/2 = \operatorname{tr}(Q^\top Y^\top AYQ)/2 = \operatorname{tr}(Y^\top AYQQ^\top)/2 = \operatorname{tr}(Y^\top AY)/2 = f(Y)$. 因此将 $\{YQ \in \mathbb{R}^{n\times p} : Q^\top Q = I_p\}$ 视为一个元素 (即等价类), 记为 $[Y]$. 下面计算 $\partial f(Y)$ 和 $\partial^2 f(Y)[Z]$. $\partial f(Y)$ 的 (i,j) 元素计算如下:

$$\begin{aligned}
(\partial f(Y))_{ij} &= \frac{\partial}{\partial y_{ij}} f(Y) = \frac{\partial}{\partial y_{ij}} \frac{1}{2}\operatorname{tr}(Y^\top AY) = \frac{1}{2}\frac{\partial}{\partial y_{ij}} \sum_{k,l,m} y_{kl}^\top a_{lm} y_{mk} \\
&= \frac{1}{2}\frac{\partial}{\partial y_{ij}} \sum_{k,l,m} y_{lk} a_{lm} y_{mk} = \frac{1}{2}\sum_{k,l,m} \frac{\partial y_{lk}}{\partial y_{ij}} a_{lm} y_{mk} + \frac{1}{2}\sum_{k,l,m} y_{lk} a_{lm} \frac{\partial y_{mk}}{\partial y_{ij}} \\
&= \frac{1}{2}\sum_{m} \frac{\partial y_{ij}}{\partial y_{ij}} a_{im} y_{mj} + \frac{1}{2}\sum_{l} y_{lj} a_{li} \frac{\partial y_{ij}}{\partial y_{ij}} = \frac{1}{2}\sum_{m} a_{im} y_{mj} + \frac{1}{2}\sum_{l} y_{lj} a_{li} \\
&= \frac{1}{2}\sum_{m} a_{im} y_{mj} + \frac{1}{2}\sum_{l} a_{il}^\top y_{lj} \\
&= \frac{1}{2}(AY)_{ij} + \frac{1}{2}(A^\top Y)_{ij}.
\end{aligned}$$

这里对标量值 $y_{kl}^\top$ 使用符号 "$\top$", 它表示 y_{lk}. 计算 $A^\top B$ 的 (i,j) 元素时, 这一记号很方便. 由矩阵乘矩阵的定义, 有 $(AB)_{ij} = \sum_k a_{ik} b_{kj}$. 使用上述记号, 对 $A^\top B$ 的 (i,j) 元素有 $(A^\top B)_{ij} = \sum_k a_{ik}^\top b_{kj} = \sum_k a_{ki} b_{kj}$. 由于 $A = A^\top$, 得

$$\partial f(Y) = AY. \tag{2.73}$$

接着, $\partial^2 f(Y)[Z]$ 的 (i,j) 元素为

$$\begin{aligned}
(\partial^2 f(Y)[Z])_{ij} &= \sum_{k,l} \frac{\partial^2}{\partial y_{ij}\partial y_{kl}} f(Y) \times z_{kl} = \sum_{k,l} \frac{\partial}{\partial y_{ij}} \left(\frac{\partial}{\partial y_{kl}} f(Y) \right) \times z_{kl} \\
&= \sum_{k,l} \frac{\partial}{\partial y_{ij}} \left(\frac{1}{2}\sum_m a_{km} y_{ml} + \frac{1}{2}\sum_m a_{km}^\top y_{ml} \right) \times z_{kl} \\
&= \sum_{k,l} \left(\frac{1}{2}\sum_m a_{km} \frac{\partial y_{ml}}{\partial y_{ij}} + \frac{1}{2}\sum_m a_{km}^\top \frac{\partial y_{ml}}{\partial y_{ij}} \right) \times z_{kl} \\
&= \sum_{k} \left(\frac{1}{2} a_{ki} \frac{\partial y_{ij}}{\partial y_{ij}} + \frac{1}{2} a_{ki}^\top \frac{\partial y_{ij}}{\partial y_{ij}} \right) \times z_{kj} \\
&= \sum_{k} \left(\frac{1}{2} a_{ki} z_{kj} + \frac{1}{2} a_{ki}^\top z_{kj} \right) \\
&= \sum_{k} \left(\frac{1}{2} a_{ik}^\top z_{kj} + \frac{1}{2} a_{ik} z_{kj} \right) \\
&= \left(\frac{1}{2}(A^\top Z)_{ij} + \frac{1}{2}(AZ)_{ij} \right).
\end{aligned}$$

由于 $A = A^\top$, 得

$$\partial^2 \bar{f}(Y)[Z] = AZ. \tag{2.74}$$

将 (2.73) 和 (2.74) 代入 Newton–Grassmann 方程 (2.72), 得

$$(Y_{k\perp}^{\top} A Y_{k\perp}) C_k - C_k (Y_k^{\top} A Y_k) = -Y_{k\perp}^{\top} A Y_k, \tag{2.75}$$

其中 C 为待求的未知矩阵. 这就是 (2.59) 中所述的 Sylvester 方程. 求解 (2.75) 得到 C_k 后, 下一迭代值 Y_{k+1} 例如可取为 $Y_{k+1} = Q_k$, 其中 Q_k 是 $Y_{k\perp} C_k$ 的 (薄) QR 分解.

3 Krylov 子空间方法的分类与理论

摘要 Krylov 子空间方法大致分为三类: 用于 Hermitian 线性方程组的方法, 用于复对称线性方程组的方法, 以及用于非 Hermitian 线性方程组的方法. 非 Hermitian 线性方程组包含复对称线性方程组, 因为复对称矩阵是非 Hermitian 的且是对称的. 复对称线性方程组的 Krylov 子空间方法利用系数矩阵的对称性, 因而比非 Hermitian 线性方程组的 Krylov 子空间方法更高效. 本章还介绍用于提高 Krylov 子空间方法收敛速度的预条件技术.

3.1 Hermitian 线性方程组

本节推导共轭梯度 (CG) 方法, 共轭残差 (CR) 方法和最小残差 (MINRES) 方法. 这些方法用于矩阵 A 为 Hermitian (或实对称) 矩阵的情形. CG 方法通常用于 A 为 Hermitian 正定矩阵的情形, CR 方法和 MINRES 方法则用于 Hermitian (不定) 矩阵. 在精确算术下, CR 方法和 MINRES 方法产生相同的近似解. 我们将看到, 这些方法在每次迭代都找到最佳近似解. 具体而言, 对 Hermitian 正定矩阵, CG 方法找到使误差的某个加权范数达到极小值的最佳近似解. 另一方面, CR 方法和 MINRES 方法找到使残差的 2-范数达到极小值的最佳近似解.

3.1.1 共轭梯度 (CG) 方法

CG 方法 [95] 由 Hestenes (1906–1991) 和 Stiefel (1909–1978) 于 1952 年提出, 是最著名的 Krylov 子空间方法. 在精确算术下, 与其他 Krylov 子空间方法一样, CG 方法在有限次迭代内给出精确解. 这意味着 CG 方法具有双重特点: 既是 Section 1.3 中的直接方法, 也是 Section 1.6 中的迭代方法. 关于 CG 方法发展的编年史, 见 Golub 和 O'Leary 的精彩综述 [80]. 下面从 Lanczos 过程 (Algorithm 1.11) 推导 CG 方法. 由 (1.42), Lanczos 过程的矩阵形式为

$$AR_n = R_{n+1}T_{n+1,n}, \tag{3.1}$$

其中

$$R_n := [\boldsymbol{r}_0, \boldsymbol{r}_1, \dots, \boldsymbol{r}_{n-1}],$$

$$T_{n+1,n} = \begin{bmatrix} t_{0,0} & t_{0,1} & & \\ -\alpha_0^{-1} & t_{1,1} & \ddots & \\ & -\alpha_1^{-1} & \ddots & t_{n-2,n-1} \\ & & \ddots & t_{n-1,n-1} \\ & & & -\alpha_{n-1}^{-1} \end{bmatrix},$$

$$t_{k-1,k} := \frac{(\boldsymbol{r}_{k-1}, A\boldsymbol{r}_k)}{(\boldsymbol{r}_{k-1}, \boldsymbol{r}_{k-1})}, \quad (k = 1, 2, \dots, n-1),$$

$$t_{k,k} := \frac{(\boldsymbol{r}_k, A\boldsymbol{r}_k)}{(\boldsymbol{r}_k, \boldsymbol{r}_k)}, \quad (k = 0, 1, \dots, n-1).$$

这里尺度参数 (α_k) 尚未确定. 设 $\boldsymbol{x}_k$ 为 $A\boldsymbol{x} = \boldsymbol{b}$ 的第 k 个近似解, 且这些 $\boldsymbol{x}_k$ 满足下列等式:

$$R_n = [\boldsymbol{b} - A\boldsymbol{x}_0, \boldsymbol{b} - A\boldsymbol{x}_1, \dots, \boldsymbol{b} - A\boldsymbol{x}_{n-1}] = \boldsymbol{b}\mathbf{1}_n^\top - AX_n,$$

其中 $\mathbf{1}_n := [1, 1, \dots, 1]^\top \in \mathbb{R}^n$, $X_n := [\boldsymbol{x}_0, \boldsymbol{x}_1, \dots, \boldsymbol{x}_{n-1}]$. 注意, R_n 就是 (3.1) 中给出的矩阵. 于是有

$$\begin{aligned} &AR_n = R_{n+1}T_{n+1,n} = (\boldsymbol{b}\mathbf{1}_{n+1}^\top - AX_{n+1})T_{n+1,n} \\ \Leftrightarrow\ &R_n = A^{-1}(\boldsymbol{b}\mathbf{1}_{n+1}^\top - AX_{n+1})T_{n+1,n} = \boldsymbol{x}\mathbf{1}_{n+1}^\top T_{n+1,n} - X_{n+1}T_{n+1,n}. \end{aligned} \tag{3.2}$$

现在用条件 $\mathbf{1}_{n+1}^{\top}T_{n+1,n}=[0,0,\ldots,0]$ 确定尺度参数 α_k, 即 $\alpha_0^{-1}=t_{0,0}$, 且对 $k=2,3,\ldots,n$, 有 $\alpha_{k-1}^{-1}=t_{k-1,k-1}+t_{k-2,k-1}$. 于是, 这一条件和 (3.2) 给出

$$R_n=-X_{n+1}T_{n+1,n}, \tag{3.3}$$

其中 (3.1) 中的矩阵 $T_{n+1,n}$ 改写为

$$T_{n+1,n}=\begin{bmatrix} \alpha_0^{-1} & t_{0,1} & & \\ -\alpha_0^{-1} & \alpha_1^{-1}-t_{0,1} & \ddots & \\ & -\alpha_1^{-1} & \ddots & t_{n-2,n-1} \\ & & \ddots & \alpha_{n-1}^{-1}-t_{n-2,n-1} \\ & & & -\alpha_{n-1}^{-1} \end{bmatrix}$$

且矩阵 $T_{n+1,n}$ 具有下列 LDU 分解:

$$T_{n+1,n}=\underbrace{\begin{bmatrix} 1 & & \\ -1 & 1 & \\ \ddots & \ddots & \\ & -1 & 1 \end{bmatrix}}_{L}\underbrace{\begin{bmatrix} \alpha_0^{-1} & & & \\ & \alpha_1^{-1} & & \\ & & \ddots & \\ & & & \alpha_{n-1}^{-1} \end{bmatrix}}_{D^{-1}}\underbrace{\begin{bmatrix} 1 & \alpha_0 t_{0,1} & & \\ & 1 & \ddots & \\ & & \ddots & \alpha_{n-1}t_{n-1,n} \\ & & & 1 \end{bmatrix}}_{U}, \tag{3.4}$$

其中 L 为 $n+1$ 行 n 列的下双对角矩阵, D^{-1} 为 n 行 n 列的对角矩阵, U 为 n 行 n 列的上双对角矩阵. 因此得到

$$R_n=-X_{n+1}LD^{-1}U. \tag{3.5}$$

现在定义 $P_n(=[\boldsymbol{p}_0,\boldsymbol{p}_1,\ldots,\boldsymbol{p}_{n-1}]):=R_nU^{-1}$. 则 (3.5) 可改写为

$$P_nD=X_{n+1}(-L), \tag{3.6}$$

它等价于

$$\boldsymbol{x}_{k+1}=\boldsymbol{x}_k+\alpha_k\boldsymbol{p}_k,\quad k=0,1,\ldots,n-1. \tag{3.7}$$

由关系 $\boldsymbol{r}_k=\boldsymbol{b}-A\boldsymbol{x}_k$ 和 (3.7) 得

$$\boldsymbol{r}_{k+1}=\boldsymbol{r}_k-\alpha_kA\boldsymbol{p}_k,\quad k=0,1,\ldots,n-1. \tag{3.8}$$

定义 $P_n=R_nU^{-1}$ 给出 $P_nU=R_n$, 它等价于

$$\boldsymbol{p}_0=\boldsymbol{r}_0, \tag{3.9}$$

$$\boldsymbol{p}_k=\boldsymbol{r}_k+\beta_{k-1}\boldsymbol{p}_{k-1},\quad k=1,2,\ldots,n-1, \tag{3.10}$$

其中 $\beta_{k-1}:=-\alpha_{k-1}t_{k-1,k}$. 如上所述, 计算 α_k 需要 $A\boldsymbol{r}_k$, 因为 $\alpha_k=(t_{k,k}+t_{k-1,k})^{-1}$; 这意味着若使用迭代式 (3.7)–(3.10), 每次迭代需要两次矩阵向量乘法, 即 $A\boldsymbol{r}_k$ 和 $A\boldsymbol{p}_k$. 下面将说明不需要 $A\boldsymbol{r}_k$. 为此先推导 α_k 的计算公式. 由 (3.1), (3.4) 及 P_n 的定义, 有

$$AP_n=R_{n+1}LD^{-1}.$$

由此得到

$$P_n^{\mathrm{H}} A P_n D = U^{-\mathrm{H}} R_n^{\mathrm{H}} R_{n+1} L. \tag{3.11}$$

由于 (3.1) 中的 $R_n = [\boldsymbol{r}_0, \boldsymbol{r}_1, \ldots, \boldsymbol{r}_{n-1}]$ 由 Lanczos 过程生成, 当 $i \neq j$ 时有 $\boldsymbol{r}_i^{\mathrm{H}} \boldsymbol{r}_j = 0$. 用矩阵形式表示, 即当 $i \neq j$ 时 $R_n^{\mathrm{H}} R_{n+1}$ 的 (i, j) 元素为零. 由于 L 和 $U^{-\mathrm{H}}$ 都是对角元素全为一的下三角矩阵, 这一事实给出 (3.11) 中 $U^{-\mathrm{H}} R_n^{\mathrm{H}} R_{n+1} L$ 的 $(k+1, k+1)$ 元素为 $\boldsymbol{r}_k^{\mathrm{H}} \boldsymbol{r}_k$. 因此由 (3.11) 的 $(k+1, k+1)$ 元素有 $\boldsymbol{p}_k^{\mathrm{H}} A \boldsymbol{p}_k \alpha_k = \boldsymbol{r}_k^{\mathrm{H}} \boldsymbol{r}_k$, 即

$$\alpha_k = \frac{\boldsymbol{r}_k^{\mathrm{H}} \boldsymbol{r}_k}{\boldsymbol{p}_k^{\mathrm{H}} A \boldsymbol{p}_k}. \tag{3.12}$$

由于 A 为 Hermitian 矩阵, 对所有 k, α_k 都是实数, 稍后推导 β_k 的计算公式时将使用这一点. 接着给出 β_k 的计算公式. 由 (3.1) 得

$$R_n^{\mathrm{H}} A R_n = R_n^{\mathrm{H}} R_{n+1} T_{n+1,n}, \tag{3.13}$$

$R_n^{\mathrm{H}} A R_n$ 的 $(k+1, k)$ 元素为 $\boldsymbol{r}_k^{\mathrm{H}} A \boldsymbol{r}_{k-1}$. 如 (3.11) 所示, $R_n^{\mathrm{H}} R_{n+1}$ 在 $k = 1, 2, \ldots, n$ 时的 (k, k) 元素为 $\boldsymbol{r}_{k-1}^{\mathrm{H}} \boldsymbol{r}_{k-1}$, 其余元素为零. 由此可见, $R_n^{\mathrm{H}} R_{n+1} T_{n+1,n}$ 的 $(k+1, k)$ 元素为 $\boldsymbol{r}_k^{\mathrm{H}} \boldsymbol{r}_k \times (-\alpha_{k-1})^{-1}$. 因此得到 $\boldsymbol{r}_k^{\mathrm{H}} A \boldsymbol{r}_{k-1} = \boldsymbol{r}_k^{\mathrm{H}} \boldsymbol{r}_k \times (-\alpha_{k-1})^{-1}$. 由于前述 α_{k-1} 为实数, $\boldsymbol{r}_k^{\mathrm{H}} A \boldsymbol{r}_{k-1}$ 也是实数. 因而 $\boldsymbol{r}_k^{\mathrm{H}} A \boldsymbol{r}_{k-1} = (\boldsymbol{r}_k^{\mathrm{H}} A \boldsymbol{r}_{k-1})^{\mathrm{H}} = \boldsymbol{r}_{k-1}^{\mathrm{H}} A^{\mathrm{H}} \boldsymbol{r}_k = \boldsymbol{r}_{k-1}^{\mathrm{H}} A \boldsymbol{r}_k$. 由此得 $\boldsymbol{r}_{k-1}^{\mathrm{H}} A \boldsymbol{r}_k = \boldsymbol{r}_k^{\mathrm{H}} \boldsymbol{r}_k \times (-\alpha_{k-1})^{-1}$. 回顾 $\beta_{k-1} = -\alpha_{k-1} t_{k-1,k}$, 得

$$\beta_{k-1} = -\alpha_{k-1} t_{k-1,k} = \frac{\boldsymbol{r}_k^{\mathrm{H}} \boldsymbol{r}_k}{\boldsymbol{r}_{k-1}^{\mathrm{H}} A \boldsymbol{r}_k} \frac{\boldsymbol{r}_{k-1}^{\mathrm{H}} A \boldsymbol{r}_k}{\boldsymbol{r}_{k-1}^{\mathrm{H}} \boldsymbol{r}_{k-1}} = \frac{\boldsymbol{r}_k^{\mathrm{H}} \boldsymbol{r}_k}{\boldsymbol{r}_{k-1}^{\mathrm{H}} \boldsymbol{r}_{k-1}}. \tag{3.14}$$

由 (3.7)–(3.10), (3.12) 和 (3.14) 得到 CG 方法的算法, 见 Algorithm 3.1. CG 方法的性质见 Proposition 3.1.

Algorithm 3.1 CG 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N$, $\beta_{-1} = 0$, $\boldsymbol{p}_{-1} = \boldsymbol{0}$, $\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Output: $\boldsymbol{x}_n$

1: **for** $n = 0, 1, 2, \ldots,$ 直至收敛 **do**
2: $\quad \boldsymbol{p}_n = \boldsymbol{r}_n + \beta_{n-1} \boldsymbol{p}_{n-1}$
3: $\quad \alpha_n = \dfrac{(\boldsymbol{r}_n, \boldsymbol{r}_n)}{(\boldsymbol{p}_n, A\boldsymbol{p}_n)}$
4: $\quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n$
5: $\quad \boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n A \boldsymbol{p}_n$
6: $\quad \beta_n = \dfrac{(\boldsymbol{r}_{n+1}, \boldsymbol{r}_{n+1})}{(\boldsymbol{r}_n, \boldsymbol{r}_n)}$
7: **end for**

Proposition 3.1: 设 $\boldsymbol{p}_n, \boldsymbol{r}_n$ 为 CG 方法中的向量, 则:

1. $(\boldsymbol{r}_i, \boldsymbol{r}_j) = 0 \quad \text{for } i \neq j,$
2. $(\boldsymbol{p}_i, A\boldsymbol{p}_j) = 0 \quad \text{for } i \neq j.$

Proof: 第一个性质由 (3.1) 来自 Lanczos 过程这一事实立即得到. 对第二个性质, 由 (3.11) 有

$$P_n^{\mathrm{H}} A P_n = U^{-\mathrm{H}} R_n^{\mathrm{H}} R_{n+1} L D^{-1}.$$

由于 A 为 Hermitian 矩阵, $P_n^{\mathrm{H}} A P_n$ 为 Hermitian 矩阵. 因此 $U^{-\mathrm{H}} R_n^{\mathrm{H}} R_{n+1} L D^{-1}$ 为 Hermitian 矩阵. 由于 $i \neq j$ 时 $R_n^{\mathrm{H}} R_{n+1}$ 的 (i,j) 元素为零, 矩阵 $U^{-\mathrm{H}} R_n^{\mathrm{H}} R_{n+1} L D^{-1}$ 是下三角矩阵. 这表明 $P_n^{\mathrm{H}} A P_n$ 既是 Hermitian 矩阵, 又是下三角矩阵. 因而 $P_n^{\mathrm{H}} A P_n$ 为对角矩阵, 这等价于第二个性质.

■

由 Proposition 3.1, 有 CG 方法的下列性质: **Corollary 3.2:** *设 $\boldsymbol{p}_n$ 和 $\boldsymbol{r}_n$ 为 CG 方法中的向量, 则:*

1. $\boldsymbol{r}_n \perp \mathcal{K}_n(A, \boldsymbol{r}_0)$,
2. $A\boldsymbol{p}_n \perp \mathcal{K}_n(A, \boldsymbol{r}_0)$.

Proof: 由 (3.8) 和 (3.9) 有 $\boldsymbol{r}_1 = \boldsymbol{r}_0 - \alpha_0 A \boldsymbol{r}_0$. 这意味着 $\mathrm{span}\{\boldsymbol{r}_0, \boldsymbol{r}_1\} = \mathcal{K}_2(A, \boldsymbol{r}_0)$. 类似地, $\mathrm{span}\{\boldsymbol{r}_0, \boldsymbol{r}_1, \ldots, \boldsymbol{r}_{n-1}\} = \mathcal{K}_n(A, \boldsymbol{r}_0)$. 由 Proposition 3.1 的第一个性质, 得 $\boldsymbol{r}_n \perp \boldsymbol{r}_0, \boldsymbol{r}_1, \ldots, \boldsymbol{r}_{n-1}$. 因此 $\boldsymbol{r}_n \perp \mathcal{K}_n(A, \boldsymbol{r}_0)$. 接着, 由 (3.8) 和 Proposition 3.1 的第一个性质, 得 $A\boldsymbol{p}_n = (\boldsymbol{r}_n - \boldsymbol{r}_{n+1})/\alpha_n \perp \boldsymbol{r}_0, \boldsymbol{r}_1, \ldots, \boldsymbol{r}_{n-1}$, 因而 $A\boldsymbol{p}_n \perp \mathcal{K}_n(A, \boldsymbol{r}_0)$.

■

由 Corollary 3.2 的证明容易看出

$$\boldsymbol{r}_n \in \mathcal{K}_{n+1}(A, \boldsymbol{r}_0), \quad \boldsymbol{p}_n \in \mathcal{K}_{n+1}(A, \boldsymbol{r}_0), \quad \boldsymbol{x}_n - \boldsymbol{x}_0 \in \mathcal{K}_n(A, \boldsymbol{r}_0). \tag{3.15}$$

若系数矩阵 A 为 Hermitian (或实对称) 正定矩阵, 则 CG 方法给出关于误差 A-范数的最优近似解, 如下所述.

Theorem 3.3:
设 $A\boldsymbol{x} = \boldsymbol{b}$ 为 Hermitian 正定线性方程组. 则 CG 方法产生使误差的 A-范数达到极小值的近似解:

$$\min_{\boldsymbol{x}_n \in \boldsymbol{x}_0 + \mathcal{K}_n(A, \boldsymbol{r}_0)} \|\boldsymbol{e}_n\|_A,$$

其中 $\|\boldsymbol{e}_n\|_A = (\boldsymbol{e}_n^{\mathrm{H}} A \boldsymbol{e}_n)^{1/2}$, $\boldsymbol{e}_n = \boldsymbol{x} - \boldsymbol{x}_n$ 为 CG 方法第 n 个近似解的误差.

Proof: 由于 A 为 Hermitian 正定矩阵, Cholesky 分解存在, 即 $A = LL^{\mathrm{H}}$. (实对称正定矩阵的 Cholesky 分解见 Section 1.4.1.) 回顾 CG 方法的近似解满足

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + \boldsymbol{z}_n, \quad \boldsymbol{z}_n \in \mathcal{K}_n(A, \boldsymbol{r}_0),$$

其中 $\boldsymbol{z}_n$ 由 $\boldsymbol{r}_n \perp \mathcal{K}_n(A, \boldsymbol{r}_0)$ 确定, 见 Corollary 3.2. 设 V_n 为列向量对应于 $\mathcal{K}_n(A, \boldsymbol{r}_0)$ 的标准正交基向量的矩阵. 则

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + V_n \boldsymbol{y}_n, \quad \boldsymbol{y}_n \in \mathbb{C}^n, \tag{3.16}$$

其中 $\boldsymbol{y}_n$ 由 $V_n^{\mathrm{H}} \boldsymbol{r}_n = \boldsymbol{0}$ 确定. 由 (3.16) 和 $\boldsymbol{e}_n = \boldsymbol{x} - \boldsymbol{x}_n$, 得

$$\boldsymbol{e}_n = \boldsymbol{e}_0 - V_n \boldsymbol{y}_n,$$

其中 $\boldsymbol{y}_n$ 由 $V_n^{\mathrm{H}} L L^{\mathrm{H}} \boldsymbol{e}_n = \boldsymbol{0}$ 确定, 因为 $V_n^{\mathrm{H}} \boldsymbol{r}_n = \boldsymbol{0}$, 且 $\boldsymbol{r}_n = \boldsymbol{b} - A\boldsymbol{x}_n = A\boldsymbol{x} - A\boldsymbol{x}_n = A\boldsymbol{e}_n = LL^{\mathrm{H}}\boldsymbol{e}_n$. 这里使用了 Cholesky 分解 $A = LL^{\mathrm{H}}$. 等价地, 有

$$L^{\mathrm{H}}\boldsymbol{e}_n = L^{\mathrm{H}}\boldsymbol{e}_0 - L^{\mathrm{H}} V_n \boldsymbol{y}_n,$$

其中 $\boldsymbol{y}_n$ 由 $(L^{\mathrm{H}} V_n)^{\mathrm{H}} L^{\mathrm{H}} \boldsymbol{e}_n = \boldsymbol{0}$ 确定. 这意味着 CG 方法寻找满足下列条件的近似解

$$\min_{\boldsymbol{y}_n \in \mathbb{C}^n} \|L^{\mathrm{H}}\boldsymbol{e}_n\|, \tag{3.17}$$

因为 $(L^{\mathrm{H}} V_n)^{\mathrm{H}} L^{\mathrm{H}} \boldsymbol{e}_n = \boldsymbol{0}$ 等价于正规方程 $(L^{\mathrm{H}} V_n)^{\mathrm{H}} (L^{\mathrm{H}} V_n)\boldsymbol{y}_n = (L^{\mathrm{H}} V_n)^{\mathrm{H}} L^{\mathrm{H}} \boldsymbol{e}_0$. 极小化问题 (3.17) 等价于

$$\min_{\boldsymbol{x}_n \in \boldsymbol{x}_0 + \mathcal{K}_n(A, \boldsymbol{r}_0)} \|L^{\mathrm{H}}\boldsymbol{e}_n\|.$$

由于 $\|L^{\mathrm{H}}\boldsymbol{e}_n\|^2 = (L^{\mathrm{H}}\boldsymbol{e}_n)^{\mathrm{H}}(L^{\mathrm{H}}\boldsymbol{e}_n) = \boldsymbol{e}_n^{\mathrm{H}} L L^{\mathrm{H}} \boldsymbol{e}_n = \boldsymbol{e}_n^{\mathrm{H}} A \boldsymbol{e}_n$, 最终有

$$\min_{\boldsymbol{x}_n \in \boldsymbol{x}_0 + \mathcal{K}_n(A, \boldsymbol{r}_0)} \|\boldsymbol{e}_n\|_A,$$

证明完毕.

■

Theorem 3.3 给出 CG 方法的一种最优性. 另一方面, 尚无法由此看出 CG 方法收敛得多快. 为稍后考察收敛速度, 先介绍 CG 方法的矩阵多项式表示. CG 方法的残差向量 $\boldsymbol{r}_n$ 和辅助向量 $\boldsymbol{p}_n$ 可用两个多项式 R_n 和 P_n 表示,

$$\boldsymbol{r}_n = R_n(A)\boldsymbol{r}_0, \quad \boldsymbol{p}_n = P_n(A)\boldsymbol{r}_0, \tag{3.18}$$

其中 $R_n(\lambda)$ 表示 Lanczos 多项式, 它满足下列三项递推关系:

$$R_0(\lambda) = 1, \tag{3.19}$$

$$R_1(\lambda) = (1 - \alpha_0\lambda)R_0(\lambda), \tag{3.20}$$

$$\begin{aligned} R_n(\lambda) = &\left(1 + \frac{\beta_{n-2}}{\alpha_{n-2}}\alpha_{n-1} - \alpha_{n-1}\lambda\right) R_{n-1}(\lambda) \\ &- \frac{\beta_{n-2}}{\alpha_{n-2}}\alpha_{n-1} R_{n-2}(\lambda), \quad n = 2, 3, \ldots, \end{aligned} \tag{3.21}$$

其中 $R_n(0) = 1$. R_n 和 P_n 满足下列两项递推关系:

$$R_0(\lambda) = 1, \quad P_0(\lambda) = 1, \tag{3.22}$$

$$R_n(\lambda) = R_{n-1}(\lambda) - \alpha_{n-1}\lambda P_{n-1}(\lambda), \tag{3.23}$$

$$P_n(\lambda) = R_n(\lambda) + \beta_{n-1} P_{n-1}(\lambda) \quad n = 1, 2, \ldots. \tag{3.24}$$

CG 方法的收敛速度见 Theorem 3.4.

Theorem 3.4:

CG 方法误差的 A-范数满足

$$\|\boldsymbol{e}_n\|_A \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^n \|\boldsymbol{e}_0\|_A, \tag{3.25}$$

其中 κ 为 A 的条件数.

Proof: 由于 CG 方法的残差向量可表示为 $\boldsymbol{r}_n = R(A)\boldsymbol{r}_0$, 且 $\boldsymbol{r}_n = A\boldsymbol{e}_n$, 得到 $\boldsymbol{e}_n = R_n(A)\boldsymbol{e}_0$. 由 (3.25) 有

$$\|\boldsymbol{e}_n\|_A = \|R_n(A)\boldsymbol{e}_0\|_A = \min_{p_n\in\mathcal{P}_n} \|p_n(A)\boldsymbol{e}_0\|_A,$$

其中 $\mathcal{P}_n$ 为次数为 n 且满足 $\mathcal{P}_0 = 1$ 的多项式的集合. 利用 Hermitian 正定矩阵 A 的对角化, 有 $A = PDP^{\mathrm{H}}$, 其中 P 为酉矩阵, D 为对角元素是 A 的特征值的对角矩阵. 令 $A^{1/2} = PD^{1/2}P^{\mathrm{H}}$, 其中 $D^{1/2}$ 的对角元素为 A 的特征值的平方根. 注意, A 的所有特征值均为正, 且 $A^{1/2}$ 为 Hermitian 矩阵, 即 $(A^{1/2})^{\mathrm{H}} = A^{1/2}$. 可见 $A^{1/2}A^{1/2} = PD^{1/2}P^{\mathrm{H}}PD^{1/2}P^{\mathrm{H}} = PD^{1/2}D^{1/2}P^{\mathrm{H}} = PDP^{\mathrm{H}} = A$. 于是有

$$\begin{aligned}
\|\boldsymbol{e}_n\|_A &= \min_{p_n\in\mathcal{P}_n} \|p_n(A)\boldsymbol{e}_0\|_A, \\
&= \min_{p_n\in\mathcal{P}_n} \sqrt{(p_n(A)\boldsymbol{e}_0)^{\mathrm{H}}A(p_n(A)\boldsymbol{e}_0)} \\
&= \min_{p_n\in\mathcal{P}_n} \sqrt{(p_n(A)\boldsymbol{e}_0)^{\mathrm{H}}(A^{1/2})^{\mathrm{H}}A^{1/2}(p_n(A)\boldsymbol{e}_0)} \\
&= \min_{p_n\in\mathcal{P}_n} \sqrt{(A^{1/2}p_n(A)\boldsymbol{e}_0)^{\mathrm{H}}(A^{1/2}p_n(A)\boldsymbol{e}_0)} \\
&= \min_{p_n\in\mathcal{P}_n} \|A^{1/2}p_n(A)\boldsymbol{e}_0\|.
\end{aligned}$$

利用 $A = PDP^{\mathrm{H}}$ 和 $A^{1/2} = PD^{1/2}P^{\mathrm{H}}$, 得

$$\begin{aligned}
\|\boldsymbol{e}_n\|_A &= \min_{p_n\in\mathcal{P}_n} \|A^{1/2}p_n(A)\boldsymbol{e}_0\| = \min_{p_n\in\mathcal{P}_n} \|PD^{1/2}P^{\mathrm{H}}p_n(PDP^{\mathrm{H}})\boldsymbol{e}_0\| \\
&= \min_{p_n\in\mathcal{P}_n} \|PD^{1/2}P^{\mathrm{H}}Pp_n(D)P^{\mathrm{H}}\boldsymbol{e}_0\| = \min_{p_n\in\mathcal{P}_n} \|PD^{1/2}p_n(D)P^{\mathrm{H}}\boldsymbol{e}_0\| \\
&= \min_{p_n\in\mathcal{P}_n} \|Pp_n(D)D^{1/2}P^{\mathrm{H}}\boldsymbol{e}_0\| = \min_{p_n\in\mathcal{P}_n} \|Pp_n(D)P^{\mathrm{H}}PD^{1/2}P^{\mathrm{H}}\boldsymbol{e}_0\| \\
&= \min_{p_n\in\mathcal{P}_n} \|Pp_n(D)P^{\mathrm{H}}A^{1/2}\boldsymbol{e}_0\| \\
&\leq \min_{p_n\in\mathcal{P}_n} \|Pp_n(D)P^{\mathrm{H}}\|\|A^{1/2}\boldsymbol{e}_0\| \\
&= \min_{p_n\in\mathcal{P}_n} \|p_n(D)\|\|\boldsymbol{e}_0\|_A \\
&= \min_{p_n\in\mathcal{P}_n} \max_{i=1,2,\ldots,N} |p_n(\lambda_i)| \times \|\boldsymbol{e}_0\|_A.
\end{aligned}$$

因此有

$$\frac{\|\boldsymbol{e}_n\|_A}{\|\boldsymbol{e}_0\|_A} \leq \min_{p_n\in\mathcal{P}_n} \max_{i=1,2,\ldots,N} |p_n(\lambda_i)|.$$

设 a 和 b 分别为 A 的最小和最大特征值. 则所有特征值 λ_i 都属于闭区间 $[a,b]$. 因此有下列不等式:

$$\frac{\|\boldsymbol{e}_n\|_A}{\|\boldsymbol{e}_0\|_A} \leq \min_{p_n\in\mathcal{P}_n} \max_{i=1,2,\ldots,N} |p_n(\lambda_i)| \leq \min_{p_n\in\mathcal{P}_n} \max_{\lambda\in[a,b]} |p_n(\lambda)|.$$

极小极大问题的解可用 Chebyshev 多项式改写如下:

$$\min_{p_n\in\mathcal{P}_n}\max_{\lambda\in[a,b]}|p_n(\lambda)|=\max_{\lambda\in[a,b]}\frac{|T_n(\frac{2\lambda-b-a}{b-a})|}{T_n(\frac{-b-a}{b-a})},$$

其中

$$T_n(x)=\frac{1}{2}\left[\left(x+\sqrt{x^2-1}\right)^n+\left(x-\sqrt{x^2-1}\right)^n\right]\quad\text{for }|x|\geq 1. \tag{3.26}$$

由 Chebyshev 多项式的性质, $|T_n(\dfrac{2\lambda-b-a}{b-a})|\leq 1$, 从而有

$$\min_{p_n\in\mathcal{P}_n}\max_{\lambda\in[a,b]}|p_n(\lambda)|\leq\frac{1}{|T_n(\frac{-b-a}{b-a})|}=\frac{1}{|T_n(\frac{-\kappa-1}{\kappa-1})|},$$

其中 $\kappa=b/a$ 是 A 的条件数. 利用 (3.26), 得

$$\begin{aligned}\min_{p_n\in\mathcal{P}_n}\max_{\lambda\in[a,b]}|p_n(\lambda)|&\leq\frac{1}{|T_n(\frac{-\kappa-1}{\kappa-1})|}=\frac{2}{\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^n+\left(\frac{\sqrt{\kappa}+1}{\sqrt{\kappa}-1}\right)^n}\leq\frac{2}{\left(\frac{\sqrt{\kappa}+1}{\sqrt{\kappa}-1}\right)^n}\\&=2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^n,\end{aligned}$$

证明完毕.

■

Theorem 3.4 表明, CG 方法的收敛速度依赖于系数矩阵 A 的条件数, 即条件数越小, CG 方法收敛越快. 因此很自然地考虑下列等价线性方程组:

$$A\boldsymbol{x}=\boldsymbol{b}\Leftrightarrow\widetilde{A}\widetilde{\boldsymbol{x}}=\widetilde{\boldsymbol{b}},$$

使得 $\widetilde{A}$ 的条件数远小于 A 的条件数, 然后不对 $A\boldsymbol{x}=\boldsymbol{b}$, 而对变换后的线性方程组 $\widetilde{A}\widetilde{\boldsymbol{x}}=\widetilde{\boldsymbol{b}}$ 应用 CG 方法. 所得算法称为预条件 (PCG) 方法. 构造 $\widetilde{A}$ 的技术称为预条件技术, 将在 Section 3.5 中讨论. 现在介绍预条件 CG 方法的算法. 令 $A\approx K=LL^{\mathrm{H}}$. 则预期 $\widetilde{A}=L^{-1}AL^{-\mathrm{H}}$ 接近条件数为一的单位矩阵. 现在考虑对下列变换后的线性方程组应用 CG 方法

$$\widetilde{A}\widetilde{\boldsymbol{x}}=\widetilde{\boldsymbol{b}},$$

其中 $\widetilde{A}=L^{-1}AL^{-\mathrm{H}}$, $\widetilde{\boldsymbol{x}}=L^{\mathrm{H}}\boldsymbol{x}$, $\widetilde{\boldsymbol{b}}=L^{-1}\boldsymbol{b}$. 则预期用于 $\widetilde{A}\widetilde{\boldsymbol{x}}=\widetilde{\boldsymbol{b}}$ 的 CG 方法, 比用于 $A\boldsymbol{x}=\boldsymbol{b}$ 的 CG 方法收敛快得多. 由 Algorithm 3.1, 用于 $\widetilde{A}\widetilde{\boldsymbol{x}}=\widetilde{\boldsymbol{b}}$ 的 CG 方法的迭代式为

$$\begin{aligned}\widetilde{\boldsymbol{p}}_n&=\widetilde{\boldsymbol{r}}_n+\beta_{n-1}\widetilde{\boldsymbol{p}}_{n-1},\\\alpha_n&=\frac{(\widetilde{\boldsymbol{r}}_n,\widetilde{\boldsymbol{r}}_n)}{(\widetilde{\boldsymbol{p}}_n,\widetilde{A}\widetilde{\boldsymbol{p}}_n)},\\\widetilde{\boldsymbol{x}}_{n+1}&=\widetilde{\boldsymbol{x}}_n+\alpha_n\widetilde{\boldsymbol{p}}_n,\\\widetilde{\boldsymbol{r}}_{n+1}&=\widetilde{\boldsymbol{r}}_n-\alpha_n\widetilde{A}\widetilde{\boldsymbol{p}}_n,\\\beta_n&=\frac{(\widetilde{\boldsymbol{r}}_{n+1},\widetilde{\boldsymbol{r}}_{n+1})}{(\widetilde{\boldsymbol{r}}_n,\widetilde{\boldsymbol{r}}_n)}.\end{aligned}$$

作下列改写:

$$\widetilde{\boldsymbol{x}}\Rightarrow L^{\mathrm{H}}\boldsymbol{x},\quad\widetilde{\boldsymbol{p}}\Rightarrow L^{\mathrm{H}}\boldsymbol{p},\quad\widetilde{\boldsymbol{r}}\Rightarrow L^{-1}\boldsymbol{r},$$

得到

$$\begin{aligned}
\boldsymbol{p}_n &= L^{-\mathrm{H}}L^{-1}\boldsymbol{r}_n + \beta_{n-1}L^{-\mathrm{H}}L^{\mathrm{H}}\boldsymbol{p}_{n-1} = L^{-\mathrm{H}}L^{-1}\boldsymbol{r}_n + \beta_{n-1}\boldsymbol{p}_{n-1},\\
\alpha_n &= \frac{(L^{-1}\boldsymbol{r}_n, L^{-1}\boldsymbol{r}_n)}{(L^{\mathrm{H}}\boldsymbol{p}_n, L^{-1}AL^{-\mathrm{H}}L^{\mathrm{H}}\boldsymbol{p}_n)} = \frac{(L^{-\mathrm{H}}L^{-1}\boldsymbol{r}_n, \boldsymbol{r}_n)}{(\boldsymbol{p}_n, A\boldsymbol{p}_n)},\\
\boldsymbol{x}_{n+1} &= L^{-\mathrm{H}}L^{\mathrm{H}}\boldsymbol{x}_n + \alpha_n L^{-\mathrm{H}}L^{\mathrm{H}}\boldsymbol{p}_n = \boldsymbol{x}_n + \alpha_n\boldsymbol{p}_n,\\
\boldsymbol{r}_{n+1} &= LL^{-1}\boldsymbol{r}_n - \alpha_n LL^{-1}AL^{-\mathrm{H}}L^{\mathrm{H}}\boldsymbol{p}_n = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n,\\
\beta_n &= \frac{(L^{-1}\boldsymbol{r}_{n+1}, L^{-1}\boldsymbol{r}_{n+1})}{(L^{-\mathrm{H}}L^{-1}\boldsymbol{r}_n, \boldsymbol{r}_n)} = \frac{(L^{-\mathrm{H}}L^{-1}\boldsymbol{r}_{n+1}, \boldsymbol{r}_{n+1})}{(L^{-\mathrm{H}}L^{-1}\boldsymbol{r}_n, \boldsymbol{r}_n)}.
\end{aligned}$$

由于 $K^{-1} = L^{-\mathrm{H}}L^{-1}$, 得到 Algorithm 3.2 所述的 PCG 方法.

Algorithm 3.2 预条件 CG 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N$, $\beta_{-1} = 0$, $\boldsymbol{p}_{-1} = \boldsymbol{0}$, $\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Output: $\boldsymbol{x}_n$

1: **for** $n = 0, 1, 2, \ldots,$ 直至收敛 **do**
2: $\quad \boldsymbol{p}_n = K^{-1}\boldsymbol{r}_n + \beta_{n-1}\boldsymbol{p}_{n-1}$
3: $\quad \alpha_n = \dfrac{(K^{-1}\boldsymbol{r}_n, \boldsymbol{r}_n)}{(\boldsymbol{p}_n, A\boldsymbol{p}_n)}$
4: $\quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n\boldsymbol{p}_n$
5: $\quad \boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n$
6: $\quad \beta_n = \dfrac{(K^{-1}\boldsymbol{r}_{n+1}, \boldsymbol{r}_{n+1})}{(K^{-1}\boldsymbol{r}_n, \boldsymbol{r}_n)}$
7: **end for**

下面列出关于 CG 方法的几点说明.

1. 由 Theorem 3.3, 在精确算术下, CG 方法在 N 次迭代内找到精确解.

2. 即使系数矩阵不正定, 仍可使用 CG 方法. 此时, Theorem 3.3 中的最优性不再成立, CG 方法可能发生中断, 即除以零, 因为对任意 $\boldsymbol{p}_n \neq \boldsymbol{0}$ 都有 $(\boldsymbol{p}_n, A\boldsymbol{p}_n) > 0$ 这一性质不再成立, 见 Algorithm 3.1 的第 3 行. 实际计算中不会发生中断 (除以零), 但可能发生近中断, 即 $(\boldsymbol{p}_n, A\boldsymbol{p}_n) \approx 0$, 导致数值不稳定.

3. 在精确算术下, 若不发生中断, 用于 Hermitian 不定矩阵的 CG 方法也在 N 次迭代内找到精确解.

接下来两小节介绍 Hermitian 线性方程组的 CR 方法和 MINRES 方法.

3.1.2 共轭残差 (CR) 方法

共轭残差 (CR) 方法 [177] 由 CG 方法的提出者之一 Stiefel 于 1955 年提出. CR 方法的特点是关于残差范数具有最优性, 稍后将予以说明. 可以像推导 CG 方法一样推导 CR 方法, 但这里利用 CG 方法的算法给出 CR 方法的简洁推导. 这一推导的缺点是, 假设 A 为 Hermitian 正定矩阵, 而 CR 方法可用于 Hermitian 不定线性方程组. 另一方面, 我们将看到这一推导比 CG 方法的推导简单得多. 与 CG 方法推导相似的 CR 方法推导见 Section 3.2.2: 将复对称线性方程组的 COCR 方法的推导限制到实对称情形即可得到. 现在从 CG 方法推导 CR 方

法. 若 A 为 Hermitian 正定矩阵, 则 A 存在平方根: A 的 Cholesky 分解 (见 Section 1.4.1) 为 LL^{H}, 利用奇异值分解又可将 L 分解为 $U\Sigma V^{\mathrm{H}}$.[1] 于是有

$$A = LL^{\mathrm{H}} = (U\Sigma V^{\mathrm{H}})(V\Sigma U^{\mathrm{H}}) = U\Sigma^2 U^{\mathrm{H}} = (U\Sigma U^{\mathrm{H}})(U\Sigma U^{\mathrm{H}}) = A^{\frac{1}{2}}A^{\frac{1}{2}}.$$

因此 A 的平方根为 $A^{\frac{1}{2}} = U\Sigma U^{\mathrm{H}}$. 这里说明, 若 A 为 Hermitian 正定矩阵, 则对下列线性方程组应用 CG 方法即可得到 CR 方法的算法:

$$A\widetilde{\boldsymbol{x}} = A^{\frac{1}{2}}\boldsymbol{b}, \quad \widetilde{\boldsymbol{x}} = A^{\frac{1}{2}}\boldsymbol{x}. \tag{3.27}$$

由 CG 方法的算法可得

$$\begin{aligned}
\widetilde{\boldsymbol{p}}_n &= \widetilde{\boldsymbol{r}}_n + \beta_{n-1}\widetilde{\boldsymbol{p}}_{n-1},\\
\alpha_n &= \frac{(\widetilde{\boldsymbol{r}}_n, \widetilde{\boldsymbol{r}}_n)}{(\widetilde{\boldsymbol{p}}_n, A\widetilde{\boldsymbol{p}}_n)},\\
\widetilde{\boldsymbol{x}}_{n+1} &= \widetilde{\boldsymbol{x}}_n + \alpha_n\widetilde{\boldsymbol{p}}_n,\\
\widetilde{\boldsymbol{r}}_{n+1} &= \widetilde{\boldsymbol{r}}_n - \alpha_n A\widetilde{\boldsymbol{p}}_n,\\
\beta_n &= \frac{(\widetilde{\boldsymbol{r}}_{n+1}, \widetilde{\boldsymbol{r}}_{n+1})}{(\widetilde{\boldsymbol{r}}_n, \widetilde{\boldsymbol{r}}_n)}.
\end{aligned}$$

原方程组 $A\boldsymbol{x} = \boldsymbol{b}$ 的残差向量为 $\boldsymbol{r}_n = \boldsymbol{b} - A\boldsymbol{x}_n$, 由 (3.27) 有 $\widetilde{\boldsymbol{r}}_n = A^{\frac{1}{2}}\boldsymbol{b} - A\widetilde{\boldsymbol{x}}_n = A^{\frac{1}{2}}(\boldsymbol{b} - A\boldsymbol{x}_n) = A^{\frac{1}{2}}\boldsymbol{r}_n$. 将 $\widetilde{\boldsymbol{r}}_n = A^{\frac{1}{2}}\boldsymbol{r}_n$, $\widetilde{\boldsymbol{p}}_n = A^{\frac{1}{2}}\boldsymbol{p}_n$, $\widetilde{\boldsymbol{x}}_n = A^{\frac{1}{2}}\boldsymbol{x}_n$ 代入上述递推式, 得到 CR 方法的算法 (Algorithm 3.3).

Algorithm 3.3 CR 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N$, $\beta_{-1} = 0$, $\boldsymbol{p}_{-1} = \boldsymbol{0}$, $\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Output: $\boldsymbol{x}_n$

1: **for** $n = 0, 1, 2, \ldots,$ 直至收敛 **do**
2: $\quad \boldsymbol{p}_n = \boldsymbol{r}_n + \beta_{n-1}\boldsymbol{p}_{n-1}$
3: $\quad A\boldsymbol{p}_n = A\boldsymbol{r}_n + \beta_{n-1}A\boldsymbol{p}_{n-1}$
4: $\quad \alpha_n = \dfrac{(\boldsymbol{r}_n, A\boldsymbol{r}_n)}{(A\boldsymbol{p}_n, A\boldsymbol{p}_n)}$
5: $\quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n\boldsymbol{p}_n$
6: $\quad \boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n$
7: $\quad \beta_n = \dfrac{(\boldsymbol{r}_{n+1}, A\boldsymbol{r}_{n+1})}{(\boldsymbol{r}_n, A\boldsymbol{r}_n)}$
8: **end for**

由 Algorithm 3.3 的第 7 行, 若 Hermitian 矩阵 A 不定, CR 方法可能发生中断, 即 $(\boldsymbol{r}_n, A\boldsymbol{r}_n) = 0$. 利用归纳法可证明, CR 方法生成的迭代向量 $\boldsymbol{r}_i, \boldsymbol{p}_i$ 满足

$$(\boldsymbol{r}_i, A\boldsymbol{r}_j) = 0 \quad \text{for } i \neq j, \tag{3.28}$$

$$(A\boldsymbol{p}_i, A\boldsymbol{p}_j) = 0 \quad \text{for } i \neq j. \tag{3.29}$$

将两个性质 ((3.28) 和 (3.29)) 与 CG 方法的性质比较, CR 方法寻找满足下列条件的近似解

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + \boldsymbol{z}_n, \quad \boldsymbol{z}_n \in \mathcal{K}_n(A, \boldsymbol{r}_0),$$

[1] U 和 V 为酉矩阵, Σ 为对角元素非负的对角矩阵.

其中 $\boldsymbol{z}_n$ 由 $\boldsymbol{r}_n \perp A\mathcal{K}_n(A,\boldsymbol{r}_0)(:=\mathcal{K}_n(A,A\boldsymbol{r}_0))$ 确定, 或等价地,

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + V_n\boldsymbol{y}_n, \quad \boldsymbol{y}_n \in \mathbb{C}^n,$$

其中 $\boldsymbol{y}_n$ 由 $(AV_n)^{\mathrm{H}}\boldsymbol{r}_n = \boldsymbol{0}$ 确定. 沿用 Theorem 3.3 的证明, 得

$$\min_{\boldsymbol{x}_n \in \boldsymbol{x}_0+\mathcal{K}_n(A,\boldsymbol{r}_0)} \|\boldsymbol{e}_n\|_{A^2}.$$

由于 $\|\boldsymbol{e}_n\|_{A^2}^2 = \boldsymbol{e}_n^{\mathrm{H}}A^2\boldsymbol{e}_n = (A\boldsymbol{e}_n)^{\mathrm{H}}A\boldsymbol{e}_n = \boldsymbol{r}_n^{\mathrm{H}}\boldsymbol{r}_n = \|\boldsymbol{r}_n\|^2$, CR 方法寻找满足下列条件的近似解

$$\min_{\boldsymbol{x}_n \in \boldsymbol{x}_0+\mathcal{K}_n(A,\boldsymbol{r}_0)} \|\boldsymbol{r}_n\|.$$

因此, CR 方法产生关于残差范数的最优近似解. 与预条件 CG 方法的推导类似, 对变换后的线性方程组 $\widetilde{A}\widetilde{\boldsymbol{x}} = \widetilde{\boldsymbol{b}}$ 应用 CR 方法, 可推导出预条件 CR 方法. 所得算法见 Algorithm 3.4.

Algorithm 3.4 预条件 CR 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \beta_{-1} = 0,\ \boldsymbol{p}_{-1} = \boldsymbol{0},\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Output: $\boldsymbol{x}_n$

1: **for** $n = 0, 1, 2, \ldots,$ 直至收敛 **do**

2: $\quad \boldsymbol{p}_n = K^{-1}\boldsymbol{r}_n + \beta_{n-1}\boldsymbol{p}_{n-1}$

3: $\quad (A\boldsymbol{p}_n = AK^{-1}\boldsymbol{r}_n + \beta_{n-1}A\boldsymbol{p}_{n-1})$

4: $\quad \alpha_n = \dfrac{(K^{-1}\boldsymbol{r}_n, AK^{-1}\boldsymbol{r}_n)}{(A\boldsymbol{p}_n, K^{-1}A\boldsymbol{p}_n)}$

5: $\quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n\boldsymbol{p}_n$

6: $\quad \boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n$

7: $\quad (K^{-1}\boldsymbol{r}_{n+1} = K^{-1}\boldsymbol{r}_n - \alpha_n K^{-1}A\boldsymbol{p}_n)$

8: $\quad \beta_n = \dfrac{(K^{-1}\boldsymbol{r}_{n+1}, AK^{-1}\boldsymbol{r}_{n+1})}{(K^{-1}\boldsymbol{r}_n, AK^{-1}\boldsymbol{r}_n)}$

9: **end for**

注意, Algorithm 3.4 中加入第 3 和第 7 行, 是为了减少形如 $A\boldsymbol{v}$ 的矩阵向量乘法及形如 $K\boldsymbol{v} = \boldsymbol{w}$ 的线性方程组求解次数. 实际上, 每次迭代似乎都需要计算

$$A\boldsymbol{p}_n, \quad K^{-1}\boldsymbol{r}_{n+1}, \quad A(K^{-1}\boldsymbol{r}_n), \quad K^{-1}(A\boldsymbol{p}_n).$$

但由第 3 和第 7 行, 除第一次迭代需要计算 $K^{-1}\boldsymbol{r}_0$ 外, 不必计算 $A\boldsymbol{p}_n$ 和 $K^{-1}\boldsymbol{r}_{n+1}$. 因此, 每次迭代实质上只需计算下列形式:

$$A\boldsymbol{v}, \quad K^{-1}\boldsymbol{w}$$

这里 $\boldsymbol{v} := K^{-1}\boldsymbol{r}_{n+1}$ 由第 7 行得到, $\boldsymbol{w} := A\boldsymbol{p}_n$ 由第 3 行得到.

3.1.3 最小残差 (MINRES) 方法

最小残差 (MINRES) 方法 [143] 由 Paige 和 Saunders 于 1975 年提出. MINRES 方法生成在仿射空间 $\boldsymbol{x}_0+\mathcal{K}_n(A,\boldsymbol{r}_0)$ 上使 $\|\boldsymbol{b}-A\boldsymbol{x}_n\|$ 达到极小值的 $\boldsymbol{x}_n$. 因此在精确算术下, MINRES 方法与 CR 方法产生相同的近似解. 这一极小化可利用 Lanczos 过程实现. 这里给出 MINRES 方法的推导过程. 设 V_n 为 $\mathcal{K}_n(A,\boldsymbol{r}_0)$ 的标准正交基. MINRES 方法在仿射空间 $\boldsymbol{x}_0+\mathcal{K}_n(A,\boldsymbol{r}_0)$ 上寻找 $\boldsymbol{x}_n$, 即

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + V_n\boldsymbol{y}_n, \quad \boldsymbol{y}_n \in \mathbb{C}^n. \tag{3.30}$$

相应的残差向量为

$$\boldsymbol{r}_n = \boldsymbol{r}_0 - AV_n\boldsymbol{y}_n.$$

由 (1.42) 中 Lanczos 过程的矩阵形式, 得

$$\boldsymbol{r}_n = \boldsymbol{r}_0 - V_{n+1}T_{n+1,n}\boldsymbol{y}_n = V_{n+1}(\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}_n),$$

其中 $\beta := \|\boldsymbol{r}_0\|$. 残差向量的 2-范数写成

$$\begin{aligned}
\|\boldsymbol{r}_n\| &= \|V_{n+1}(\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}_n)\| \\
&= \sqrt{[V_{n+1}(\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}_n)]^{\mathrm{H}}V_{n+1}(\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}_n)} \\
&= \sqrt{(\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}_n)^{\mathrm{H}}V_{n+1}^{\mathrm{H}}V_{n+1}(\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}_n)} \\
&= \sqrt{(\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}_n)^{\mathrm{H}}(\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}_n)} \\
&= \|\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}_n\|.
\end{aligned}$$

MINRES 方法寻找使残差向量的 2-范数达到极小值的近似解, 即选择 $\boldsymbol{y}_n$ 使得

$$\boldsymbol{y}_n := \arg\min_{\boldsymbol{y}\in\mathbb{C}^n} \|\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}\|. \tag{3.31}$$

利用作为酉矩阵的一种 Givens 旋转求解 (3.31) 中的最小二乘问题, 就得到 MINRES 方法. 下面说明 Givens 旋转如何用于求解 (3.31). 为简单起见, 考虑 $n = 4$ 的情形. $\beta\boldsymbol{e}_1 - T_{5,4}\boldsymbol{y}$ 写成

$$\beta\boldsymbol{e}_1 - T_{5,4}\boldsymbol{y} = \begin{bmatrix}\beta\\0\\0\\0\\0\end{bmatrix} - \begin{bmatrix}t_{11} & t_{12} & & \\ t_{21} & t_{22} & t_{23} & \\ & t_{32} & t_{33} & t_{34} \\ & & t_{43} & t_{44} \\ & & & t_{54}\end{bmatrix}\begin{bmatrix}y_1\\y_2\\y_3\\y_4\end{bmatrix},$$

其中 $t_{ij} = t_{ji}$, $1 \le i, j \le 4$. 设 G_1 为 Givens 旋转矩阵, 即

$$G_1 = \begin{bmatrix} c_1 & s_1 & & & \\ -\bar{s}_1 & c_1 & & & \\ & & 1 & & \\ & & & 1 & \\ & & & & 1\end{bmatrix} \quad \text{with } c_1 = \frac{|t_{11}|}{\sqrt{|t_{11}|^2 + |t_{21}|^2}}, \quad \bar{s}_1 = \frac{t_{21}}{t_{11}}c_1,$$

其中 s_1 取为 $\bar{s}_1$ 的共轭. 有一种例外: 若 $t_{11} = 0$, 则令 $s_1 = \bar{s}_1 = 1$. 于是 G_1 为酉矩阵, 因而 $\|\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}\| = \|G_1(\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y})\|$. 由此得到

$$G_1(\beta\boldsymbol{e}_1 - T_{5,4}\boldsymbol{y}) = \begin{bmatrix}g_1^{(1)}\\g_2^{(1)}\\0\\0\\0\end{bmatrix} - \begin{bmatrix}t_{11}^{(1)} & t_{12}^{(1)} & t_{13}^{(1)} & \\ & t_{22}^{(1)} & t_{23}^{(1)} & \\ & t_{32} & t_{33} & t_{34} \\ & & t_{43} & t_{44} \\ & & & t_{54}\end{bmatrix}\begin{bmatrix}y_1\\y_2\\y_3\\y_4\end{bmatrix},$$

其中 $g_2^{(1)} = -\bar{s}_1\beta$. 接着定义酉矩阵 G_2 为

$$G_2 = \begin{bmatrix} 1 & & & & \\ & c_2 & s_2 & & \\ & -\bar{s}_2 & c_2 & & \\ & & & 1 & \\ & & & & 1 \end{bmatrix} \quad \text{with } c_2 = \frac{|t_{22}^{(1)}|}{\sqrt{|t_{22}^{(1)}|^2 + |t_{32}|^2}}, \quad \bar{s}_2 = \frac{t_{32}}{t_{22}^{(1)}} c_2.$$

有一种例外: 若 $t_{22}^{(1)} = 0$, 则令 $s_2 = \bar{s}_2 = 1$. 于是 G_2 和 G_2G_1 为酉矩阵, 因而 $\|\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}\| = \|G_2G_1(\beta\boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y})\|$. 由此得到

$$G_2G_1(\beta\boldsymbol{e}_1 - T_{5,4}\boldsymbol{y}) = \begin{bmatrix} g_1^{(1)} \\ g_2^{(2)} \\ g_3^{(2)} \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} t_{11}^{(1)} & t_{12}^{(1)} & t_{13}^{(1)} & \\ & t_{22}^{(2)} & t_{23}^{(2)} & t_{24}^{(2)} \\ & & t_{33}^{(2)} & t_{34}^{(2)} \\ & & t_{43} & t_{44} \\ & & & t_{54} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix},$$

其中 $g_3^{(2)} = (-1)^2\bar{s}_2\bar{s}_1\beta$. 类似地, 利用 G_3 和 G_4, 最终得到

$$\begin{aligned} Q_4(\beta\boldsymbol{e}_1 - T_{5,4}\boldsymbol{y}) &= \begin{bmatrix} g_1^{(1)} \\ g_2^{(2)} \\ g_3^{(3)} \\ g_4^{(4)} \\ g_5^{(5)} \end{bmatrix} - \begin{bmatrix} t_{11}^{(1)} & t_{12}^{(1)} & t_{13}^{(1)} & \\ & t_{22}^{(2)} & t_{23}^{(2)} & t_{24}^{(2)} \\ & & t_{33}^{(3)} & t_{34}^{(3)} \\ & & & t_{44}^{(4)} \\ & & & \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} \\ &= \begin{bmatrix} \boldsymbol{g}_4 \\ g_5^{(5)} \end{bmatrix} - \begin{bmatrix} R_4 \\ \boldsymbol{0}^\top \end{bmatrix} \boldsymbol{y}_4, \end{aligned}$$

其中 $Q_4 = G_4G_3G_2G_1$, $g_5^{(5)} = (-1)^4\bar{s}_4\bar{s}_3\bar{s}_2\bar{s}_1\beta$. 因此, 通过回代求解下列方程, 得到 $\boldsymbol{y}_4$:

$$R_4\boldsymbol{y}_4 = \boldsymbol{g}_4 \tag{3.32}$$

有趣的是, 不必求解方程 $R_4\boldsymbol{y}_4 = \boldsymbol{g}_4$, 即可得到 $\min\limits_{\boldsymbol{y}\in\mathbb{C}^4} \|\beta\boldsymbol{e}_1 - T_{5,4}\boldsymbol{y}\|$ 的值, 因为该值满足

$$\begin{aligned} \|\boldsymbol{r}_4\| &= \min_{\boldsymbol{y}\in\mathbb{C}^4} \|\beta\boldsymbol{e}_1 - T_{5,4}\boldsymbol{y}\| = \min_{\boldsymbol{y}\in\mathbb{C}^4} \|Q_4(\beta\boldsymbol{e}_1 - T_{5,4}\boldsymbol{y})\| \\ &= \left\| \begin{bmatrix} \boldsymbol{g}_4 \\ g_5^{(5)} \end{bmatrix} - \begin{bmatrix} R_4 \\ \boldsymbol{0}^\top \end{bmatrix} \boldsymbol{y}_4 \right\| = \left\| \begin{bmatrix} \boldsymbol{0} \\ g_5^{(5)} \end{bmatrix} \right\| \\ &= |g_5^{(5)}|. \end{aligned}$$

因此, $|g_5^{(5)}|$ 对应于 $\boldsymbol{r}_4$ 的残差范数, 可用作停止准则. 为得到近似解, 由 (3.30) 和 (3.32) 有

$$\boldsymbol{x}_4 = \boldsymbol{x}_0 + V_4R_4^{-1}\boldsymbol{g}_4. \tag{3.33}$$

这里引入矩阵 $P_4 := V_4R_4^{-1}$, 其列为 $[\boldsymbol{p}_1, \boldsymbol{p}_2, \boldsymbol{p}_3, \boldsymbol{p}_4]$. 于是由 $[\boldsymbol{p}_1, \boldsymbol{p}_2, \boldsymbol{p}_3, \boldsymbol{p}_4]R_4 = [\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3, \boldsymbol{v}_4]$, 有下列递推关系:

$$\boldsymbol{p}_1 = \boldsymbol{v}_1/t_{11}^{(1)}, \tag{3.34}$$

$$\boldsymbol{p}_2 = (\boldsymbol{v}_2 - t_{12}^{(1)}\boldsymbol{p}_1)/t_{22}^{(2)}, \tag{3.35}$$

$$\boldsymbol{p}_3 = (\boldsymbol{v}_3 - t_{13}^{(1)}\boldsymbol{p}_1 - t_{23}^{(2)}\boldsymbol{p}_2)/t_{33}^{(3)}, \tag{3.36}$$

$$\boldsymbol{p}_4 = (\boldsymbol{v}_4 - t_{24}^{(2)}\boldsymbol{p}_2 - t_{34}^{(3)}\boldsymbol{p}_3)/t_{44}^{(4)}. \tag{3.37}$$

由 (3.33), $\boldsymbol{x}_4 = \boldsymbol{x}_0 + P_4\boldsymbol{g}_4 = \boldsymbol{x}_0 + \sum_{i=1}^{4} g_i\boldsymbol{p}_i$, 因此

$$\boldsymbol{x}_i = \boldsymbol{x}_{i-1} + g_i\boldsymbol{p}_i, \quad i = 1, \ldots, 4, \tag{3.38}$$

其中 g_i 是 $\boldsymbol{g}_4$ 的第 i 个元素. MINRES 方法的近似解 $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_4$ 由 (3.38) 及递推式 (3.34)–(3.37) 得到. $n = 4$ 时的推导容易推广到第 n 次迭代, 得到 Algorithm 3.5. 由 Algorithm 3.5, MINRES 方法从不发生中断, 因为除非 $\widetilde{\boldsymbol{v}}_{n+1} = \boldsymbol{0}$, 否则 $\beta_n \neq 0$.

Algorithm 3.5 MINRES 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \beta_0 = 0,\ \boldsymbol{v}_0 = \boldsymbol{0},\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Output: $\boldsymbol{x}_n$

$\boldsymbol{g} = (\|\boldsymbol{r}_0\|, 0, \ldots, 0)^\top,\ \boldsymbol{v}_1 = \boldsymbol{r}_0/\|\boldsymbol{r}_0\|$
for $n = 1, 2, \ldots$ **do**
    (Lanczos 过程)
    $\alpha_n = (\boldsymbol{v}_n, A\boldsymbol{v}_n)$
    $\widetilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \alpha_n\boldsymbol{v}_n - \beta_{n-1}\boldsymbol{v}_{n-1}$
    $\beta_n = (\widetilde{\boldsymbol{v}}_{n+1}, \widetilde{\boldsymbol{v}}_{n+1})^{1/2}$
    $\boldsymbol{v}_{n+1} = \widetilde{\boldsymbol{v}}_{n+1}/\beta_n$
    $t_{n-1,n} = \beta_{n-1},\ t_{n,n} = \alpha_n$
    $t_{n+1,n} = \beta_n$
    (Givens 旋转)
    for $i = \max\{1, n-2\}, \ldots, n-1$ **do**
        $$\begin{bmatrix} t_{i,n} \\ t_{i+1,n} \end{bmatrix} = \begin{bmatrix} c_i & s_i \\ -\bar{s}_i & c_i \end{bmatrix} \begin{bmatrix} t_{i,n} \\ t_{i+1,n} \end{bmatrix}$$
    end for
    $$c_n = \frac{|t_{n,n}|}{\sqrt{|t_{n,n}|^2 + |t_{n+1,n}|^2}}$$
    $$\bar{s}_n = \frac{t_{n+1,n}}{t_{n,n}} c_n$$
    $t_{n,n} = c_n t_{n,n} + s_n t_{n+1,n}$
    $t_{n+1,n} = 0$
    $$\begin{bmatrix} g_n \\ g_{n+1} \end{bmatrix} = \begin{bmatrix} c_n & s_n \\ -\bar{s}_n & c_n \end{bmatrix} \begin{bmatrix} g_n \\ 0 \end{bmatrix}$$
    (更新 $\boldsymbol{x}_n$)
    $\boldsymbol{p}_n = (\boldsymbol{v}_n - t_{n-2,n}\boldsymbol{p}_{n-2}$
        $-t_{n-1,n}\boldsymbol{p}_{n-1})/t_{n,n}$
    $\boldsymbol{x}_n = \boldsymbol{x}_{n-1} + g_n\boldsymbol{p}_n$
    (检查收敛)
    若 $|g_{n+1}|/\|\boldsymbol{b}\| \leq \epsilon$, 则停止
end for

如上所述, MINRES 方法从不发生中断. Paige 和 Saunders 还在 [143] 中提出了 CG 方法的一种变体 SYMMLQ 方法, 以避免 Hermitian 不定线性方程组求解中的中断. 下面作简要说明. 回顾 CG 方法的近似解 $\boldsymbol{x}_n$ 为

$$\begin{aligned}\boldsymbol{x}_n &= \boldsymbol{x}_0 + V_n \boldsymbol{y}_n,\\ \boldsymbol{r}_n &= \boldsymbol{r}_0 - V_{n+1} T_{n+1,n} \boldsymbol{y}_n = V_{n+1}(\beta \boldsymbol{e}_1 - T_{n+1,n}\boldsymbol{y}_n),\end{aligned}$$

其中 $\boldsymbol{y}_n$ 由 $\boldsymbol{r}_n \perp \mathcal{K}_n(A, \boldsymbol{r}_0)$ 确定 (见 Corollary 3.2), 即 $V^{\mathrm{H}}\boldsymbol{r}_n = \boldsymbol{0}$. 由此得到

$$T_n \boldsymbol{y}_n = \beta \boldsymbol{e}_1, \tag{3.39}$$

其中 T_n 是删除 $T_{n+1,n}$ 最后一行得到的三对角矩阵. CG 方法通过 T_n 的 LDL^{H} 分解得到, 即

$$T_n = LDL^{\mathrm{H}},$$

其中 L 为单位下双对角矩阵, D 为对角矩阵. 若 A 为 Hermitian 正定矩阵, 则 T_n 也是 Hermitian 正定矩阵. 因此这一分解从不发生中断. 另一方面, 当 A 不定时, 可能因中断而无法得到 T_n 的 LDL^{H} 分解. 为绕过这一问题, Paige 和 Saunders 提议使用 T_n 的 LQ 分解, 即

$$T_n = L_n Q_n, \quad Q_n^{\mathrm{H}} Q_n = I,$$

其中 L_n 为下三角矩阵, 该分解通过 Givens 旋转得到, 因而不发生中断. 所得算法称为 SYMMLQ 方法. 当系数矩阵 A 为 Hermitian 正定矩阵时, 在精确算术下 SYMMLQ 方法给出与 CG 方法相同的结果; 在不定情形下, SYMMLQ 方法在求解 (3.39) 时成功避免了中断. 关于简要历史说明, 包括 SYMMLQ 方法与 1962 年 (较少为人所知的) Fridman 方法之间的关系, 见 [160] 的 Section 2.4. 另一种重要的补救方法是 Bank 和 Chan 的 “复合步” 技术 [16], 以及 [58], [126] 中的类似技术 (双曲对). 复合步的关键思想是将 (3.39) 中的 T_n 分解为

$$T_n = \widetilde{L}\widetilde{D}\widetilde{L}^{\mathrm{H}},$$

其中 $\widetilde{L}$ 为单位下分块双对角矩阵, $\widetilde{D}$ 为块大小为 1×1 或 2×2 的分块对角矩阵, 即

$$\widetilde{D} = \begin{bmatrix} D_1 & & \\ & \ddots & \\ & & D_m \end{bmatrix},$$

其中 D_i 为标量或 2×2 矩阵. 这一分解也不发生中断, 例如见 [16] 的 Theorem 2.2. 复合步技术的算法说明 (复合步 BiCG 方法) 见 Section 3.3.2.

3.2 复对称线性方程组

本节考虑下列形式的复对称线性方程组

$$A\boldsymbol{x} = \boldsymbol{b},$$

其中 A 为复对称矩阵, 即 $A = A^{\top}$ 且 $A \neq A^{\mathrm{H}}$. 注意, 复对称矩阵 A 不是 Hermitian 矩阵. 下面是复对称矩阵的一个例子:

$$A = \begin{bmatrix} 1+i & 2+i \\ 2+i & 3 \end{bmatrix}.$$

关于复对称线性方程组的应用, 见 Section 2.1.1.3 中的 Helmholtz 方程和 Section 2.2.1 中的大规模电子结构计算. 本节介绍求解复对称线性方程组的 COCG 方法, COCR 方法和 QMR_SYM 方法. 若将这些方法应用于实对称线性方程组, 则它们分别退化为 CG 方法, CR 方法和 MINRES 方法. 关于复对称线性方程组的其他 Krylov 子空间方法及应用, 例如见 [1], [30], [37], [40], [83], [84], [124], [141] 及其中的参考文献.

3.2.1 共轭正交共轭梯度 (COCG) 方法

COCG 方法 [197] 是求解复对称线性方程组最著名的方法. 将点积 $(\boldsymbol{a},\boldsymbol{b})$ 改为 $(\bar{\boldsymbol{a}},\boldsymbol{b})$, 可从 CG 方法形式上导出该算法. 这里 $\bar{\boldsymbol{a}}$ 表示 $\boldsymbol{a}$ 的复共轭. 经过这一改写, COCG 方法的算法见 Algorithm 3.6.

Algorithm 3.6 COCG 方法 (注意: $(\bar{\boldsymbol{a}},\boldsymbol{b})=\boldsymbol{a}^{\top}\boldsymbol{b}$)

Input: $\boldsymbol{x}_0\in\mathbb{C}^N,\ \beta_{-1}=0,\ \boldsymbol{p}_{-1}=\boldsymbol{0},\ \boldsymbol{r}_0=\boldsymbol{b}-A\boldsymbol{x}_0$

Output: $\boldsymbol{x}_n$

1: **for** $n=0,1,2,\ldots,$ 直至收敛 **do**

2: $\quad\boldsymbol{p}_n=\boldsymbol{r}_n+\beta_{n-1}\boldsymbol{p}_{n-1}$

3: $\quad\alpha_n=\dfrac{(\bar{\boldsymbol{r}}_n,\boldsymbol{r}_n)}{(\bar{\boldsymbol{p}}_n,A\boldsymbol{p}_n)}$

4: $\quad\boldsymbol{x}_{n+1}=\boldsymbol{x}_n+\alpha_n\boldsymbol{p}_n$

5: $\quad\boldsymbol{r}_{n+1}=\boldsymbol{r}_n-\alpha_nA\boldsymbol{p}_n$

6: $\quad\beta_n=\dfrac{(\bar{\boldsymbol{r}}_{n+1},\boldsymbol{r}_{n+1})}{(\bar{\boldsymbol{r}}_n,\boldsymbol{r}_n)}$

7: **end for**

COCG 方法的残差向量 $\boldsymbol{r}_n$ 满足下列关系:

$$\boldsymbol{r}_n\perp\overline{\mathcal{K}_n(A,\boldsymbol{r}_0)},$$

其中 $\overline{\mathcal{K}_n(A,\boldsymbol{r}_0)}:=\mathcal{K}_n(\bar{A},\bar{\boldsymbol{r}}_0)$. 由 Algorithm 3.6 容易看出, 若矩阵 A 为实对称矩阵, 则 COCG 方法与 CG 方法相同. 预条件 COCG 方法见 Algorithm 3.7.

Algorithm 3.7 预条件 COCG 方法 (注意: $(\bar{\boldsymbol{a}},\boldsymbol{b})=\boldsymbol{a}^{\top}\boldsymbol{b}$)

Input: $\boldsymbol{x}_0\in\mathbb{C}^N,\ \boldsymbol{r}_0=\boldsymbol{b}-A\boldsymbol{x}_0,\ \boldsymbol{p}_{-1}=\boldsymbol{0},\ \beta_{-1}=0$

Output: $\boldsymbol{x}_n$

1: **for** $n=0,1,2,\ldots,$ 直至收敛 **do**

2: $\quad\boldsymbol{p}_n=K^{-1}\boldsymbol{r}_n+\beta_{n-1}\boldsymbol{p}_{n-1}$

3: $\quad\alpha_n=\dfrac{(\overline{K^{-1}\boldsymbol{r}_n},\boldsymbol{r}_n)}{(\bar{\boldsymbol{p}}_n,A\boldsymbol{p}_n)}$

4: $\quad\boldsymbol{x}_{n+1}=\boldsymbol{x}_n+\alpha_n\boldsymbol{p}_n$

5: $\quad\boldsymbol{r}_{n+1}=\boldsymbol{r}_n-\alpha_nA\boldsymbol{p}_n$

6: $\quad\beta_n=\dfrac{(\overline{K^{-1}\boldsymbol{r}_{n+1}},\boldsymbol{r}_{n+1})}{(\overline{K^{-1}\boldsymbol{r}_n},\boldsymbol{r}_n)}$

7: **end for**

3.2.2 共轭正交共轭残差 (COCR) 方法

COCR 方法 [171] 也是求解复对称线性方程组的 Krylov 子空间方法. 将点积 $(\boldsymbol{a},\boldsymbol{b})$ 改为

$(\bar{\boldsymbol{a}}, \boldsymbol{b})$, 可由 CR 方法形式上导出该算法. COCR 方法产生的近似解使残差向量满足

$$\boldsymbol{r}_n \perp \overline{A\mathcal{K}_n(A, \boldsymbol{r}_0)},$$

其中 $\overline{A\mathcal{K}_n(A, \boldsymbol{r}_0)} := \bar{A}\mathcal{K}_n(\bar{A}, \bar{\boldsymbol{r}}_0)$. 由此, 若矩阵 A 为实对称矩阵, COCR 方法就与 CR 方法相同. 这表明, 若 A 复对称且接近实矩阵, 则可以预期残差的 2-范数趋向于几乎单调下降, 或表现出平滑的收敛行为. 下面推导 COCR 方法. 首先在 Algorithm 3.8 中引入共轭 A-正交化过程.

Algorithm 3.8 不作归一化的共轭 A-正交化过程 (注意: $(\bar{\boldsymbol{a}}, \boldsymbol{b}) = \boldsymbol{a}^\top \boldsymbol{b}$)

Input: $\boldsymbol{v}_0 = \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Input: $t_{0,0} = (\overline{A\boldsymbol{v}_0}, A\boldsymbol{v}_0)/(\bar{\boldsymbol{v}}_0, A\boldsymbol{v}_0)$

Input: $\boldsymbol{v}_1 = -\alpha_0(A\boldsymbol{v}_0 - t_{0,0}\boldsymbol{v}_0)$

$t_{0,0} = (\overline{A\boldsymbol{v}_1}A\boldsymbol{v}_1)/(\bar{\boldsymbol{v}}_1, A\boldsymbol{v}_1)$

$\boldsymbol{v}_1 = -\alpha_0(A\boldsymbol{v}_1 - t_{0,0}\boldsymbol{v}_1)$

for $n = 1, 2, \ldots, N-1$ **do**

  $t_{n-1,n} = (\overline{A\boldsymbol{v}_{n-1}}, A\boldsymbol{v}_n)/(\bar{\boldsymbol{v}}_{n-1}, A\boldsymbol{v}_{n-1})$

  $t_{n,n} = (\overline{A\boldsymbol{v}_n}, A\boldsymbol{v}_n)/(\bar{\boldsymbol{v}}_n, A\boldsymbol{v}_n)$

  $\boldsymbol{v}_{n+1} = -\alpha_n(A\boldsymbol{v}_n - t_{n,n}\boldsymbol{v}_n - t_{n-1,n}\boldsymbol{v}_{n-1})$

end for

与 Lanczos 过程 (Algorithm 1.11) 类似, Algorithm 3.8 中的向量 $\boldsymbol{v}_i$ 满足下列关系:

$$\operatorname{span}\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\} = \mathcal{K}_n(A, \boldsymbol{r}_0),$$

$$(\bar{\boldsymbol{v}}_i, A\boldsymbol{v}_j) = 0 \quad \text{for } i \neq j. \tag{3.40}$$

下面从 Algorithm 3.8 推导 COCR 方法. 对给定的初始猜测 $\boldsymbol{x}_0$, 令 $\boldsymbol{r}_0 := \boldsymbol{b} - A\boldsymbol{x}_0$ 为初始残差向量. 则 Algorithm 3.8 可表示为下列矩阵形式:

$$A\underbrace{[\boldsymbol{r}_0, \boldsymbol{r}_1, \ldots, \boldsymbol{r}_{n-1}]}_{R_n} = \underbrace{[\boldsymbol{r}_0, \boldsymbol{r}_1, \ldots, \boldsymbol{r}_{n-1}, \boldsymbol{r}_n]}_{R_{n+1}} \underbrace{\begin{bmatrix} t_{0,0} & t_{0,1} & & \\ -\alpha_0^{-1} & t_{1,1} & \ddots & \\ & -\alpha_1^{-1} & \ddots & t_{n-2,n-1} \\ & & \ddots & t_{n-1,n-1} \\ & & & -\alpha_{n-1}^{-1} \end{bmatrix}}_{T_{n+1,n}},$$

其中 $t_{k-1,k} = \dfrac{(\overline{A\boldsymbol{r}_{k-1}}, A\boldsymbol{r}_k)}{(\bar{\boldsymbol{r}}_{k-1}, A\boldsymbol{r}_{k-1})}$, $t_{k,k} = \dfrac{(\overline{A\boldsymbol{r}_k}, A\boldsymbol{r}_k)}{(\bar{\boldsymbol{r}}_k, A\boldsymbol{r}_k)}$. 由上述矩阵形式得到

$$AR_n = R_{n+1}T_{n+1,n}. \tag{3.41}$$

由于矩阵 R_n 由 Algorithm 3.8 生成, 由 A-正交性质 (3.40) 有 $R_n^\top AR_n = D_n$, 其中 D_n 为 n 行 n 列的对角矩阵. 标量参数 $\alpha_0, \ldots, \alpha_{n-1}$ 仍未知. 我们将说明, 若确定这些标量参数,

使得能够从 $\boldsymbol{r}_0,\ldots,\boldsymbol{r}_{n-1}$ 的信息中提取近似解 $\boldsymbol{x}_0,\ldots,\boldsymbol{x}_{n-1}$, 就得到 COCR 方法的算法. 设 $X_n:=[\boldsymbol{x}_0,\ldots,\boldsymbol{x}_{n-1}]$, $\mathbf{1}$ 为 $[1,\ldots,1]^\top$. 则 $R_n:=[\boldsymbol{r}_0,\boldsymbol{r}_1,\ldots,\boldsymbol{r}_{n-1}]$ 与 X_n 有如下关系:

$$R_n=[\boldsymbol{b}-A\boldsymbol{x}_0,\boldsymbol{b}-A\boldsymbol{x}_1,\ldots,\boldsymbol{b}-A\boldsymbol{x}_{n-1}]=\boldsymbol{b}\mathbf{1}_n^\top-AX_n. \tag{3.42}$$

将 (3.42) 代入 (3.41), 得

$$AR_n=(\boldsymbol{b}\mathbf{1}_{n+1}^\top-AX_{n+1})T_{n+1,n}.$$

于是,

$$R_n=(\boldsymbol{x}\mathbf{1}_{n+1}^\top-X_{n+1})T_{n+1,n},$$

其中 $\boldsymbol{x}$ 为线性方程组 $A\boldsymbol{x}=\boldsymbol{b}$ 的精确解. 若如下形式的条件

$$\boldsymbol{x}\mathbf{1}_{n+1}^\top T_{n+1,n}=O_n \tag{3.43}$$

成立, 则 $R_n=-X_{n+1}T_{n+1,n}$. 关系 $R_n=-X_{n+1}T_{n+1,n}$ 表明, 可以由残差向量得到近似解向量. 反之, 若这一关系不成立, 就不能由残差向量得到近似解向量. 由于 $T_{n+1,n}$ 中的未知参数 $\alpha_0,\ldots,\alpha_{n-1}$ 尚未确定, 现确定这些参数使 (3.43) 成立. 由此得到

$$\begin{aligned}&\alpha_0^{-1}=t_{0,0},\\&\alpha_k^{-1}=t_{k,k}+t_{k-1,k},\quad 1\le k\le n-1.\end{aligned}$$

将上述递推关系代入 $T_{n+1,n}$, 有

$$T_{n+1,n}=\begin{bmatrix}\alpha_0^{-1}&t_{0,1}&&&\\-\alpha_0^{-1}&\alpha_1^{-1}-t_{0,1}&\ddots&&\\&-\alpha_1^{-1}&\ddots&t_{n-2,n-1}&\\&&\ddots&\alpha_{n-1}^{-1}-t_{n-2,n-1}\\&&&-\alpha_{n-1}^{-1}\end{bmatrix}.$$

于是, $T_{n+1,n}$ 可分解如下:

$$\begin{aligned}T_{n+1,n}&=\begin{bmatrix}1&&\\-1&\ddots&\\&\ddots&1\\&&-1\end{bmatrix}\begin{bmatrix}\alpha_0^{-1}&&\\&\ddots&\\&&\alpha_{n-1}^{-1}\end{bmatrix}\begin{bmatrix}1&\alpha_0t_{0,1}&&\\&1&\ddots&\\&&\ddots&\alpha_{n-2}t_{n-2,n-1}\\&&&1\end{bmatrix}\\&=B_{n+1,n}^{(L)}\Omega_n^{-1}B_n^{(U)}.\end{aligned}$$

由 (3.41) 和上述分解得到

$$AR_n=R_{n+1}T_{n+1,n}=R_{n+1}B_{n+1,n}^{(L)}\Omega_n^{-1}B_n^{(U)}. \tag{3.44}$$

这里 $P_n:=R_n(B_n^{(U)})^{-1}$. 由 (3.44) 有

$$AP_n=AR_n(B_n^{(U)})^{-1}=R_{n+1}B_{n+1,n}^{(L)}\Omega_n^{-1}. \tag{3.45}$$

上述矩阵形式等价于下列递推关系:

$$\boldsymbol{r}_k=\boldsymbol{r}_{k-1}-\alpha_{k-1}A\boldsymbol{p}_{k-1},\quad 1\le k\le n. \tag{3.46}$$

由 $P_n = R_n(B_n^{(U)})^{-1}$ 得

$$\boldsymbol{p}_k = \boldsymbol{r}_k + \beta_{k-1}\boldsymbol{p}_{k-1}, \quad 1 \le k \le n, \tag{3.47}$$

其中 $\beta_{k-1} := -\alpha_{k-1}t_{k-1,k}$. 现在给出近似解的计算公式. 由 (3.45),

$$\begin{aligned} AP_n &= R_{n+1}B_{n+1,n}^{(L)}\Omega_n^{-1} \\ &= (\boldsymbol{b}\mathbf{1}^\top - AX_{n+1})B_{n+1,n}^{(L)}\Omega_n^{-1} \\ &= -AX_{n+1}B_{n+1,n}^{(L)}\Omega_n^{-1}. \end{aligned}$$

因此得到

$$P_n\Omega_n = X_{n+1}(-B_{n+1,n}^{(L)}),$$

它等价于

$$\boldsymbol{x}_k = \boldsymbol{x}_{k-1} + \alpha_{k-1}\boldsymbol{p}_{k-1}, \quad 1 \le k \le n. \tag{3.48}$$

现在给出 α_k 和 β_k 更实用的计算公式. 由 (3.41) 得

$$(AR_n)^\top AR_n = R_n^\top AR_{n+1}T_{n+1,n}.$$

由上述关系的 $(k+1,k)$ 元素有 $(\overline{A\boldsymbol{r}_k}, A\boldsymbol{r}_{k-1}) = -\alpha_{k-1}^{-1}(\bar{\boldsymbol{r}}_k, A\boldsymbol{r}_k)$. 因此由 $\beta_{k-1} = -\alpha_{k-1}t_{k-1,k}$ 得

$$\beta_k = -\alpha_k t_{k,k+1} = \frac{(\bar{\boldsymbol{r}}_{k+1}, A\boldsymbol{r}_{k+1})}{(\overline{A\boldsymbol{r}_{k+1}}, A\boldsymbol{r}_k)} \times \frac{(\overline{A\boldsymbol{r}_k}, A\boldsymbol{r}_{k+1})}{(\bar{\boldsymbol{r}}_k, A\boldsymbol{r}_k)} = \frac{(\bar{\boldsymbol{r}}_{k+1}, A\boldsymbol{r}_{k+1})}{(\bar{\boldsymbol{r}}_k, A\boldsymbol{r}_k)}. \tag{3.49}$$

这里使用了关系 $(\bar{\boldsymbol{r}}_k, A\boldsymbol{r}_{k+1}) = (\bar{\boldsymbol{r}}_{k+1}, A\boldsymbol{r}_k)$, 因为

$$(\bar{\boldsymbol{r}}_k, A\boldsymbol{r}_{k+1}) = (\bar{\boldsymbol{r}}_k, A\boldsymbol{r}_{k+1})^\top = (\boldsymbol{r}_k^\top A\boldsymbol{r}_{k+1})^\top = (\bar{\boldsymbol{r}}_{k+1}, A^\top\boldsymbol{r}_k) = (\bar{\boldsymbol{r}}_{k+1}, A\boldsymbol{r}_k). \tag{3.50}$$

由 (3.45), 并回顾 $P_n = R_n(B_n^{(U)})^{-1}$, 得

$$(AP_n)^\top AP_n = (AP_n)^\top R_{n+1}B_{n+1,n}^{(L)}\Omega_n^{-1} = (B_n^{(U)})^{-\top}R_n^\top AR_{n+1}B_{n+1,n}^{(L)}\Omega_n^{-1}.$$

$(B_n^{(U)})^{-\top}$ 为下三角矩阵, A-正交性质 (3.40) 表明 $R_n^\top AR_{n+1}$ 的非对角元素为零. 因而 $(AP_n)^\top AP_n$ 为如下形式的下三角矩阵

$$(AP_n)^\top AP_n = \begin{bmatrix} d_0 & & & \\ * & \ddots & & \\ \vdots & \ddots & \ddots & \\ * & \cdots & * & d_{n-1} \end{bmatrix},$$

其中 $d_k = (\bar{\boldsymbol{r}}_k, \boldsymbol{r}_k)\alpha_k^{-1}$. 因此得到

$$\alpha_k = \frac{(\bar{\boldsymbol{r}}_k, A\boldsymbol{r}_k)}{(\overline{A\boldsymbol{p}_k}, A\boldsymbol{p}_k)}. \tag{3.51}$$

此外, 由于 $(AP_n)^\top AP_n$ 是复对称的, 有

$$(AP_n)^\top AP_n = \begin{bmatrix} d_0 & & & \\ * & \ddots & & \\ \vdots & \ddots & \ddots & \\ * & \cdots & * & d_{n-1} \end{bmatrix} = \begin{bmatrix} d_0 & * & \cdots & * \\ & \ddots & \ddots & \vdots \\ & & \ddots & * \\ & & & d_{n-1} \end{bmatrix},$$

这意味着 $(AP_n)^\top AP_n = \mathrm{diag}(d_0, \ldots, d_{n-1})$. 因此 $\boldsymbol{p}_i$ 和 $\boldsymbol{p}_j$ 共轭 A^2-正交, 即

$$(\overline{A\boldsymbol{p}_i}, A\boldsymbol{p}_j) = 0 \quad \text{for } i \neq j.$$

由 (3.46)–(3.51) 得到 COCR 方法的算法, 见 Algorithm 3.9.

Algorithm 3.9 COCR 方法 (注意: $(\bar{\boldsymbol{a}}, \boldsymbol{b}) = \boldsymbol{a}^\top \boldsymbol{b}$)

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \beta_{-1} = 0,\ \boldsymbol{p}_{-1} = \boldsymbol{0},\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Output: $\boldsymbol{x}_n$

1: **for** $n = 0, 1, 2, \ldots,$ 直至收敛 **do**

2: $\quad \boldsymbol{p}_n = \boldsymbol{r}_n + \beta_{n-1}\boldsymbol{p}_{n-1}$

3: $\quad A\boldsymbol{p}_n = A\boldsymbol{r}_n + \beta_{n-1}A\boldsymbol{p}_{n-1}$

4: $\quad \alpha_n = \dfrac{(\bar{\boldsymbol{r}}_n, A\boldsymbol{r}_n)}{(\overline{A\boldsymbol{p}_n}, A\boldsymbol{p}_n)}$

5: $\quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n$

6: $\quad \boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n$

7: $\quad \beta_n = \dfrac{(\bar{\boldsymbol{r}}_{n+1}, A\boldsymbol{r}_{n+1})}{(\bar{\boldsymbol{r}}_n, A\boldsymbol{r}_n)}$

8: **end for**

对线性方程组 $K_1^{-1}AK_1^{-\top}\widetilde{\boldsymbol{x}} = K_1^{-1}\boldsymbol{b}$ ($\widetilde{\boldsymbol{x}} = K_1^\top \boldsymbol{x}$) 应用 COCR 方法, 通过下列改写即可得到预条件 COCR 方法:

$$\widetilde{\boldsymbol{x}}_n \Rightarrow K_1^\top \boldsymbol{x}_n, \quad \widetilde{\boldsymbol{p}}_n \Rightarrow K_1^\top \boldsymbol{p}_n, \quad \widetilde{\boldsymbol{r}}_n \Rightarrow K_1^{-1}\boldsymbol{r}_n,$$

其中 $A \approx K = K_1 K_1^\top$. 于是得到 Algorithm 3.10 所述的预条件 COCR 方法.

Algorithm 3.10 预条件 COCR 方法 (注意: $(\bar{\boldsymbol{a}}, \boldsymbol{b}) = \boldsymbol{a}^\top \boldsymbol{b}$)

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0,\ \boldsymbol{p}_{-1} = \boldsymbol{0},\ \beta_{-1} = 0$

Output: $\boldsymbol{x}_n$

1: **for** $n = 0, 1, 2, \ldots,$ 直至收敛 **do**

2: $\quad \boldsymbol{p}_n = K^{-1}\boldsymbol{r}_n + \beta_{n-1}\boldsymbol{p}_{n-1}$

3: $\quad (A\boldsymbol{p}_n = AK^{-1}\boldsymbol{r}_n + \beta_{n-1}A\boldsymbol{p}_{n-1})$

4: $\quad \alpha_n = \dfrac{(\overline{K^{-1}\boldsymbol{r}_n}, AK^{-1}\boldsymbol{r}_n)}{(\overline{A\boldsymbol{p}_n}, K^{-1}A\boldsymbol{p}_n)}$

5: $\quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n$

6: $\quad \boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n$

7: $\quad (K^{-1}\boldsymbol{r}_{n+1} = K^{-1}\boldsymbol{r}_n - \alpha_n K^{-1}A\boldsymbol{p}_n)$

8: $\quad \beta_n = \dfrac{(\overline{K^{-1}\boldsymbol{r}_{n+1}}, AK^{-1}\boldsymbol{r}_{n+1})}{(\overline{K^{-1}\boldsymbol{r}_n}, AK^{-1}\boldsymbol{r}_n)}$

9: **end for**

注意, Algorithm 3.10 中加入第 3 和第 7 行, 是为了减少形如 $A\boldsymbol{v}$ 的矩阵向量乘法及形如 $K\boldsymbol{v} = \boldsymbol{w}$ 的线性方程组求解次数, 这与 Section 3.1.2 中预条件 CR 方法所述的技术相同.

3.2.3 准最小残差 (QMR_SYM) 方法

van der Vorst 的著作 [196], 第 112 页中命名的 QMR_SYM 方法 [61], 也是求解复对称线性方程组的一种著名 Krylov 子空间方法, 可从 Section 1.9.3 所述的复对称 Lanczos 过程导出. 本小节介绍 QMR_SYM 方法的推导过程. 设 V_n 为 N 行 n 列的矩阵, 其列向量由复对称 Lanczos 过程 (Algorithm 1.10) 生成. 则 QMR_SYM 方法从下式寻找近似解

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + V_n \boldsymbol{y}_n, \quad \boldsymbol{y}_n \in \mathbb{C}^n. \tag{3.52}$$

这里 $\boldsymbol{y}_n$ 由稍后所述的条件确定. 注意 $\boldsymbol{x}_n$ 位于仿射空间 $\boldsymbol{x}_0 + \mathcal{K}_n(A, \boldsymbol{r}_0)$ 中. 相应的残差向量为

$$\boldsymbol{r}_n = \boldsymbol{r}_0 - AV_n \boldsymbol{y}_n.$$

由复对称 Lanczos 过程的矩阵形式 (1.41) 得

$$\boldsymbol{r}_n = \boldsymbol{r}_0 - V_{n+1} T_{n+1,n} \boldsymbol{y}_n = V_{n+1}(\beta \boldsymbol{e}_1 - T_{n+1,n} \boldsymbol{y}_n)$$

其中 $\beta := \|\boldsymbol{r}_0\|$. 上述推导过程与 Section 3.1.3 中 MINRES 方法的推导相似. 区别在于, 若像 MINRES 方法一样选择 $\boldsymbol{y}_n$ 使残差范数达到极小值, 则可能导致大量计算开销, 因为

$$\|\boldsymbol{r}_n\| = \|V_{n+1}(\beta \boldsymbol{e}_1 - T_{n+1,n} \boldsymbol{y}_n)\| \neq \|(\beta \boldsymbol{e}_1 - T_{n+1,n} \boldsymbol{y}_n)\|, \tag{3.53}$$

这是由于 $V_{n+1}^{\mathrm{H}} V_{n+1} \neq I$. 这意味着, 若试图得到最小残差近似解, 就需要极小化 $\|V_{n+1}(\beta \boldsymbol{e}_1 - T_{n+1,n} \boldsymbol{y}_n)\|$. 结果是计算开销和内存需求随迭代次数增加而增长. 因此, QMR_SYM 方法不按最小残差途径设计, 而按准最小残差途径设计, 即选择 $\boldsymbol{y}_n$ 使得

$$\boldsymbol{y}_n := \arg\min_{\boldsymbol{y} \in \mathbb{C}^n} \|\beta \boldsymbol{e}_1 - T_{n+1,n} \boldsymbol{y}\|. \tag{3.54}$$

所得向量 $\beta \boldsymbol{e}_1 - T_{n+1,n} \boldsymbol{y}_n$ 称为准残差向量. 这里注意, 若 V_n 满足关系 $V_n^{\mathrm{H}} V_n = I_n$, 则上述选择使残差范数达到极小值. 由于 A 复对称, 除一些特殊情形外, 一般的 V_n 不满足关系 $V_n^{\mathrm{H}} V_n = I_n$. 文献 [61] 给出的一种特殊情形具有下列形式

$$A = B + i\sigma I, \quad B = B^{\top} \in \mathbb{R}^{N \times N}, \quad \sigma \in \mathbb{R}.$$

为实现极小化 (3.54), Section 3.1.3 所述的 Givens 旋转起重要作用. 用满足 $Q_n^{\mathrm{H}} Q_n = I_n$ 和下式的 Givens 旋转 $Q_n = G_n \cdots G_1$ 乘以 $\beta \boldsymbol{e}_1 - T_{n+1,n} \boldsymbol{y}$

$$Q_n T_{n+1,n} = \begin{bmatrix} R_n \\ \boldsymbol{0}^{\top} \end{bmatrix},$$

有

$$\min_{\boldsymbol{y} \in \mathbb{C}^n} \|\beta \boldsymbol{e}_1 - T_{n+1,n} \boldsymbol{y}\| = \min_{\boldsymbol{y} \in \mathbb{C}^n} \left\| \begin{bmatrix} \boldsymbol{g}_n \\ g_{n+1} \end{bmatrix} - \begin{bmatrix} R_n \\ \boldsymbol{0}^{\top} \end{bmatrix} \boldsymbol{y} \right\|, \tag{3.55}$$

其中

$$\begin{bmatrix} \boldsymbol{g}_n \\ g_{n+1} \end{bmatrix} = \beta Q_n \boldsymbol{e}_1$$

且 R_n 为上三角矩阵. 于是 (3.55) 的解为 $\boldsymbol{y}_n = R_n^{-1}\boldsymbol{g}_n$, 结合 (3.52), 得到近似解 $\boldsymbol{x}_n = \boldsymbol{x}_0 + V_n R_n^{-1}\boldsymbol{g}_n$. 与 MINRES 方法类似, 引入 $P_n := V_n R_n^{-1}$ 可给出三项递推关系. 于是有 $\boldsymbol{x}_n$ 更实用的计算公式:

$$\boldsymbol{x}_i = \boldsymbol{x}_{i-1} + g_i \boldsymbol{p}_i, \quad i = 1, \ldots, n,$$

其中 g_i 和 $\boldsymbol{p}_i$ 分别为 $\boldsymbol{g}_n$ 的第 i 个元素和 P_n 的第 i 列. QMR_SYM 方法的完整算法见 Algorithm 3.11. 与 MINRES 方法不同, 由于关系 (3.53), 不能从 (3.55) 得到残差范数 $\|\boldsymbol{r}_n\|$. 因此问题是如何计算 $\|\boldsymbol{r}_n\|$ 以评价近似解的质量. 一种简单办法是计算 $\|\boldsymbol{b} - A\boldsymbol{x}_n\|$, 但每次迭代需要额外一次矩阵向量乘法. 另一方面, 可以避免这次矩阵向量乘法计算. 下面给出这一技术. 与 MINRES 方法类似, QMR_SYM 方法的近似解由下式给出

$$\begin{aligned}
\boldsymbol{p}_n &= \frac{1}{t_{n,n}}(\boldsymbol{v}_n - t_{n-2,n}\boldsymbol{p}_{n-2} - t_{n-1,n}\boldsymbol{p}_{n-1}),\\
\boldsymbol{x}_n &= \boldsymbol{x}_{n-1} + g_n \boldsymbol{p}_n.
\end{aligned}$$

由于 $\boldsymbol{r}_n = \boldsymbol{b} - A\boldsymbol{x}_n$, 有

$$\boldsymbol{r}_n = \boldsymbol{r}_{n-1} - g_n A\boldsymbol{p}_n.$$

这里 $A\boldsymbol{p}_n$ 可按下式更新

$$A\boldsymbol{p}_n = \frac{1}{t_{n,n}}(A\boldsymbol{v}_n - t_{n-2,n}A\boldsymbol{p}_{n-2} - t_{n-1,n}A\boldsymbol{p}_{n-1}).$$

由于复对称 Lanczos 过程中已计算 $A\boldsymbol{v}_n$, 可以不直接计算 $\|\boldsymbol{b} - A\boldsymbol{x}_n\|$ 而计算残差的 2-范数 $\|\boldsymbol{r}_n\|$, 从而以较低开销评价 $\|\boldsymbol{r}_n\|$ 来检查收敛. 这些递推式加入 Algorithm 3.11 的第 24–26 行.

Algorithm 3.11 QMR_SYM 方法 (注意: $(\bar{\boldsymbol{a}}, \boldsymbol{b}) = \boldsymbol{a}^\top \boldsymbol{b}$)

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \beta_0 = 0,\ \boldsymbol{v}_0 = \boldsymbol{p}_{-1} = \boldsymbol{p}_0 = \boldsymbol{0},\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Output: $\boldsymbol{x}_n$

$\boldsymbol{g} = (\|\boldsymbol{r}_0\|, 0, \ldots, 0)^\top,\ \boldsymbol{v}_1 = \boldsymbol{r}_0/\|\boldsymbol{r}_0\|$

for $n = 1, 2, \ldots$ **do**

(复对称 Lanczos 过程)

$\alpha_n = (\bar{\boldsymbol{v}}_n, A\boldsymbol{v}_n)$

$\widetilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \alpha_n \boldsymbol{v}_n - \beta_{n-1}\boldsymbol{v}_{n-1}$

$\beta_n = (\overline{\widetilde{\boldsymbol{v}}_{n+1}}, \widetilde{\boldsymbol{v}}_{n+1})^{1/2}$

$\boldsymbol{v}_{n+1} = \widetilde{\boldsymbol{v}}_{n+1}/\beta_n$

$t_{n-1,n} = \beta_{n-1},\ t_{n,n} = \alpha_n$

$t_{n+1,n} = \beta_n$

(Givens 旋转)

for $i = \max\{1, n-2\}, \ldots, n-1$ **do**

$$\begin{bmatrix} t_{i,n} \\ t_{i+1,n} \end{bmatrix} = \begin{bmatrix} c_i & s_i \\ -\bar{s}_i & c_i \end{bmatrix} \begin{bmatrix} t_{i,n} \\ t_{i+1,n} \end{bmatrix}$$

end for

$$c_n = \frac{|t_{n,n}|}{\sqrt{|t_{n,n}|^2 + |t_{n+1,n}|^2}}$$

$$\bar{s}_n = \frac{t_{n+1,n}}{t_{n,n}} c_n$$

$t_{n,n} = c_n t_{n,n} + s_n t_{n+1,n}$

$t_{n+1,n} = 0$

$$\begin{bmatrix} g_n \\ g_{n+1} \end{bmatrix} = \begin{bmatrix} c_n & s_n \\ -\bar{s}_n & c_n \end{bmatrix} \begin{bmatrix} g_n \\ 0 \end{bmatrix}$$

(更新 $\boldsymbol{x}_n$)

$\boldsymbol{p}_n = (\boldsymbol{v}_n - t_{n-2,n}\boldsymbol{p}_{n-2}$

$\quad -t_{n-1,n}\boldsymbol{p}_{n-1})/t_{n,n}$

$\boldsymbol{x}_n = \boldsymbol{x}_{n-1} + g_n \boldsymbol{p}_n$

(检查收敛)

$A\boldsymbol{p}_n = (A\boldsymbol{v}_n - t_{n-2,n}A\boldsymbol{p}_{n-2}$

$\quad -t_{n-1,n}A\boldsymbol{p}_{n-1})/t_{n,n}$

$\boldsymbol{r}_n = \boldsymbol{r}_{n-1} - g_n A\boldsymbol{p}_n$

若 $\|\boldsymbol{r}_n\|/\|\boldsymbol{b}\| \le \epsilon$, 则停止

end for

3.3 非 Hermitian 线性方程组

回顾 CG 方法和 CR 方法 (或 MINRES 方法) 具有的以下两个良好性质:

1. 残差向量满足 Ritz-Galerkin 途径 (1.24) 或最小残差途径 (1.26).
2. 残差向量由短递推生成.

如 Section 3.1 所述, 第一个性质决定近似解的最优性, 第二个性质在使每次迭代的计算开销和

内存需求保持不变方面起重要作用. 回顾 Ritz–Galerkin 途径 (1.24) 对应于 CG 方法, 最小残差途径 (1.26) 对应于 CR 方法和 MINRES 方法. 这里产生一个问题: 对于非 Hermitian 线性方程组, 是否存在同时满足这两个性质的 Krylov 子空间方法? 答案是否定的, 这已在 [57] 中证明, 称为 Faber–Manteuffel 定理. 更具体地, 若矩阵 A 具有形式

$$A = e^{i\phi}(icI + G) \quad (c \geq 0,\ 0 \leq \phi \leq 2\pi,\ B^{-1}G^{\mathrm{H}}B = G),$$

则可构造同时满足这两个性质的算法. 详见 [57] 和 Greenbaum 的著作 [81], Chapter 6. 另见 [121] 及其中的参考文献. 为求解非 Hermitian 线性方程组, 可以考虑如下变换后的线性方程组:

$$A^{\mathrm{H}}A\boldsymbol{x} = A^{\mathrm{H}}\boldsymbol{b}. \tag{3.56}$$

若 A 非奇异, 则 $A^{\mathrm{H}}A$ 为 Hermitian 正定矩阵. 因而可对变换后的线性方程组应用 CG 方法. 然而, $A^{\mathrm{H}}A$ 的条件数可能为原矩阵 A 的条件数的两倍. 因此, 由 Theorem 3.4, 收敛速度可能较慢. 此外, 若条件数很大, 则由误差分析 (1.3) 可知, 所得近似解的精度可能很低. 注意, 误差分析 (1.3) 不依赖于求解线性方程组的数值算法. 这意味着, 由任何数值算法计算的近似解的精度都可能变低. 另一方面, 若 $A^{\mathrm{H}}A$ 的条件数不很大, 则有些情况下对 (3.56) 应用 CG 方法是一种可取的方法. 事实上, 若 A 为酉矩阵, 则 $A^{\mathrm{H}}A = I$, 这表明 CG 方法只需一次迭代就给出解. 因此可以推测, 当 A 接近酉矩阵时, 这一途径尤其有用. 这一途径中著名的算法为 CGNE 方法和 CGNR 方法, 将在 Section 3.4.1 中介绍. 下面推导非 Hermitian 线性方程组的 Krylov 子空间方法. 将看到 BiCG 方法, BiCR 方法和 QMR 方法可分别视为 CG 方法, CR 方法和 MINRES 方法的推广, 它们基于双 Lanczos 过程 (或类似双 Lanczos 的过程); GMRES 方法可视为 MINRES 方法的推广, 且 GMRES 方法基于 Arnoldi 过程. 还将看到如何通过乘积型 Krylov 子空间方法和 IDR(s) 方法改进 BiCG 方法.

3.3.1 双共轭梯度 (BiCG) 方法

与 Section 3.1.1 中 CG 方法的推导过程类似, BiCG 方法 [58], [119] 可由 Section 1.9.2 中的双 Lanczos 过程导出. 本小节给出一种简洁的推导方式, 以说明非 Hermitian 线性方程组的 BiCG 方法可从预条件 COCG 方法 (Algorithm 3.7) 导出. 首先考虑下列 $2N \times 2N$ 复对称线性方程组:

$$\begin{bmatrix} O & A \\ A^{\top} & O \end{bmatrix} \begin{bmatrix} \bar{\boldsymbol{x}}^* \\ \boldsymbol{x} \end{bmatrix} = \begin{bmatrix} \boldsymbol{b} \\ \bar{\boldsymbol{b}}^* \end{bmatrix}, \quad \text{or } \widetilde{A}\widetilde{\boldsymbol{x}} = \widetilde{\boldsymbol{b}}. \tag{3.57}$$

由方程组 (3.57) 可看出, 它在数学上等价于

$$A\boldsymbol{x} = \boldsymbol{b} \quad \text{and} \quad A^{\top}\bar{\boldsymbol{x}}^* = \bar{\boldsymbol{b}}^* \quad (\Leftrightarrow A^{\mathrm{H}}\boldsymbol{x}^* = \boldsymbol{b}^*).$$

若采用下列预条件子, 将 COCG 算法

$$\widetilde{M} = \begin{bmatrix} O & I \\ I & O \end{bmatrix}, \quad I:\ \text{identity matrix}, \tag{3.58}$$

应用于 (3.57), 则所得算法在第 n 次迭代时可写为

$$\begin{aligned}
\widetilde{\boldsymbol{p}}_n^{\mathrm{COCG}} &= \widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_n^{\mathrm{COCG}} + \beta_{n-1}\widetilde{\boldsymbol{p}}_{n-1}^{\mathrm{COCG}}, \\
\alpha_n &= \frac{(\overline{\widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_n^{\mathrm{COCG}}}, \widetilde{\boldsymbol{r}}_n^{\mathrm{COCG}})}{(\overline{\widetilde{\boldsymbol{p}}_n^{\mathrm{COCG}}}, \widetilde{A}\widetilde{\boldsymbol{p}}_n^{\mathrm{COCG}})}, \\
\widetilde{\boldsymbol{x}}_{n+1}^{\mathrm{COCG}} &= \widetilde{\boldsymbol{x}}_n^{\mathrm{COCG}} + \alpha_n\widetilde{\boldsymbol{p}}_n^{\mathrm{COCG}}, \\
\widetilde{\boldsymbol{r}}_{n+1}^{\mathrm{COCG}} &= \widetilde{\boldsymbol{r}}_n^{\mathrm{COCG}} - \alpha_n\widetilde{A}\widetilde{\boldsymbol{p}}_n^{\mathrm{COCG}}, \\
\beta_n &= \frac{(\overline{\widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_{n+1}^{\mathrm{COCG}}}, \widetilde{\boldsymbol{r}}_{n+1}^{\mathrm{COCG}})}{(\overline{\widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_n^{\mathrm{COCG}}}, \widetilde{\boldsymbol{r}}_n^{\mathrm{COCG}})}.
\end{aligned}$$

将 (3.58) 中的 $\widetilde{M}^{-1}(=\widetilde{M})$ 及向量

$$\widetilde{\boldsymbol{x}}_n^{\mathrm{COCG}} := \begin{bmatrix}\boldsymbol{x}_n \\ \bar{\boldsymbol{x}}_n^*\end{bmatrix}, \quad \widetilde{\boldsymbol{r}}_n^{\mathrm{COCG}} := \begin{bmatrix}\boldsymbol{r}_n \\ \bar{\boldsymbol{r}}_n^*\end{bmatrix}, \quad \widetilde{\boldsymbol{p}}_n^{\mathrm{COCG}} := \begin{bmatrix}\bar{\boldsymbol{p}}_n^* \\ \boldsymbol{p}_n\end{bmatrix}$$

代入前述递推式, 并使用下列结果:

$$\begin{aligned}
(\overline{\widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_n^{\mathrm{COCG}}}, \widetilde{\boldsymbol{r}}_n^{\mathrm{COCG}}) &= \begin{bmatrix}\bar{\boldsymbol{r}}_n^{*\top} & \boldsymbol{r}_n^{\top}\end{bmatrix}\begin{bmatrix}\boldsymbol{r}_n \\ \bar{\boldsymbol{r}}_n^*\end{bmatrix} \\
&= \bar{\boldsymbol{r}}_n^{*\top}\boldsymbol{r}_n + \boldsymbol{r}_n^{\top}\bar{\boldsymbol{r}}_n^* \\
&= \boldsymbol{r}_n^{*\mathrm{H}}\boldsymbol{r}_n + (\boldsymbol{r}_n^{\top}\bar{\boldsymbol{r}}_n^*)^{\top} \\
&= (\boldsymbol{r}_n^*, \boldsymbol{r}_n) + \boldsymbol{r}_n^{*\mathrm{H}}\boldsymbol{r}_n \\
&= 2(\boldsymbol{r}_n^*, \boldsymbol{r}_n) \\
(\overline{\widetilde{\boldsymbol{p}}_n^{\mathrm{COCG}}}, \widetilde{A}\widetilde{\boldsymbol{p}}_n^{\mathrm{COCG}}), &= \begin{bmatrix}((\bar{\boldsymbol{p}}_n^*)^{\top}\boldsymbol{p}_n)^{\top}\end{bmatrix}\begin{bmatrix}A\boldsymbol{p}_n \\ A^{\top}\bar{\boldsymbol{p}}_n^*\end{bmatrix} \\
&= (\boldsymbol{p}_n^*)^{\mathrm{H}}A\boldsymbol{p}_n + \boldsymbol{p}_n^{\top}A^{\top}\bar{\boldsymbol{p}}_n^*, \\
&= (\boldsymbol{p}_n^*)^{\mathrm{H}}A\boldsymbol{p}_n + (\boldsymbol{p}_n^{\top}A^{\top}\bar{\boldsymbol{p}}_n^*)^{\top}, \\
&= 2(\boldsymbol{p}_n^*, A\boldsymbol{p}_n),
\end{aligned}$$

就容易得到 Algorithm 3.12 所述的求解非 Hermitian 线性方程组的 BiCG 算法. 若矩阵 A 为实非对称矩阵, 则上述推导过程对应于 van der Vorst 的著作 [196], Chapter 7 中的推导, 其中实非对称线性方程组的 BiCG 方法由 CG 方法导出. 由 Algorithm 3.12 可见, 若系数矩阵 A 为 Hermitian 矩阵, 则选择 $\boldsymbol{r}_0^* = \boldsymbol{r}_0$ 就退化为 CG 方法. 与 Proposition 3.1 所示的 CG 迭代类似, BiCG 迭代具有下列关系:

Proposition 3.5: 设 $\boldsymbol{p}_n, \boldsymbol{r}_n$ 为 BiCG 方法中的向量. 则,

1. $(\boldsymbol{r}_i^*, \boldsymbol{r}_j) = 0 \quad \text{for } i \neq j,$
2. $(\boldsymbol{p}_i^*, A\boldsymbol{p}_j) = 0 \quad \text{for } i \neq j.$

Algorithm 3.12 BiCG 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N$, $\beta_{-1} = 0$, $\boldsymbol{p}_{-1} = \boldsymbol{p}_{-1}^* = \boldsymbol{0}$, $\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Input: 选择 $\boldsymbol{r}_0^* \in \mathbb{C}^N$, 例如 $\boldsymbol{r}_0^* = \boldsymbol{r}_0$

Output: $\boldsymbol{x}_n$

1: **for** $n = 0, 1, 2, \ldots,$ 直至收敛 **do**

2: $\quad \boldsymbol{p}_n = \boldsymbol{r}_n + \beta_{n-1}\boldsymbol{p}_{n-1}, \quad \boldsymbol{p}_n^* = \boldsymbol{r}_n^* + \bar{\beta}_{n-1}\boldsymbol{p}_{n-1}^*$

3: $\quad \alpha_n = \dfrac{(\boldsymbol{r}_n^*, \boldsymbol{r}_n)}{(\boldsymbol{p}_n^*, A\boldsymbol{p}_n)}$

4: $\quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n$

5: $\quad \boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n, \quad \boldsymbol{r}_{n+1}^* = \boldsymbol{r}_n^* - \bar{\alpha}_n A^{\mathrm{H}}\boldsymbol{p}_n^*$

6: $\quad \beta_n = \dfrac{(\boldsymbol{r}_{n+1}^*, \boldsymbol{r}_{n+1})}{(\boldsymbol{r}_n^*, \boldsymbol{r}_n)}$

7: **end for**

由 Proposition 3.5, 下列性质成立:

Corollary 3.6: 设 $\boldsymbol{p}_n, \boldsymbol{r}_n$ 为 BiCG 方法中的向量. 则,

1. $\boldsymbol{r}_n \perp \mathcal{K}_n(A^{\mathrm{H}}, \boldsymbol{r}_0^*)$,
2. $A\boldsymbol{p}_n \perp \mathcal{K}_n(A^{\mathrm{H}}, \boldsymbol{r}_0^*)$.

下面给出 BiCG 方法中 α_n 和 β_n 的其他计算公式, 它们将用于推导 Section 3.3.8 中的乘积型 Krylov 子空间方法. 由于 $\boldsymbol{r}_n^* \in \mathcal{K}_{n+1}(A^{\mathrm{H}}, \boldsymbol{r}_0^*)$, 向量 $\boldsymbol{r}_n^*$ 可写为

$$\begin{aligned}\boldsymbol{r}_n^* &= c_n(A^{\mathrm{H}})^n\boldsymbol{r}_0^* + \underbrace{c_{n-1}(A^{\mathrm{H}})^{n-1}\boldsymbol{r}_0^* + \cdots + c_0\boldsymbol{r}_0^*}_{\boldsymbol{z}} \\ &= c_n(A^{\mathrm{H}})^n\boldsymbol{r}_0^* + \boldsymbol{z}, \quad \boldsymbol{z} \in \mathcal{K}_n(A^{\mathrm{H}}, \boldsymbol{r}_0^*).\end{aligned}$$

由上式及 Corollary 3.6 得

$$\begin{aligned}(\boldsymbol{r}_n^*, \boldsymbol{r}_n) &= (c_n(A^{\mathrm{H}})^n\boldsymbol{r}_0^* + \boldsymbol{z}, \boldsymbol{r}_n) = \bar{c}_n((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, \boldsymbol{r}_n) + (\boldsymbol{z}, \boldsymbol{r}_n) \\ &= \bar{c}_n((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, \boldsymbol{r}_n) \\ (\boldsymbol{r}_n^*, A\boldsymbol{p}_n) &= (c_n(A^{\mathrm{H}})^n\boldsymbol{r}_0^* + \boldsymbol{z}, A\boldsymbol{p}_n) = \bar{c}_n((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, A\boldsymbol{p}_n) + (\boldsymbol{z}, A\boldsymbol{p}_n) \\ &= \bar{c}_n((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, A\boldsymbol{p}_n).\end{aligned}$$

因此, BiCG 方法 (Algorithm 3.12) 的 α_n 可改写为

$$\alpha_n = \frac{(\boldsymbol{r}_n^*, \boldsymbol{r}_n)}{(\boldsymbol{p}_n^*, A\boldsymbol{p}_n)} = \frac{((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, \boldsymbol{r}_n)}{((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, A\boldsymbol{p}_n)}. \tag{3.59}$$

类似地, 给出 β_n 的其他计算公式. 由 Corollary 3.6, 两个向量 $(A^{\mathrm{H}})^n\boldsymbol{r}_0^* \in \mathcal{K}_{n+1}(A^{\mathrm{H}}, \boldsymbol{r}_0^*)$ 和 $A\boldsymbol{p}_{n+1}$ 正交, 因此

$$\begin{aligned}0 &= ((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, A\boldsymbol{p}_{n+1}) \\ &= ((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, A\boldsymbol{r}_{n+1} + \beta_n A\boldsymbol{p}_n) \\ &= ((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, A\boldsymbol{r}_{n+1}) + \beta_n((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, A\boldsymbol{p}_n) \\ &= ((A^{\mathrm{H}})^{n+1}\boldsymbol{r}_0^*, \boldsymbol{r}_{n+1}) + \beta_n((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, A\boldsymbol{p}_n),\end{aligned}$$

立即得到

$$\beta_n = -\frac{((A^{\mathrm{H}})^{n+1}\boldsymbol{r}_0^*, \boldsymbol{r}_{n+1})}{((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, A\boldsymbol{p}_n)}.$$

结合 (3.59) 与上述结果, 得

$$\beta_n = -\alpha_n \frac{((A^{\mathrm{H}})^{n+1}\boldsymbol{r}_0^*, \boldsymbol{r}_{n+1})}{((A^{\mathrm{H}})^n\boldsymbol{r}_0^*, \boldsymbol{r}_n)}. \tag{3.60}$$

3.3.2 复合步双共轭梯度 (CSBiCG) 方法

由 BiCG 方法的算法 (Algorithm 3.12), 有两类可能的 (近) 中断:

1. $\rho_n := (\boldsymbol{r}_n^*, \boldsymbol{r}_n) = 0$ 而 $\boldsymbol{r}_n \neq \boldsymbol{0}$ (Lanczos 中断),
2. $\sigma_n := (\boldsymbol{p}_n^*, A\boldsymbol{p}_n) = 0$ (主元中断).

Lanczos 中断可通过 [145], [189] 中的前瞻 Lanczos 过程避免. Bank 和 Chan 关注主元中断, 在假设不发生 Lanczos 中断的条件下, 提出复合步双共轭梯度 (CSBiCG) 方法来消除主元中断 [16], [17]. 下面根据 [17] 对该算法作简要说明. 若 $\sigma_n = 0$, 就无法得到 $\boldsymbol{r}_{n+1} = \boldsymbol{r}_n - (\rho_n/\sigma_n)A\boldsymbol{p}_n$ 和 $\boldsymbol{r}_{n+1}^* = \boldsymbol{r}_n^* - \overline{(\rho_n/\sigma_n)}A^{\mathrm{H}}\boldsymbol{p}_n^*$. 考虑辅助向量:

$$\begin{aligned}\boldsymbol{z}_{n+1} &= \sigma_n\boldsymbol{r}_n - \rho_n A\boldsymbol{p}_n,\\ \boldsymbol{z}_{n+1}^* &= \bar{\sigma}_n\boldsymbol{r}_n - \bar{\rho}_n A^{\mathrm{H}}\boldsymbol{p}_n^*.\end{aligned}$$

于是即使 $\sigma_n = 0$, $\boldsymbol{z}_{n+1}$ 和 $\boldsymbol{z}_{n+1}^*$ 也存在, 且 $\boldsymbol{z}_{n+1}$ 和 $\boldsymbol{z}_{n+1}^*$ 分别属于 $\mathcal{K}_{n+2}(A, \boldsymbol{r}_0)$ 和 $\mathcal{K}_{n+2}(A^{\mathrm{H}}, \boldsymbol{r}_0^*)$. 随后复合步技术按下式考虑 $\boldsymbol{r}_{n+2}$ 和 $\boldsymbol{r}_{n+2}^*$:

$$\begin{aligned}\boldsymbol{r}_{n+2} &= \boldsymbol{r}_n - c_1 A\boldsymbol{p}_n - c_2 A\boldsymbol{z}_{n+1},\\ \boldsymbol{r}_{n+2}^* &= \boldsymbol{r}_n^* - c_1^* A^{\mathrm{H}}\boldsymbol{p}_n^* - c_2^* A^{\mathrm{H}}\boldsymbol{z}_{n+1}^*,\end{aligned}$$

其中 c_1, c_2, c_1^*, c_2^* 确定为使得

$$\boldsymbol{r}_{n+2} \perp \operatorname{span}\{\boldsymbol{p}_n^*, \boldsymbol{z}_{n+1}^*\}, \qquad \boldsymbol{r}_{n+2}^* \perp \operatorname{span}\{\boldsymbol{p}_n, \boldsymbol{z}_{n+1}\},$$

即 $(\boldsymbol{r}_{n+2}, \boldsymbol{p}_n^*) = (\boldsymbol{r}_{n+2}, \boldsymbol{z}_{n+1}^*) = (\boldsymbol{r}_{n+2}^*, \boldsymbol{p}_n) = (\boldsymbol{r}_{n+2}^*, \boldsymbol{z}_{n+1}) = 0$. 类似地, 令

$$\begin{aligned}\boldsymbol{p}_{n+2} &= \boldsymbol{r}_{n+2} + d_1\boldsymbol{p}_n + d_2\boldsymbol{z}_{n+1},\\ \boldsymbol{p}_{n+2}^* &= \boldsymbol{r}_{n+2}^* + d_1^*\boldsymbol{p}_n^* + d_2^*\boldsymbol{z}_{n+1}^*,\end{aligned}$$

并确定 d_1, d_2, d_1^*, d_2^* 使得

$$A\boldsymbol{p}_{n+2} \perp \operatorname{span}\{\boldsymbol{p}_n^*, \boldsymbol{z}_{n+1}^*\}, \qquad A^{\mathrm{H}}\boldsymbol{p}_{n+2}^* \perp \operatorname{span}\{\boldsymbol{p}_n, \boldsymbol{z}_{n+1}\}.$$

于是 [17], Theorem 4.4 说明, 当 $i \neq j$ 时有 $(\boldsymbol{r}_i^*, \boldsymbol{r}_j) = (\boldsymbol{p}_i^*, A\boldsymbol{p}_j) = 0$, 这意味着复合步残差向量 $\boldsymbol{r}_{n+2}$ 和 $A\boldsymbol{p}_{n+2}$ 满足

$$\boldsymbol{r}_{n+2} \perp \mathcal{K}_{n+2}(A^{\mathrm{H}}, \boldsymbol{r}_0^*), \quad A\boldsymbol{p}_{n+2} \perp \mathcal{K}_{n+2}(A^{\mathrm{H}}, \boldsymbol{r}_0^*).$$

这些与 BiCG 方法具有的性质相同, 见 Corollary 3.6. 所得算法见 Algorithm 3.13. 由 Algorithm 3.13, 矩阵向量乘法的次数与 BiCG 方法相同. 复合步技术也用于其他 Krylov 子空间方法 [35], [113]. 在 Algorithm 3.13 的每一步, 需要选择 1×1 步 (BiCG 步) 或 2×2 步 (复合步). 文献 [17] 提出如下选择:

if $\|\boldsymbol{z}_{n+1}\| \le \|\boldsymbol{r}_n\||\sigma_n|$ **then**
    1×1 步
else
    $\nu_{n+2} = \|\delta_n \boldsymbol{r}_n - \rho_n^3 \zeta_{n+1} \boldsymbol{q}_n - \theta_{n+1} \rho_n^2 \boldsymbol{y}_{n+1}\|$
    if $\nu_{n+2}|\sigma_n| < \|\boldsymbol{z}_{n+1}\||\delta_n|$ **then**
        2×2 步
    else
        1×1 步
    end if
end if

这一选择不仅避免 (近) 主元中断, 也使残差范数的收敛历程得到一定程度的平滑.

Algorithm 3.13 复合步 BiCG 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Input: 选择 $\boldsymbol{r}_0^* \in \mathbb{C}^N$, 例如 $\boldsymbol{r}_0^* = \boldsymbol{r}_0$

Output: $\boldsymbol{x}_n$

$\boldsymbol{p}_0 = \boldsymbol{r}_0, \quad \boldsymbol{p}_0^* = \boldsymbol{r}_0^*$

$\boldsymbol{q}_0 = A\boldsymbol{p}_0, \quad \boldsymbol{q}_0^* = A^{\mathrm{H}}\boldsymbol{p}_0^*$

$\rho_0 = (\boldsymbol{r}_0^*, \boldsymbol{r}_0)$

$n = 0$

while 直至收敛 **do**

  $\sigma_n = (\boldsymbol{p}_n^*, \boldsymbol{q}_n)$

  $\boldsymbol{z}_{n+1} = \sigma_n \boldsymbol{r}_n - \rho_n \boldsymbol{q}_n, \quad \boldsymbol{z}_{n+1}^* = \bar{\sigma}_n \boldsymbol{r}_n^* - \bar{\rho}_n \boldsymbol{q}_n^*$

  $\boldsymbol{y}_{n+1} = A\boldsymbol{z}_{n+1}, \quad \boldsymbol{y}_{n+1}^* = A^{\mathrm{H}}\boldsymbol{z}_{n+1}^*$

  $\theta_{n+1} = (\boldsymbol{z}_{n+1}^*, \boldsymbol{z}_{n+1})$

  $\zeta_{n+1} = (\boldsymbol{z}_{n+1}^*, \boldsymbol{y}_{n+1})$

  if 1×1 步 **then**

    $\alpha = \rho_n / \sigma_n$

    $\rho_{n+1} = \theta_{n+1} / \sigma_n^2$

    $\beta_{n+1} = \rho_{n+1} / \rho_n$

    $\boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n$

    $\boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n \boldsymbol{q}_n, \quad \boldsymbol{r}_{n+1}^* = \boldsymbol{r}_n^* - \bar{\alpha}_n \boldsymbol{q}_n^*$

    $\boldsymbol{p}_{n+1} = \boldsymbol{z}_{n+1}/\sigma_n + \beta_{n+1}\boldsymbol{p}_n, \quad \boldsymbol{p}_{n+1}^* = \boldsymbol{z}_{n+1}^*/\bar{\sigma}_n + \bar{\beta}_{n+1}\boldsymbol{p}_n^*$

    $\boldsymbol{q}_{n+1} = \boldsymbol{y}_{n+1}/\sigma_n + \beta_{n+1}\boldsymbol{q}_n, \quad \boldsymbol{q}_{n+1}^* = \boldsymbol{y}_{n+1}^*/\bar{\sigma}_n + \bar{\beta}_{n+1}\boldsymbol{q}_n^*$

    $n = n + 1$

  end if

  if 2×2 步 **then**

    $\delta_n = \sigma_n \zeta_{n+1} \rho_n^2 - \theta_{n+1}^2$

    $\alpha_n = \zeta_{n+1}\rho_n^3/\delta_n, \quad \alpha_{n+1} = \theta_{n+1}\rho_n^2/\delta_n$

    $\boldsymbol{x}_{n+2} = \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n + \alpha_{n+1}\boldsymbol{z}_{n+1}$

    $\boldsymbol{r}_{n+2} = \boldsymbol{r}_n - \alpha_n \boldsymbol{q}_n - \alpha_{n+1}\boldsymbol{y}_{n+1}, \quad \boldsymbol{r}_{n+2}^* = \boldsymbol{r}_n^* - \bar{\alpha}_n \boldsymbol{q}_n^* - \bar{\alpha}_{n+1}\boldsymbol{y}_{n+1}$

    $\rho_{n+2} = (\boldsymbol{r}_{n+2}^*, \boldsymbol{r}_{n+2})$

    $\beta_{n+1} = \rho_{n+2}/\rho_n, \quad \beta_{n+2} = \rho_{n+2}\sigma_n/\theta_{n+1}$

    $\boldsymbol{p}_{n+2} = \boldsymbol{r}_{n+2} + \beta_{n+1}\boldsymbol{p}_n + \beta_{n+2}\boldsymbol{z}_{n+1}, \quad \boldsymbol{p}_{n+2}^* = \boldsymbol{r}_{n+2}^* + \bar{\beta}_{n+1}\boldsymbol{p}_n^* + \bar{\beta}_{n+2}\boldsymbol{z}_{n+1}^*$

    $\boldsymbol{q}_{n+2} = A\boldsymbol{p}_{n+2}, \quad \boldsymbol{q}_{n+2}^* = A^{\mathrm{H}}\boldsymbol{p}_{n+2}^*$

    $n = n + 2$

  end if

end while

下述定理给出 (复合步) BiCG 方法的最佳逼近结果:

Theorem 3.7:

[[16]] 考虑对实非对称线性方程组应用 (复合步) BiCG 方法, 初始猜测为 $\boldsymbol{x}_0 = \boldsymbol{0}$, 且 $\boldsymbol{r}_0^* = \boldsymbol{r}_0$. 假设对所有 $\boldsymbol{v}, \boldsymbol{w} \in \mathbb{R}^N$, 有

$$|\boldsymbol{w}^\top A\boldsymbol{v}| \le \Gamma\|\boldsymbol{v}\|\|\boldsymbol{w}\|,$$

其中 Γ 为与 $\boldsymbol{v}$ 和 $\boldsymbol{w}$ 无关的常数. 进一步假设, 对于 (复合步) BiCG 方法中计算近似解 $\boldsymbol{x}_n$ 的那些步骤, 有

$$\inf_{\substack{\boldsymbol{v}\in\mathcal{K}_n(A,\boldsymbol{r}_0)\\ \|\boldsymbol{v}\|=1}} \sup_{\substack{\boldsymbol{w}\in\mathcal{K}_n(A^\top,\boldsymbol{r}_0^*)\\ \|\boldsymbol{w}\|\le 1}} \boldsymbol{w}^\top A\boldsymbol{v} \ge \delta_n \ge \delta > 0.$$

则

$$\|\boldsymbol{x} - \boldsymbol{x}_n\| \le (1 + \Gamma/\delta) \inf_{\boldsymbol{v}\in\mathcal{K}_n(A,\boldsymbol{r}_0)} \|\boldsymbol{x} - \boldsymbol{v}\|.$$

Theorem 3.7 是 [16] 中 Theorem 4.1 的简化结果. 由 Theorem 3.7 可见, 当 Γ/δ 接近 0 时, (复合步) BiCG 方法产生在误差范数下接近最佳近似解的近似解.

3.3.3 双共轭残差 (BiCR) 方法

BiCR 方法 [169] 是 CR 方法向非 Hermitian 线性方程组的推广.[2] 本小节说明 BiCR 方法可由 Algorithm 3.10 中的预条件 COCR 方法导出. 采用预条件子 (3.58), 对 (3.57) 应用 COCR 算法, 所得算法在第 n 次迭代时可写为

$$\begin{aligned}
\widetilde{\boldsymbol{p}}_n^{\mathrm{COCR}} &= \widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_n^{\mathrm{COCR}} + \beta_{n-1}\widetilde{\boldsymbol{p}}_{n-1}^{\mathrm{COCR}},\\
\alpha_n &= \frac{(\overline{\widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_n^{\mathrm{COCR}}}, \widetilde{A}\widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_n^{\mathrm{COCR}})}{(\overline{\widetilde{A}\widetilde{\boldsymbol{p}}_n^{\mathrm{COCR}}}, \widetilde{M}^{-1}\widetilde{A}\widetilde{\boldsymbol{p}}_n^{\mathrm{COCR}})},\\
\widetilde{\boldsymbol{x}}_{n+1}^{\mathrm{COCR}} &= \widetilde{\boldsymbol{x}}_n^{\mathrm{COCR}} + \alpha_n\widetilde{\boldsymbol{p}}_n^{\mathrm{COCR}},\\
\widetilde{\boldsymbol{r}}_{n+1}^{\mathrm{COCR}} &= \widetilde{\boldsymbol{r}}_n^{\mathrm{COCR}} - \alpha_n\widetilde{A}\widetilde{\boldsymbol{p}}_n^{\mathrm{COCR}},\\
\beta_n &= \frac{(\overline{\widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_{n+1}^{\mathrm{COCR}}}, \widetilde{A}\widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_{n+1}^{\mathrm{COCR}})}{(\overline{\widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_n^{\mathrm{COCR}}}, \widetilde{A}\widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_n^{\mathrm{COCR}})}.
\end{aligned}$$

将 (3.58) 中的 $\widetilde{M}^{-1}(=\widetilde{M})$ 以及向量

$$\widetilde{\boldsymbol{x}}_n^{\mathrm{COCR}} := \begin{bmatrix}\boldsymbol{x}_n\\ \bar{\boldsymbol{x}}_n^*\end{bmatrix}, \quad \widetilde{\boldsymbol{r}}_n^{\mathrm{COCR}} := \begin{bmatrix}\boldsymbol{r}_n\\ \bar{\boldsymbol{r}}_n^*\end{bmatrix}, \quad \widetilde{\boldsymbol{p}}_n^{\mathrm{COCR}} := \begin{bmatrix}\bar{\boldsymbol{p}}_n^*\\ \boldsymbol{p}_n\end{bmatrix}$$

[2] Chronopulous 和 Ma 于 1989 年的未发表论文 [38] 中提到了该方法, Chronopulous 博士于 2020 年 4 月 6 日通过电子邮件将这一事实告知作者.

代入前述递推式, 并使用下列结果:

$$
\begin{aligned}
(\overline{\widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_n^{\mathrm{COCR}}}, \widetilde{A}\widetilde{M}^{-1}\widetilde{\boldsymbol{r}}_n^{\mathrm{COCR}}) &= \begin{bmatrix} \bar{\boldsymbol{r}}_n^{*\top} & \boldsymbol{r}_n^{\top} \end{bmatrix} \begin{bmatrix} A\boldsymbol{r}_n \\ A^{\top}\bar{\boldsymbol{r}}_n^{*} \end{bmatrix} \\
&= \bar{\boldsymbol{r}}_n^{*\top} A\boldsymbol{r}_n + \boldsymbol{r}_n^{\top} A^{\top}\bar{\boldsymbol{r}}_n^{*} \\
&= \boldsymbol{r}_n^{*\mathrm{H}} A\boldsymbol{r}_n + (\boldsymbol{r}_n^{\top} A^{\top}\bar{\boldsymbol{r}}_n^{*})^{\top} \\
&= (\boldsymbol{r}_n^{*}, A\boldsymbol{r}_n) + \boldsymbol{r}_n^{*\mathrm{H}} A\boldsymbol{r}_n \\
&= 2(\boldsymbol{r}_n^{*}, A\boldsymbol{r}_n)
\end{aligned}
$$

以及

$$
\begin{aligned}
(\overline{\widetilde{A}\widetilde{\boldsymbol{p}}_n^{\mathrm{COCR}}}, \widetilde{M}^{-1}\widetilde{A}\widetilde{\boldsymbol{p}}_n^{\mathrm{COCR}}) &= \begin{bmatrix} (A\boldsymbol{p}_n)^{\top} & (A^{\top}\bar{\boldsymbol{p}}_n^{*})^{\top} \end{bmatrix} \begin{bmatrix} A^{\top}\bar{\boldsymbol{p}}_n^{*} \\ A\boldsymbol{p}_n \end{bmatrix} \\
&= (A\boldsymbol{p}_n)^{\top} A^{\top}\bar{\boldsymbol{p}}_n^{*} + (A^{\top}\bar{\boldsymbol{p}}_n^{*})^{\top} A\boldsymbol{p}_n \\
&= (\boldsymbol{p}_n^{\top} A^{\top} A^{\top}\bar{\boldsymbol{p}}_n^{*})^{\top} + (A^{\mathrm{H}}\boldsymbol{p}_n^{*})^{\mathrm{H}} A\boldsymbol{p}_n \\
&= (A^{\mathrm{H}}\boldsymbol{p}_n^{*})^{\mathrm{H}} A\boldsymbol{p}_n + (A^{\mathrm{H}}\boldsymbol{p}_n^{*}, A\boldsymbol{p}_n) \\
&= 2(A^{\mathrm{H}}\boldsymbol{p}_n^{*}, A\boldsymbol{p}_n),
\end{aligned}
$$

就容易得到求解非 Hermitian 线性方程组的 BiCR 算法.

Algorithm 3.14 BiCR 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N$, $\beta_{-1} = 0$, $\boldsymbol{p}_{-1} = \boldsymbol{p}_{-1}^{*} = \boldsymbol{0}$, $\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Input: 选择 $\boldsymbol{r}_0^{*} \in \mathbb{C}^N$, 例如 $\boldsymbol{r}_0^{*} = \boldsymbol{r}_0$

Output: $\boldsymbol{x}_n$

1: **for** $n = 0, 1, 2, \ldots,$ 直至收敛 **do**

2: $\quad \boldsymbol{p}_n = \boldsymbol{r}_n + \beta_{n-1}\boldsymbol{p}_{n-1}, \quad \boldsymbol{p}_n^{*} = \boldsymbol{r}_n^{*} + \bar{\beta}_{n-1}\boldsymbol{p}_{n-1}^{*}$

3: $\quad A\boldsymbol{p}_n = A\boldsymbol{r}_n + \beta_{n-1}A\boldsymbol{p}_{n-1}$

4: $\quad \alpha_n = \dfrac{(\boldsymbol{r}_n^{*}, A\boldsymbol{r}_n)}{(A^{\mathrm{H}}\boldsymbol{p}_n^{*}, A\boldsymbol{p}_n)}$

5: $\quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n\boldsymbol{p}_n$

6: $\quad \boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n, \quad \boldsymbol{r}_{n+1}^{*} = \boldsymbol{r}_n^{*} - \bar{\alpha}_n A^{\mathrm{H}}\boldsymbol{p}_n^{*}$

7: $\quad \beta_n = \dfrac{(\boldsymbol{r}_{n+1}^{*}, A\boldsymbol{r}_{n+1})}{(\boldsymbol{r}_n^{*}, A\boldsymbol{r}_n)}$

8: **end for**

与 Proposition 3.5 类似, BiCR 迭代具有以下性质:

Proposition 3.8: 设 $\boldsymbol{p}_n, \boldsymbol{r}_n$ 为 BiCR 方法中的向量. 则,

1. $(\boldsymbol{r}_i^{*}, A\boldsymbol{r}_j) = 0 \quad \text{for } i \neq j,$
2. $(A^{\mathrm{H}}\boldsymbol{p}_i^{*}, A\boldsymbol{p}_j) = 0 \quad \text{for } i \neq j.$

若系数矩阵 A 为 Hermitian 矩阵或实对称矩阵, 则选择 $\boldsymbol{r}_0^{*} = \boldsymbol{r}_0$ 就退化为 CR 方法, 后者产生残差 2-范数意义下的最优近似解. 因此, 若 A 接近 Hermitian 矩阵或实对称矩阵, 可以预期 BiCR 方法具有平滑的收敛行为, 并产生接近最小残差的解. 为说明这一点, 考虑非对称线性方程组 (2.18), 参数为 $N = 20$, $a_1 = a_2 = a_3 = 1$, 两组不同的数值 $b_1 = b_2 = b_3 = 0.1$ (接

近对称的矩阵) 和 $b_1 = b_2 = b_3 = 10$ (远离对称的矩阵), 以及 $c = 700$. 横轴为矩阵向量乘法的次数. BiCG 方法和 BiCR 方法所用的停止准则为 $\|\boldsymbol{r}_n\|/\|\boldsymbol{b}\| \leq 10^{-10}$. 当 $b_1 = b_2 = b_3 = 0.1$ (接近对称的矩阵) 时的结果见 Figure 3.1. 从 Figure 3.1 可见, BiCR 方法表现出的收敛行为比 BiCG 方法平滑得多. 当 $b_1 = b_2 = b_3 = 0.1$ (远离对称的矩阵) 时的结果见 Figure 3.2. 从 Figure 3.2 可见, BiCG 方法和 BiCR 方法具有相似的收敛行为, BiCR 方法未表现出平滑的收敛行为.

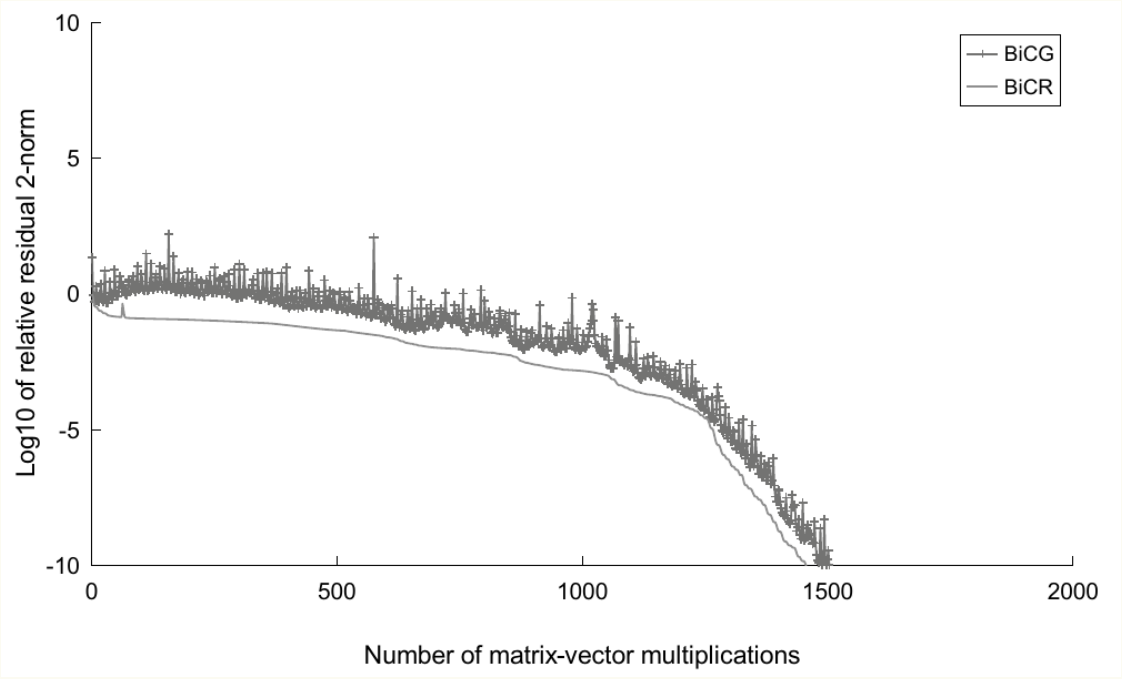


图 3.1: 当 $b_1 = b_2 = b_3 = 0.1$ 时 BiCG 和 BiCR 的收敛历程

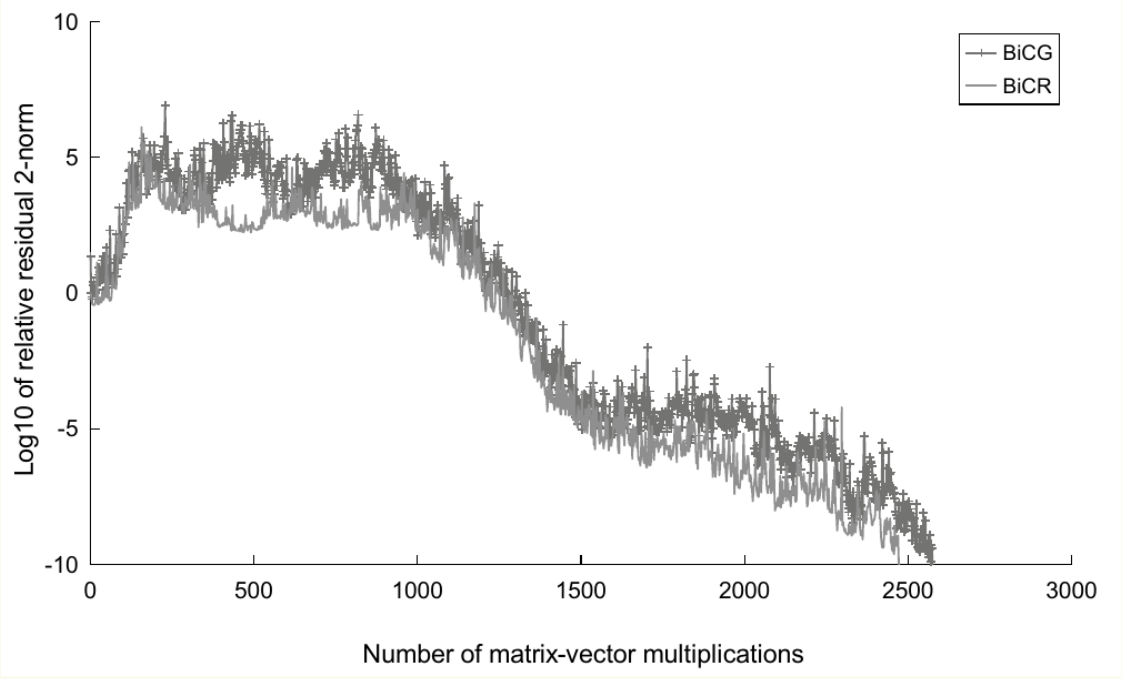


图 3.2: 当 $b_1 = b_2 = b_3 = 10$ 时 BiCG 和 BiCR 的收敛历程

3.3.4 准最小残差 (QMR) 方法

本小节介绍准最小残差 (QMR) 方法 [66] 的思想. QMR 方法的推导过程与 Section 3.2.3

中 QMR_SYM 方法的推导过程几乎相同. 主要区别是不采用复对称 Lanczos 过程, 而采用双 Lanczos 过程. 设 V_n 为通过 Algorithm 1.9 给出的双 Lanczos 过程得到的 $\mathcal{K}_n(A, \boldsymbol{r}_0)$ 的双正交基. 则 QMR 方法寻找近似解 $\boldsymbol{x}_n$, 使 $\boldsymbol{x}_n$ 位于仿射空间 $\boldsymbol{x}_0 + \mathcal{K}_n(A, \boldsymbol{r}_0)$ 中, 即,

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + V_n \boldsymbol{y}_n, \quad \boldsymbol{y}_n \in \mathbb{C}^n.$$

相应的残差向量为

$$\boldsymbol{r}_n = \boldsymbol{r}_0 - AV_n \boldsymbol{y}_n.$$

由双 Lanczos 过程的矩阵形式 (1.40) 得

$$\boldsymbol{r}_n = \boldsymbol{r}_0 - V_{n+1} T_{n+1,n} \boldsymbol{y}_n = V_{n+1}(\beta \boldsymbol{e}_1 - T_{n+1,n} \boldsymbol{y}_n),$$

其中 $\beta := \|\boldsymbol{r}_0\|$. 上述推导过程与 MINRES 方法的推导过程很相似; 然而, 在此情形下, 若选择 $\boldsymbol{y}_n$ 使残差范数达到极小值, 则可能导致如 Section 3.2.3 所述的大量计算开销和内存需求. 与 QMR_SYM 方法类似, QMR 方法选择 $\boldsymbol{y}_n$ 使得

$$\boldsymbol{y}_n := \arg\min_{\boldsymbol{y} \in \mathbb{C}^n} \|\beta \boldsymbol{e}_1 - T_{n+1,n} \boldsymbol{y}\|. \tag{3.61}$$

这一极小化问题可由 Givens 旋转高效求解. 按照 (3.31) 的求解及 Section 3.2.3, 即得到 QMR 方法, 其算法见 Algorithm 3.15. 最后应注意, Algorithm 3.15 与 BiCG 方法及 BiCR 方法一样, 存在发生中断 (除以零) 的可能性. 在有限精度运算中, 这样的中断很罕见; 但可能发生近中断, 并导致数值不稳定. 因此, 为避免 (近) 中断问题, [66] 中的 QMR 方法使用由 Taylor [189] 和 Parlett 等 [145] 提出的前瞻 Lanczos 过程.

Algorithm 3.15 QMR 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \beta_0 = 0,\ \gamma_0 = 0,\ \boldsymbol{v}_0 = \boldsymbol{0},\ \boldsymbol{w}_0 = \boldsymbol{0},\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Output: $\boldsymbol{x}_n$

$\boldsymbol{g} = (\|\boldsymbol{r}_0\|, 0, \ldots, 0)^\top, \quad \boldsymbol{v}_1 = \boldsymbol{r}_0/\|\boldsymbol{r}_0\|$

for $n = 1, 2, \ldots$ **do**

(双 Lanczos 过程)

$\alpha_n = (\boldsymbol{w}_n, A\boldsymbol{v}_n)$,

$\widetilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \alpha_n \boldsymbol{v}_n - \beta_{n-1}\boldsymbol{v}_{n-1}$

$\widetilde{\boldsymbol{w}}_{n+1} = A^{\mathrm{H}}\boldsymbol{w}_n - \bar{\alpha}_n \boldsymbol{w}_n - \gamma_{n-1}\boldsymbol{w}_{n-1}$

$\gamma_n = \|\widetilde{\boldsymbol{v}}_{n+1}\|_2$

$\boldsymbol{v}_{n+1} = \widetilde{\boldsymbol{v}}_{n+1}/\gamma_n$

$\beta_n = (\widetilde{\boldsymbol{w}}_{n+1}, \boldsymbol{v}_{n+1})$

$\boldsymbol{w}_{n+1} = \widetilde{\boldsymbol{w}}_{n+1}/\bar{\beta}_n$

(Givens 旋转)

for $i = \max\{1, n-2\}, \ldots, n-1$ **do**

$$\begin{bmatrix} t_{i,n} \\ t_{i+1,n} \end{bmatrix} = \begin{bmatrix} c_i & s_i \\ -\bar{s}_i & c_i \end{bmatrix} \begin{bmatrix} t_{i,n} \\ t_{i+1,n} \end{bmatrix}$$

end for

$$c_n = \frac{|t_{n,n}|}{\sqrt{|t_{n,n}|^2 + |t_{n+1,n}|^2}}$$

$$\bar{s}_n = \frac{t_{n+1,n}}{t_{n,n}} c_n$$

$t_{n,n} = c_n t_{n,n} + s_n t_{n+1,n}$

$t_{n+1,n} = 0$

$$\begin{bmatrix} g_n \\ g_{n+1} \end{bmatrix} = \begin{bmatrix} c_n & s_n \\ -\bar{s}_n & c_n \end{bmatrix} \begin{bmatrix} g_n \\ 0 \end{bmatrix}$$

(更新 $\boldsymbol{x}_n$)

$\boldsymbol{p}_n = (\boldsymbol{v}_n - t_{n-2,n}\boldsymbol{p}_{n-2},$

$\quad -t_{n-1,n}\boldsymbol{p}_{n-1})/t_{n,n}$

$\boldsymbol{x}_n = \boldsymbol{x}_{n-1} + g_n \boldsymbol{p}_n$

(检查收敛)

$A\boldsymbol{p}_n = (A\boldsymbol{v}_n - t_{n-2,n}A\boldsymbol{p}_{n-2}$

$\quad -t_{n-1,n}A\boldsymbol{p}_{n-1})/t_{n,n}$

$\boldsymbol{r}_n = \boldsymbol{r}_{n-1} - g_n A\boldsymbol{p}_n$

若 $\|\boldsymbol{r}_n\|/\|\boldsymbol{b}\| \le \epsilon$, 则停止

end for

3.3.5 广义最小残差 (GMRES) 方法

GMRES 方法 [152] 的推导过程与 Section 3.1.3 中的 MINRES 方法密切相关. GMRES 方法生成使 $\|\boldsymbol{b} - A\boldsymbol{x}_n\|$ 在仿射空间 $\boldsymbol{x}_0 + \mathcal{K}_n(A, \boldsymbol{r}_0)$ 上达到极小值的 $\boldsymbol{x}_n$. 这可通过 Section 1.9.1 中的 Arnoldi 过程高效实现. 下面给出 GMRES 方法的推导过程. 设 V_n 为 N 行 n 列的矩阵, 其列为 Arnoldi 过程生成的 $\mathcal{K}_n(A, \boldsymbol{r}_0)$ 的标准正交基向量. 于是, 由于 $\boldsymbol{x}_n$ 位于仿射空间

$\boldsymbol{x}_0 + \mathcal{K}_n(A, \boldsymbol{r}_0)$ 中, 有

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + V_n \boldsymbol{y}_n, \quad \boldsymbol{y}_n \in \mathbb{C}^n.$$

相应的残差向量写为

$$\boldsymbol{r}_n = \boldsymbol{r}_0 - AV_n \boldsymbol{y}_n.$$

由 (1.38) 中 Arnoldi 过程的矩阵表示得

$$\boldsymbol{r}_n = V_{n+1}(\beta \boldsymbol{e}_1 - H_{n+1,n} \boldsymbol{y}_n), \tag{3.62}$$

其中 $\beta := \|\boldsymbol{r}_0\|$. 由于 $V_{n+1}^{\mathrm{H}} V_{n+1} = I_{n+1}$, 即单位矩阵, 残差向量的 2-范数为

$$\|\boldsymbol{r}_n\| = \|V_{n+1}(\beta \boldsymbol{e}_1 - H_{n+1,n} \boldsymbol{y}_n)\| = \|\beta \boldsymbol{e}_1 - H_{n+1,n} \boldsymbol{y}_n\|.$$

因此, 按下式选择 $\boldsymbol{y}_n$ 可使残差向量的 2-范数达到极小值:

$$\boldsymbol{y}_n := \arg\min_{\boldsymbol{y} \in \mathbb{C}^n} \|\beta \boldsymbol{e}_1 - H_{n+1,n} \boldsymbol{y}\|. \tag{3.63}$$

向量 $\boldsymbol{y}_n$ 可通过 Givens 旋转得到. 按照 (3.31) 的求解就得到 GMRES 方法. 下面在 Algorithm 3.16 中描述 GMRES 算法, 它基于改进 Gram–Schmidt 型 Arnoldi 过程 (Algorithm 1.8).

Algorithm 3.16 改进 Gram-Schmidt 型 GMRES 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Output: $\boldsymbol{x}_n$

1: $\boldsymbol{g} = (\|\boldsymbol{r}_0\|, 0, \ldots, 0)^\top, \quad \boldsymbol{v}_1 = \boldsymbol{r}_0/\|\boldsymbol{r}_0\|$
2: **for** $n = 1, 2, \ldots$ **do**
3: (Arnoldi 过程)
4: $\boldsymbol{t} = A\boldsymbol{v}_n$
5: **for** $i = 1, 2, \ldots, n$ **do**
6: $h_{i,n} = (\boldsymbol{v}_i, \boldsymbol{t})$
7: $\boldsymbol{t} = \boldsymbol{t} - h_{i,n}\boldsymbol{v}_i$
8: **end for**
9: $h_{n+1,n} = \|\boldsymbol{t}\|$
10: $\boldsymbol{v}_{n+1} = \boldsymbol{t}/h_{n+1,n}$
11: (Givens 旋转)
12: **for** $i = 1, 2, \ldots, n-1$ **do**
13: $\begin{bmatrix} h_{i,n} \\ h_{i+1,n} \end{bmatrix} = \begin{bmatrix} c_i & s_i \\ -\bar{s}_i & c_i \end{bmatrix} \begin{bmatrix} h_{i,n} \\ h_{i+1,n} \end{bmatrix}$
14: **end for**
15: $c_n = \dfrac{|h_{n,n}|}{\sqrt{|h_{n,n}|^2 + |h_{n+1,n}|^2}}$
16: $\bar{s}_n = \dfrac{h_{n+1,n}}{h_{n,n}} c_n$
17: $h_{n,n} = c_n h_{n,n} + s_n h_{n+1,n}$
18: $h_{n+1,n} = 0$
19: $\begin{bmatrix} g_n \\ g_{n+1} \end{bmatrix} = \begin{bmatrix} c_n & s_n \\ -\bar{s}_n & c_n \end{bmatrix} \begin{bmatrix} g_n \\ 0 \end{bmatrix}$
20: (检查收敛)
21: **if** $|g_{n+1}|/\|\boldsymbol{b}\| \le \epsilon$ **then**
22: $\boldsymbol{x}_n = \boldsymbol{x}_0 + V_n H_n^{-1} \boldsymbol{g}$
23: **end if**
24: **end for**

注意, 在 Algorithm 3.16 第 22 行计算 $\boldsymbol{c} := H_n^{-1}\boldsymbol{g}$, 对应于求解形如 $H_n\boldsymbol{c} = \boldsymbol{g}$ 的线性方程组. 经过 Givens 旋转后, H_n 不再是 Hessenberg 矩阵, 而变为上三角矩阵. 因此该线性方程组可通过 (1.6) 所述的回代容易地求解. 下面介绍 GMRES 方法的收敛性分析.

Proposition 3.9: 设 A 为 N 行 N 列的可对角化矩阵, 即 $A = X\Lambda X^{-1}$, 其中 $\Lambda = \mathrm{diag}\{\lambda_1, \ldots, \lambda_N\}$, 设 $\boldsymbol{r}_n$ 为 m 步 GMRES 残差向量. 则

$$\|\boldsymbol{r}_n\| \le \kappa(X)\epsilon_n\|\boldsymbol{r}_0\|,$$

其中 $\kappa(X) = \|X\|\|X^{-1}\|$, 且

$$\epsilon^{(n)} = \min_{p \in \mathbb{P}_n, p(0)=1} \max_{i=1,\ldots,N} |p(\lambda_i)|.$$

这里 $\mathbb{P}_m$ 表示所有次数至多为 m 的多项式构成的集合.

Proof: GMRES 方法寻找近似解 $\boldsymbol{x}_n$ 使得

$$\min_{\boldsymbol{x}_n \in \boldsymbol{x}_0 + \mathcal{K}_n(A, \boldsymbol{r}_0)} \|\boldsymbol{r}_n\|.$$

由于 $\boldsymbol{r}_n \in \mathcal{K}_{n+1}(A, \boldsymbol{r}_0)$, GMRES 残差向量可写为矩阵多项式形式 $\boldsymbol{r}_n = (I + c_1 A + c_2 A^2 + \cdots + c_n A^n)\boldsymbol{r}_0$, 其中 c_i 通过极小化 $\|\boldsymbol{r}_n\|$ 确定; 等价地, $\boldsymbol{r}_n = p_n^{(\mathrm{opt})}(A)\boldsymbol{r}_0$, 其中 $p_n^{(\mathrm{opt})}(0) = 1$, 且 $p_n^{(\mathrm{opt})}$ 通过极小化 $\|\boldsymbol{r}_n\|$ 确定, 即,

$$\min_{p \in \mathbb{P}_n, p(0)=1} \|p_n(A)\boldsymbol{r}_0\|.$$

因此, 对任意 $p \in \mathbb{P}_n$ 且 $p(0) = 1$, 残差 2-范数 $\|\boldsymbol{r}_n\|$ 有如下界

$$\begin{aligned}
\|\boldsymbol{r}_n\| &= \|p_n^{(\mathrm{opt})}(A)\boldsymbol{r}_0\| \\
&\le \|p_n(A)\boldsymbol{r}_0\| \\
&= \|X p_n(\Lambda) X^{-1}\boldsymbol{r}_0\| \\
&\le \|X\|\|X^{-1}\|\|p_n(\Lambda)\|\|\boldsymbol{r}_0\| \\
&= \kappa(X) \max_{i=1,\ldots,N} |p_n(\lambda_i)|\|\boldsymbol{r}_0\|
\end{aligned}$$

由于上述不等式对任意 $p \in \mathbb{P}_n$ 且 $p(0) = 1$ 都成立, 有

$$\begin{aligned}
\|\boldsymbol{r}_n\| &\le \kappa(X) \left(\min_{p \in \mathbb{P}_n, p(0)=1} \max_{i=1,\ldots,N} |p(\lambda_i)| \right) \|\boldsymbol{r}_0\| \\
&= \kappa(X)\epsilon_n \|\boldsymbol{r}_0\|,
\end{aligned}$$

证明完毕.

■

由 Proposition 3.9, 若 X 的条件数较小, 则 ϵ_n 的值高度依赖于收敛速度. 进一步, 若 X 的条件数较小且特征值良好聚集, 则 GMRES 方法表现出良好的收敛行为. 若矩阵 A 对称, 则 X 的条件数为 1. 因此 GMRES 方法 (或 MINRES 方法) 的收敛性仅依赖于特征值的分布. 使用 ϵ_n 的一个上界, 即可不使用全部特征值而估计收敛行为. 详情见 Saad 的优秀著作 [151], 第 206–207 页. GMRES 方法的良好特点在于, 与基于 BiCG 方法和 BiCR 方法的 Krylov 子空间方法不同, GMRES 方法从不发生中断, 残差 2-范数单调下降, 且具有残差 2-范数意义下的最优性. 另一方面, GMRES 方法的一个缺点是其计算开销和内存需求随迭代次数线性增加. 例如, 若迭代次数为 10000, 则需存储一个 $N \times 10000$ 矩阵, 而 (3.63) 中最小二乘问题的规模为 10001×10000. 为克服这一缺点, 提出了重启的 GMRES 方法, 见 Algorithm 3.17. 记为 GMRES(m) 的重启 GMRES 方法的思想是, 运行 GMRES 方法直至达到预定最大迭代次数 m, 然后以此前 GMRES 方法在第 m 次迭代产生的近似解作为初始猜测, 重新启动 GMRES 方法.

Algorithm 3.17 GMRES(m) 方法

Input: 初始猜测 $\boldsymbol{x}_0 \in \mathbb{C}^N$ 和重启数 m

Output: $\boldsymbol{x}_m$

1: 以 $\boldsymbol{x}_0$ 运行 GMRES 方法 (Algorithm 3.16) 的 m 次迭代, 得到 $\boldsymbol{x}_m$.
2: **while** 收敛 **do**
3: 置 $\boldsymbol{x}_0 = \boldsymbol{x}_m$.
4: 以 $\boldsymbol{x}_0$ 运行 GMRES 方法 (Algorithm 3.16) 的 m 次迭代, 得到 $\boldsymbol{x}_m$.
5: **end while**

考虑 GMRES(m) 方法的重启频率 m 固定的情形. 这意味着反复对线性方程组应用 m 步 GMRES 方法, 直至收敛. 另一方面, 非固定的重启频率是指先运行 GMRES(m_1), 然后运行 GMRES(m_2) 方法, GMRES(m_3) 方法, 如此继续, 直至收敛. 若 $m_1 = m_2 = \cdots = m$, 则对应于固定的重启频率. 文献 [117] 及一些作者研究了对 $k = 1, 2, \ldots$ 确定 m_k 的高效方法, 例如 [14], [137], [176], [211]. GMRES 方法的另一个重要发展是增广. 增广有两种途径: 一种利用此前的近似解, 例如 LGMRES 方法 [15]; 另一种利用对应于模较小特征值的近似特征向量, 例如 GMRES-E 方法 [134]. 下面介绍 LGMRES 方法和 GMRES-E 方法的基本思想. 一组 m 次迭代 (例如 Algorithm 3.17 中第 3 和第 4 行) 称为一个循环, 将 LGMRES 方法在第 j 个循环后的近似解记为 $\boldsymbol{x}_m^{(j)}$. 令 $\boldsymbol{z}^{(k)} := \boldsymbol{x}_m^{(j-1)} - \boldsymbol{x}_m^{(j-2)}$. 则对给定的 l, LGMRES 方法按下列方式寻找近似解:

$$\boldsymbol{x}_m^{(j)} = \boldsymbol{x}_m^{(j-1)} + q_{m-1}^{(j)}(A)\boldsymbol{r}_m^{(j-1)} + \sum_{k=j-l}^{j-1} \alpha_{j,k}(A)\boldsymbol{z}^{(k)},$$

其中 $q_{m-1}^{(j)}(A)$ 为 $m-1$ 次多项式, $\alpha_{j,k}(A)$ 为 k 次多项式. 这些多项式通过极小化 $\boldsymbol{x}_m^{(j)}$ 的残差 2-范数确定. 相关算法例如见 [13], [103], [104], [106]. 将 GMRES-E 方法在第 j 个循环后的近似解记为 $\boldsymbol{x}_m^{(j)}$, $\boldsymbol{p}_k^{(j)}$ 为对应于模第 k 小的近似特征值 (调和 Ritz 值) 的近似特征向量 (调和 Ritz 向量). 则对给定的 p, GMRES-E 方法按下列方式寻找近似解:

$$\boldsymbol{x}_m^{(j)} = \boldsymbol{x}_m^{(j-1)} + q_{m-1}^{(j)}(A)\boldsymbol{r}_m^{(j-1)} + \sum_{k=1}^{p} \beta_{j,k}(A)\boldsymbol{p}_k^{(j-1)},$$

其中 $q_{m-1}^{(j)}(A)$ 为 $m-1$ 次多项式, $\beta_{j,k}(A)$ 为 k 次多项式. 这些多项式通过极小化 $\boldsymbol{x}_m^{(j)}$ 的残差 2-范数确定. 相关算法例如见 [11], [135]. 文献 [31] 将 LGMRES 方法和 GMRES-E 方法结合, 提出了使用切换控制器的 LGMRES-E 方法. 粗略地说, 切换规则是在收敛良好时选择 LGMRES 方法, 在发现收敛缓慢时选择 GMRES-E 方法. 增广并不只用于 GMRES 方法. 事实上, 文献 [74], [89], [90] 建立了包括 BiCG 方法, BiCR 方法及其他重要 Krylov 空间方法在内的 Krylov 子空间方法的增广与消去框架. 最后给出 GMRES 方法和 GMRES(m) 方法的一个数值例子的结果. 考虑非对称线性方程组 (2.18), 参数为 $N = 20$, $a_1 = a_2 = a_3 = 1$, $b_1 = b_2 = b_3 = 1$, 以及 $c = 1$. 结果见 Figure 3.3, 其中 GMRES 方法和 GMRES(m) 方法所用的停止准则为 $\|\boldsymbol{r}_n\|/\|\boldsymbol{b}\| \le 10^{-10}$, 横轴为矩阵向量乘法的次数. 由 Figure 3.3 可见, GMRES 方法, GMRES(10) 方法和 GMRES(20) 方法的残差 2-范数单调下降, 这有理论保证. 从矩阵向量乘法次数看, GMRES 方法优于重启 GMRES 方法 (GMRES(10), GMRES(20)). 对 GMRES 方法而言, 由于它在 Krylov 子空间上寻找最佳近似解, 所需的矩阵向量乘法次数最少. 另一

方面, 如前所述, GMRES 方法的计算开销随迭代次数线性增加. 这意味着, 当所需迭代次数很大时, 矩阵向量乘法可能不再是最耗时的部分. 从 CPU 时间看, GMRES 方法需要 0.39 秒, GMRES(10) 需要 0.31 秒, GMRES(20) 需要 0.26 秒.[3] 因此, 就 CPU 时间而言, GMRES(20) 方法在所有方法中最好, 它平衡了增加的计算开销与所需的迭代次数.

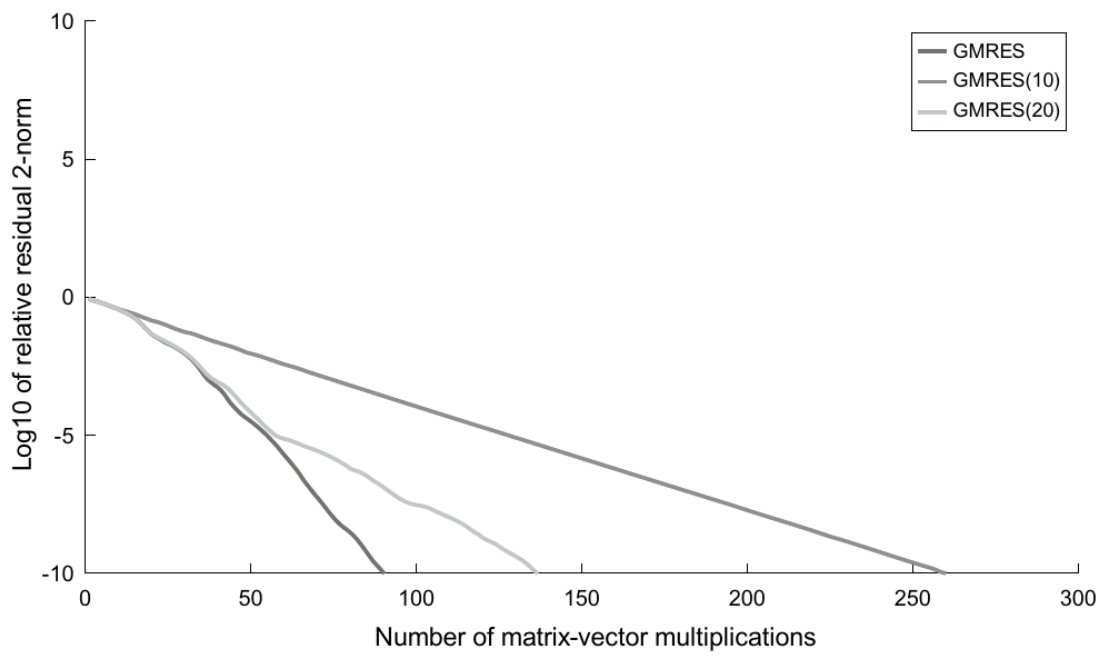


图 3.3: GMRES, GMRES(10) 和 GMRES(20) 的收敛历程

3.3.6 广义共轭残差 (GCR) 方法

广义共轭残差 (GCR) 方法 [54] 是另一种基于 Arnoldi 过程的 Krylov 求解器. 在精确运算中, GCR 方法产生与 GMRES 方法相同的近似解. 该算法由下列以 $A^{\mathrm{H}}A$ 为权且 $\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$ 的 Arnoldi 过程得到: 置 $\boldsymbol{v}_1 = \dfrac{\boldsymbol{r}_0}{\|\boldsymbol{r}_0\|_{A^{\mathrm{H}}A}}$,

对 $n = 1, 2, \ldots,$ 执行:

$$h_{i,n} = (\boldsymbol{v}_i, A\boldsymbol{v}_n)_{A^{\mathrm{H}}A}, \quad i = 1, 2, \ldots, n,$$

$$\widetilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \sum_{i=1}^{n} h_{i,n}\boldsymbol{v}_i, \tag{3.64}$$

$$h_{n+1,n} = \|\widetilde{\boldsymbol{v}}_{n+1}\|_{A^{\mathrm{H}}A},$$

$$\boldsymbol{v}_{n+1} = \frac{\widetilde{\boldsymbol{v}}_{n+1}}{h_{n+1,n}}. \tag{3.65}$$

结束 该过程生成 $A^{\mathrm{H}}A$-标准正交向量, 即,

$$(\boldsymbol{v}_i, A^{\mathrm{H}}A\boldsymbol{v}_j) = (A\boldsymbol{v}_i, A\boldsymbol{v}_j) = \delta_{i,j}.$$

设 V_n 为以 $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$ 为列的矩阵. 则 GCR 方法生成

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + V_n\boldsymbol{y}_n$$

[3]代码: GNU Octave 6.2.0 版, 操作系统: macOS Moterey 12.2.1 版, CPU: Apple M1 pro.

使残差 2-范数 $\|\boldsymbol{r}_n\|$ 达到极小值. 相应的残差向量可写为

$$\boldsymbol{r}_n = \boldsymbol{r}_0 - AV_n\boldsymbol{y}_n = \boldsymbol{r}_0 - \sum_{i=1}^{n} y_i A\boldsymbol{v}_i. \tag{3.66}$$

由上述关系有

$$\boldsymbol{r}_n = \boldsymbol{r}_{n-1} - y_n A\boldsymbol{v}_n. \tag{3.67}$$

未知参数 y_i $(i = 1, 2, \ldots, n)$ 通过极小化残差 2-范数确定, 即由 (3.66), 这可通过下式实现

$$y_i = (A\boldsymbol{v}_i, \boldsymbol{r}_n), \quad \text{for } i = 1, 2, \ldots, n.$$

由于 $A\boldsymbol{v}_i$ 为标准正交向量, 有

$$y_i = (A\boldsymbol{v}_i, \boldsymbol{r}_0), \quad \text{for } i = 1, 2, \ldots, n.$$

这里定义向量 $\boldsymbol{p}_n := -y_n\widetilde{\boldsymbol{v}}_{n+1}$. 则由 (3.64) 得

$$\boldsymbol{p}_n = -y_n A\boldsymbol{v}_n - \sum_{i=1}^{n} (\boldsymbol{v}_i, -y_n A\boldsymbol{v}_n)_{A^{\mathrm{H}}A}\boldsymbol{v}_i. \tag{3.68}$$

由 (3.67) 有 $-y_n A\boldsymbol{v}_n = \boldsymbol{r}_n - \boldsymbol{r}_{n-1}$. 将此关系代入 (3.68), 得

$$\boldsymbol{p}_n = \boldsymbol{r}_n - \boldsymbol{r}_{n-1} - \sum_{i=1}^{n} (\boldsymbol{v}_i, \boldsymbol{r}_n - \boldsymbol{r}_{n-1})_{A^{\mathrm{H}}A}\boldsymbol{v}_i \tag{3.69}$$

$$= \boldsymbol{r}_n - \sum_{i=1}^{n} (\boldsymbol{v}_i, \boldsymbol{r}_n)_{A^{\mathrm{H}}A}\boldsymbol{v}_i - \left(\boldsymbol{r}_{n-1} - \sum_{i=1}^{n} (\boldsymbol{v}_i, \boldsymbol{r}_{n-1})_{A^{\mathrm{H}}A}\boldsymbol{v}_i \right).$$

由于 $\boldsymbol{r}_{n-1}$ 位于 $\mathcal{K}_n(A, \boldsymbol{r}_0)$ 中, 可展开为 $\boldsymbol{r}_{n-1} = \sum_{i=1}^{n} c_i\boldsymbol{v}_i$. 于是,

$$(\boldsymbol{v}_j, \boldsymbol{r}_{n-1})_{A^{\mathrm{H}}A} = \sum_{i=1}^{n} c_i(\boldsymbol{v}_j, \boldsymbol{v}_i) = c_i$$

因为 $(\boldsymbol{v}_j, \boldsymbol{v}_i) = \delta_{j,i}$. 因而有

$$\boldsymbol{r}_{n-1} = \sum_{i=1}^{n} (\boldsymbol{v}_i, \boldsymbol{r}_{n-1})_{A^{\mathrm{H}}A}\boldsymbol{v}_i. \tag{3.70}$$

由 (3.69) 和 (3.70), 向量 $\boldsymbol{p}_n$ 可写为

$$\boldsymbol{p}_n = \boldsymbol{r}_n - \sum_{i=1}^{n} (\boldsymbol{v}_i, \boldsymbol{r}_n)_{A^{\mathrm{H}}A}\boldsymbol{v}_i. \tag{3.71}$$

另一方面, $\boldsymbol{p}_n$ 和 $\boldsymbol{v}_{n+1}$ 的关系为

$$\boldsymbol{p}_n = -\frac{y_n}{|y_n|}\|A\boldsymbol{p}_n\|\boldsymbol{v}_{n+1},$$

因为由 $\boldsymbol{p}_n$ 的定义及 (3.65) 得

$$\begin{aligned}\boldsymbol{p}_n &= -y_n\widetilde{\boldsymbol{v}}_{n+1}\\ &= -y_n\|\widetilde{\boldsymbol{v}}_{n+1}\|_{A^{\mathrm{H}}A}\boldsymbol{v}_{n+1}\\ &= -y_n\sqrt{(A\widetilde{\boldsymbol{v}}_{n+1}, A\widetilde{\boldsymbol{v}}_{n+1})}\,\boldsymbol{v}_{n+1}\\ &= -y_n\sqrt{\left(-\frac{1}{y_n}A\boldsymbol{p}_n, -\frac{1}{y_n}A\boldsymbol{p}_n\right)}\,\boldsymbol{v}_{n+1}\\ &= -\frac{y_n}{|y_n|}\sqrt{(A\boldsymbol{p}_n, A\boldsymbol{p}_n)}\,\boldsymbol{v}_{n+1}.\end{aligned}$$

因此, 使用 $\boldsymbol{v}_n = -(|y_{n-1}|/y_{n-1}) \times \boldsymbol{p}_{n-1}/\sqrt{(A\boldsymbol{p}_{n-1}, A\boldsymbol{p}_{n-1})}$, 有下列关系:

$$\begin{aligned}y_nA\boldsymbol{v}_n &= (A\boldsymbol{v}_n, \boldsymbol{r}_0)A\boldsymbol{v}_n = \frac{(A\boldsymbol{p}_{n-1}, \boldsymbol{r}_0)}{(A\boldsymbol{p}_{n-1}, A\boldsymbol{p}_{n-1})}A\boldsymbol{p}_{n-1},\\ (\boldsymbol{v}_i, \boldsymbol{r}_n)_{A^{\mathrm{H}}A}\boldsymbol{v}_i &= \frac{(A\boldsymbol{p}_{i-1}, A\boldsymbol{r}_n)}{(A\boldsymbol{p}_{i-1}, A\boldsymbol{p}_{i-1})}\boldsymbol{p}_{i-1}.\end{aligned}$$

因此, 将上述结果代入 (3.67) 和 (3.71), 得

$$\begin{aligned}\boldsymbol{r}_n &= \boldsymbol{r}_{n-1} - \frac{(A\boldsymbol{p}_{n-1}, \boldsymbol{r}_0)}{(A\boldsymbol{p}_{n-1}, A\boldsymbol{p}_{n-1})}A\boldsymbol{p}_{n-1},\\ \boldsymbol{p}_n &= \boldsymbol{r}_n - \sum_{i=1}^{n}\frac{(A\boldsymbol{p}_{i-1}, A\boldsymbol{r}_n)}{(A\boldsymbol{p}_{i-1}, A\boldsymbol{p}_{i-1})}\boldsymbol{p}_{i-1}.\end{aligned}$$

现在, 由 $\boldsymbol{p}_n$ 的定义, 对某个常数 c 有 $\boldsymbol{p}_{n-1} = c\boldsymbol{v}_n$. 因而 $\boldsymbol{p}_{n-1}$ 与 $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{n-1}$ 关于 $A^{\mathrm{H}}A$ 正交. 由 (3.66) 得

$$(A\boldsymbol{p}_{n-1}, \boldsymbol{r}_{n-1}) = (A\boldsymbol{p}_{n-1}, \boldsymbol{r}_0) - \sum_{i=1}^{n-1} y_i(A\boldsymbol{p}_{n-1}, A\boldsymbol{v}_i) = (A\boldsymbol{p}_{n-1}, \boldsymbol{r}_0).$$

现在定义 α_{n-1} 和 $\beta_{n-1,i}$ 为

$$\alpha_{n-1} = \frac{(A\boldsymbol{p}_{n-1}, \boldsymbol{r}_{n-1})}{(A\boldsymbol{p}_{n-1}, A\boldsymbol{p}_{n-1})}, \tag{3.72}$$

$$\beta_{n-1,i} = -\frac{(A\boldsymbol{p}_{i-1}, A\boldsymbol{r}_n)}{(A\boldsymbol{p}_{i-1}, A\boldsymbol{p}_{i-1})}. \tag{3.73}$$

于是有

$$\boldsymbol{r}_n = \boldsymbol{r}_{n-1} - \alpha_{n-1}A\boldsymbol{p}_{n-1}, \tag{3.74}$$

$$\boldsymbol{p}_n = \boldsymbol{r}_n + \sum_{i=1}^{n}\beta_{n-1,i}\boldsymbol{p}_{i-1}. \tag{3.75}$$

由残差向量的递推关系得到

$$\boldsymbol{x}_n = \boldsymbol{x}_{n-1} + \alpha_{n-1}\boldsymbol{p}_{n-1}. \tag{3.76}$$

为减少矩阵向量乘法的次数, 使用下列递推式:

$$A\boldsymbol{p}_n = A\boldsymbol{r}_n + \sum_{i=1}^{n}\beta_{n-1,i}A\boldsymbol{p}_{i-1}. \tag{3.77}$$

由 (3.72)–(3.77), 得到 Algorithm 3.18 中的 GCR 算法.

Algorithm 3.18 GCR 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \beta_{-1} = 0,\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0,\ \boldsymbol{p}_0 = \boldsymbol{r}_0$

Output: $\boldsymbol{x}_n$

1: **for** $n = 0, 1, 2, \ldots,$ 直至收敛 **do**

2: $\quad \alpha_n = \dfrac{(A\boldsymbol{p}_n, \boldsymbol{r}_n)}{(A\boldsymbol{p}_n, A\boldsymbol{p}_n)}$

3: $\quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n$

4: $\quad \boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n$

5: $\quad \beta_{n,i} = -\dfrac{(A\boldsymbol{p}_{i-1}, A\boldsymbol{r}_{n+1})}{(A\boldsymbol{p}_{i-1}, A\boldsymbol{p}_{i-1})}, \quad 1 \le i \le n+1$

6: $\quad \boldsymbol{p}_{n+1} = \boldsymbol{r}_{n+1} + \sum_{i=1}^{n+1} \beta_{n,i} \boldsymbol{p}_{i-1}$

7: $\quad (A\boldsymbol{p}_{n+1} = A\boldsymbol{r}_{n+1} + \sum_{i=1}^{n+1} \beta_{n,i} A\boldsymbol{p}_{i-1})$

8: **end for**

下面给出 GCR 方法的性质.

Proposition 3.10: GCR 迭代具有下列性质:

(G1) $(A\boldsymbol{p}_i, A\boldsymbol{p}_j) = 0, \quad i \neq j,$

(G2) $(\boldsymbol{r}_i, A\boldsymbol{p}_j) = 0, \quad i > j,$

(G3) $(\boldsymbol{r}_i, A\boldsymbol{p}_i) = (\boldsymbol{r}_i, A\boldsymbol{r}_i),$

(G4) $(\boldsymbol{r}_i, A\boldsymbol{r}_j) = 0, \quad i > j,$

(G5) $(\boldsymbol{r}_i, A\boldsymbol{p}_i) = (\boldsymbol{r}_0, A\boldsymbol{p}_i), \quad i \geq j.$

证明见 [54], Theorem 3.1. 由上述性质可见, 前面的推导基于性质 (G1). 在精确运算中, GCR 方法和 GMRES 方法一样, 至多在 N 次迭代内收敛. 然而, GCR 方法与 GMRES 方法具有相同的缺点, 即计算工作量和所需内存随迭代次数增加. 因此 [54] 提出了称为 GCR(k) 方法的重启 GCR 版本. 其算法见 Algorithm 3.19. 另一种选择是只保留 k 个方向向量:

$$\boldsymbol{p}_{n+1} = \boldsymbol{r}_{n+1} + \sum_{i=1}^{\min\{k,n+1\}} \beta_{n,i} \boldsymbol{p}_{i-1}.$$

这一方法称为 Vinsome [201] 提出的 Orthomin 方法. 后来 [54] 将 Orthomin 方法称为 Orthomin(k) 方法. Orthomin(k) 方法的算法见 Algorithm 3.20.

Algorithm 3.19 GCR(k) 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0,\ \boldsymbol{p}_0 = \boldsymbol{r}_0$

Output: $\boldsymbol{x}_{k+1}$

for $n = 0, 1, \ldots, k$ 直至收敛 **do**

$\alpha_n = \dfrac{(A\boldsymbol{p}_n, \boldsymbol{r}_n)}{(A\boldsymbol{p}_n, A\boldsymbol{p}_n)}$

$\boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n$

$\boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n$

$\beta_{n,i} = -\dfrac{(A\boldsymbol{p}_{i-1}, A\boldsymbol{r}_{n+1})}{(A\boldsymbol{p}_{i-1}, A\boldsymbol{p}_{i-1})}, \quad 1 \le i \le n+1$

$\boldsymbol{p}_{n+1} = \boldsymbol{r}_{n+1} + \sum_{i=1}^{n+1} \beta_{n,i} \boldsymbol{p}_{i-1}$

$(A\boldsymbol{p}_{n+1} = A\boldsymbol{r}_{n+1} + \sum_{i=1}^{n+1} \beta_{n,i} A\boldsymbol{p}_{i-1})$

end for

$\boldsymbol{x}_0 = \boldsymbol{x}_{k+1}$

重复

Algorithm 3.20 Orthomin(k) 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0,\ \boldsymbol{p}_0 = \boldsymbol{r}_0$

Output: $\boldsymbol{x}_n$

for $n = 0, 1, \ldots,$ 直至 $\|\boldsymbol{r}_n\| \le \epsilon \|\boldsymbol{b}\|$ **do**

$\alpha_n = \dfrac{(A\boldsymbol{p}_n, \boldsymbol{r}_n)}{(A\boldsymbol{p}_n, A\boldsymbol{p}_n)}$

$\boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n$

$\boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n$

$\beta_{n,i} = -\dfrac{(A\boldsymbol{p}_{i-1}, A\boldsymbol{r}_{n+1})}{(A\boldsymbol{p}_{i-1}, A\boldsymbol{p}_{i-1})}, \quad n-k+2 \le i \le n+1$

$\boldsymbol{p}_{n+1} = \boldsymbol{r}_{n+1} + \sum_{i=1}^{\min\{k, n+1\}} \beta_{n,i} \boldsymbol{p}_{i-1}$

$(A\boldsymbol{p}_{n+1} = A\boldsymbol{r}_{n+1} + \sum_{i=1}^{n+1} \beta_{n,i} A\boldsymbol{p}_{i-1})$

end for

GCR 方法, GCR(k) 方法和 Orthomin(k) 方法的收敛性分析见 Proposition 3.11.

Proposition 3.11: 设 M 为 $A \in \mathbb{R}^{N\times N}$ 的对称部分, R 为 A 的斜对称部分, 即 $M = (A + A^\top)/2$, $R = (A - A)^\top/2$. 若 M 正定, 则 GCR 方法, GCR(k) 方法或 Orthomin(k) 方法生成的残差序列 $\{\boldsymbol{r}_n\}$ 满足

$$\|\boldsymbol{r}_n\| \le \left(1 - \frac{\lambda_{\min}(M)^2}{\lambda_{\max}(A^\top A)}\right)^{n/2} \|\boldsymbol{r}_0\|$$

以及

$$\|\boldsymbol{r}_n\| \le \left(1 - \frac{\lambda_{\min}(M)^2}{\lambda_{\min}(M)\lambda_{\max}(M) + \rho(R)^2}\right)^{n/2} \|\boldsymbol{r}_0\|,$$

其中 $\lambda_{\min}(M)$ 和 $\lambda_{\max}(M)$ 为对称正定矩阵 M 的最小和最大特征值, $\rho(R)$ 为 R 的谱半径.

谱半径见 (1.19). 证明见 [54], Theorem 4.4. 由 Proposition 3.11 可见, 若 A 接近 I, 则这三种方法收敛很快, 因为 $\lambda_{\min}(M) \approx \lambda_{\max}(M) \approx 1$ 且 $R \approx O$.

3.3.7 完全正交化方法 (FOM)

FOM [149] 是 CG 方法的推广. 与 CG 方法一样, FOM 寻找满足下列条件的近似解

$$\begin{aligned} &\boldsymbol{x}_n = \boldsymbol{x}_0 + \boldsymbol{z}_n, \quad \boldsymbol{z}_n \in \mathcal{K}_n(A, \boldsymbol{r}_0), \\ &\boldsymbol{r}_n \perp \mathcal{K}_n(A, \boldsymbol{r}_0). \end{aligned}$$

FOM 使用 Section 1.9.1 中的 Arnoldi 方法生成 $\mathcal{K}_n(A, \boldsymbol{r}_0)$ 的标准正交基向量. 因而残差向量 $\boldsymbol{r}_n$ 与 (3.62) 具有相同的形式, 即,

$$\boldsymbol{r}_n = V_{n+1}(\beta \boldsymbol{e}_1 - H_{n+1,n}\boldsymbol{y}_n). \tag{3.78}$$

GMRES 方法与 FOM 的区别在于, GMRES 方法寻找使残差向量的 2-范数 $\|\boldsymbol{r}_n\|$ 达到极小值的 $\boldsymbol{y}_n$, 而 FOM 寻找使残差向量与 n 维 Krylov 子空间正交的 $\boldsymbol{y}_n$, 即 $\boldsymbol{r}_n \perp \mathcal{K}_n(A, \boldsymbol{r}_0)$. 设 $\boldsymbol{v}_i$ 为 V_{n+1} 的第 i 列. 则

$$\begin{aligned} V_n^{\mathrm{H}}\boldsymbol{r}_n = \boldsymbol{0} &\Leftrightarrow \boldsymbol{v}_i^{\mathrm{H}}\boldsymbol{r}_n = 0 \quad \text{for } i = 1, 2, \ldots, n \\ &\Leftrightarrow \boldsymbol{r}_n^{\mathrm{H}}(c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + c_n\boldsymbol{v}_n) = 0 \quad \text{for all } c_i \in \mathbb{C} \\ &\Leftrightarrow \boldsymbol{r}_n \perp \mathcal{K}_n(A, \boldsymbol{r}_0). \end{aligned}$$

注意, 令 $c_i = 1$, 且对 $i \neq j$ 令 $c_j = 0$, 就可说明断言 $\boldsymbol{v}_i^{\mathrm{H}}\boldsymbol{r}_n = 0 \Leftarrow \boldsymbol{r}_n^{\mathrm{H}}(c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + c_n\boldsymbol{v}_n) = 0$. 由此, 寻找使 $\boldsymbol{r}_n \perp \mathcal{K}_n(A, \boldsymbol{r}_0)$ 的 $\boldsymbol{y}_n$, 等价于确定使 $V_n^{\mathrm{H}}\boldsymbol{r}_n = \boldsymbol{0}$ 的 $\boldsymbol{y}_n$. 于是由 (3.78) 得

$$\boldsymbol{0} = V_n^{\mathrm{H}}\boldsymbol{r}_n = V_n^{\mathrm{H}}V_{n+1}(\beta\boldsymbol{e}_1 - H_{n+1,n}\boldsymbol{y}_n) = \beta\boldsymbol{e}_1 - H_{n,n}\boldsymbol{y}_n.$$

因此 $\boldsymbol{y}_n$ 可通过求解下列线性方程组得到:

$$H_{n,n}\boldsymbol{y}_n = \beta\boldsymbol{e}_1.$$

由于 $H_{n,n}$ 为 Hessenberg 矩阵, 可通过 LU 分解以 $O(n^2)$ 的计算开销求解该线性方程组. GMRES 方法出现后, FOM 的吸引力有所降低, 因为与 GMRES 方法不同, FOM 可能因使用不选主元的 LU 分解而发生中断. 另一方面, 在考虑重启时, 用 FOM 求解 Chapter 4 中的移位线性方程组仍有吸引力.

3.3.8 乘积型 Krylov 子空间方法

如 Section 3.3.1 所示, BiCG 方法求解 $A\boldsymbol{x} = \boldsymbol{b}$ 时需要 A^{H}. P. Sonneveld 发现, 将 BiCG 多项式平方可得到一种不使用 A^{H} 信息的算法, 所得算法称为 CGS 方法 [173]. 虽然收敛性取决于线性方程组, 但当 BiCG 方法表现出平滑的收敛行为时, CGS 方法比 BiCG 方法快得多. 然而, CGS 方法有时表现出很不规则的收敛, 可能发生近似解精度损失或发散. H. A. van der Vorst 观察到, BiCG 多项式的平方可替换为一次最小残差多项式与 BiCG 多项式的乘积, 由此得到 BiCGSTAB 方法 [195], 它往往比 CGS 方法具有更平滑的收敛行为. M. H. Gutknecht 提出二次最小残差多项式与 BiCG 多项式相乘, 称为 BiCGSTAB2 方法 [87]. G. L. G. Sleijpen

和 D. R. Fokkema 提出 ℓ 次最小残差多项式与 BiCG 多项式相乘, 称为 BiCGSTAB(ℓ) 方法 [162]. S.-L. Zhang 构造了包含 CGS 方法, BiCGSTAB 方法和 BiCGSTAB2 方法的框架. 在这一框架中, 文献 [212] 提出了 GPBiCG 方法. 这些方法称为 (基于 Bi-CG 方法的) 乘积型方法, 这一名称首先由 S.-L. Zhang 提出. 本节根据 Zhang 的著作 [72] 介绍 Zhang 的框架. 如 (3.18) 所示, CG 方法的残差向量可写为 Lanczos 多项式与初始残差向量 $\boldsymbol{r}_0$ 的乘积. 类似地, BiCG 方法的残差向量也可写为

$$\boldsymbol{r}_n^{\mathrm{BiCG}} = R_n(A)\boldsymbol{r}_0,$$

其中

$$R_0(\lambda) = 1, \quad P_0(\lambda) = 1, \tag{3.79}$$

$$R_n(\lambda) = R_{n-1}(\lambda) - \alpha_{n-1}\lambda P_{n-1}(\lambda), \tag{3.80}$$

$$P_n(\lambda) = R_n(\lambda) + \beta_{n-1}P_{n-1}(\lambda) \quad n = 1, 2, \ldots \tag{3.81}$$

CG 方法与 BiCG 方法的 (矩阵) 多项式表示的区别在于 Lanczos 多项式中的 α_i 和 β_i. 基于 BiCG 方法的乘积型方法的第 n 个残差向量, 定义为 n 次多项式与 BiCG 方法的第 n 个残差向量的乘积, 如下所示:

$$\boldsymbol{r}_n = H_n(A)\boldsymbol{r}_n^{\mathrm{BiCG}} = H_n(A)R_n(A)\boldsymbol{r}_0. \tag{3.82}$$

3.3.8.1 残差多项式的重构

S.-L. Zhang 提出将 (3.82) 中的 H_n 定为下列三项递推关系:

$$H_0(\lambda) := 1, \tag{3.83}$$

$$H_1(\lambda) := (1 - \zeta_0\lambda)H_0(\lambda), \tag{3.84}$$

$$H_n(\lambda) := (1 + \eta_{n-1} - \zeta_{n-1}\lambda)H_{n-1}(\lambda) - \eta_{n-1}H_{n-2}(\lambda), \quad n = 2, 3, \ldots \tag{3.85}$$

可见 H_n 与 Lanczos 多项式 (3.19)–(3.21) 相似. 下面将 H_n 改写为耦合的两项递推关系的形式. 引入 G_{n-1} 为

$$G_{n-1}(\lambda) := \frac{H_{n-1}(\lambda) - H_n(\lambda)}{\lambda},$$

并将 (3.85) 改写为

$$-(H_{n-1}(\lambda) - H_n(\lambda)) = -\zeta_{n-1}\lambda H_{n-1}(\lambda) - \eta_{n-1}(H_{n-2}(\lambda) - H_{n-1}(\lambda)).$$

于是 (3.83)–(3.85) 中的 $H_n(\lambda)$ 可改写为

$$H_0(\lambda) = 1, \quad G_0(\lambda) = \zeta_0, \tag{3.86}$$

$$H_n(\lambda) = H_{n-1}(\lambda) - \lambda G_{n-1}(\lambda), \tag{3.87}$$

$$G_n(\lambda) = \zeta_n H_n(\lambda) + \eta_n G_{n-1}(\lambda), \quad n = 1, 2, \ldots \tag{3.88}$$

3.3.8.2 乘积型方法迭代量的递推公式

由于 (3.79)–(3.81) 及 (3.86)–(3.88) 已给出 R_n, P_n, H_n, G_n 的递推关系, 可以使用这些递推关系, 由前一个残差向量 $\boldsymbol{r}_n = H_n(A)R_n(A)\boldsymbol{r}_0$ 计算残差向量 $\boldsymbol{r}_{n+1} = H_{n+1}(A)R_{n+1}(A)\boldsymbol{r}_0$. 事实上, $H_{n+1}R_{n+1}$ 可作如下展开:

$$H_{n+1}R_{n+1} = H_nR_{n+1} - \eta_n\lambda G_{n-1}R_{n+1} - \zeta_n\lambda H_nR_{n+1} \tag{3.89}$$

$$= H_nR_n - \alpha_n\lambda H_nP_n - \lambda G_nR_{n+1}, \tag{3.90}$$

$$H_nR_{n+1} = H_nR_n - \alpha_n\lambda H_nP_n, \tag{3.91}$$

$$\lambda G_nR_{n+2} = H_nR_{n+1} - H_{n+1}R_{n+1} - \alpha_{n+1}\lambda H_nP_{n+1} + \alpha_{n+1}\lambda H_{n+1}P_{n+1}, \tag{3.92}$$

$$H_{n+1}P_{n+1} = H_{n+1}R_{n+1} + \beta_nH_nP_n - \beta_n\lambda G_nP_n, \tag{3.93}$$

$$\lambda H_nP_{n+1} = \lambda H_nR_{n+1} + \beta_n\lambda H_nP_n, \tag{3.94}$$

$$\lambda G_nP_n = \zeta_n\lambda H_nP_n + \eta_n(H_{n-1}R_n - H_nR_n + \beta_{n-1}\lambda G_{n-1}P_{n-1}), \tag{3.95}$$

$$G_nR_{n+1} = \zeta_nH_nR_n + \eta_nG_{n-1}R_n - \alpha_n\lambda G_nP_n. \tag{3.96}$$

定义新的辅助向量为

$$\begin{aligned}
\boldsymbol{t}_n &:= H_n(A)\boldsymbol{r}_{n+1}^{\mathrm{BiCG}}, \qquad \boldsymbol{y}_n := AG_{n-1}(A)\boldsymbol{r}_{n+1}^{\mathrm{BiCG}},\\
\boldsymbol{p}_n &:= H_n(A)\boldsymbol{p}_n^{\mathrm{BiCG}}, \qquad \boldsymbol{w}_n := AH_n(A)\boldsymbol{p}_{n+1}^{\mathrm{BiCG}},\\
\boldsymbol{u}_n &:= AG_n(A)\boldsymbol{p}_n^{\mathrm{BiCG}}, \quad \boldsymbol{z}_n := G_n(A)\boldsymbol{r}_{n+1}^{\mathrm{BiCG}}.
\end{aligned}$$

则由 (3.89)–(3.96) 有下列递推公式:

$$\boldsymbol{r}_{n+1} = \boldsymbol{t}_n - \eta_n\boldsymbol{y}_n - \zeta_nA\boldsymbol{t}_n \tag{3.97}$$

$$= \boldsymbol{r}_n - \alpha_nA\boldsymbol{p}_n - A\boldsymbol{z}_n, \tag{3.98}$$

$$\boldsymbol{t}_n = \boldsymbol{r}_n - \alpha_nA\boldsymbol{p}_n, \tag{3.99}$$

$$\boldsymbol{y}_{n+1} = \boldsymbol{t}_n - \boldsymbol{r}_{n+1} - \alpha_{n+1}\boldsymbol{w}_n + \alpha_{n+1}A\boldsymbol{p}_{n+1}, \tag{3.100}$$

$$\boldsymbol{p}_{n+1} = \boldsymbol{r}_{n+1} + \beta_n(\boldsymbol{p}_n - \boldsymbol{u}_n), \tag{3.101}$$

$$\boldsymbol{w}_n = A\boldsymbol{t}_n + \beta_nA\boldsymbol{p}_n, \tag{3.102}$$

$$\boldsymbol{u}_n = \zeta_nA\boldsymbol{p}_n + \eta_n(\boldsymbol{t}_{n-1} - \boldsymbol{r}_n + \beta_{n-1}\boldsymbol{u}_{n-1}), \tag{3.103}$$

$$\boldsymbol{z}_n = \zeta_n\boldsymbol{r}_n + \eta_n\boldsymbol{z}_{n-1} - \alpha_n\boldsymbol{u}_n. \tag{3.104}$$

针对残差向量和近似解, 给出了两个递推关系 (3.97) 和 (3.98). 第一个递推关系 (3.97) 将用于确定两个参数 ζ_n 和 η_n, 第二个递推关系 (3.98) 可用于按下式得到近似解:

$$\boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n\boldsymbol{p}_n + \boldsymbol{z}_n.$$

3.3.8.3 α_n 和 β_n 的计算

由于 H_n 最高次项的系数为 $(-1)^n \prod_{i=0}^{n-1} \zeta_i$, 得到

$$(\boldsymbol{r}_0^*, \boldsymbol{r}_n) = (\bar{H}_n(A^{\mathrm{H}})\boldsymbol{r}_0^*, \boldsymbol{r}_n^{\mathrm{BiCG}}) = \left((-1)^n \prod_{i=0}^{n-1} \zeta_i\right) ((A^{\mathrm{H}})^n \boldsymbol{r}_0^*, \boldsymbol{r}_n^{\mathrm{BiCG}}),$$

$$(\boldsymbol{r}_0^*, A\boldsymbol{p}_n) = (\bar{H}_n(A^{\mathrm{H}})\boldsymbol{r}_0^*, A\boldsymbol{p}_n^{\mathrm{BiCG}}) = \left((-1)^n \prod_{i=0}^{n-1} \zeta_i\right) ((A^{\mathrm{H}})^n \boldsymbol{r}_0^*, A\boldsymbol{p}_n^{\mathrm{BiCG}}).$$

因此由 (3.59) 和 (3.60) 得

$$\alpha_n = \frac{(\boldsymbol{r}_0^*, \boldsymbol{r}_n)}{(\boldsymbol{r}_0^*, A\boldsymbol{p}_n)}, \qquad \beta_n = \frac{\alpha_n}{\zeta_n} \times \frac{(\boldsymbol{r}_0^*, \boldsymbol{r}_{n+1})}{(\boldsymbol{r}_0^*, \boldsymbol{r}_n)}.$$

3.3.8.4 实现细节

通过确定两个参数 ζ_n 和 η_n, 可从 Zhang 的框架导出 CGS 方法, BiCGSTAB 方法, BiCGSTAB2 方法和 GPBiCG 方法等 Krylov 子空间方法. 这些 Krylov 子空间方法与参数的关系列于 Table 3.1.

表 3.1: 乘积型方法中 ζ_n 和 η_n 的选择

CGS	$\zeta_n = \alpha_n, \quad \eta_n = \dfrac{\beta_{n-1}}{\alpha_{n-1}}\alpha_n$
BiCGSTAB	$\zeta_n = \underset{\zeta_n \in \mathbb{C}}{\arg\min} \lVert \boldsymbol{r}_{n+1} \rVert, \quad \eta_n = 0$
GPBiCG	$\{\zeta_n, \eta_n\} = \underset{\zeta_n, \eta_n \in \mathbb{C}}{\arg\min} \lVert \boldsymbol{r}_{n+1} \rVert$
BiCGSTAB2	偶数次迭代采用 BiCGSTAB 参数 奇数次迭代采用 GPBiCG 参数

3.3.8.5 CGS 方法的选择

在递推关系 (3.87) 和 (3.88) 中置 $\zeta_n = \alpha_n$ 且 $\eta_n = (\beta_{n-1}/\alpha_{n-1})\alpha_n$, 就得到 CGS 方法, 即多项式 H_n 和 G_n 变为 Lanczos 多项式:

$$H_n(\lambda) = R_n(\lambda), \qquad G_n(\lambda) = \alpha_n P_n(\lambda).$$

这一事实得到关系 $\boldsymbol{p}_n - \boldsymbol{u}_n = \boldsymbol{z}_n/\alpha_n$, 因为由辅助向量的定义有

$$\begin{aligned}
\boldsymbol{p}_n - \boldsymbol{u}_n &= H_n(A)P_n(A)\boldsymbol{r}_0 - AG_n(A)P_n(A)\boldsymbol{r}_0 \\
&= R_n(A)P_n(A)\boldsymbol{r}_0 - \alpha_n AP_n(A)P_n(A)\boldsymbol{r}_0 \\
&= (R_n(A) - \alpha_n AP_n(A))P_n(A)\boldsymbol{r}_0 \\
&= R_{n+1}(A)P_n(A)\boldsymbol{r}_0 \\
&= P_n(A)R_{n+1}(A)\boldsymbol{r}_0 \\
&= P_n(A)\boldsymbol{r}_{n+1}^{\mathrm{BiCG}} \\
&= \frac{1}{\alpha_n} \times G_n(A)\boldsymbol{r}_{n+1}^{\mathrm{BiCG}} \\
&= \frac{1}{\alpha_n} \times \boldsymbol{z}_n.
\end{aligned}$$

注意, 由 (3.98) 和 (3.99) 有 $\boldsymbol{t}_{n-1}-\boldsymbol{r}_n=A\boldsymbol{z}_{n-1}$. 使用递推公式 (3.98) 更新 $\boldsymbol{r}_{n+1}$ 后, 在递推公式 (3.97)–(3.104) 中就不再需要辅助向量 $\boldsymbol{t}_n,\boldsymbol{y}_n$ 和 $\boldsymbol{w}_n$. 现在引入下列辅助向量:

$$\widetilde{\boldsymbol{u}}_n:=A^{-1}\boldsymbol{u}_n/\alpha_n,\qquad \widetilde{\boldsymbol{z}}_n:=\boldsymbol{z}_n/\alpha_n,$$

以代替 $\boldsymbol{u}_n$ 和 $\boldsymbol{z}_n$. 使用关系 $(\boldsymbol{r}_0^*,A\boldsymbol{p}_n)=(\boldsymbol{r}_0^*,\boldsymbol{u}_n/\alpha_n)$ 即可得到 CGS 方法, 其算法见 Algorithm 3.21. 这里已将 $\widetilde{\boldsymbol{u}}_n$ 和 $\widetilde{\boldsymbol{z}}_n$ 改写为 $\boldsymbol{u}_n$ 和 $\boldsymbol{z}_n$.

Algorithm 3.21 CGS 方法

Input: $\boldsymbol{x}_0\in\mathbb{C}^N,\ \beta_{-1}=0,\ \boldsymbol{u}_{-1}=\boldsymbol{z}_{-1}=\boldsymbol{0},\ \boldsymbol{r}_0=\boldsymbol{b}-A\boldsymbol{x}_0$

Input: 选择 $\boldsymbol{r}_0^*\in\mathbb{C}^N$, 例如 $\boldsymbol{r}_0^*=\boldsymbol{r}_0$

Output: $\boldsymbol{x}_n$

for $n=0,1,\ldots$ **do**

$\quad\boldsymbol{p}_n=\boldsymbol{r}_n+\beta_{n-1}\boldsymbol{z}_{n-1}$

$\quad\boldsymbol{u}_n=\boldsymbol{p}_n+\beta_{n-1}(\boldsymbol{z}_{n-1}+\beta_{n-1}\boldsymbol{u}_{n-1})$

$\quad\alpha_n=\dfrac{(\boldsymbol{r}_0^*,\boldsymbol{r}_n)}{(\boldsymbol{r}_0^*,A\boldsymbol{u}_n)}$

$\quad\boldsymbol{z}_n=\boldsymbol{p}_n-\alpha_nA\boldsymbol{u}_n$

$\quad\boldsymbol{x}_{n+1}=\boldsymbol{x}_n+\alpha_n(\boldsymbol{p}_n+\boldsymbol{z}_n)$

$\quad\boldsymbol{r}_{n+1}=\boldsymbol{r}_n-\alpha_nA(\boldsymbol{p}_n+\boldsymbol{z}_n)$

$\quad\beta_n=\dfrac{(\boldsymbol{r}_0^*,\boldsymbol{r}_{n+1})}{(\boldsymbol{r}_0^*,\boldsymbol{r}_n)}$

end for

CGS 方法的残差向量写为 $\boldsymbol{r}_{n+1}=R_{n+1}(A)R_{n+1}(A)\boldsymbol{r}_0$. CGS 方法是 (Bi)CG 平方方法的缩写, 名称来自项 $R_{n+1}(A)R_{n+1}(A)$. 由此, 可以预期 CGS 方法的收敛速度为 BiCG 方法的两倍. 另一方面, 当 BiCG 方法像通常那样表现出不规则的收敛行为时, CGS 方法的收敛行为会很不规则, 导致近似解的精度低得多, 或不收敛.

3.3.8.6 BiCGSTAB 方法的选择

若选择 $\eta_n=0$, 并选择 ζ_n 使残差向量的 2-范数 $\|\boldsymbol{r}_{n+1}\|=\|\boldsymbol{t}_n-\zeta_nA\boldsymbol{t}_n\|$ 达到极小值, 就得到 BiCGSTAB 方法. 为实现这一极小化, 选择 ζ_n 使下列性质成立:

$$\boldsymbol{r}_{n+1}\perp A\boldsymbol{t}_n,$$

或者等价地, 由 (1.34) 得

$$\zeta_n=\frac{\boldsymbol{t}_n^{\mathrm{H}}A\boldsymbol{t}_n}{(A\boldsymbol{t}_n)^{\mathrm{H}}(A\boldsymbol{t}_n)}.$$

由于 $\eta_n=0$, 得到

$$\boldsymbol{u}_n=\zeta_nA\boldsymbol{p}_n,\qquad \boldsymbol{z}_n=\zeta_n\boldsymbol{t}_n.$$

BiCGSTAB 方法的算法见 Algorithm 3.22. BiCGSTAB 方法是 BiCG 稳定化方法的缩写. 下面说明稳定化的含义. 由于 $\eta_n=0$, (3.87) 和 (3.88) 中的递推关系化为

$$H_n(\lambda)=H_{n-1}(\lambda)-\zeta_{n-1}\lambda H_{n-1}(\lambda)=(1-\zeta_{n-1}\lambda)H_{n-1}(\lambda).$$

这意味着 BiCGSTAB 方法的残差向量为

$$\boldsymbol{r}_{n+1} = H_{n+1}(A)R_{n+1}(A)\boldsymbol{r}_0 = (I - \zeta_n A)\boldsymbol{r}_n^{\mathrm{BiCG}}.$$

这表明, 由于选择 ζ_n 使 $\|\boldsymbol{r}_{n+1}\|$ 达到极小值, 可以预期 BiCG 方法的残差向量得到稳定化, 即可以预期 BiCGSTAB 方法比 BiCG 方法具有更平滑的收敛行为.

Algorithm 3.22 BiCGSTAB 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \beta_{-1} = 0,\ \boldsymbol{p}_{-1} = \boldsymbol{0},\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Input: 选择 $\boldsymbol{r}_0^* \in \mathbb{C}^N$, 例如 $\boldsymbol{r}_0^* = \boldsymbol{r}_0$

Output: $\boldsymbol{x}_n$

for $n = 0, 1, \ldots$ **do**

$\quad \boldsymbol{p}_n = \boldsymbol{r}_n + \beta_{n-1}(\boldsymbol{p}_{n-1} - \zeta_{n-1}A\boldsymbol{p}_{n-1})$

$\quad \alpha_n = \dfrac{(\boldsymbol{r}_0^*, \boldsymbol{r}_n)}{(\boldsymbol{r}_0^*, A\boldsymbol{p}_n)}$

$\quad \boldsymbol{t}_n = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n$

$\quad \zeta_n = \dfrac{(A\boldsymbol{t}_n, \boldsymbol{t}_n)}{(A\boldsymbol{t}_n, A\boldsymbol{t}_n)}$

$\quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n\boldsymbol{p}_n + \zeta_n\boldsymbol{t}_n$

$\quad \boldsymbol{r}_{n+1} = \boldsymbol{t}_n - \zeta_n A\boldsymbol{t}_n$

$\quad \beta_n = \dfrac{\alpha_n}{\zeta_n} \times \dfrac{(\boldsymbol{r}_0^*, \boldsymbol{r}_{n+1})}{(\boldsymbol{r}_0^*, \boldsymbol{r}_n)}$

end for

3.3.8.7 GPBiCG 方法的选择

对于 GPBiCG 方法, 通过极小化残差向量的 2-范数 $\|\boldsymbol{r}_{n+1}\| = \|\boldsymbol{t}_n - \eta_n\boldsymbol{y}_n - \zeta_n A\boldsymbol{t}_n\|$ 来选择两个参数 η_n 和 ζ_n. 下列正交条件可实现这一极小化:

$$\boldsymbol{r}_{n+1} \perp A\boldsymbol{t}_n, \qquad \boldsymbol{r}_{n+1} \perp \boldsymbol{y}_n,$$

或者等价地, 在 (1.31) 和 (1.32) 中置 $\boldsymbol{d} = \boldsymbol{t}_n$, $M = [\boldsymbol{y}_n, A\boldsymbol{t}_n]$, $\boldsymbol{x} = [\eta_n, \zeta_n]^\top$, 得到

$$\zeta_n = \frac{(\boldsymbol{y}_n, \boldsymbol{y}_n)(A\boldsymbol{t}_n, \boldsymbol{t}_n) - (\boldsymbol{y}_n, \boldsymbol{t}_n)(A\boldsymbol{t}_n, \boldsymbol{y}_n)}{(A\boldsymbol{t}_n, A\boldsymbol{t}_n)(\boldsymbol{y}_n, \boldsymbol{y}_n) - (\boldsymbol{y}_n, A\boldsymbol{t}_n)(A\boldsymbol{t}_n, \boldsymbol{y}_n)},$$
$$\eta_n = \frac{(A\boldsymbol{t}_n, A\boldsymbol{t}_n)(\boldsymbol{y}_n, \boldsymbol{t}_n) - (\boldsymbol{y}_n, A\boldsymbol{t}_n)(A\boldsymbol{t}_n, \boldsymbol{t}_n)}{(A\boldsymbol{t}_n, A\boldsymbol{t}_n)(\boldsymbol{y}_n, \boldsymbol{y}_n) - (\boldsymbol{y}_n, A\boldsymbol{t}_n)(A\boldsymbol{t}_n, \boldsymbol{y}_n)}.$$

GPBiCG 方法的算法见 Algorithm 3.23. 注意, GPBiCG 方法使用两个参数 ζ_n 和 η_n 来极小化残差 2-范数, 而 BiCGSTAB 方法使用一个参数 ζ_n 来极小化残差 2-范数, 且 $\eta_n = 0$. 由此, 可以预期 GPBiCG 方法比 BiCGSTAB 方法表现出更平滑的收敛行为.

Algorithm 3.23 GPBiCG 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N$, $\beta_{-1} = 0$, $\boldsymbol{p}_{-1} = \boldsymbol{t}_{-1} = \boldsymbol{u}_{-1} = \boldsymbol{w}_{-1} = \boldsymbol{z}_{-1} = \boldsymbol{0}$, $\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Input: 选择 $\boldsymbol{r}_0^* \in \mathbb{C}^N$, 例如 $\boldsymbol{r}_0^* = \boldsymbol{r}_0$

Output: $\boldsymbol{x}_n$

for $n = 0, 1, \ldots$ **do**

$\boldsymbol{p}_n = \boldsymbol{r}_n + \beta_{n-1}(\boldsymbol{p}_{n-1} - \boldsymbol{u}_{n-1})$

$\alpha_n = \dfrac{(\boldsymbol{r}_0^*, \boldsymbol{r}_n)}{(\boldsymbol{r}_0^*, A\boldsymbol{p}_n)}$

$\boldsymbol{y}_n = \boldsymbol{t}_{n-1} - \boldsymbol{r}_n - \alpha_n \boldsymbol{w}_{n-1} + \alpha_n A\boldsymbol{p}_n$

$\boldsymbol{t}_n = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n$

$\zeta_n = \dfrac{(\boldsymbol{y}_n, \boldsymbol{y}_n)(A\boldsymbol{t}_n, \boldsymbol{t}_n) - (\boldsymbol{y}_n, \boldsymbol{t}_n)(A\boldsymbol{t}_n, \boldsymbol{y}_n)}{(A\boldsymbol{t}_n, A\boldsymbol{t}_n)(\boldsymbol{y}_n, \boldsymbol{y}_n) - (\boldsymbol{y}_n, A\boldsymbol{t}_n)(A\boldsymbol{t}_n, \boldsymbol{y}_n)}$

$\eta_n = \dfrac{(A\boldsymbol{t}_n, A\boldsymbol{t}_n)(\boldsymbol{y}_n, \boldsymbol{t}_n) - (\boldsymbol{y}_n, A\boldsymbol{t}_n)(A\boldsymbol{t}_n, \boldsymbol{t}_n)}{(A\boldsymbol{t}_n, A\boldsymbol{t}_n)(\boldsymbol{y}_n, \boldsymbol{y}_n) - (\boldsymbol{y}_n, A\boldsymbol{t}_n)(A\boldsymbol{t}_n, \boldsymbol{y}_n)}$

if $n = 0$ **then**

$\zeta_n = \dfrac{(A\boldsymbol{t}_n, \boldsymbol{t}_n)}{(A\boldsymbol{t}_n, A\boldsymbol{t}_n)}, \quad \eta_n = 0$

end if

$\boldsymbol{u}_n = \zeta_n A\boldsymbol{p}_n + \eta_n(\boldsymbol{t}_{n-1} - \boldsymbol{r}_n + \beta_{n-1}\boldsymbol{u}_{n-1})$

$\boldsymbol{z}_n = \zeta_n \boldsymbol{r}_n + \eta_n \boldsymbol{z}_{n-1} - \alpha_n \boldsymbol{u}_n$

$\boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n + \boldsymbol{z}_n$

$\boldsymbol{r}_{n+1} = \boldsymbol{t}_n - \eta_n \boldsymbol{y}_n - \zeta_n A\boldsymbol{t}_n$

$\beta_n = \dfrac{\alpha_n}{\zeta_n} \times \dfrac{(\boldsymbol{r}_0^*, \boldsymbol{r}_{n+1})}{(\boldsymbol{r}_0^*, \boldsymbol{r}_n)}$

$\boldsymbol{w}_n = A\boldsymbol{t}_n + \beta_n A\boldsymbol{p}_n$

end for

Fujino [71] 从 Zhang 的框架提出 GPBiCG(m, ℓ) 方法. K. Aihara [4] 提出了 GPBiCG 方法和 BiCGSTAB(ℓ) 方法的统一推广, 称为 GPBiCGstab(L) 方法.

3.3.8.8 数值例子

本小节给出一些说明性的数值例子. 考虑非对称线性方程组 (2.18), 参数为 $N = 20$, $a_1 = a_2 = a_3 = 1$, $b_1 = b_2 = b_3 = 1$, 以及两个不同参数 $c = 300$ 和 $c = 900$. 所有 Krylov 子空间方法均采用停止准则 $\|\boldsymbol{r}_n\|/\|\boldsymbol{b}\| \leq 10^{-10}$. 当 $c = 300$ 时, CGS 方法, BiCGSTAB 方法和 GPBiCG 方法的收敛历程见 Figure 3.4. 横轴不是迭代次数, 而是矩阵向量乘法的次数. (CGS 方法, BiCGSTAB 方法和 GPBiCG 方法每次迭代均需两次矩阵向量乘法.) 由 Figure 3.4, GPBiCG 方法比 BiCGSTAB 方法收敛更快, 而从矩阵向量乘法次数, 或等价的迭代次数看, CGS 方法最好. 另一方面, CGS 方法表现出不规则的收敛行为, 这可能导致精度损失. GPBiCG 方法和 BiCGSTAB 方法表现出相对平滑的收敛行为, 这可能由残差 2-范数的局部极小化所致. 从精度看, CGS 方法, BiCGSTAB 方法和 GPBiCG 方法的真实相对残差 2-范数的 $\log_{10}$ 分别为 -9.79, -10.50, -10.17. 由此, 所有方法都产生了足够精确的近似解. 接着, $c = 900$ 时的收敛历程见 Figure 3.5. 由 Figure 3.5 可见, CGS 方法表现出很不规则的收敛行为, 收敛也最慢; 从矩阵向量乘法次数看, GPBiCG 方法最好. 从精度看, CGS 方法, BiCGSTAB 方法和 GPBiCG 方法的真实相对残差 2-范数的 $\log_{10}$ 分别为 -6.37, -10.03, -9.79. 这意味

着 CGS 方法损失了约四位精度. 这种精度损失称为残差间隙. 因此, 强烈建议在满足停止准则后检查真实相对残差 2-范数, 即 $\|\boldsymbol{b}-A\boldsymbol{x}_n\|/\|\boldsymbol{b}\|$ (不是 $\|\boldsymbol{r}_n\|/\|\boldsymbol{b}\|$!).

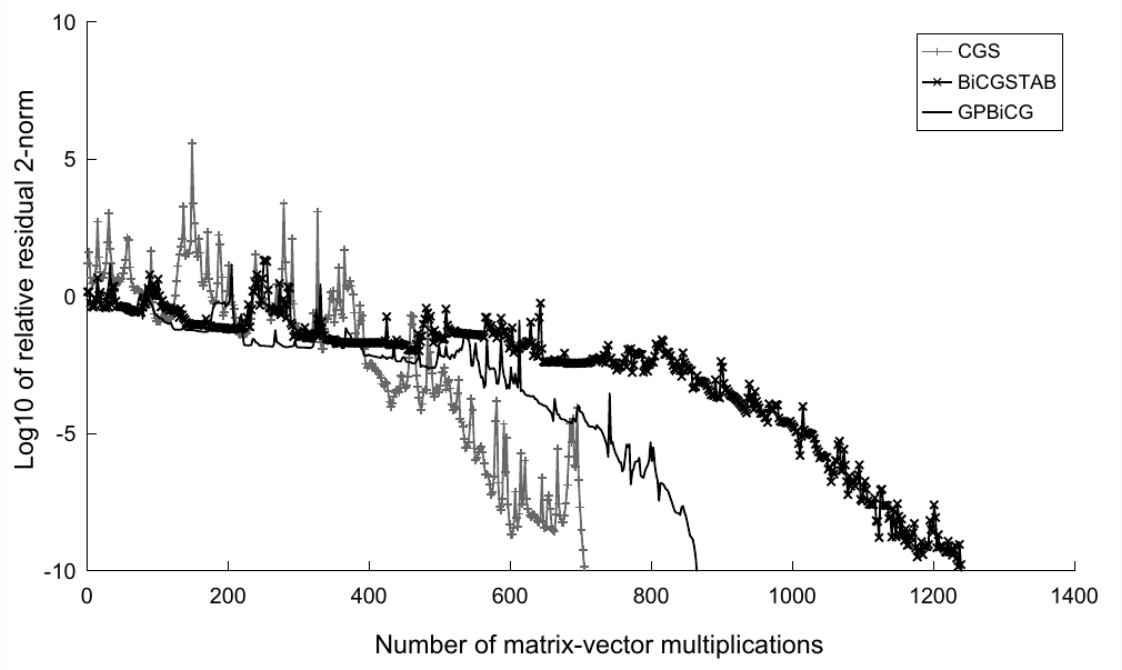


图 3.4: 当 $N=20, c=300$ 时 CGS, BiCGSTAB 和 GPBiCG 的收敛历程

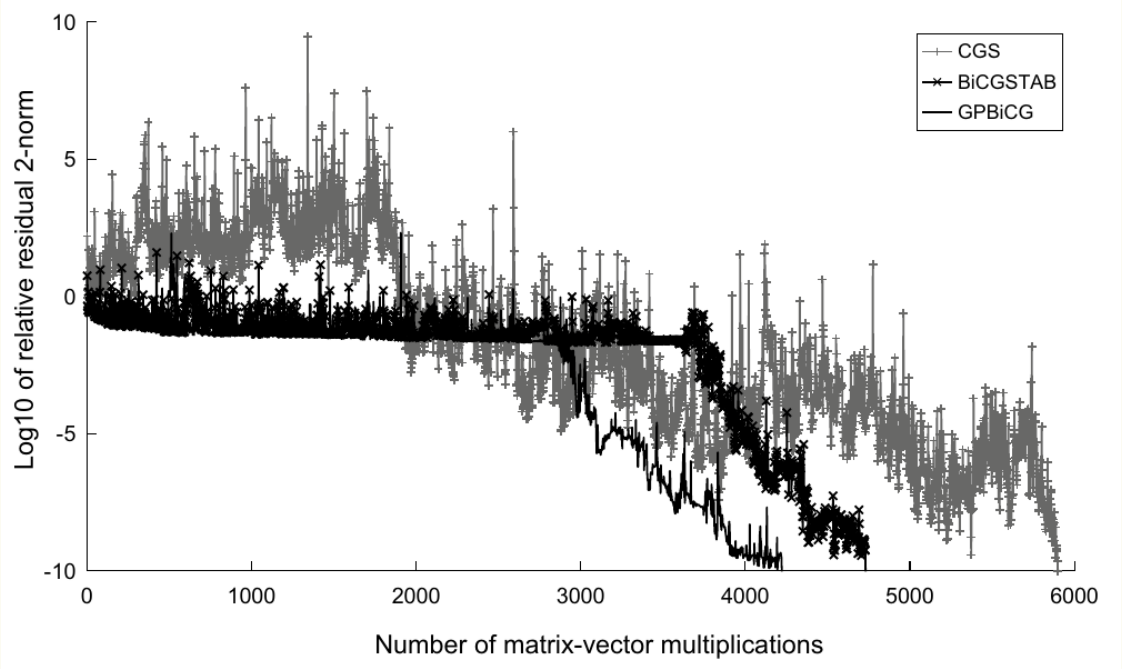


图 3.5: 当 $N=20, c=900$ 时 CGS, BiCGSTAB 和 GPBiCG 的收敛历程

3.3.8.9 其他相关方法

Section 3.3.4 中的 QMR 方法与 BiCG 方法一样需要 A^{H} 的信息. 与 CGS 方法的思想类似, R. W. Freund 提出了 QMR 方法的平方变型, 称为 TFQMR (无转置 QMR) 方法 [63], 它往往表现出平滑的收敛行为. 文献 [34] 基于 BiCGSTAB 方法和 QMR 方法的思想提出 QMRCGSTAB 方法. 当系数矩阵 A 接近 Hermitian 矩阵时, Section 3.3.3 中的 BiCR 方法可能比 BiCG 方法表现出更平滑的收敛行为, 因为当 A 为 Hermitian 矩阵时, 选择 $\boldsymbol{r}_0^*=\boldsymbol{r}_0$ 的

BiCR 方法变为 CR 方法, 而 CR 方法的残差 2-范数单调下降. 因此, 自然可以将基于 BiCG 方法的乘积型 Krylov 子空间方法替换为基于 BiCR 方法的乘积型方法. 事实上, 可构造基于 BiCR 方法的乘积型方法, 文献 [165] 导出了所得算法 (CRS, BiCRSTAB, GPBiCR). K. Abe 和 G. L. G. Sleijpen 独立提出了基于 BiCR 方法的乘积型方法, 见 [2]. Y.-F. Jing 等提出了类似于 BiCR 方法的一种途径, 称为 BiCOR 方法 [114], 其乘积型变型见 [32].

3.3.9 诱导降维 (IDR(s)) 方法

诱导降维 (IDR) 方法由 Wesseling 和 Sonneveld 于 1979 年前后建立 [205]. IDR 方法在数学上等价于 BiCGSTAB 方法, 而 BiCGSTAB 方法可视为 IDR 方法的稳定化变型. 文献 [195] 的致谢中写道, Sonneveld 建议 van der Vorst 重新考虑 Sonneveld 的 IDR. 虽然 IDR 方法未受到进一步关注, 但大约 30 年后, Sonneveld 和 van Gijzen 提出了一种称为 IDR(s) 方法的创新算法 [174]. 本节介绍 IDR(s) 方法的理论与推导. 本节推导看似冗长, 但使 IDR(s) 方法的原理更易理解.

Theorem 3.12:

[IDR 定理 [174], [205], p.550] 设 A 为 $\mathbb{C}^{N\times N}$ 中的任意矩阵, $\boldsymbol{v}_0$ 为 $\mathbb{C}^N$ 中的任意非零向量, $\mathcal{G}_0$ 为完全 Krylov 空间, 即 $\mathcal{G}_0 := \mathcal{K}_N(A, \boldsymbol{v}_0)$. 设 $\mathcal{S}$ 表示 $\mathbb{C}^N$ 的任意 (真) 子空间, 使 $\mathcal{S}$ 和 $\mathcal{G}_0$ 不具有共同的非平凡不变子空间,[a] 并定义序列 $\mathcal{G}_j$, $j = 1, 2, \ldots$ 为

$$\mathcal{G}_j = (I - \omega_j A)(\mathcal{G}_{j-1} \cap \mathcal{S}), \tag{3.105}$$

其中 ω_j 为非零标量. 则:

(1) $\mathcal{G}_j \subset \mathcal{G}_{j-1}$ 对所有 $j > 0$ 成立.

(2) $\mathcal{G}_j = \{\mathbf{0}\}$ 对某个 $j \le N$ 成立.

[a] 这意味着 $\mathcal{S} \cap \mathcal{G}_0$ 不包含 A 的任何特征向量, 平凡不变子空间为 $\{\mathbf{0}\}$.

Proof: 本证明基于 [174], Theorem 2.1. 第一个断言用归纳法证明. 首先证明 $\mathcal{G}_1 \subset \mathcal{G}_0$. 由 $\mathcal{G}_0 = \mathcal{K}_N(A, \boldsymbol{v}_0)$ 得 $(I - \omega_1 A)\mathcal{G}_0 \subset \mathcal{G}_0$, 因为对给定的 $\boldsymbol{v} \in (I - \omega_1 A)\mathcal{G}_0$, 有

$$\begin{aligned}
\boldsymbol{v} &= (I - \omega_1 A)(c_0\boldsymbol{v}_0 + c_1 A\boldsymbol{v}_0 + \cdots + c_{N-1}A^{N-1}\boldsymbol{v}_0) \\
&= c_0(I - \omega_1 A)\boldsymbol{v}_0 + c_1(I - \omega_1 A)A\boldsymbol{v}_0 + \cdots + c_{N-1}(I - \omega_1 A)A^{N-1}\boldsymbol{v}_0 \\
&= d_0\boldsymbol{v}_0 + d_1 A\boldsymbol{v}_0 + \cdots + d_{N-1}A^{N-1}\boldsymbol{v}_0 + d_N A^N\boldsymbol{v}_0 \\
&= \widetilde{d}_0\boldsymbol{v}_0 + \widetilde{d}_1 A\boldsymbol{v}_0 + \cdots + \widetilde{d}_{N-1}A^{N-1}\boldsymbol{v}_0 \\
&\in \mathcal{G}_0.
\end{aligned}$$

因此有 $\mathcal{G}_1 \subset \mathcal{G}_0$, 因为

$$\mathcal{G}_1 = (I - \omega_1 A)(\mathcal{G}_0 \cap \mathcal{S}) \subset (I - \omega_1 A)\mathcal{G}_0 \subset \mathcal{G}_0.$$

接着, 在假设 $\mathcal{G}_j \subset \mathcal{G}_{j-1}$ 下, 对所有 $j > 0$ 证明 $\mathcal{G}_{j+1} \subset \mathcal{G}_j$ (即 $\boldsymbol{x} \in \mathcal{G}_{j+1} \Rightarrow \boldsymbol{x} \in \mathcal{G}_j$). 设 $\boldsymbol{x} \in \mathcal{G}_{j+1}$. 则对某个 $\boldsymbol{y} \in \mathcal{G}_j \cap \mathcal{S}$, $\boldsymbol{x}$ 可写为 $\boldsymbol{x} = (I - \omega_{j+1}A)\boldsymbol{y}$. 由假设 $\mathcal{G}_j \subset \mathcal{G}_{j-1}$, 得 $\boldsymbol{y} \in \mathcal{G}_j \cap \mathcal{S} \subset \mathcal{G}_{j-1} \cap \mathcal{S}$, 因而 $(I - \omega_j A)\boldsymbol{y} \in \mathcal{G}_j$. 由于 $\boldsymbol{y} \in \mathcal{G}_j$ 且 $\boldsymbol{y} - \omega_j A\boldsymbol{y} \in \mathcal{G}_j$, 有 $A\boldsymbol{y} \in \mathcal{G}_j$, 从而

$(I-\omega_{j+1}A)\boldsymbol{y}=\boldsymbol{x}\in\mathcal{G}_j$. 下面证明第二个断言. 性质 $\mathcal{G}_{j+1}\subset\mathcal{G}_j$ 意味着 $\dim(\mathcal{G}_{j+1})<\dim(\mathcal{G}_j)$ 或 $\dim(\mathcal{G}_{j+1})=\dim(\mathcal{G}_j)$. 若 $\mathcal{G}_j\cap\mathcal{S}\neq\mathcal{G}_j$, 则 $\dim(\mathcal{G}_j\cap\mathcal{S})<\dim(\mathcal{G}_j)$, 从而有 $\dim(\mathcal{G}_{j+1})<\dim(\mathcal{G}_j)$, 因为 $\dim(\mathcal{G}_{j+1})=\dim((I-\omega_jA)(\mathcal{G}_j\cap\mathcal{S}))\leq\dim(\mathcal{G}_j\cap\mathcal{S})<\dim(\mathcal{G}_j)$. 另一方面, 若 $\mathcal{G}_j\cap\mathcal{S}=\mathcal{G}_j$, 则 $\mathcal{G}_j\subset\mathcal{S}$, 且有 $\mathcal{G}_{j+1}=(I-\omega_{j+1}A)(\mathcal{G}_j\cap\mathcal{S})=(I-\omega_{j+1}A)\mathcal{G}_j\subset\mathcal{G}_j$. (最后一个包含关系来自第一个断言 $\mathcal{G}_{j+1}\subset\mathcal{G}_j$.) 这表明 $A\mathcal{G}_j\subset\mathcal{G}_j$, 即 $\mathcal{G}_j$ 为 A 的不变子空间. 回顾 $\mathcal{G}_j\subset\mathcal{S}$ 和 $\mathcal{G}_j\subset\mathcal{G}_0$, 得 $\mathcal{G}_j\subset\mathcal{G}_0\cap\mathcal{S}$. 由 $\mathcal{G}_0\cap\mathcal{S}$ 不具有共同的 A 的非平凡不变子空间这一假设, 子空间 $\mathcal{G}_j$ 为 A 的平凡不变子空间, 即 $\mathcal{G}_j=\{\boldsymbol{0}\}$. 因此, 或者 $\dim(\mathcal{G}_{j+1})<\dim(\mathcal{G}_j)$, 或者 $\dim(\mathcal{G}_j)=0$, 这表明对某个 $k\leq N$ 有 $\dim(\mathcal{G}_k)=0$, 即 $\mathcal{G}_k=\{\boldsymbol{0}\}$.

■

Corollary 3.13: 对给定的 $s\geq1$, 设 $\mathcal{S}$ 为 $\mathbb{C}^N$ 的子空间且 $\dim(\mathcal{S})=N-s$. 若 $\mathcal{G}_{i-1}+\mathcal{S}=\mathbb{C}^N$ 且 $(I-\omega_jA)$ 非奇异, 则

$$\dim(\mathcal{G}_j)=\dim(\mathcal{G}_{j-1})-s. \tag{3.106}$$

Proof: 设 W_1,W_2 为 $\mathbb{C}^N$ (或一般向量空间 V) 的子空间. 线性代数中众所周知, $\dim(W_1)+\dim(W_2)=\dim(W_1+W_2)+\dim(W_1\cap W_2)$. 令 $W_1=\mathcal{G}_{i-1}$, $W_2=\mathcal{S}$. 则由假设 $\mathcal{G}_{i-1}+\mathcal{S}=\mathbb{C}^N$, 有 $\dim(\mathcal{G}_{i-1}+\mathcal{S})=N$. 因而 $\dim(\mathcal{G}_i)=\dim((I-\omega_jA)(\mathcal{G}_{i-1}\cap\mathcal{S}))=\dim(\mathcal{G}_{i-1}\cap\mathcal{S})=\dim(\mathcal{G}_{i-1})+\dim(\mathcal{S})-N=\dim(\mathcal{G}_{i-1})-s$.

■

Remark 3.14: 由 Corollary 3.13, 若对 $i=1,2,\ldots,n$ 有 $\mathcal{G}_{i-1}+\mathcal{S}=\mathbb{C}^N$, 则 $\dim(\mathcal{G}_n)=\dim(\mathcal{G}_0)-ns$. 因而, 若 N/s 为整数且 $\dim(\mathcal{G}_0)=N$, 则有 $\dim(\mathcal{G}_{N/s})=0$, 即 $\mathcal{G}_{N/s}=\{\boldsymbol{0}\}$. Remark 3.15 将引用本注以给出 IDR(s) 方法所需矩阵向量乘法次数的最大值.

3.3.9.1 IDR(s) 方法的推导

对于给定数 $s\geq1$, IDR(s) 方法生成满足下列条件的残差向量 $\boldsymbol{r}_0,\boldsymbol{r}_1,\ldots$:

1. $\boldsymbol{r}_{j(s+1)},\boldsymbol{r}_{j(s+1)+1},\ldots,\boldsymbol{r}_{j(s+1)+s}\in\mathcal{G}_j,\quad j=0,1,2,\ldots$;
2. 选择 (3.105) 中的子空间 $\mathcal{S}$, 使 $\dim\mathcal{S}=N-s$.

为简单起见, 考虑 $s=2$ 的情形, 即 IDR(2) 方法. 在此情形下, IDR(2) 方法产生下列残差向量:

$$\boldsymbol{r}_0,\boldsymbol{r}_1,\boldsymbol{r}_2\in\mathcal{G}_0,\quad \boldsymbol{r}_3,\boldsymbol{r}_4,\boldsymbol{r}_5\in\mathcal{G}_1,\quad \boldsymbol{r}_6,\boldsymbol{r}_7,\boldsymbol{r}_8\in\mathcal{G}_2,\ldots$$

这里说明如何从给定残差向量 $\boldsymbol{r}_0,\boldsymbol{r}_1,\boldsymbol{r}_2\in\mathcal{G}_0$ 产生 $\boldsymbol{r}_3,\boldsymbol{r}_4,\boldsymbol{r}_5\in\mathcal{G}_1$. 由于 $s=2$, 由 Remark 3.14 可知, 需选择 $\mathcal{S}$ 使 $\dim(\mathcal{S})=N-2$. 标准选择为

$$\mathcal{S}=\operatorname{span}\{\boldsymbol{s}_1,\boldsymbol{s}_2\}^{\perp} \tag{3.107}$$

其中 $\boldsymbol{s}_1$ 和 $\boldsymbol{s}_2$ 为任意线性无关向量, 符号 $\perp$ 表示正交补. 现在令 $S=[\boldsymbol{s}_1,\boldsymbol{s}_2]\in\mathbb{C}^{N\times2}$, 其中向量 $\boldsymbol{s}_1$ 和 $\boldsymbol{s}_2$ 线性无关. 则对于向量 $\boldsymbol{v}\in\mathbb{C}^N$, 下列事实成立:

$$\boldsymbol{v}\perp\mathcal{S}\Leftrightarrow S^{\mathrm{H}}\boldsymbol{v}=\boldsymbol{0}. \tag{3.108}$$

下面使用下列定义:

$$\Delta\boldsymbol{r}_k:=\boldsymbol{r}_{k+1}-\boldsymbol{r}_k,\quad dR_k:=[\Delta\boldsymbol{r}_k,\Delta\boldsymbol{r}_{k+1}],\quad \boldsymbol{c}_k=(S^{\mathrm{H}}dR_k)^{-1}S^{\mathrm{H}}\boldsymbol{r}_{k+2}. \tag{3.109}$$

下列向量 $\boldsymbol{v}_0$ 位于 $\mathcal{G}_0 \cap \mathcal{S}$ 中:

$$\boldsymbol{v}_0 = \boldsymbol{r}_2 - dR_0\boldsymbol{c}_0 \in \mathcal{G}_0 \cap \mathcal{S}. \tag{3.110}$$

事实上, $\boldsymbol{v}_0 \in \mathcal{G}_0$, 因为 $\boldsymbol{v}_0 \in \mathrm{span}\{\boldsymbol{r}_2, \Delta\boldsymbol{r}_0, \Delta\boldsymbol{r}_1\}$, 而由假设 $\boldsymbol{r}_0, \boldsymbol{r}_1, \boldsymbol{r}_2 \in \mathcal{G}_0$ 有 $\boldsymbol{r}_2, \Delta\boldsymbol{r}_0, \Delta\boldsymbol{r}_1 \in \mathcal{G}_0$; 又有 $\boldsymbol{v}_0 \in \mathcal{S}$, 因为容易看出 $S^{\mathrm{H}}\boldsymbol{v}_0 = \boldsymbol{0}$, 并使用关系 (3.108). 由 (3.105), 可按下式产生 $\boldsymbol{r}_3 \in \mathcal{G}_1$

$$\boldsymbol{r}_3 = (I - \omega_0 A)\boldsymbol{v}_0 \in \mathcal{G}_1, \tag{3.111}$$

其中通常选择 ω_0 使 $\|\boldsymbol{r}_3\|$ 达到极小值, 即 $\omega_0 = (A\boldsymbol{v}_0, \boldsymbol{v}_0)/(A\boldsymbol{v}_0, A\boldsymbol{v}_0)$. 为产生 $\boldsymbol{r}_4 \in \mathcal{G}_1$, 需要一个使 $\boldsymbol{v}_1 \in \mathcal{G}_0 \cap \mathcal{S}$ 的向量 $\boldsymbol{v}_1$. 为此, 利用 $\boldsymbol{r}_3$ 和 dR_1 按下式构造 $\boldsymbol{v}_1$:

$$\boldsymbol{v}_1 = \boldsymbol{r}_3 - dR_1\boldsymbol{c}_1 \in \mathcal{G}_0 \cap \mathcal{S}.$$

由与上面类似的讨论, 可见 $\boldsymbol{v}_1 \in \mathcal{G}_0$ 且 $\boldsymbol{v}_1 \in \mathcal{S}$. 因此有

$$\boldsymbol{r}_4 = (I - \omega_0 A)\boldsymbol{v}_1 \in \mathcal{G}_1.$$

类似地,

$$\boldsymbol{v}_2 = \boldsymbol{r}_4 - dR_2\boldsymbol{c}_2 \in \mathcal{G}_0 \cap \mathcal{S}, \quad \boldsymbol{r}_5 = (I - \omega_0 A)\boldsymbol{v}_2 \in \mathcal{G}_1.$$

现在, 已从给定残差向量 $\boldsymbol{r}_0, \boldsymbol{r}_1, \boldsymbol{r}_2 \in \mathcal{G}_0$ 得到 $\boldsymbol{r}_3, \boldsymbol{r}_4, \boldsymbol{r}_5 \in \mathcal{G}_1$. 类似地, 下列算法从 $\boldsymbol{r}_3, \boldsymbol{r}_4, \boldsymbol{r}_5 \in \mathcal{G}_1$ 产生 $\boldsymbol{r}_6, \boldsymbol{r}_7, \boldsymbol{r}_8 \in \mathcal{G}_2$:

$$\begin{aligned}
\boldsymbol{v}_3 &= \boldsymbol{r}_5 - dR_3\boldsymbol{c}_3 \in \mathcal{G}_1 \cap \mathcal{S}, \quad \boldsymbol{r}_6 = (I - \omega_1 A)\boldsymbol{v}_3 \in \mathcal{G}_2, \\
\boldsymbol{v}_4 &= \boldsymbol{r}_6 - dR_4\boldsymbol{c}_4 \in \mathcal{G}_1 \cap \mathcal{S}, \quad \boldsymbol{r}_7 = (I - \omega_1 A)\boldsymbol{v}_4 \in \mathcal{G}_2, \\
\boldsymbol{v}_5 &= \boldsymbol{r}_7 - dR_5\boldsymbol{c}_5 \in \mathcal{G}_1 \cap \mathcal{S}, \quad \boldsymbol{r}_8 = (I - \omega_1 A)\boldsymbol{v}_5 \in \mathcal{G}_2,
\end{aligned}$$

其中通常选择 ω_1 使 $\|\boldsymbol{r}_6\|$ 达到极小值, 即 $\omega_1 = (A\boldsymbol{v}_3, \boldsymbol{v}_3)/(A\boldsymbol{v}_3, A\boldsymbol{v}_3)$. 现在说明如何从 $\boldsymbol{x}_0, \boldsymbol{x}_1, \boldsymbol{x}_2$ 提取近似解 $\boldsymbol{x}_3, \boldsymbol{x}_4, \boldsymbol{x}_5$. 下面使用下列定义:

$$\Delta\boldsymbol{x}_k := \boldsymbol{x}_{k+1} - \boldsymbol{x}_k, \qquad dX_k := [\Delta\boldsymbol{x}_k, \Delta\boldsymbol{x}_{k+1}].$$

于是由 (3.110) 和 (3.111), 按下式得到 $\boldsymbol{x}_3$:

$$\begin{aligned}
\boldsymbol{r}_3 = (I - \omega_0 A)\boldsymbol{v}_0 &= \boldsymbol{r}_2 - dR_0\boldsymbol{c}_0 - \omega_0 A\boldsymbol{v}_0 \\
&\Leftrightarrow \boldsymbol{b} - A\boldsymbol{x}_3 = \boldsymbol{b} - A\boldsymbol{x}_2 - dR_0\boldsymbol{c}_0 - \omega_0 A\boldsymbol{v}_0 \\
&\Leftrightarrow \boldsymbol{x}_3 = \boldsymbol{x}_2 + A^{-1}dR_0\boldsymbol{c}_0 + \omega_0\boldsymbol{v}_0 \\
&\Leftrightarrow \boldsymbol{x}_3 = \boldsymbol{x}_2 - dX_0\boldsymbol{c}_0 + \omega_0\boldsymbol{v}_0.
\end{aligned}$$

类似地,

$$\begin{aligned}
\boldsymbol{x}_4 &= \boldsymbol{x}_3 - dX_1\boldsymbol{c}_1 + \omega_0\boldsymbol{v}_1, \\
\boldsymbol{x}_5 &= \boldsymbol{x}_4 - dX_2\boldsymbol{c}_2 + \omega_0\boldsymbol{v}_2.
\end{aligned}$$

$\boldsymbol{x}_6, \boldsymbol{x}_7, \boldsymbol{x}_8$ 可按下式得到:

$$\begin{aligned}
\boldsymbol{x}_6 &= \boldsymbol{x}_5 - dX_3\boldsymbol{c}_3 + \omega_1\boldsymbol{v}_3, \\
\boldsymbol{x}_7 &= \boldsymbol{x}_7 - dX_4\boldsymbol{c}_4 + \omega_1\boldsymbol{v}_4, \\
\boldsymbol{x}_8 &= \boldsymbol{x}_8 - dX_5\boldsymbol{c}_5 + \omega_1\boldsymbol{v}_5.
\end{aligned}$$

最后说明如何确定初始残差向量 $\boldsymbol{r}_0, \boldsymbol{r}_1, \boldsymbol{r}_2 \in \mathcal{G}_0$ 及 $\boldsymbol{x}_0, \boldsymbol{x}_1, \boldsymbol{x}_2$. 按如下方式给出这些向量: 计算

$$\begin{aligned}
&\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0, \\
&\boldsymbol{r}_1 = (I - c_0 A)\boldsymbol{r}_0, \quad \boldsymbol{x}_1 = \boldsymbol{x}_0 + c_0 \boldsymbol{r}_0, \\
&\boldsymbol{r}_2 = (I - c_1 A)\boldsymbol{r}_1, \quad \boldsymbol{x}_2 = \boldsymbol{x}_1 + c_1 \boldsymbol{r}_1,
\end{aligned}$$

其中 $\boldsymbol{x}_0$ 为任意向量, c_0 和 c_1 选择为使 $\|\boldsymbol{r}_1\|$ 和 $\|\boldsymbol{r}_2\|$ 达到极小值. 由上述讨论, $s = 2$ 的 IDR(s) 方法容易推广到一般数 $s \geq 1$ 的 IDR(s) 方法, 其算法见 Algorithm 3.24. 符号 "$\Delta X(:, i)$" 表示矩阵 ΔX 的第 i 列.

Algorithm 3.24 IDR(s) 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ P \in \mathbb{C}^{N\times s},\ s \geq 1$

Output: $\boldsymbol{x}_i$

$\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

(其他初始残差 $\boldsymbol{r}_1, \ldots, \boldsymbol{r}_s \in \mathcal{G}_0$)

for $i = 1$ 至 s **do**

  $\boldsymbol{v} = A\boldsymbol{r}_{i-1}, \quad \omega = \dfrac{\boldsymbol{v}^{\mathrm{H}}\boldsymbol{r}_{i-1}}{\boldsymbol{v}^{\mathrm{H}}\boldsymbol{v}}$

  $\Delta X(:,i) = \omega\boldsymbol{r}_{i-1}, \quad \Delta R(:,i) = -\omega\boldsymbol{v}$

  $\boldsymbol{x}_i = \boldsymbol{x}_{i-1} + \Delta X(:,i); \quad \boldsymbol{r}_i = \boldsymbol{r}_{i-1} + \Delta R(:,i)$

end for

$j = 1, \quad i = s$

$M = P^{\mathrm{H}}\Delta R, \quad \boldsymbol{h} = P^{\mathrm{H}}\boldsymbol{r}_i$

while $\dfrac{\|\boldsymbol{r}_i\|}{\|\boldsymbol{b}\|} > \epsilon$ **do**

  for $k = 0$ 至 s **do**

    由 $M\boldsymbol{c} = \boldsymbol{h}$ 解出 $\boldsymbol{c}$

    $\boldsymbol{q} = -\Delta R\boldsymbol{c}$

    $\boldsymbol{v} = \boldsymbol{r}_i + \boldsymbol{q}$

    if $k = 0$ **then**

      $\boldsymbol{t} = A\boldsymbol{v}, \quad \omega = \dfrac{\boldsymbol{t}^{\mathrm{H}}\boldsymbol{v}}{\boldsymbol{t}^{\mathrm{H}}\boldsymbol{t}}$

      $\Delta R(:,j) = \boldsymbol{q} - \omega\boldsymbol{t}$

      $\Delta X(:,j) = -\Delta X\boldsymbol{c} + \omega\boldsymbol{v}$

    else

      $\Delta X(:,j) = -\Delta X\boldsymbol{c} + \omega\boldsymbol{v}$

      $\Delta R(:,j) = -A\Delta X(:,j)$

    end if

  (更新近似解 $\boldsymbol{x}_i$)

    $\boldsymbol{r}_{i+1} = \boldsymbol{r}_i + \Delta R(:,j)$

    $\boldsymbol{x}_{i+1} = \boldsymbol{x}_i + \Delta X(:,j)$

    $\Delta\boldsymbol{m} = P^{\mathrm{H}}\Delta R(:,j)$

    $M(:,j) = \Delta\boldsymbol{m}$

    $\boldsymbol{h} = \boldsymbol{h} + \Delta\boldsymbol{m}$

    $i = i + 1, \quad j = j + 1$

    $j = (j-1)\%s + 1$ (%: 取模运算, 即 $a\%n = r$, 其中 $a = mn + r$.)

  end for

end while

Remark 3.15: 为从 $\mathcal{G}_{j-1}$ 中的残差向量产生 $\mathcal{G}_j$ 中的残差向量, IDR(s) 方法需要 $s+1$ 次矩阵向量乘法. 由 Remark 3.14, $\mathcal{G}_{N/s}$ 中的残差向量为零. 因此, 在精确运算中, IDR(s) 方法得到精确解所需的矩阵向量乘法次数至多为 $(s+1)\times N/s = N + N/s$. 注意, Section 3.3.8 中乘积型 Krylov 子空间方法的矩阵向量乘法次数至多为 $2N$.

由下面的注, IDR(s) 方法可视为 BiCGSTAB 方法的推广.

Remark 3.16: IDR(1) 方法和 BiCGSTAB 方法在偶数步给出相同的残差向量.

Remark 3.17: 从正交性看, IDR(s) 方法产生的残差向量属于下列所谓的 Sonneveld 子空间:

$$\{\Omega_j(A)\boldsymbol{v} : \boldsymbol{v} \perp \mathcal{B}_j(A^{\mathrm{H}}, S)\}. \tag{3.112}$$

这里 $\Omega_j(A) := \prod_{i=1}^{j}(I - \omega_i A)$, $\mathcal{B}_j(A^{\mathrm{H}}, S)$ 为使用 A 和 $S := [\boldsymbol{s}_1, \ldots, \boldsymbol{s}_s]$ 生成的 $\mathbb{C}^N$ 的子空间, 定义为

$$\mathcal{B}_j(A^{\mathrm{H}}, S) := \mathcal{K}_j(A^{\mathrm{H}}, \boldsymbol{s}_1) + \cdots + \mathcal{K}_j(A^{\mathrm{H}}, \boldsymbol{s}_s).$$

这里符号 "+" 表示子空间之和.[4] $\mathcal{B}_j(A^{\mathrm{H}}, R_0^*)$ 将在块 Krylov 子空间方法的 Definition 3.18 中重新定义. 关于 (3.112) 这一观点的详情, 见 [161], [163].

使用 $s = 2$ 的 IDR(s) 方法, 对 (3.112) 中 $j = 1, 2$ 两种情形, 粗略说明 Remark 3.17. 首先考虑 $j = 1$ 的情形. 由 (3.105), $\mathcal{G}_1$ 由下式生成

$$\mathcal{G}_1 = (I - \omega_1 A)(\mathcal{G}_0 \cap \mathcal{S}),$$

其中 $\mathcal{S} = \mathrm{span}\{\boldsymbol{s}_1, \boldsymbol{s}_2\}^{\perp}$, 另见 (3.107). 则 $\boldsymbol{r} \in \mathcal{G}_1$ 可写为

$$\boldsymbol{r} = (I - \omega_1 A)\boldsymbol{v}, \quad \boldsymbol{v} \in \mathcal{G}_0, \quad \boldsymbol{v} \perp \mathrm{span}\{\boldsymbol{s}_1, \boldsymbol{s}_2\}.$$

注意, $\boldsymbol{v} \in \mathcal{S}$ 意味着 $\boldsymbol{v} \perp \mathrm{span}\{\boldsymbol{s}_1, \boldsymbol{s}_2\}$. 由于 $\mathcal{B}_1(A^{\mathrm{H}}, S) = \mathcal{K}_1(A^{\mathrm{H}}, \boldsymbol{s}_1) + \mathcal{K}_1(A^{\mathrm{H}}, \boldsymbol{s}_2) = \mathrm{span}\{\boldsymbol{s}_1, \boldsymbol{s}_2\}$, 有

$$\boldsymbol{r} \in \{(I - \omega_1 A)\boldsymbol{v} : \boldsymbol{v} \perp \mathcal{B}_1(A^{\mathrm{H}}, S)\} = \{\Omega_1(A)\boldsymbol{v} : \boldsymbol{v} \perp \mathcal{B}_1(A^{\mathrm{H}}, S)\},$$

这对应于 (3.112) 中 $j = 1$ 的情形. 接着考虑 $j = 2$ 的情形. $\mathcal{G}_2$ 由下式生成

$$\mathcal{G}_2 = (I - \omega_2 A)(\mathcal{G}_1 \cap \mathcal{S}).$$

则残差向量 $\widetilde{\boldsymbol{r}} \in \mathcal{G}_2$ 可写为

$$\widetilde{\boldsymbol{r}} = (I - \omega_2 A)\widetilde{\boldsymbol{v}}, \quad \widetilde{\boldsymbol{v}} \in \mathcal{G}_1, \quad \widetilde{\boldsymbol{v}} \perp \mathrm{span}\{\boldsymbol{s}_1, \boldsymbol{s}_2\}. \tag{3.113}$$

由于 $\widetilde{\boldsymbol{v}} \in \mathcal{G}_1 = (I - \omega_1 A)(\mathcal{G}_0 \cap \mathcal{S})$, 向量 $\widetilde{\boldsymbol{v}}$ 可写为

$$\widetilde{\boldsymbol{v}} = (I - \omega_1 A)\boldsymbol{v}, \quad \boldsymbol{v} \in \mathcal{G}_0, \quad \boldsymbol{v} \perp \mathrm{span}\{\boldsymbol{s}_1, \boldsymbol{s}_2\}. \tag{3.114}$$

由 (3.113) 和 (3.114), 有

$$\widetilde{\boldsymbol{r}} = (I - \omega_2 A)(I - \omega_1 A)\boldsymbol{v}, \tag{3.115}$$

其中

$$\widetilde{\boldsymbol{v}} = (I - \omega_1 A)\boldsymbol{v} \perp \mathrm{span}\{\boldsymbol{s}_1, \boldsymbol{s}_2\}, \quad \boldsymbol{v} \perp \mathrm{span}\{\boldsymbol{s}_1, \boldsymbol{s}_2\}.$$

注意

$$\begin{aligned}(I - \omega_1 A)\boldsymbol{v} \perp \mathrm{span}\{\boldsymbol{s}_1, \boldsymbol{s}_2\} &\Leftrightarrow ((I - \omega_1 A)\boldsymbol{v})^{\mathrm{H}}\boldsymbol{s}_i = 0 \quad (i = 1, 2)\\ &\Leftrightarrow \boldsymbol{v}^{\mathrm{H}}(I - \omega_1 A)^{\mathrm{H}}\boldsymbol{s}_i = 0 \quad (i = 1, 2)\\ &\Leftrightarrow \boldsymbol{v} \perp \mathrm{span}\{(I - \omega_1 A)^{\mathrm{H}}\boldsymbol{s}_1, (I - \omega_1 A)^{\mathrm{H}}\boldsymbol{s}_2\}.\end{aligned}$$

[4] 给定两个子空间 W_1 和 W_2, W_1 和 W_2 的和定义为 $W_1 + W_2 := \{\boldsymbol{w}_1 + \boldsymbol{w}_2 : \boldsymbol{w}_1 \in W_1, \boldsymbol{w}_2 \in W_2\}$.

$\boldsymbol{v} \perp \mathrm{span}\{(I-\omega_1 A)^{\mathrm{H}}\boldsymbol{s}_1, (I-\omega_1 A)^{\mathrm{H}}\boldsymbol{s}_2\}$ 及 $\boldsymbol{v} \perp \mathrm{span}\{\boldsymbol{s}_1, \boldsymbol{s}_2\}$ 表明

$$\boldsymbol{v} \perp \boldsymbol{s}_1, \quad \boldsymbol{v} \perp \boldsymbol{s}_2, \quad \boldsymbol{v} \perp A^{\mathrm{H}}\boldsymbol{s}_1, \quad \boldsymbol{v} \perp A^{\mathrm{H}}\boldsymbol{s}_1.$$

因此,

$$\begin{aligned} \boldsymbol{v} \perp \mathrm{span}\{\boldsymbol{s}_1, A^{\mathrm{H}}\boldsymbol{s}_1, \boldsymbol{s}_2, A^{\mathrm{H}}\boldsymbol{s}_2\}. \\ = \mathrm{span}\{\boldsymbol{s}_1, A^{\mathrm{H}}\boldsymbol{s}_1\} + \mathrm{span}\{\boldsymbol{s}_2, A^{\mathrm{H}}\boldsymbol{s}_2\} \\ = \mathcal{K}_2(A^{\mathrm{H}}, \boldsymbol{s}_1) + \mathcal{K}_2(A^{\mathrm{H}}, \boldsymbol{s}_2) \\ = \mathcal{B}_2(A^{\mathrm{H}}, S). \end{aligned}$$

于是, 结合 (3.115) 与上述结果, 有

$$\widetilde{\boldsymbol{r}} = (I-\omega_2 A)(I-\omega_1 A)\boldsymbol{v}, \quad \boldsymbol{v} \perp \mathcal{B}_2(A^{\mathrm{H}}, S), \tag{3.116}$$

由此得到

$$\widetilde{\boldsymbol{r}} \in \{\Omega_2(A)\boldsymbol{v} : \boldsymbol{v} \perp \mathcal{B}_2(A^{\mathrm{H}}, S)\}. \tag{3.117}$$

这对应于 (3.112) 中 $j = 2$ 的情形. 下面介绍与 IDR(s) 方法有关的一些文献. 文献 [88] 对 IDR(s) 方法给出了优美的解释, 其中 [88], Figure 3.1 对 IDR 定理的可视化有助于直观理解 IDR 定理 (Theorem 3.12). 同一批作者在 [199] 中提出了更稳定且更精确的 IDR(s) 变型, 文献 [47] 提出了采用准最小残差策略的 IDR(s) 方法. 文献 [209] 提出了采用部分标准正交化的 IDR(s) 变型. 使用 BiCGSTAB(ℓ) 方法思想的 IDR(s) 推广称为 IDRstab 方法 [164] 和 GBiCGSTAB(s, L) 方法 [185]. IDRstab 方法的变型见 [5]–[7].

3.3.9.2 数值例子

本小节通过一个数值实验展示 IDR(s) 方法的典型收敛行为. 考虑非对称线性方程组 (2.18), 参数为 $N = 20$, $a_1 = a_2 = a_3 = 1$, $b_1 = b_2 = b_3 = 1$, 以及 $c = 300$. 停止准则为 $\|\boldsymbol{r}_n\|/\|\boldsymbol{b}\| \leq 10^{-10}$. IDR($s$) 方法 ($s = 1, 2, 4$) 和 BiCGSTAB 方法的收敛历程见 Figure 3.6. 横轴为矩阵向量乘法的次数. 由 Figure 3.6 可见, 随着 IDR(s) 方法中的数 s 增大, 所需矩阵向量乘法次数减少. IDR(4) 方法和 IDR(2) 方法比 BiCGSTAB 方法收敛更快. 从精度看, BiCGSTAB 方法, IDR(1) 方法, IDR(2) 方法和 IDR(4) 方法的真实相对残差 2-范数分别为 -10.50, -10.04, -10.2, -8.76.

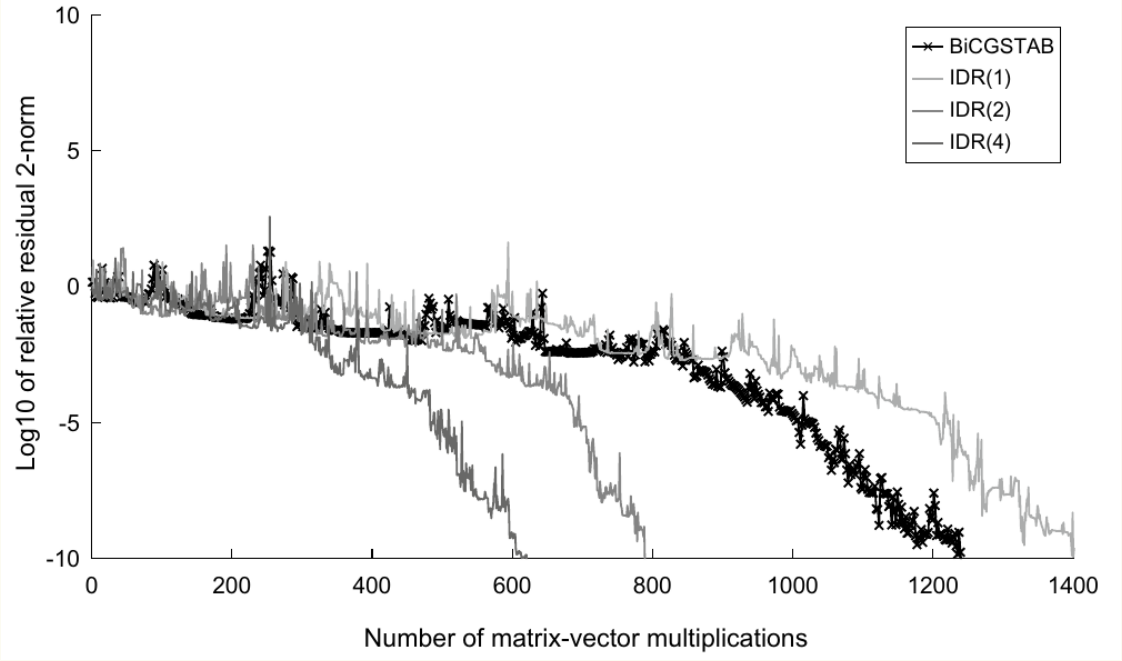


图 3.6: BiCGSTAB, IDR(1), IDR(2) 和 IDR(4) 的收敛历程

3.3.10 块诱导降维 (块 IDR(s)) 方法

块 IDR(s) 方法 [46] 是 IDR(s) 方法的推广, 用以求解形如下式的块线性方程组

$$AX = B, \tag{3.118}$$

其中 $A \in \mathbb{C}^{N\times N}$, $B \in \mathbb{C}^{N\times m}$, 且 m 通常远小于 N. 在介绍块 IDR(s) 方法的理论之前, 先在下列定义之后简要描述块 Krylov 子空间方法的框架:

Definition 3.18 (块 Krylov 子空间): 设 $V \in \mathbb{C}^{N\times m}$. 则由下式定义的 $\mathcal{K}_n(A,V) \subset \mathbb{C}^{N\times m}$

$$\begin{aligned}\mathcal{K}_n(A,V) &:= \{VC_0 + AVC_1 + \cdots + A^{n-1}VC_{n-1} : C_0, C_1, \ldots, C_{n-1} \in \mathbb{C}^{m\times m}\} \\ &\quad (= \text{block span}\{V, AV, \ldots, A^{n-1}V\})\end{aligned}$$

称为关于 A 和 V 的块 Krylov 子空间.

Definition 3.19 (Krylov 子空间之和): 设 $V = [\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_m] \in \mathbb{C}^{N\times m}$. 则 $\mathcal{B}_n(A,V) \subset \mathbb{C}^N$ 定义为

$$\begin{aligned}\mathcal{B}_n(A,V) &:= \mathcal{K}_n(A,\boldsymbol{v}_1) + \mathcal{K}_n(A,\boldsymbol{v}_2) + \cdots + \mathcal{K}_n(A,\boldsymbol{v}_m) \\ &= \{V\boldsymbol{c}_0 + AV\boldsymbol{c}_1 + \cdots + A^{n-1}V\boldsymbol{c}_{n-1} : \boldsymbol{c}_0, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{n-1} \in \mathbb{C}^m\}.\end{aligned}$$

Definition 3.20 (块 Krylov 子空间的阶): 满足下式的最小数 n

$$\dim(\mathcal{B}_n(A,V)) = \dim(\mathcal{B}_{n+1}(A,V))$$

称为 A 关于 V 的块阶, 记为 $\nu(V,A)$.

Remark 3.21: Definition 3.18 和 Definition 3.20 是 Definition 1.5 和 Definition 1.6 的推广. 事实上, 若 $m = 1$, 这些定义就与 Definition 1.5 和 Definition 1.6 相同.

现在描述块 Krylov 子空间方法的框架. 设 $X_0 \in \mathbb{C}^{N\times s}$ 为初始猜测, $R_0 := B - AX_0$ 为相应的初始残差矩阵. 则块 Krylov 子空间方法按下式寻找第 n 个近似解 X_n:

$$X_n = X_0 + Z_n, \quad Z_n \in \mathcal{K}_n(A, R_0).$$

相应的残差矩阵为

$$R_n = B - AX_n = R_0 - AZ_n \in \mathcal{K}_{n+1}(A, R_0).$$

确定 Z_n 就得到各种块 Krylov 子空间方法, 例如,

- 块 CG 方法: $R_n \perp \mathcal{B}_n(A, R_0)$.
- 块 COCG 方法: $R_n \perp \overline{\mathcal{B}_n(A, R_0)}$.
- 块 BiCG 方法: $R_n \perp \mathcal{B}_n(A^{\mathrm{H}}, R_0^*)$.

这里 R_0^* 为初始影子残差矩阵,[5] 符号 $R_n \perp \mathcal{B}_n(A, V)$ 表示 R_n 的所有列向量都与 $\mathcal{B}_n(A, V)$ 正交. 这里简要回顾块 Krylov 子空间方法的历史. 1980 年, O'Leary 在 [142] 中提出块 BiCG 方法 (及块 CG 方法). 1990 年, Vital 在 [202] 中提出块 GMRES 方法. 1997 年提出了基于 QMR 方法的块 Krylov 子空间方法 [65], [157]. 2003 年, Guennouni 等提出块 BiCGSTAB 方法 [86]. 2011 年, Du 等提出块 IDR(s) 方法 [46]. 其他块 Krylov 子空间方法见 [153], [181], [182], [210]. 现在转向块 IDR(s) 方法的推导. 块 IDR(s) 方法基于下述定理 (见 [46]), 它是 Theorem 3.12 的轻微修改.

Theorem 3.22:

设 A 为 $\mathbb{C}^{N\times N}$ 中的任意矩阵, $\boldsymbol{v}_0$ 为 $\mathbb{C}^N$ 中的任意非零向量, 并设 $\mathcal{G}_0 = \mathcal{B}_{\nu(R_0,A)}(A, R_0)$. 设 $\mathcal{S}$ 表示 $\mathbb{C}^N$ 的任意 (真) 子空间, 使 $\mathcal{S}$ 和 $\mathcal{G}_0$ 不具有共同的非平凡不变子空间,[a] 并定义序列 $\mathcal{G}_j$, $j = 1, 2, \ldots$ 为

$$\mathcal{G}_j = (I - \omega_j A)(\mathcal{G}_{j-1} \cap \mathcal{S}), \tag{3.119}$$

其中 ω_j 为非零标量. 则:

(1) $\mathcal{G}_j \subset \mathcal{G}_{j-1}$ 对所有 $j > 0$ 成立.

(2) $\mathcal{G}_j = \{\mathbf{0}\}$ 对某个 $j \le N$ 成立.

[a] 这意味着 $\mathcal{S} \cap \mathcal{G}_0$ 不包含 A 的任何特征向量, 平凡不变子空间为 $\{\mathbf{0}\}$.

Proof: Theorem 3.12 和 Theorem 3.22 的唯一区别在于 $\mathcal{G}_0$ 的定义, 即 $\mathcal{G}_0 = \mathcal{K}_N(A, \boldsymbol{v}_0)$ 或 $\mathcal{G}_0 = \mathcal{B}_{\nu(R_0,A)}(A, R_0)$. 在 Theorem 3.12-(1) 的证明中, $\mathcal{G}_0 = \mathcal{K}_N(A, \boldsymbol{v}_0)$ 仅用于证明 $\mathcal{G}_1 \subset \mathcal{G}_0$. 因此只需证明 $\mathcal{G}_1 \subset \mathcal{G}_0$. 由 $\mathcal{G}_0 = \mathcal{B}_{\nu(R_0,A)}(A, R_0)$ 有 $(I - \omega_1 A)\mathcal{G}_0 \subset \mathcal{G}_0$, 因为对于给定的 $\boldsymbol{v} \in (I - \omega_1 A)\mathcal{G}_0 = (I - \omega_1 A)\mathcal{B}_{\nu(R_0,A)}(A, R_0)$, 有

$$\begin{aligned}
\boldsymbol{v} &= (I - \omega_1 A)(R_0\boldsymbol{c}_0 + AR_0\boldsymbol{c}_1 + \cdots + A^{\nu(R_0,A)-1}R_0\boldsymbol{c}_{\nu(R_0,A)-1}) \\
&= (I - \omega_1 A)R_0\boldsymbol{c}_0 + (I - \omega_1 A)AR_0\boldsymbol{c}_1 + \cdots + (I - \omega_1 A)A^{\nu(R_0,A)-1}R_0\boldsymbol{c}_{\nu(R_0,A)-1} \\
&= R_0\boldsymbol{d}_0 + AR_0\boldsymbol{d}_1 + \cdots + A^{\nu(R_0,A)-1}R_0\boldsymbol{d}_{\nu(R_0,A)-1} + A^{\nu(R_0,A)}R_0\boldsymbol{d}_{\nu(R_0,A)} \\
&= R_0\widetilde{\boldsymbol{d}}_0 + AR_0\widetilde{\boldsymbol{d}}_1 + \cdots + A^{\nu(R_0,A)-1}R_0\widetilde{\boldsymbol{d}}_{\nu(R_0,A)-1} \\
&\in \mathcal{G}_0.
\end{aligned}$$

[5] 实际中通常取 $R_0^* = R_0$, 或将 R_0^* 设为随机矩阵.

因此 $\mathcal{G}_1 = (I - \omega_1 A)(\mathcal{G}_0 \cap \mathcal{S}) \subset (I - \omega_1 A)\mathcal{G}_0 \subset \mathcal{G}_0$. 按照 Theorem 3.12-(2) 的证明并使用 $\dim \mathcal{G}_0 \leq N$, 即可证明 Theorem 3.22-(2).

■

与 Corollary 3.13 类似, 有下列事实:

Corollary 3.23: 对于给定的 $s, m \geq 1$, 设 $\mathcal{S}$ 为 $\mathbb{C}^N$ 的子空间, 且 $\dim(\mathcal{S}) = N - sm$. 若 $\mathcal{G}_{i-1} + \mathcal{S} = \mathbb{C}^N$ 且 $(I - \omega_j A)$ 非奇异, 则

$$\dim(\mathcal{G}_j) = \dim(\mathcal{G}_{j-1}) - sm.$$

Corollary 3.13 与 Corollary 3.23 的区别在于参数 m. 参数 m 对应于 $AX = B$ 的右端项个数, 其中 B 为 N 行 m 列的矩阵. 下面解释块 IDR(s) 方法. 块 IDR(s) 方法对 $k = 0, 1, \ldots$ 生成残差矩阵 $R_k = B - AX_k$, 使得:

1. 对 $j = 0, 1, 2, \ldots$, $R_{j(s+1)}, R_{j(s+1)+1}, \ldots, R_{j(s+1)+s}$ 的所有列向量属于 (3.119) 中的 $\mathcal{G}_j$;
2. 选择 (3.119) 中的子空间 $\mathcal{S}$, 使 $\dim \mathcal{S} = N - sm$.

设 $S = [\boldsymbol{s}_1, \boldsymbol{s}_2, \ldots, \boldsymbol{s}_{sm}] \in \mathbb{C}^{N \times sm}$. 则子空间 $\mathcal{S}$ 的标准选择如下:

$$\mathcal{S} = \mathrm{span}\{S\}^{\perp} = \mathrm{span}\{\boldsymbol{s}_1, \boldsymbol{s}_2, \ldots, \boldsymbol{s}_{sm}\}^{\perp},$$

其中 $S \in \mathbb{C}^{N \times sm}$ 的所有列向量线性无关, 因此容易看出 $\dim \mathcal{S} = N - sm$. 现在简要说明 $s = 2$ 的块 IDR(s) 方法. 对 $j = 0, 1, 2$, 块 IDR(2) 方法产生下列残差矩阵:

$$R_0, R_1, R_2, \quad R_3, R_4, R_5, \quad R_6, R_7, R_8,$$

其中 R_0, R_1, R_2 的所有列向量属于 $\mathcal{G}_0$, R_3, R_4, R_5 的所有列向量属于 $\mathcal{G}_1$, R_3, R_4, R_5 的所有列向量属于 $\mathcal{G}_2$. 下面在 R_0, R_1, R_2 的所有列向量属于 $\mathcal{G}_0$ 的假设下, 说明如何从 R_0, R_1, R_2 得到 R_3, R_4, R_5. 与 IDR(s) 方法类似, 下列定义对块 IDR(s) 方法有用:

$$\Delta R_k := R_{k+1} - R_k, \quad dR_k^{\blacksquare} := [\Delta R_k, \Delta R_{k+1}], \quad C_k = (S^{\mathrm{H}} dR_k^{\blacksquare})^{-1} S^{\mathrm{H}} R_{k+2}. \tag{3.120}$$

下列矩阵 V_0 的所有列向量位于 $\mathcal{G}_0 \cap \mathcal{S}$ 中:

$$V_0 = R_2 - dR_0^{\blacksquare} C_0, \quad \mathrm{Im}(V_0) \subset \mathcal{G}_0 \cap \mathcal{S}$$

因为由 R_0, R_1, R_2 的所有列向量属于 $\mathcal{G}_0$ 这一假设, $R_2 \in \mathbb{C}^{N \times m}$ 和 $dR_0 \in \mathbb{C}^{N \times 2m}$ 的所有列向量属于 $\mathcal{G}_0$; 又由关系 $S^{\mathrm{H}} V_0 = O$, V_0 的所有列向量也属于 $\mathcal{S}$, 其中 O 为 $2m \times m$ 零矩阵. 这里 $\mathrm{Im}(V_0)$ 为 V_0 的像, 即 V_0 的所有列向量张成的子空间. 由 (3.119), 列向量属于 $\mathcal{G}_1$ 的残差矩阵 R_3 可按下式产生

$$R_3 = (I - \omega_0 A) V_0, \quad \mathrm{Im}(R_3) \subset \mathcal{G}_1, \tag{3.121}$$

其中通常选择 ω_0 使 $\|R_3\|_{\mathrm{F}}$ 达到极小值,[6] 即,

$$\omega_0 = \frac{\mathrm{Tr}(T_0^{\mathrm{H}} V_0)}{\mathrm{Tr}(T_0^{\mathrm{H}} T_0)}, \quad T_0 := A V_0.$$

Frobenius 范数的极小化来自下列事实:

[6] 对 $R = (r_{i,j}) \in \mathbb{C}^{N \times m}$, 符号 $\|R\|_{\mathrm{F}} := (\sum_{i=1}^{N} \sum_{j=1}^{m} \bar{r}_{i,j} r_{i,j})^{1/2}$ 称为 R 的 Frobenius 范数; 对方阵 $A = (a_{i,j}) \in \mathbb{C}^{N \times N}$, 符号 $\mathrm{Tr}(A) := \sum_{i=1}^{N} a_{i,i}$ 称为 A 的迹.

Proposition 3.24: 设 $V, W \in \mathbb{C}^{m\times n}$. 则下列极小化问题:

$$\min_{\omega\in\mathbb{C}} \|V - \omega W\|_{\mathrm{F}}$$

可由下式求解

$$\omega = \frac{\mathrm{Tr}(W^{\mathrm{H}}V)}{\mathrm{Tr}(W^{\mathrm{H}}W)}.$$

Proof: 设 $V = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n]$, $W = [\boldsymbol{w}_1, \ldots, \boldsymbol{w}_n]$, 则使用

$$\boldsymbol{v} := \mathrm{vec}(V) = \begin{bmatrix} \boldsymbol{v}_1 \\ \vdots \\ \boldsymbol{v}_n \end{bmatrix}, \quad \boldsymbol{w} := \mathrm{vec}(W) = \begin{bmatrix} \boldsymbol{w}_1 \\ \vdots \\ \boldsymbol{w}_n \end{bmatrix}$$

得到 $\|V - \omega W\|_{\mathrm{F}} = \|\boldsymbol{v} - \omega\boldsymbol{w}\|$. 因而由 (1.34), 极小化参数为

$$\omega = \frac{\boldsymbol{w}^{\mathrm{H}}\boldsymbol{v}}{\boldsymbol{w}^{\mathrm{H}}\boldsymbol{w}} = \frac{\mathrm{Tr}(W^{\mathrm{H}}V)}{\mathrm{Tr}(W^{\mathrm{H}}W)},$$

证明完毕.

■

与 IDR(s) 方法中 $\boldsymbol{r}_3$ 和 $\boldsymbol{r}_4$ 的推导类似, 按下式得到 R_3 和 R_4:

$$\begin{aligned}
V_1 &= R_3 - dR_1^{\blacksquare}C_1, \quad \mathrm{Im}(V_1) \subset \mathcal{G}_0 \cap \mathcal{S}, \\
R_4 &= (I - \omega_0 A)V_1, \quad \mathrm{Im}(R_4) \subset \mathcal{G}_1, \\
V_2 &= R_4 - dR_2^{\blacksquare}C_2, \quad \mathrm{Im}(V_2) \subset \mathcal{G}_0 \cap \mathcal{S}, \\
R_5 &= (I - \omega_0 A)V_2, \quad \mathrm{Im}(R_5) \subset \mathcal{G}_1.
\end{aligned}$$

由上述计算, 得到残差矩阵 R_k ($k = 0, \ldots, 5$), 使得

$$\begin{aligned}
&\mathrm{Im}(R_k) \subset \mathcal{G}_0 \quad \text{for } k = 0, 1, 2, \\
&\mathrm{Im}(R_k) \subset \mathcal{G}_1 \quad \text{for } k = 3, 4, 5.
\end{aligned}$$

接着说明如何从 X_0, X_1, X_2 得到近似解矩阵 X_3, X_4, X_5. 令

$$\Delta X_k := X_{k+1} - X_k, \qquad dX_k^{\blacksquare} := [\Delta X_k, \Delta X_{k+1}].$$

则按下式得到 X_3:

$$\begin{aligned}
R_3 = (I - \omega_0 A)V_0 &= R_2 - dR_0^{\blacksquare}C_0 - \omega_0 AV_0 \\
\Leftrightarrow B - AX_3 &= B - AX_2 - dR_0^{\blacksquare}C_0 - \omega_0 AV_0 \\
\Leftrightarrow X_3 &= X_2 + A^{-1}dR_0^{\blacksquare}C_0 + \omega_0 V_0 \\
\Leftrightarrow X_3 &= X_2 - dX_0^{\blacksquare}C_0 + \omega_0 V_0.
\end{aligned}$$

类似地,

$$\begin{aligned}
X_4 &= X_3 - dX_1^{\blacksquare}C_1 + \omega_0 V_1, \\
X_5 &= X_4 - dX_2^{\blacksquare}C_2 + \omega_0 V_2.
\end{aligned}$$

已说明在 $s = 2$ 的块 IDR(s) 方法中, 如何从 $k = 0, 1, 2$ 的 R_k 和 X_k 得到 $k = 3, 4, 5$ 的 R_k 和 X_k. 按照上述推导, 得到 Algorithm 3.25 中的块 IDR(s) 方法. 此外, 块 IDR(s) 方法的预条件版本见 Algorithm 3.26. 注意, 符号 "$B(:, \ell)$" 表示矩阵 B 的第 ℓ 列, 符号 "$\Delta X(:, im+1 : (i+1)m)$" 表示矩阵 ΔX 的第 $(im+1)$, $(im+2)$, …, $(i+1)m$ 列.

Remark 3.25: 由 Corollary 3.23, 维数的减少量通常为 sm, 即 $\dim(\mathcal{G}_j) = \dim(\mathcal{G}_{j-1}) - sm$, 块 IDR($s$) 方法需要 $m(s+1)$ 次矩阵向量乘法. 因此, 在精确运算中, 块 IDR(s) 方法得到精确解矩阵所需的矩阵向量乘法总次数至多为 $m(s+1) \times N/sm = N + N/s$, 不依赖于块大小 m. 注意, 若对 $AX = B$ 应用 IDR(s) 方法 (即 $A\boldsymbol{x}_1 = \boldsymbol{b}_1, A\boldsymbol{x}_2 = \boldsymbol{b}_2, \ldots, A\boldsymbol{x}_m = \boldsymbol{b}_m$), 则 IDR($s$) 方法所需的矩阵向量乘法次数至多为 $m(N + N/s)$, 是块 IDR(s) 方法的 m 倍.

由下面的注, 块 IDR(s) 方法可视为块 BiCGSTAB 方法的推广.

Remark 3.26: 块 IDR(1) 方法和块 BiCGSTAB 方法在偶数步给出相同的残差向量.

Algorithm 3.25 块 IDR(s) 方法

Input: $X_0 \in \mathbb{C}^{N\times s},\ R_0 = B - AX_0,\ P \in \mathbb{C}^{n\times sm}$

Output: X_i

for $i = 0$ 至 $s-1$ **do**
  $V = AR_i, \quad \omega = \dfrac{\mathrm{Tr}(V^{\mathrm{H}}R_i)}{\mathrm{Tr}(V^{\mathrm{H}}V)}$
  $\Delta X(:, im+1:(i+1)m) = \omega R_i, \quad \Delta R(:, im+1:(i+1)m) = -\omega V$
  $X_{i+1} = X_i + \Delta X(:, im+1:(i+1)m), \quad R_{i+1} = R_i + \Delta R(:, im+1:(i+1)m)$
end for
$j = 1, \quad i = s$
$M = P^{\mathrm{H}}\Delta R, \quad H = P^{\mathrm{H}}R_i$
while $\max\limits_{\ell\in\{1,2,\ldots,m\}} \dfrac{\|R_i(:,\ell)\|_{\mathrm{F}}}{\|B(:,\ell)\|_{\mathrm{F}}} > \epsilon$ **do**
  for $k = 0$ 至 s **do**
    由 $MC = H$ 解出 C
    $Q = -\Delta RC,$
    $V = R_i + Q$
    if $k = 0$ **then**
      $T = AV, \quad \omega = \dfrac{\mathrm{Tr}(T^{\mathrm{H}}V)}{\mathrm{Tr}(T^{\mathrm{H}}T)}$
      $\Delta R(:, (j-1)m+1:jm) = Q - \omega T$
      $\Delta X(:, (j-1)m+1:jm) = -\Delta XC + \omega V$
    else
      $\Delta X(:, (j-1)m+1:jm) = -\Delta XC + \omega V$
      $\Delta R(:, (j-1)m+1:jm) = -A\Delta X(:, (j-1)m+1:jm)$
    end if
    $R_{i+1} = R_i + \Delta R(:, (j-1)m+1:jm)$
    $X_{i+1} = X_i + \Delta X(:, (j-1)m+1:jm)$
    $\Delta M = P^{\mathrm{H}}\Delta R(:, (j-1)m+1:jm)$
    $M(:, (j-1)m+1:jm) = \Delta M$
    $H = H + \Delta M$
    $i = i+1, \quad j = j+1$
    $j = (j-1)\%s + 1$ (%: 取模运算, 即 $a\%n = r$, 其中 $a = mn + r$.)
  end for
end while

Algorithm 3.26 预条件块 IDR(s) 方法

Input: $X_0 \in \mathbb{C}^{N\times s},\ R_0 = B - AX_0,\ P \in \mathbb{C}^{n\times sm}$

Input: $X_0 = KX_0$ (K: 预条件矩阵)

Output: X_i

for $i = 0$ 至 $s-1$ **do**
  $W = K^{-1}R_i$
  $V = AW, \quad \omega = \dfrac{\mathrm{Tr}((K_1^{-1}V)^{\mathrm{H}}K_1^{-1}R_i)}{\mathrm{Tr}((K_1^{-1}V)^{\mathrm{H}}K_1^{-1}V)}$
  $\Delta X(:, im+1:(i+1)m) = \omega R_i, \quad \Delta R(:, im+1:(i+1)m) = -\omega V$
  $X_{i+1} = X_i + \Delta X(:, im+1:(i+1)m), \quad R_{i+1} = R_i + \Delta R(:, im+1:(i+1)m)$
end for
$j = 1, \quad i = s$
$M = P^{\mathrm{H}}\Delta R, \quad H = P^{\mathrm{H}}R_i$
while $\max\limits_{\ell\in\{1,2,\ldots,m\}} \dfrac{\|R_i(:,\ell)\|_{\mathrm{F}}}{\|B(:,\ell)\|_{\mathrm{F}}} > \epsilon$ **do**
  for $k = 0$ 至 s **do**
    由 $MC = H$ 解出 C
    $Q = -\Delta RC$
    $V = R_i + Q$
    if $k = 0$ **then**
      $T = AK^{-1}V, \quad \omega = \dfrac{\mathrm{Tr}((K_1^{-1}T)^{\mathrm{H}}K_1^{-1}V)}{\mathrm{Tr}((K_1^{-1}T)^{\mathrm{H}}K_1^{-1}T)}$
      $\Delta R(:, (j-1)m+1:jm) = Q - \omega T$
      $\Delta X(:, (j-1)m+1:jm) = -\Delta XC + \omega V$
    else
      $\Delta X(:, (j-1)m+1:jm) = -\Delta XC + \omega V$
      $\Delta R(:, (j-1)m+1:jm) = -AK^{-1}\Delta X(:, (j-1)m+1:jm)$
    end if
    $R_{i+1} = R_i + \Delta R(:, (j-1)m+1:jm)$
    $X_{i+1} = X_i + \Delta X(:, (j-1)m+1:jm)$
    $\Delta M = P^{\mathrm{H}}\Delta R(:, (j-1)m+1:jm)$
    $M(:, (j-1)m+1:jm) = \Delta M$
    $H = H + \Delta M$
    $i = i+1, \quad j = j+1$
    $j = (j-1)\%s + 1$ (%: 取模运算, 即 $a\%n = r$, 其中 $a = mn + r$.)
  end for
end while
$X_i = K^{-1}X_i$

3.4 其他 Krylov 子空间方法

本节介绍基于正规方程和增广线性方程组的 Krylov 子空间方法: CGNE 方法, CGNR 方法和 LSQR 方法.

3.4.1 正规方程的 Krylov 子空间方法

如 Section 3.3 的引言所述, 由于 $A^{\mathrm{H}}A$ 为 Hermitian 正定矩阵, 可将 CG 方法应用于正规方程 (3.56). 本小节介绍其细节. 不直接考虑非 Hermitian 线性方程组 $A\boldsymbol{x}=\boldsymbol{b}$, 而考虑下列正规方程:

$$A^{\mathrm{H}}A\boldsymbol{x}=A^{\mathrm{H}}\boldsymbol{b} \tag{3.122}$$

或

$$AA^{\mathrm{H}}\boldsymbol{y}=\boldsymbol{b},\quad \boldsymbol{x}=A^{\mathrm{H}}\boldsymbol{y}. \tag{3.123}$$

如前所述, 由于 $A^{\mathrm{H}}A$ 和 AA^{H} 为 Hermitian 正定矩阵, 用 CG 方法求解正规方程是自然的. 将 CG 方法应用于 (3.122), 就得到 Algorithm 3.27 中的 CGNR 方法 (或 CGLS 方法) [95].

Algorithm 3.27 CGNR 方法

Input: $\boldsymbol{x}_0\in\mathbb{C}^N,\ \beta_{-1}=0,\ \boldsymbol{p}_{-1}=\boldsymbol{0},\ \boldsymbol{r}_0=\boldsymbol{b}-A\boldsymbol{x}_0$

Output: $\boldsymbol{x}_n$

1: **for** $n=0,1,\ldots,$ 直至收敛 **do**
2: $\quad\boldsymbol{p}_n=A^{\mathrm{H}}\boldsymbol{r}_n+\beta_{n-1}\boldsymbol{p}_{n-1}$
3: $\quad\alpha_n=\dfrac{(A^{\mathrm{H}}\boldsymbol{r}_n,A^{\mathrm{H}}\boldsymbol{r}_n)}{(A\boldsymbol{p}_n,A\boldsymbol{p}_n)}$
4: $\quad\boldsymbol{x}_{n+1}=\boldsymbol{x}_n+\alpha_n\boldsymbol{p}_n$
5: $\quad\boldsymbol{r}_{n+1}=\boldsymbol{r}_n-\alpha_nA\boldsymbol{p}_n$
6: $\quad\beta_n=\dfrac{(A^{\mathrm{H}}\boldsymbol{r}_{n+1},A^{\mathrm{H}}\boldsymbol{r}_{n+1})}{(A^{\mathrm{H}}\boldsymbol{r}_n,A^{\mathrm{H}}\boldsymbol{r}_n)}$
7: **end for**

由 CGNR 方法的算法和 Theorem 3.3 易知, CGNR 方法使误差的 $A^{\mathrm{H}}A$-范数极小, 或使残差向量的 2-范数极小:

$$\min_{\boldsymbol{x}_n\in\boldsymbol{x}_0+\mathcal{K}_n(A^{\mathrm{H}}A,A^{\mathrm{H}}\boldsymbol{r}_0)}\|\boldsymbol{x}-\boldsymbol{x}_n\|_{A^{\mathrm{H}}A}=\min_{\boldsymbol{x}_n\in\boldsymbol{x}_0+\mathcal{K}_n(A^{\mathrm{H}}A,A^{\mathrm{H}}\boldsymbol{r}_0)}\|\boldsymbol{r}_n\|,$$

其中 $\boldsymbol{r}_n:=\boldsymbol{b}-A\boldsymbol{x}_n$. 另一方面, 将 CG 方法应用于 (3.123), 就得到 Algorithm 3.28 中的 CGNE 方法 [41] (或 Craig 方法). 由 CGNE 方法的算法和 (3.16) 易知, CGNE 方法使线性方程组 $AA^{\mathrm{H}}\boldsymbol{y}=\boldsymbol{b}$ 的误差的 AA^{H}-范数极小:

$$\min_{\boldsymbol{y}_n\in\boldsymbol{y}_0+\mathcal{K}_n(AA^{\mathrm{H}},\boldsymbol{b}-AA^{\mathrm{H}}\boldsymbol{y}_0)}\|\boldsymbol{y}-\boldsymbol{y}_n\|_{AA^{\mathrm{H}}}.$$

Algorithm 3.28 CGNE 方法

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N,\ \beta_{-1} = 0,\ \boldsymbol{p}_{-1} = \boldsymbol{0},\ \boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$

Output: $\boldsymbol{x}_n$

1: **for** $n = 0, 1, \ldots,$ 直至收敛 **do**

2: $\quad \boldsymbol{p}_n = A^{\mathrm{H}}\boldsymbol{r}_n + \beta_{n-1}\boldsymbol{p}_{n-1}$

3: $\quad \alpha_n = \dfrac{(\boldsymbol{r}_n, \boldsymbol{r}_n)}{(\boldsymbol{p}_n, \boldsymbol{p}_n)}$

4: $\quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n$

5: $\quad \boldsymbol{r}_{n+1} = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n$

6: $\quad \beta_n = \dfrac{(\boldsymbol{r}_{n+1}, \boldsymbol{r}_{n+1})}{(\boldsymbol{r}_n, \boldsymbol{r}_n)}$

7: **end for**

由上述事实和 $\boldsymbol{y}_n = A^{-\mathrm{H}}\boldsymbol{x}_n$ 可知, CGNE 方法生成的 $\boldsymbol{x}_n$ 满足

$$\begin{aligned}
&\min_{\boldsymbol{y}_n \in \boldsymbol{y}_0 + \mathcal{K}_n(AA^{\mathrm{H}}, \boldsymbol{b} - AA^{\mathrm{H}}\boldsymbol{y}_0)} \|\boldsymbol{y} - \boldsymbol{y}_n\|_{AA^{\mathrm{H}}} \\
&= \min_{A^{-\mathrm{H}}\boldsymbol{x}_n \in A^{-\mathrm{H}}\boldsymbol{x}_0 + \mathcal{K}_n(AA^{\mathrm{H}}, \boldsymbol{b} - A\boldsymbol{x}_0)} \|A^{-\mathrm{H}}\boldsymbol{x} - A^{-\mathrm{H}}\boldsymbol{x}_n\|_{AA^{\mathrm{H}}} \\
&= \min_{A^{-\mathrm{H}}\boldsymbol{x}_n \in A^{-\mathrm{H}}\boldsymbol{x}_0 + \mathcal{K}_n(AA^{\mathrm{H}}, \boldsymbol{b} - A\boldsymbol{x}_0)} \|\boldsymbol{x} - \boldsymbol{x}_n\| \\
&= \min_{\boldsymbol{x}_n \in \boldsymbol{x}_0 + A^{\mathrm{H}}\mathcal{K}_n(AA^{\mathrm{H}}, \boldsymbol{b} - A\boldsymbol{x}_0)} \|\boldsymbol{x} - \boldsymbol{x}_n\| \\
&= \min_{\boldsymbol{x}_n \in \boldsymbol{x}_0 + \mathcal{K}_n(A^{\mathrm{H}}A, A^{\mathrm{H}}\boldsymbol{r}_0)} \|\boldsymbol{x} - \boldsymbol{x}_n\|,
\end{aligned}$$

其中 $\boldsymbol{r}_0 := \boldsymbol{b} - A\boldsymbol{x}_0$. 因此, CGNE 方法使线性方程组 $A\boldsymbol{x} = \boldsymbol{b}$ 的误差的 2-范数极小. 在一些特殊情形下, CGNR 方法和 CGNE 方法的收敛表现很好 [64], [138]. 然而, 由于 $A^{\mathrm{H}}A$ (或 AA^{H}) 的条件数为 A 的两倍, 由 Theorem 3.4, 这些方法可能收敛缓慢. 若系数矩阵病态, 通常优先采用 LSQR 方法 [144]. 该算法基于 Golub–Kahan 双对角化过程 [78], 而这一过程是将 Lanczos 过程应用于下列形式的增广线性方程组得到的

$$\begin{bmatrix} I & A \\ A^{\mathrm{H}} & O \end{bmatrix} \begin{bmatrix} \boldsymbol{r} \\ \boldsymbol{x} \end{bmatrix} = \begin{bmatrix} \boldsymbol{b} \\ \boldsymbol{0} \end{bmatrix}.$$

将其记为 $\widetilde{A}\widetilde{\boldsymbol{x}} = \widetilde{\boldsymbol{b}}$. 该过程从一个单位向量开始:

$$\boldsymbol{w}_1 := \begin{bmatrix} \boldsymbol{u}_1 \\ \boldsymbol{0} \end{bmatrix} = \frac{1}{\|\boldsymbol{b}\|} \begin{bmatrix} \boldsymbol{b} \\ \boldsymbol{0} \end{bmatrix}.$$

令 $h_{0,1} := \|\boldsymbol{b}\|$. 则有 $h_{01}\boldsymbol{u}_1 = \boldsymbol{b}$. 第一步, 将 Lanczos 过程 (Algorithm 1.11) 应用于 $\widetilde{A}$, 得

$$\widetilde{\boldsymbol{w}}_2 = \widetilde{A}\boldsymbol{w}_1 - \alpha_1 \boldsymbol{w}_1 = \begin{bmatrix} \boldsymbol{0} \\ A^{\mathrm{H}}\boldsymbol{u}_1 \end{bmatrix},$$

$$\boldsymbol{w}_2 = \begin{bmatrix} \boldsymbol{0} \\ \boldsymbol{v}_1 \end{bmatrix} = \frac{1}{\|A^{\mathrm{H}}\boldsymbol{u}_1\|} \begin{bmatrix} \boldsymbol{0} \\ A^{\mathrm{H}}\boldsymbol{u}_1 \end{bmatrix}.$$

利用 $h_{1,1} := \|A^{\mathrm{H}}\boldsymbol{u}_1\|$, 得

$$h_{1,1}\boldsymbol{v}_1 = A^{\mathrm{H}}\boldsymbol{u}_1. \tag{3.124}$$

第二步, 有

$$\widetilde{\boldsymbol{w}}_3 = \widetilde{A}\boldsymbol{w}_2 - \alpha_2\boldsymbol{w}_2 - \beta_1\boldsymbol{w}_1 = \begin{bmatrix} A\boldsymbol{v}_1 - \beta_1\boldsymbol{u}_1 \\ \boldsymbol{0} \end{bmatrix},$$

$$\boldsymbol{w}_3 = \begin{bmatrix} \boldsymbol{u}_2 \\ \boldsymbol{0} \end{bmatrix} = \frac{1}{\|A\boldsymbol{v}_1 - h_{1,1}\boldsymbol{u}_1\|} \begin{bmatrix} A\boldsymbol{v}_1 - h_{1,1}\boldsymbol{u}_1 \\ \boldsymbol{0} \end{bmatrix}.$$

利用 $h_{2,1} := \|A\boldsymbol{v}_1 - h_{1,1}\boldsymbol{u}_1\|$, 将关于 $\boldsymbol{w}_3$ 的等式改写为

$$h_{2,1}\boldsymbol{u}_2 = A\boldsymbol{v}_1 - h_{1,1}\boldsymbol{u}_1. \tag{3.125}$$

第三步, 得

$$\widetilde{\boldsymbol{w}}_4 = \widetilde{A}\boldsymbol{w}_3 - \alpha_3\boldsymbol{w}_3 - \beta_2\boldsymbol{w}_2 = \begin{bmatrix} \boldsymbol{0} \\ A^{\mathrm{H}}\boldsymbol{u}_2 - \beta_2\boldsymbol{v}_1 \end{bmatrix},$$

$$\boldsymbol{w}_4 = \begin{bmatrix} \boldsymbol{0} \\ \boldsymbol{v}_2 \end{bmatrix} = \frac{1}{\|A^{\mathrm{H}}\boldsymbol{u}_2 - h_{2,1}\boldsymbol{v}_1\|} \begin{bmatrix} \boldsymbol{0} \\ A^{\mathrm{H}}\boldsymbol{u}_2 - h_{2,1}\boldsymbol{v}_1 \end{bmatrix}.$$

将关于 $\boldsymbol{w}_4$ 的等式改写为

$$h_{2,2}\boldsymbol{v}_2 = A^{\mathrm{H}}\boldsymbol{u}_2 - h_{2,1}\boldsymbol{v}_1 \tag{3.126}$$

这里使用 $h_{2,2} := \|A^{\mathrm{H}}\boldsymbol{u}_2 - h_{2,1}\boldsymbol{v}_1\|$. 因此, 由 (3.124)–(3.126), 得下列矩阵形式:

$$A\boldsymbol{v}_1 = [\boldsymbol{u}_1, \boldsymbol{u}_2] \begin{bmatrix} h_{1,1} \\ h_{2,1} \end{bmatrix} \Leftrightarrow AV_1 = U_2 B_{2,1},$$

$$A^{\mathrm{H}}[\boldsymbol{u}_1, \boldsymbol{u}_2] = [\boldsymbol{v}_1, \boldsymbol{v}_2] \begin{bmatrix} h_{1,1} & h_{2,1} \\ 0 & h_{2,2} \end{bmatrix} \Leftrightarrow A^{\mathrm{H}}U_2 = V_1 B_{2,1}^{\top} + h_{2,2}\boldsymbol{v}_2\boldsymbol{e}_2^{\top},$$

其中 $U_2 = [\boldsymbol{u}_1, \boldsymbol{u}_2]$, $V_1 = \boldsymbol{v}_1$, $B_{2,1}^{\top} = [h_{1,1}, h_{2,1}]$. 上述递推关系对应于 Golub–Kahan 双对角化过程 [78] 的前几步. Golub–Kahan 双对角化过程的算法见 Algorithm 3.29.

Algorithm 3.29 Golub–Kahan 双对角化过程

Input: $A \in \mathbb{C}^{N\times N}$, $\boldsymbol{b} \in \mathbb{C}^N$

Output: $\boldsymbol{u}_i$, $\boldsymbol{v}_i$ $(i = 1, 2, \ldots)$

$h_{0,1} = \|\boldsymbol{b}\|$, $\boldsymbol{u}_1 = \boldsymbol{b}/h_{0,1}$
$h_{1,1} = \|A^{\mathrm{H}}\boldsymbol{u}_1\|$, $\boldsymbol{v}_1 = A^{\mathrm{H}}\boldsymbol{u}_1/h_{1,1}$
for $n = 1, 2, \ldots$ **do**
    $\widetilde{\boldsymbol{u}}_{n+1} = A\boldsymbol{v}_n - h_{n,n}\boldsymbol{u}_n$
    $h_{n+1,n} = \|\widetilde{\boldsymbol{u}}_{n+1}\|$
    $\boldsymbol{u}_{n+1} = \widetilde{\boldsymbol{u}}_{n+1}/h_{n+1,n}$
    $\widetilde{\boldsymbol{v}}_{n+1} = A^{\mathrm{H}}\boldsymbol{u}_{n+1} - h_{n+1,n}\boldsymbol{v}_n$
    $h_{n+1,n+1} = \|\widetilde{\boldsymbol{v}}_{n+1}\|$
    $\boldsymbol{v}_{n+1} = \widetilde{\boldsymbol{v}}_{n+1}/h_{n+1,n+1}$
end for

Golub–Kahan 双对角化过程可写成矩阵形式

$$AV_n = U_{n+1}B_{n+1,n}, \tag{3.127}$$

$$A^{\mathrm{H}}U_{n+1} = V_n B_{n+1,n}^{\top} + h_{n+1,n+1}\boldsymbol{v}_{n+1}\boldsymbol{e}_{n+1}^{\top},$$

其中

$$U_n := [\boldsymbol{u}_1, \ldots, \boldsymbol{u}_n], \quad V_n := [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n], \quad B_{n+1,n} := \begin{bmatrix} h_{1,1} & & \\ h_{2,1} & \ddots & \\ & \ddots & h_{n,n} \\ & & h_{n+1,n} \end{bmatrix}.$$

由双对角化过程显然可知, U_n 和 V_n 满足下列性质:

$$U_n^{\mathrm{H}}U_n = V_n^{\mathrm{H}}V_n = I_n. \tag{3.128}$$

现在可以推导 LSQR 方法. LSQR 方法利用 V_n 和 $\boldsymbol{x}_0 = \boldsymbol{0}$ 生成第 n 个近似解, 即 $\boldsymbol{x}_n = V_n\boldsymbol{y}_n$, 其中 $\boldsymbol{y}$ 由相应残差向量的 2-范数极小化确定

$$\boldsymbol{r}_n = \boldsymbol{b} - AV_n\boldsymbol{y}_n. \tag{3.129}$$

将 (3.127) 代入 (3.129), 得

$$\begin{aligned} \boldsymbol{r}_n &= \boldsymbol{b} - U_{n+1}B_{n+1,n}\boldsymbol{y}_n \\ &= U_{n+1}(h_{0,1}\boldsymbol{e}_1 - B_{n+1,n}\boldsymbol{y}_n). \end{aligned}$$

由性质 (3.128) 得

$$\min_{\boldsymbol{y}_n \in \mathbb{C}^n} \|\boldsymbol{r}_n\| = \min_{\boldsymbol{y}_n \in \mathbb{C}^n} \|h_{0,1}\boldsymbol{e}_1 - B_{n+1,n}\boldsymbol{y}_n\|. \tag{3.130}$$

因此, 类似于 Section 3.1.3 中的 MINRES 方法, 可利用 Givens 旋转求得 (3.130) 中的 $\boldsymbol{y}_n$, 并且沿用 MINRES 方法的推导, 就得到 Algorithm 3.30 中的 LSQR 算法.

Algorithm 3.30 LSQR 方法

Input: $A \in \mathbb{C}^{N\times N},\ \boldsymbol{b} \in \mathbb{C}^N$

Output: $\boldsymbol{x}_n$

$\boldsymbol{x}_0 = \boldsymbol{0},\ h_{0,1} = \|\boldsymbol{b}\|,\ \boldsymbol{u}_1 = \boldsymbol{b}/h_{0,1}$

$h_{1,1} = \|A^{\mathrm{H}}\boldsymbol{u}_1\|,\ \boldsymbol{v}_1 = A^{\mathrm{H}}\boldsymbol{u}_1/h_{1,1}$

for $n = 1, 2, \ldots$ **do**

    (G.-K. 双对角化过程)

    $\widetilde{\boldsymbol{u}}_{n+1} = A\boldsymbol{v}_n - h_{n,n}\boldsymbol{u}_n$

    $h_{n+1,n} = \|\widetilde{\boldsymbol{u}}_{n+1}\|$

    $\boldsymbol{u}_{n+1} = \widetilde{\boldsymbol{u}}_{n+1}/h_{n+1,n}$

    $\widetilde{\boldsymbol{v}}_{n+1} = A^{\mathrm{H}}\boldsymbol{u}_{n+1} - h_{n+1,n}\boldsymbol{v}_n$

    $h_{n+1,n+1} = \|\widetilde{\boldsymbol{v}}_{n+1}\|$

    $\boldsymbol{v}_{n+1} = \widetilde{\boldsymbol{v}}_{n+1}/h_{n+1,n+1}$

    (Givens 旋转)

    for $i = \max\{1, n-1\}, \ldots, n-1$ **do**

$$\begin{bmatrix} t_{i,n} \\ t_{i+1,n} \end{bmatrix} = \begin{bmatrix} c_i & s_i \\ -\bar{s}_i & c_i \end{bmatrix} \begin{bmatrix} t_{i,n} \\ t_{i+1,n} \end{bmatrix}$$

    end for

$$c_n = \frac{|t_{n,n}|}{\sqrt{|t_{n,n}|^2 + |t_{n+1,n}|^2}}$$

$$\bar{s}_n = \frac{t_{n+1,n}}{t_{n,n}} c_n$$

$$t_{n,n} = c_n t_{n,n} + s_n t_{n+1,n}$$

$$t_{n+1,n} = 0$$

$$\begin{bmatrix} g_n \\ g_{n+1} \end{bmatrix} = \begin{bmatrix} c_n & s_n \\ -\bar{s}_n & c_n \end{bmatrix} \begin{bmatrix} g_n \\ 0 \end{bmatrix}$$

    (更新 $\boldsymbol{x}_n$)

$$\boldsymbol{p}_n = (\boldsymbol{v}_n - t_{n-1,n}\boldsymbol{p}_{n-1})/t_{n,n}$$

$$\boldsymbol{x}_n = \boldsymbol{x}_{n-1} + g_n\boldsymbol{p}_n$$

    (检查收敛性)

    if $|g_{n+1}|/\|\boldsymbol{b}\| \le \epsilon$, **then** 停止.

end for

3.5 预条件技术

迭代方法的收敛速度强烈依赖于系数矩阵的谱性质, 即特征值的分布. 因此, 自然可以尝试将原线性方程组变换为具有相同解而谱性质更有利的方程组. 若 K 为非奇异矩阵, 则变换后的线性方程组

$$K^{-1}A\boldsymbol{x} = K^{-1}\boldsymbol{b} \tag{3.131}$$

与原方程组具有相同的解. 因此, 将 Krylov 子空间方法应用于变换后的线性方程组, 就可得到原线性方程组的解. 矩阵 K 称为预条件子, 良好的预条件子满足下列性质:

- $K^{-1}A \approx I$.

- 对任意向量 $\boldsymbol{z}$, 容易得到 $K^{-1}\boldsymbol{z}$.

若 A 为 Hermitian 正定 (h.p.d.) 矩阵, 则变换后的方程组 (3.131) 在实践中并不适用, 因为系数矩阵 $K^{-1}A$ 不再为 h.p.d. 矩阵. 为保留系数矩阵的 h.p.d. 结构, 将预条件子分解为 $K = K_1K_1^{\mathrm{H}}$, 再将线性方程组变换为

$$K_1^{-1}AK_1^{-\mathrm{H}}(K_1^{\mathrm{H}}\boldsymbol{x}) = K_1^{-1}\boldsymbol{b}.$$

上述系数矩阵 $K_1^{-1}AK_1^{-\mathrm{H}}$ 为 h.p.d. 矩阵. 因此, 可将用于 h.p.d. 线性方程组的 Krylov 子空间方法, 例如 CG 方法, 应用于该方程组. 若系数矩阵 A 为非 Hermitian 矩阵, 则相应的预条件形式为

$$K_1^{-1}AK_2^{-1}(K_2\boldsymbol{x}) = K_1^{-1}\boldsymbol{b}, \tag{3.132}$$

其中 $K = K_1K_2$. 上述形式称为左右预条件. 当 $K_1 = I$ 时, 有

$$AK^{-1}(K\boldsymbol{x}) = \boldsymbol{b}. \tag{3.133}$$

这种预条件称为右预条件; 当 $K_2 = I$ 时, 得到 (3.131), 称为左预条件. GMRES 方法和 GCR 方法经常采用右预条件. 在这种情况下, 这些方法寻找使原线性方程组 $A\boldsymbol{x} = \boldsymbol{b}$ 的残差的 2-范数极小的近似解. 另一方面, 若采用左预条件, 则这些方法寻找使变换后的线性方程组 $K^{-1}A\boldsymbol{x} = K^{-1}\boldsymbol{b}$ 的残差的 2-范数极小的近似解. 关于预条件技术的综述, 例如见 [20], [146]. 另见双 Lanczos 型 Krylov 子空间方法的预条件算法 [109]. 文献 [49], [94], [136], [206] 研究了 Arnoldi 型 Krylov 子空间方法的预条件算法, 即用于奇异线性方程组的 GMRES 方法.

3.5.1 不完全矩阵分解预条件子

在 20 世纪的最后几十年中, 人们提出了许多预条件子. 在各种预条件子中, 最著名的一类基于系数矩阵的不完全分解, 可表示为 $A \approx K = LU$ (其中 L 和 U 为非奇异三角矩阵). 最简单的不完全分解之一是 Pommerell 提出的 D-ILU 预条件子 (例如见 [147], pp.77–78). 该方法的思想如下: 首先, 将系数矩阵分解为对角部分, 严格下三角部分和严格上三角部分, 即 $A = D_A + L_A + U_A$; 其次, 以 $K = (D+L_A)D^{-1}(D+U_A)$ 为预条件子, 其中 D 由 $\mathrm{diag}(K) = D_A$ 确定, 即对所有 i 有 $(K)_{ii} = (D_A)_{ii}$. 因此, 只需计算 D. D-ILU 预条件子的算法见 Algorithm 3.31, 其中 $N_0(A)$ 为矩阵 A 的非零元素的指标集, 即

$$N_0(A) := \{(i,j) : a_{i,j} \neq 0\}.$$

运行 Algorithm 3.31 后, 得到 $d_1, d_2, \ldots, d_N$. 再令

$$D = \begin{bmatrix} d_1^{-1} & & \\ & \ddots & \\ & & d_N^{-1} \end{bmatrix}$$

就得到形式为 $K = (D + L_A)D^{-1}(D + U_A)$ 的 D-ILU 预条件子.

Algorithm 3.31 D-ILU 分解

Input: $A \in \mathbb{C}^{N\times N}$, $N_0(A)$ (A 的非零元素指标集)

Output: $D^{-1} = \mathrm{diag}(d_1, d_2, \ldots, d_N)$

1: **for** $i = 1, 2, \ldots, N$ **do**
2: $\quad d_i = a_{i,i}$
3: **end for**
4: **for** $i = 1, 2, \ldots, N$ **do**
5: $\quad d_i = 1/d_i$
6: $\quad$ **for** $j = i+1, i+2, \ldots, N$ **do**
7: $\quad\quad$ if $(i,j) \in N_0(A)$ 且 $(j,i) \in N_0(A)$, then $d_j = d_j - a_{j,i}d_i a_{i,j}$
8: $\quad$ **end for**
9: **end for**

下面说明预条件 Krylov 子空间方法的实现. D-ILU 预条件 Krylov 子空间方法需要计算 K^{-1} 与向量 $\boldsymbol{v}$ 的乘积, 即 $\boldsymbol{w} = K^{-1}\boldsymbol{v} = (D+U_A)^{-1}D(D+L_A)^{-1}\boldsymbol{v}$, 向量 $\boldsymbol{w}$ 按下列步骤计算:

1. 以前代求解 $(D+L_A)\boldsymbol{y} = \boldsymbol{v}$;
2. 计算 $\boldsymbol{z} = D\boldsymbol{y}$;
3. 以回代求解 $(D+U_A)\boldsymbol{w} = \boldsymbol{z}$.

注意, 由于 D-ILU 预条件子显式包含原矩阵的非对角部分, 可以使用 Eisenstat 技巧 [53] (另见 Section 3.5.4) 更高效地实现 D-ILU 预条件 Krylov 子空间方法. Meijerink 和 van der Vorst [127] 给出了最著名且最有效的不完全 LU 分解之一. 该分解称为 ILU(0), 下面加以说明. ILU(0) 预条件子基于 Algorithm 1.1 中的 A 的 LU 分解. 为便于介绍 ILU(0) 预条件子, Algorithm 3.32 对 Algorithm 1.1 略作了修改. Algorithm 3.32 的输入为下列 A.

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & A_{1,N} \\ a_{2,1} & a_{2,2} & \cdots & A_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ a_{N,1} & a_{N,2} & \cdots & a_{N,N} \end{bmatrix},$$

输出为 $a_{i,j}$, 其中

$$A = LU,$$

这里

$$L = \begin{bmatrix} 1 & & & \\ a_{2,1} & \ddots & & \\ \vdots & \ddots & \ddots & \\ a_{N,1} & \cdots & a_{N,N-1} & 1 \end{bmatrix}, \quad U = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,N} \\ & \ddots & \ddots & \vdots \\ & & \ddots & a_{N-1,N} \\ & & & a_{N,N} \end{bmatrix}.$$

注意, 输入的 $a_{i,j}$ 为原矩阵, 输出时 $a_{i,j}$ 被覆盖, 用来存储矩阵 L 和 U.

Algorithm 3.32 LU 分解 (Algorithm 1.1 的略微修改版本)

```
for i = 2, 3, ..., N do
    for k = 1, 2, ..., i − 1 do
        a_{i,k} = a_{i,k} / a_{k,k}
        for j = k + 1, k + 2, ..., N do
            a_{i,j} = a_{i,j} − a_{i,k} × a_{k,j}
        end for
    end for
end for
```

ILU(0) 预条件子就是 Algorithm 3.32 中的 LU 分解, 但若 $(i,j) \notin N_0(A)$, 就不计算 (i,j) 元素. ILU(0) 预条件子的算法见 Algorithm 3.33. 由 Algorithm 3.33, 分解后矩阵的非零元素数通常与原矩阵相同. 因此, 当 A 为稀疏矩阵时, ILU(0) 预条件子的上三角矩阵和下三角矩阵也是稀疏矩阵. 这一事实对存储开销和计算开销都很重要.

Algorithm 3.33 ILU(0) 预条件子

```
for i = 2, 3, ..., N do
    for k = 1, 2, ..., i − 1 do
        if (i, k) ∈ N_0(A), then a_{i,k} = a_{i,k} / a_{k,k}
        for j = k + 1, k + 2, ..., N do
            if (i, j) ∈ N_0(A), then a_{i,j} = a_{i,j} − a_{i,k} × a_{k,j}
        end for
    end for
end for
```

作为 ILU(0) 预条件子的推广, 同一篇论文 [127] 还提出了 ILU(p) 预条件子, 其中 p 表示填充级别. 填充级别的定义如下: 当 $a_{i,j} \neq 0$ 时, 令位置 (i,j) 的初始填充级别为零; 当 $a_{i,j} = 0$ 时, 令位置 (i,j) 的初始填充级别为 ∞. 则位置 (i,j) 的填充级别定义为

$$\mathrm{Level}_{ij} := \min_{1 \le k \le \min\{i,j\}} \{\mathrm{Level}_{ik} + \mathrm{Level}_{kj} + 1\}. \tag{3.134}$$

例如, 考虑

$$A = \begin{bmatrix} * & * & 0 & 0 & 0 & * & 0 & 0 & 0 \\ * & * & * & 0 & 0 & 0 & * & 0 & 0 \\ 0 & * & * & * & 0 & 0 & 0 & * & 0 \\ 0 & 0 & * & * & * & 0 & 0 & 0 & * \\ 0 & 0 & 0 & * & * & * & 0 & 0 & 0 \\ * & 0 & 0 & 0 & * & * & * & 0 & 0 \\ 0 & * & 0 & 0 & 0 & * & * & * & 0 \\ 0 & 0 & * & 0 & 0 & 0 & * & * & * \\ 0 & 0 & 0 & * & 0 & 0 & 0 & * & * \end{bmatrix}, \tag{3.135}$$

其中符号 “*” 表示非零元素. 由于 A 的非零元素的级别为零, 而 A 中零元素的级别为 ∞, 得到下列以填充级别为元素的矩阵:

$$\begin{bmatrix} 0 & 0 & \infty & \infty & \infty & 0 & \infty & \infty & \infty \\ 0 & 0 & 0 & \infty & \infty & \infty & 0 & \infty & \infty \\ \infty & 0 & 0 & 0 & \infty & \infty & \infty & 0 & \infty \\ \infty & \infty & 0 & 0 & 0 & \infty & \infty & \infty & 0 \\ \infty & \infty & \infty & 0 & 0 & 0 & \infty & \infty & \infty \\ 0 & \infty & \infty & \infty & 0 & 0 & 0 & \infty & \infty \\ \infty & 0 & \infty & \infty & \infty & 0 & 0 & 0 & \infty \\ \infty & \infty & 0 & \infty & \infty & \infty & 0 & 0 & 0 \\ \infty & \infty & \infty & 0 & \infty & \infty & \infty & 0 & 0 \end{bmatrix}, \tag{3.136}$$

将 (3.134) 应用于 (3.136), 得到下列位置 (i,j) 的级别为 1 的矩阵:

$$\begin{bmatrix} 0 & 0 & \infty & \infty & \infty & 0 & \infty & \infty & \infty \\ 0 & 0 & 0 & \infty & \infty & 1 & 0 & \infty & \infty \\ \infty & 0 & 0 & 0 & \infty & \infty & 1 & 0 & \infty \\ \infty & \infty & 0 & 0 & 0 & \infty & \infty & 1 & 0 \\ \infty & \infty & \infty & 0 & 0 & 0 & \infty & \infty & 1 \\ 0 & 1 & \infty & \infty & 0 & 0 & 0 & \infty & \infty \\ \infty & 0 & 1 & \infty & \infty & 0 & 0 & 0 & \infty \\ \infty & \infty & 0 & 1 & \infty & \infty & 0 & 0 & 0 \\ \infty & \infty & \infty & 0 & 1 & \infty & \infty & 0 & 0 \end{bmatrix},$$

将 (3.134) 应用于上述矩阵, 就可确定 $\mathrm{Level}_{ij} = 2$ 的 (i,j) 元素.

$$\begin{bmatrix} 0 & 0 & \infty & \infty & \infty & 0 & \infty & \infty & \infty \\ 0 & 0 & 0 & \infty & \infty & 1 & 0 & \infty & \infty \\ \infty & 0 & 0 & 0 & \infty & 2 & 1 & 0 & \infty \\ \infty & \infty & 0 & 0 & 0 & \infty & 2 & 1 & 0 \\ \infty & \infty & \infty & 0 & 0 & 0 & \infty & 2 & 1 \\ 0 & 1 & 2 & \infty & 0 & 0 & 0 & \infty & 2 \\ \infty & 0 & 1 & 2 & \infty & 0 & 0 & 0 & \infty \\ \infty & \infty & 0 & 1 & 2 & \infty & 0 & 0 & 0 \\ \infty & \infty & \infty & 0 & 1 & 2 & \infty & 0 & 0 \end{bmatrix},$$

将 (3.134) 应用于上述矩阵, 就确定了所有填充级别, 相应矩阵为

$$\begin{bmatrix} 0 & 0 & \infty & \infty & \infty & 0 & \infty & \infty & \infty \\ 0 & 0 & 0 & \infty & \infty & 1 & 0 & \infty & \infty \\ \infty & 0 & 0 & 0 & \infty & 2 & 1 & 0 & \infty \\ \infty & \infty & 0 & 0 & 0 & 3 & 2 & 1 & 0 \\ \infty & \infty & \infty & 0 & 0 & 0 & 3 & 2 & 1 \\ 0 & 1 & 2 & 3 & 0 & 0 & 0 & 3 & 2 \\ \infty & 0 & 1 & 2 & 3 & 0 & 0 & 0 & 3 \\ \infty & \infty & 0 & 1 & 2 & 3 & 0 & 0 & 0 \\ \infty & \infty & \infty & 0 & 1 & 2 & 3 & 0 & 0 \end{bmatrix}, \tag{3.137}$$

设 $N_p(A)$ 为对应于 $\mathrm{Level}_{ij} \le p$ 的指标集, 即

$$N_p(A) := \{(i,j) : \mathrm{Level}_{ij} \le p\}.$$

例如, 由 (3.137), 当 $p = 1, 2, 3$ 时, (3.135) 的 $N_p(A)$ 如下:

$$\begin{aligned} N_1(A) &= N_0(A) \cup \{(2,6),(3,7),(4,8),(5,9),(6,2),(7,3),(8,4),(9,5)\}, \\ N_2(A) &= N_1(A) \cup \{(3,6),(4,7),(5,8),(6,9),(6,3),(7,4),(8,5),(9,6)\}, \\ N_3(A) &= N_2(A) \cup \{(4,6),(5,7),(6,8),(7,9),(6,4),(7,5),(8,6),(9,7)\}. \end{aligned}$$

ILU(p) 预条件子就是 Algorithm 3.32 中的 LU 分解, 但若 $(i,j) \notin N_p(A)$, 就不计算 (i,j) 元素. ILU(p) 预条件子的算法见 Algorithm 3.34.

Algorithm 3.34 ILU(p) 预条件子

```
for i = 2, 3, ..., N do
    for k = 1, 2, ..., i − 1 do
        if (i, k) ∈ N_p(A), then a_{i,k} = a_{i,k}/a_{k,k}
        for j = k + 1, k + 2, ..., N do
            if (i, j) ∈ N_p(A), then a_{i,j} = a_{i,j} − a_{i,k} × a_{k,j}
        end for
    end for
end for
```

ILU 的另一种成功变型是 Saad [150] 提出的采用阈值策略的 ILU. 该分解称为 ILUT(p, τ), 其中 p 用于节省存储空间, τ 为舍弃元素的准则, 即将模较小的元素强制置零.

3.5.2 近似逆预条件子

前述预条件子的性能取决于分解所得矩阵的乘积 $\widetilde{L}\widetilde{U}$ 与 A 的接近程度. 这里介绍判断良好预条件子的另一准则, 即预条件矩阵与 A^{-1} 的接近程度. 基于这一准则, Grote 和 Huckle [82] 尝试极小化下列 Frobenius 范数:

$$\min_K \|I - AK\|_{\mathrm{F}},$$

其中 $\|A\|_{\mathrm{F}}=\sqrt{\sum_{i,j}a_{i,j}^2}$. 由于上述极小化问题可写为

$$\min_{K}\|[\boldsymbol{e}_1-A\boldsymbol{k}_1,\boldsymbol{e}_2-A\boldsymbol{k}_2,\ldots,\boldsymbol{e}_N-A\boldsymbol{k}_N]\|_{\mathrm{F}},$$

就得到 N 个相互独立的最小二乘问题

$$\min_{\boldsymbol{k}_i}\|\boldsymbol{e}_i-A\boldsymbol{k}_i\|,\quad i=1,\ldots,N.$$

因此, 该预条件子的构造非常适合并行计算. Benzi 和 Tůma [24] 给出了关于近似逆的另一个出色思想, 即寻找非奇异矩阵 V 和 W, 使得

$$W^{\mathrm{H}}AV=D. \tag{3.138}$$

由 $(W^{\mathrm{H}}AV)^{-1}=V^{-1}A^{-1}W^{-\mathrm{H}}=D^{-1}$ 得, 矩阵 A 的逆为

$$A^{-1}=VD^{-1}W^{\mathrm{H}}.$$

由于等式 (3.138) 意味着当 $i\neq j$ 时 $(\boldsymbol{w}_i,A\boldsymbol{v}_j)=0$, 因而通过 A-双正交化过程计算 V 和 W. 该算法称为稀疏近似逆 (AINV) 预条件子, 见 Algorithm 3.35. 在算法中, $\boldsymbol{a}_i$ 和 $\boldsymbol{c}_i$ 分别表示 A 和 A^{H} 的第 i 列. 由 Algorithm 3.35 可见, 该算法采用 MGS 风格的 A-双正交化过程, 用于非 Hermitian 矩阵. 对于 Hermitian 矩阵和正规矩阵, 也已发展出相应的近似逆预条件子, 见 [21], [22].

3.5.3 矩阵多项式预条件子

已经介绍了两类预条件子. 一类近似系数矩阵, 即 $A\approx M$, 且 $M^{-1}\boldsymbol{v}$ 易于计算. 另一类近似系数矩阵的逆, 即 $A^{-1}\approx M$. 这里介绍与稀疏近似逆密切相关的另一种方法. 由于该方法基于矩阵多项式, 因而称为多项式预条件子.

Algorithm 3.35 AINV 预条件子

Input: $\boldsymbol{w}_i^{(0)}=\boldsymbol{v}_i^{(0)}=\boldsymbol{e}_i,\ 1\le i\le N$

for $i=1,\ldots,N$ **do**
  for $j=i,\ldots,N$ **do**
    $p_j^{(i-1)}=\boldsymbol{a}_i^{\mathrm{H}}\boldsymbol{v}_j^{(i-1)},\quad q_j^{(i-1)}=\boldsymbol{c}_i^{\mathrm{H}}\boldsymbol{w}_j^{(i-1)}$
  end for
  if $i=N$, then 退出.
  for $j=i+1,\ldots,N$ **do**
    $\boldsymbol{v}_j^{(i)}=\boldsymbol{v}_j^{(i-1)}-\left(\dfrac{p_j^{(i-1)}}{p_i^{(i-1)}}\right)\boldsymbol{v}_i^{(i-1)},\quad \boldsymbol{w}_j^{(i)}=\boldsymbol{w}_j^{(i-1)}-\left(\dfrac{q_j^{(i-1)}}{q_i^{(i-1)}}\right)\boldsymbol{w}_i^{(i-1)}$
    $v_{k,j}^{(i)}=0$ if $|v_{k,j}^{(i)}|<\mathrm{Tol},\ 1\le k\le N,\quad w_{k,j}^{(i)}=0$ if $|w_{k,j}^{(i)}|<\mathrm{Tol},\ 1\le k\le N$
  end for
end for
$\boldsymbol{z}_i=\boldsymbol{z}_i^{(i-1)},\ \boldsymbol{w}_i=\boldsymbol{w}_i^{(i-1)},\ p_i=p_i^{(i-1)},\ 1\le i\le N$
$V=[\boldsymbol{v}_1,\boldsymbol{v}_2,\ldots,\boldsymbol{v}_N],\ V=[\boldsymbol{w}_1,\boldsymbol{w}_2,\ldots,\boldsymbol{w}_N],\ D=\mathrm{diag}(p_1,p_2,\ldots,p_N)$

最著名的多项式预条件子包括 Neumann 展开和 Euler 展开. 设 A 具有形式 $I-B$, 且 $\rho(B)<1$. 则可将 A 的逆矩阵写为

$$A^{-1}=\sum_{i=0}^{\infty}B^i \qquad \text{(Neumann 展开)}$$
$$=\prod_{i=0}^{\infty}(I+B^{2^i}). \quad \text{(Euler 展开)}$$

因此, 由上述展开, 利用 Neumann (或 Euler) 展开的低阶项, 就可容易地得到并进而控制 A 的近似逆. 文献 [50] 研究了这一方法, 另见 [116], [198].

3.5.4 基于定常迭代方法的预条件子

Section 1.6 中定常迭代方法的思想也可用于为 Krylov 子空间方法构造高效预条件子. 本节介绍 Jacobi, Gauss–Seidel 和 SOR 型预条件子. Jacobi 预条件子 (或对角预条件子) 是最简单的预条件子, 通常比用不带预条件的 Krylov 子空间方法求解原线性方程组 $A\boldsymbol{x}=\boldsymbol{b}$ 更有效. 利用系数矩阵的对角元素, 按下式构造 Jacobi 预条件子:

$$K_J := \mathrm{diag}(a_{1,1}, a_{2,2}, \ldots, a_{n,n}).$$

若矩阵 A 为对角矩阵, 则 $K_J^{-1}A=I$. 这意味着 Jacobi 预条件 Krylov 子空间方法只需一步迭代就可得到解. 一般而言, 有一种方法是选取对角矩阵 D 为预条件子, 使 $D^{-1}A$ 的条件数尽可能小. 关于 $D^{-1}A$ 的条件数分析见 [59], [194]. 对称 Gauss–Seidel (SGS) 预条件子比 Jacobi 预条件子使用更多关于 A 的信息. SGS 预条件子以 Gauss–Seidel 方法命名, 后者是 Section 1.6 中的定常迭代方法之一. 该预条件子定义为

$$K_{SGS} := (D-L)D^{-1}(D-U),$$

其中 $A := D-L-U$. 若将 A 的所有对角元素缩放为一, 就得到更简单的预条件子:

$$K_{SGS} = (I-L)(I-U).$$

在这种情况下, 已知有如下高效实现方式: 使用预条件子 K_{SGS} 时, 得到下列形式的变换后线性方程组

$$(I-L)^{-1}A(I-U)^{-1}\widetilde{\boldsymbol{x}}=\widetilde{\boldsymbol{b}}, \quad \widetilde{\boldsymbol{x}}=(I-U)\boldsymbol{x}, \quad \widetilde{\boldsymbol{b}}=(I-L)^{-1}\boldsymbol{b}.$$

因此, 将 Krylov 子空间方法应用于变换后的线性方程组时, 需要计算下列矩阵向量乘积:

$$(I-L)^{-1}A(I-U)^{-1}\boldsymbol{z}.$$

当矩阵向量乘积 $A\boldsymbol{z}$ 的开销在每步迭代中占主导地位时, 这通常会使每次迭代的计算开销大约加倍. 然而, 回顾 $A=I-L-U$, 得

$$\begin{aligned}(I-L)^{-1}A(I-U)^{-1} &= (I-L)^{-1}(I-L-U)(I-U)^{-1}\\ &= (I-L)^{-1}[(I-L)+(I-U-I)](I-U)^{-1}\\ &= (I-L)^{-1}[(I-L)(I-U)^{-1}+I-(I-U)^{-1}]\\ &= (I-U)^{-1}+(I-L)^{-1}[I-(I-U)^{-1}].\end{aligned}$$

于是得到

$$\begin{aligned}(I-L)^{-1}A(I-U)^{-1}\boldsymbol{z} &= (I-U)^{-1}\boldsymbol{z}+(I-L)^{-1}[I-(I-U)^{-1}]\boldsymbol{z}\\ &= \boldsymbol{t}+(I-L)^{-1}(\boldsymbol{z}-\boldsymbol{t}),\end{aligned}$$

其中 $\boldsymbol{t} := (I-U)^{-1}\boldsymbol{z}$. 因此可见, 上述运算的开销仅约为一次矩阵向量乘法. 这一实现是 Eisenstat 技巧之一 [53]. 对称逐次超松弛 (SSOR) 预条件子可视为 SGS 预条件子的推广. 该预条件子定义为

$$K_{SSOR} := (\widetilde{D}-L)\widetilde{D}^{-1}(\widetilde{D}-U), \quad \widetilde{D} = D/\omega.$$

注意, 选取 $\omega = 1$ 就得到 K_{SGS}. 此时 Eisenstat 技巧如下: 由 $A = D - L - U$ 和 $\widetilde{A} = (\widetilde{D}-L)^{-1}A(\widetilde{D}-U)^{-1}$, 有

$$\begin{aligned}
\widetilde{A}\boldsymbol{z} &= (\widetilde{D}-L)^{-1}A(\widetilde{D}-U)^{-1}\boldsymbol{z} \\
&= (\widetilde{D}-L)^{-1}[(\widetilde{D}-L)+(D-2\widetilde{D})+(\widetilde{D}-U)](\widetilde{D}-U)^{-1}\boldsymbol{z} \\
&= (\widetilde{D}-U)^{-1}\boldsymbol{z} + (\widetilde{D}-L)^{-1}(D-2\widetilde{D})(\widetilde{D}-U)^{-1}\boldsymbol{z} + (\widetilde{D}-L)^{-1}\boldsymbol{z} \\
&= \boldsymbol{t} + (\widetilde{D}-L)^{-1}[(D-2\widetilde{D})\boldsymbol{t}+\boldsymbol{z}],
\end{aligned}$$

其中 $\boldsymbol{t} = (\widetilde{D}-U)^{-1}\boldsymbol{z}$.

3.5.5 预条件子的重排序

前面各小节考虑了用于有效求解原方程组的一些预条件技术. 另一方面, 自然也可以寻找有效的重排序以改善预条件子的性能, 例如, 对采用重排序矩阵 P 的下列方程组应用 Krylov 子空间方法:

$$K^{-1}PAP^{\top}\widetilde{\boldsymbol{x}} = \widetilde{\boldsymbol{b}},$$

其中 $\widetilde{\boldsymbol{x}} = P\boldsymbol{x}$, $\widetilde{\boldsymbol{b}} = K^{-1}P\boldsymbol{b}$. 基于这一思想, 可以采用许多重排序技术, 如 Cuthill–Mckee (CM) [42], 反向 Cuthill–Mckee (RCM) [75], 嵌套剖分 (ND) [76] 和最小度 (MD) [77]. 关于这些算法, 另见 [51], [123], [151]. CM 和 RCM 减小矩阵的带宽, MD 减少矩阵的填充元数, 而 ND 生成近似块对角矩阵, 适合并行计算. 对于对称定号矩阵, Duff 和 Meurant 给出了关于重排序效果的重要结果, 即用于直接求解器的重排序技术并未提高预条件子的质量 [52]. 对于非对称矩阵, Saad 通过实验表明, 在 ILUT 预条件之前进行 MD 和 ND 重排序, 相比原方程组没有优势, 只有 RCM 最有可能带来改进 [151], p.334. 另一方面, Benzi 等 [23] 研究了重排序对 ILU 型预条件子的影响, 得出在系数矩阵高度非对称的情形下 RCM 显著改善预条件子的结果. 文献 [25] 也讨论了对近似逆预条件子的影响的类似结果.

4 移位线性方程组的应用

摘要 如 Section 2.2 (计算物理) 和 Section 2.3 (机器学习) 所见, 需要求解一组下列形式的线性方程组

$$(A+\sigma_k I)\boldsymbol{x}^{(k)}=\boldsymbol{b},\quad k=1,2,\dots,m,$$

其中 σ_k 为标量, 这些方程组称为移位线性方程组. 使用 Section 1.3.1 中的 LU 分解时, 需要进行 m 次 LU 分解. 类似地, 使用 Section 1.6 中的定常迭代方法时, 需要求解 m 个线性方程组, 即对所有 k 求解 $(L+D+\sigma_k I)\boldsymbol{z}=\boldsymbol{v}$. 为求解这些线性方程组, 可以对每个线性方程组应用适当的 Krylov 子空间方法. 另一方面, 若第 i 个线性方程组的初始残差向量与第 j 个线性方程组的初始残差向量共线, 即两个初始残差向量之间的夹角为 0 或 π, 则生成的 Krylov 子空间相同, 这称为 Krylov 子空间的移位不变性. 这意味着可以共享仅一个 Krylov 子空间的信息来求解所有移位线性方程组, 从而高效计算移位线性方程组. 本章从 Chapter 3 中的 Krylov 子空间方法出发, 推导利用移位不变性的 Krylov 子空间方法.

4.1 移位线性方程组

Chapter 3 中的 Krylov 子空间方法对于求解下列线性方程组很有吸引力:

$$A^{(k)}\boldsymbol{x}^{(k)}=\boldsymbol{b},\quad k=1,2,\dots,m, \tag{4.1}$$

其中 $A^{(k)}$ 为下列形式的非奇异矩阵

$$A^{(k)}:=A+\sigma_k I, \tag{4.2}$$

这里 I 为单位矩阵, $\sigma_k\in\mathbb{C}$. 线性方程组 (4.1) 称为移位线性方程组. 为简便起见, 也使用下列记号:

$$(A+\sigma I)\boldsymbol{x}^{\sigma}=\boldsymbol{b}.$$

这里 $\alpha^{\sigma},\beta^{\sigma},\dots$ 和 $\boldsymbol{a}^{\sigma},\boldsymbol{b}^{\sigma},\dots$ 表示与移位线性方程组 $(A+\sigma I)\boldsymbol{x}^{\sigma}=\boldsymbol{b}$ 有关的标量和向量. 注意, α^{σ} 不是 α 的 σ 次幂. 若以初始猜测 $\boldsymbol{x}_0=\boldsymbol{0}$ 将 Krylov 子空间方法应用于 (4.1), 则由 (1.22), 该 Krylov 子空间方法在下列子空间上寻找 (4.1) 的近似解:

$$\boldsymbol{x}_n^{(k)}\in\mathcal{K}_n(A^{(k)},\boldsymbol{b}),\quad k=1,2,\dots,m. \tag{4.3}$$

由 Krylov 子空间的定义 (1.20), 下列移位不变性成立:

$$\mathcal{K}_n(A,\boldsymbol{b})=\mathcal{K}_n(A^{(k)},\boldsymbol{b}). \tag{4.4}$$

利用归纳法和 Krylov 子空间的定义很容易证明这一点. 例如, 当 $n=3$ 时, 有

$$\begin{aligned}\mathcal{K}_3(A+\sigma I,\boldsymbol{b})&=\{c_1\boldsymbol{b}+c_2(A+\sigma I)\boldsymbol{b}+c_3(A+\sigma I)^2\boldsymbol{b}:c_1,c_2,c_3\in\mathbb{C}\}\\&=\{(c_1+c_2\sigma+c_3\sigma^2)\boldsymbol{b}+(c_2+2c_3\sigma)A\boldsymbol{b}+c_3A^2\boldsymbol{b}:c_1,c_2,c_3\in\mathbb{C}\}\\&=\{d_1\boldsymbol{b}+d_2A\boldsymbol{b}+d_3A^2\boldsymbol{b}:d_1,d_2,d_3\in\mathbb{C}\}\\&=\mathcal{K}_3(A,\boldsymbol{b}).\end{aligned}$$

在第三个等式中, c_i 与 d_i 的关系为

$$\begin{bmatrix}d_1\\d_2\\d_3\end{bmatrix}=\begin{bmatrix}1&\sigma&\sigma^2\\0&1&2\sigma\\0&0&1\end{bmatrix}\begin{bmatrix}c_1\\c_2\\c_3\end{bmatrix}.$$

由于该 3×3 矩阵非奇异, 对任意 d_1,d_2,d_3 都存在 c_1,c_2,c_3. 因而第三个等式成立. 由此例可见, 移位不变性可以略作推广如下:

$$\mathcal{K}_n(A,\boldsymbol{v})=\mathcal{K}_n(A^{(k)},\boldsymbol{w}),\quad \boldsymbol{v}=c\boldsymbol{w}.$$

若对某个 c 有 $\boldsymbol{v}=c\boldsymbol{w}$, 则称这两个向量共线. 现在由移位不变性 (4.4) 可知, (4.3) 可改写为

$$\boldsymbol{x}_n^{(k)}\in\mathcal{K}_n(A,\boldsymbol{b}),\quad k=1,2,\dots,m.$$

这意味着只需一个 Krylov 子空间 $\mathcal{K}_n(A,\boldsymbol{b})$, 就可得到近似解 $\boldsymbol{x}_n^{(1)},\boldsymbol{x}_n^{(2)},\dots,\boldsymbol{x}_n^{(m)}$. 注意, 若移位不变性不成立, 则需要 m 个不同的 Krylov 子空间. 下面各节将介绍如何利用移位不变性, 以及如何将这一性质融入 Krylov 子空间方法.

4.2 移位 Hermitian 线性方程组

本节始终假设 (4.1) 中的系数矩阵 $A^{(k)}$ 为 Hermitian 矩阵,[1]即 $A^{(k)}=(A^{(k)})^{\mathrm{H}}$.

4.2.1 移位 CG 方法

本小节介绍利用移位不变性 (4.4) 的 CG 方法算法. 文献 [207], [208] 考虑了将 CG 方法应用于移位线性方程组, 其中必须存储所有 Lanczos 向量. 下面介绍基于 [67], [70] 的节省存储空间的算法. 由 Corollary 3.2, (3.15) 和 (4.4) 可知, 以 $\boldsymbol{x}_0=\boldsymbol{x}_0^{(k)}=\boldsymbol{0}$ 将 CG 方法 (Algorithm 3.1) 应用于 $A\boldsymbol{x}=\boldsymbol{b}$ 和 $A^{(k)}\boldsymbol{x}^{(k)}=\boldsymbol{b}$, 产生的残差向量 $\boldsymbol{r}_n$ 和 $\boldsymbol{r}_n^{(k)}:=\boldsymbol{b}-A^{(k)}\boldsymbol{x}_n^{(k)}$ 满足

$$\boldsymbol{r}_n,\boldsymbol{r}_n^{(k)}\in\mathcal{K}_{n+1}(A,\boldsymbol{b})\perp\mathcal{K}_n(A,\boldsymbol{b}),\quad k=1,2,\dots,m.$$

这意味着 $\boldsymbol{r}_n,\boldsymbol{r}_n^{(k)}\in\mathcal{K}_{n+1}(A,\boldsymbol{b})\cap\mathcal{K}_n(A,\boldsymbol{b})^{\perp}$, 其中 $\mathcal{K}_n(A,\boldsymbol{b})^{\perp}$ 为 $\mathcal{K}_n(A,\boldsymbol{b})$ 的正交补. 由于 $\mathcal{K}_n(A,\boldsymbol{b})\subset\mathcal{K}_{n+1}(A,\boldsymbol{b})$,

$$\dim(\mathcal{K}_{n+1}(A,\boldsymbol{b})\cap\mathcal{K}_n(A,\boldsymbol{b})^{\perp})=1. \tag{4.5}$$

因此 $\boldsymbol{r}_n$ 和所有 $\boldsymbol{r}_n^{(k)}$ 都属于 $\mathbb{C}^N$ 中同一个一维子空间, 即存在标量 $\pi_n^{(k)}\in\mathbb{C}$, 使得

$$\boldsymbol{r}_n=\pi_n^{(k)}\boldsymbol{r}_n^{(k)},\quad k=1,2,\dots,m. \tag{4.6}$$

关系 (4.6) 表明, 若已知 $\pi_n^{(k)}$, 就可由残差 $\boldsymbol{r}_n$ 产生所有残差 $\boldsymbol{r}_n^{(k)}$. 下面确定 $\pi_n^{(k)}$. 由 (3.18), 残差向量 $\boldsymbol{r}_{n+1}$ 可写为

$$\boldsymbol{r}_{n+1}=\left(1+\frac{\beta_{n-1}}{\alpha_{n-1}}\alpha_n-\alpha_nA\right)\boldsymbol{r}_n-\frac{\beta_{n-1}}{\alpha_{n-1}}\alpha_n\boldsymbol{r}_{n-1}. \tag{4.7}$$

类似地, 残差向量 $\boldsymbol{r}_{n+1}^{(k)}$ 也可写为

$$\boldsymbol{r}_{n+1}^{(k)}=\left\{1+\frac{\beta_{n-1}^{(k)}}{\alpha_{n-1}^{(k)}}\alpha_n^{(k)}-\alpha_n^{(k)}(A+\sigma_kI)\right\}\boldsymbol{r}_n^{(k)}-\frac{\beta_{n-1}^{(k)}}{\alpha_{n-1}^{(k)}}\alpha_n^{(k)}\boldsymbol{r}_{n-1}^{(k)}. \tag{4.8}$$

将 (4.6) 代入 (4.8), 得

$$\begin{aligned}\boldsymbol{r}_{n+1}=&\left\{1+\frac{\beta_{n-1}^{(k)}}{\alpha_{n-1}^{(k)}}\alpha_n^{(k)}-\alpha_n^{(k)}(A+\sigma_kI)\right\}\frac{\pi_{n+1}^{(k)}}{\pi_n^{(k)}}\boldsymbol{r}_n\\&-\frac{\beta_{n-1}^{(k)}\alpha_n^{(k)}\pi_{n+1}^{(k)}}{\alpha_{n-1}^{(k)}\pi_{n-1}^{(k)}}\boldsymbol{r}_{n-1}.\end{aligned} \tag{4.9}$$

[1]由于 $A^{(k)}$ 为 Hermitian 矩阵, 有 $\sigma\in\mathbb{R}$.

比较 (4.7) 和 (4.9) 中 $A\boldsymbol{r}_n$, $\boldsymbol{r}_n$ 和 $\boldsymbol{r}_{n-1}$ 的系数, 得

$$\alpha_n^{(k)} = \frac{\pi_n^{(k)}}{\pi_{n+1}^{(k)}}\alpha_n, \tag{4.10}$$

$$\left(1 + \frac{\beta_{n-1}^{(k)}}{\alpha_{n-1}^{(k)}}\alpha_n^{(k)} - \alpha_n^{(k)}\sigma_k\right)\frac{\pi_{n+1}^{(k)}}{\pi_n^{(k)}} = 1 + \frac{\beta_{n-1}}{\alpha_{n-1}}\alpha_n, \tag{4.11}$$

$$\frac{\beta_{n-1}}{\alpha_{n-1}}\alpha_n = \frac{\beta_{n-1}^{(k)}\alpha_n^{(k)}\pi_{n+1}^{(k)}}{\alpha_{n-1}^{(k)}\pi_{n-1}^{(k)}}. \tag{4.12}$$

将 (4.10) 代入 (4.12), 得

$$\beta_{n-1}^{(k)} = \left(\frac{\pi_{n-1}^{(k)}}{\pi_n^{(k)}}\right)^2 \beta_{n-1}. \tag{4.13}$$

将 (4.10) 和 (4.13) 代入 (4.11), 得

$$\pi_{n+1}^{(k)} = \left(1 + \frac{\beta_{n-1}}{\alpha_{n-1}}\alpha_n + \alpha_n\sigma_k\right)\pi_n^{(k)} - \frac{\beta_{n-1}}{\alpha_{n-1}}\alpha_n\pi_{n-1}^{(k)}. \tag{4.14}$$

由于 $\boldsymbol{r}_0 = \boldsymbol{r}_0^{(k)}$, 有 $\pi_0^{(k)} = 1$. 由 (3.21) 和 (4.14), 得下列关系:

$$\pi_{n+1}^{(k)} = R_{n+1}(-\sigma_k). \tag{4.15}$$

下面从 (4.8) 推导 (4.1) 的近似解向量 $\boldsymbol{x}_{n+1}^{(k)}$ 的计算公式. 令 $\boldsymbol{p}_{n-1}^{(k)} := (A + \sigma_k I)^{-1}(\boldsymbol{r}_{n-1}^{(k)} - \boldsymbol{r}_n^{(k)})/\alpha_{n-1}^{(k)}$, 则由 (4.8), 有

$$\boldsymbol{p}_n^{(k)} = \boldsymbol{r}_n^{(k)} + \beta_{n-1}^{(k)}\boldsymbol{p}_{n-1}^{(k)}, \tag{4.16}$$

$$\boldsymbol{r}_{n+1}^{(k)} = \boldsymbol{r}_n^{(k)} - \alpha_n^{(k)}(A + \sigma_k I)\boldsymbol{p}_n^{(k)}. \tag{4.17}$$

将 $\boldsymbol{r}_n^{(k)} = \boldsymbol{b} - (A + \sigma_k I)\boldsymbol{x}_n^{(k)}$ 代入 (4.17), 得

$$\boldsymbol{x}_{n+1}^{(k)} = \boldsymbol{x}_n^{(k)} + \alpha_n^{(k)}\boldsymbol{p}_n^{(k)}. \tag{4.18}$$

由 (4.6), 搜索方向 $\boldsymbol{p}_n^{(k)}$ 按下式更新

$$\boldsymbol{p}_n^{(k)} = \frac{1}{\pi_n^{(k)}}\boldsymbol{r}_n + \beta_{n-1}^{(k)}\boldsymbol{p}_{n-1}^{(k)}. \tag{4.19}$$

值得指出, 近似解的计算不使用递推关系 (4.17). 因而不需要对所有 k 计算矩阵向量乘积 $(A + \sigma_k I)\boldsymbol{p}_n^{(k)}$, 这大大节省了计算开销. 利用 (4.6), (4.10)–(4.15), (4.18) 和 (4.19), 就得到 Algorithm 4.1 中的移位 CG 方法. 在 Algorithm 4.1 中, 对给定的 $s \in \{1, 2, \ldots, m\}$, 将 CG 方法应用于称为种子系统的 $A^{(s)}\boldsymbol{x} = \boldsymbol{b}$, 并利用 CG 方法产生的残差向量 $\boldsymbol{r}_n$ 计算其他移位线性方程组 $A^{(k)}\boldsymbol{x} = \boldsymbol{b}$ 的近似解 $\boldsymbol{x}_n^{(k)}$. Algorithm 4.1 具有下列性质:

1. 需要计算 $A^{(s)}$ 与向量的乘积, 但不需要计算其他 k 对应的 $A^{(k)}$ 与向量的乘积;
2. 移位 CG 方法的近似解 $\boldsymbol{x}_n^{(k)}$ 与将 CG 方法 (以 $\boldsymbol{x}_0 = \boldsymbol{0}$) 应用于 $A^{(k)}\boldsymbol{x}^{(k)} = \boldsymbol{b}$ 得到的第 n 个近似解相同.

由 Algorithm 4.1 显然可见第一条性质成立. 由移位 CG 方法的推导过程也显然可见第二条性质成立. 在 Algorithm 4.1 中, 可利用关系 $\|\boldsymbol{r}_n^{(k)}\| = \|\boldsymbol{r}_n\|/|\pi_n^{(k)}|$ 监测近似解 $\boldsymbol{x}_n^{(k)}$ 的收敛性. 最后注意, 为计算 $A^{(s)}\boldsymbol{v}$, 实际计算 $A\boldsymbol{v} + \sigma_s\boldsymbol{v}$.

4.2.2 移位 CR 方法

由 Section 3.1.2 中的 CR 方法和 Section 4.2.1 给出的推导过程, 可以非常类似地推导移位 CR 方法. 移位 CR 方法的算法见 Algorithm 4.2. 移位 CG 方法与移位 CR 方法的主要区别是 $\alpha_n^{(s)}$ 和 $\beta_n^{(s)}$ 的计算公式. 移位 CR 方法 (Algorithm 4.2) 具有下列性质:

1. 需要计算 $A^{(s)}$ 与向量的乘积, 但不需要计算其他 k 对应的 $A^{(k)}$ 与向量的乘积, 这一性质与移位 CG 方法相同;

2. 与移位 CG 方法不同, 移位 CR 方法的近似解 $\boldsymbol{x}_n^{(k)}$ 与将 CR 方法 (以 $\boldsymbol{x}_0 = \boldsymbol{0}$) 应用于 $A^{(k)}\boldsymbol{x}^{(k)} = \boldsymbol{b}$ 得到的第 n 个近似解不同.

Algorithm 4.1 移位 CG 方法

Input: 选择种子系统 $s \in S = \{1, 2, \ldots, m\}$ 并令 $\boldsymbol{r}_0^{(s)} = \boldsymbol{b},\ \beta_{-1}^{(s)} = 0$

Input: A 和 σ_s 对于 $k = 1, 2, \ldots, m$

Input: $\boldsymbol{x}_0^{(k)} = \boldsymbol{p}_{-1}^{(k)} = \boldsymbol{0},\ \pi_0^{(s,k)} = \pi_{-1}^{(s,k)} = 1$ 对于 $k = 1, 2, \ldots, m$

Output: $\boldsymbol{x}_n^{(k)}$ 对于 $k = 1, 2, \ldots, m$

for $n = 0, 1, \ldots,$ 直至收敛 **do**

$\quad \boldsymbol{p}_n^{(s)} = \boldsymbol{r}_n^{(s)} + \beta_{n-1}^{(s)} \boldsymbol{p}_{n-1}^{(s)}$

$\quad \alpha_n^{(s)} = \dfrac{(\boldsymbol{r}_n^{(s)}, \boldsymbol{r}_n^{(s)})}{(\boldsymbol{p}_n^{(s)}, A^{(s)}\boldsymbol{p}_n^{(s)})}$

$\quad \boldsymbol{x}_{n+1}^{(s)} = \boldsymbol{x}_n^{(s)} + \alpha_n^{(s)} \boldsymbol{p}_n^{(s)}$

$\quad$ {移位系统开始}

$\quad$ **for** $k(\neq s) = 1, 2, \ldots, m$ **do**

$\quad\quad$ **if** $\|\boldsymbol{r}_n^{(k)}\| (= \|\boldsymbol{r}_n^{(s)}\| / |\pi_n^{(k)}|) > \epsilon_2 \|\boldsymbol{b}\|$ **then**

$\quad\quad\quad \pi_{n+1}^{(s,k)} = R_{n+1}^{(s)}(\sigma_s - \sigma_k)$ {← 见 (4.15)}

$$= \left[1 + \frac{\beta_{n-1}^{(s)}}{\alpha_{n-1}^{(s)}} \alpha_n^{(s)} - \alpha_n^{(s)}(\sigma_s - \sigma_k)\right] \pi_n^{(s,k)} - \frac{\beta_{n-1}^{(s)}}{\alpha_{n-1}^{(s)}} \alpha_n^{(s)} \pi_{n-1}^{(s,k)}$$

$\quad\quad\quad \beta_{n-1}^{(k)} = \left(\dfrac{\pi_{n-1}^{(s,k)}}{\pi_n^{(s,k)}}\right)^2 \beta_{n-1}^{(s)}$

$\quad\quad\quad \alpha_n^{(k)} = \dfrac{\pi_n^{(s,k)}}{\pi_{n+1}^{(s,k)}} \alpha_n^{(s)}$

$\quad\quad\quad \boldsymbol{p}_n^{(k)} = \dfrac{1}{\pi_n^{(s,k)}} \boldsymbol{r}_n^{(s)} + \beta_{n-1}^{(k)} \boldsymbol{p}_{n-1}^{(k)}$

$\quad\quad\quad \boldsymbol{x}_{n+1}^{(k)} = \boldsymbol{x}_n^{(k)} + \alpha_n^{(k)} \boldsymbol{p}_n^{(k)}$

$\quad\quad$ **end if**

$\quad$ **end for**

$\quad$ {移位系统结束}

$\quad \boldsymbol{r}_{n+1}^{(s)} = \boldsymbol{r}_n^{(s)} - \alpha_n^{(s)} A^{(s)} \boldsymbol{p}_n^{(s)}$

$\quad \beta_n^{(s)} = \dfrac{(\boldsymbol{r}_{n+1}^{(s)}, \boldsymbol{r}_{n+1}^{(s)})}{(\boldsymbol{r}_n^{(s)}, \boldsymbol{r}_n^{(s)})}$

end for

Algorithm 4.2 移位 CR 方法

Input: 选择种子系统 $s \in S = \{1, 2, \ldots, m\}$ 并令 $\boldsymbol{r}_0^{(s)} = \boldsymbol{b},\ \beta_{-1}^{(s)} = 0$

Input: $\boldsymbol{x}_0^{(k)} = \boldsymbol{p}_{-1}^{(k)} = \boldsymbol{0},\ \pi_0^{(s,k)} = \pi_{-1}^{(s,k)} = 1$ 对于 $k = 1, 2, \ldots, m$

Output: $\boldsymbol{x}_n^{(k)}$ 对于 $k = 1, 2, \ldots, m$

1: **for** $n = 0, 1, \ldots,$ 直至收敛 **do**

2: $\quad \boldsymbol{p}_n^{(s)} = \boldsymbol{r}_n^{(s)} + \beta_{n-1}^{(s)} \boldsymbol{p}_{n-1}^{(s)}$

3: $\quad (A^{(s)} \boldsymbol{p}_n^{(s)} = A^{(s)} \boldsymbol{r}_n^{(s)} + \beta_{n-1}^{(s)} A^{(s)} \boldsymbol{p}_{n-1}^{(s)})$

4: $\quad \alpha_n^{(s)} = \dfrac{(\boldsymbol{r}_n^{(s)}, A^{(s)} \boldsymbol{r}_n^{(s)})}{(A^{(s)} \boldsymbol{p}_n^{(s)}, A^{(s)} \boldsymbol{p}_n^{(s)})}$

5: $\quad \boldsymbol{x}_{n+1}^{(s)} = \boldsymbol{x}_n^{(s)} + \alpha_n^{(s)} \boldsymbol{p}_n^{(s)}$

6: $\quad$ {移位系统开始}

7: $\quad$ 执行移位 CG 方法 (Algorithm 4.1) 的第 6–15 行.

8: $\quad$ {移位系统结束}

9: $\quad \boldsymbol{r}_{n+1}^{(s)} = \boldsymbol{r}_n^{(s)} - \alpha_n^{(s)} A^{(s)} \boldsymbol{p}_n^{(s)}$

10: $\quad \beta_n^{(s)} = \dfrac{(\boldsymbol{r}_{n+1}^{(s)}, A^{(s)} \boldsymbol{r}_{n+1}^{(s)})}{(\boldsymbol{r}_n^{(s)}, A^{(s)} \boldsymbol{r}_n^{(s)})}$

11: **end for**

第二条性质源于以下事实: 对 $A\boldsymbol{x} = \boldsymbol{b}$ 和 $A^{(k)}\boldsymbol{x} = \boldsymbol{b}$ 应用 CR 方法所得的第 n 个残差向量不属于同一个一维子空间, 因为这些残差向量属于

$$\boldsymbol{r}_n^{\mathrm{CR}} \in \mathcal{K}_{n+1}(A, \boldsymbol{b}) \cap A\mathcal{K}_n(A, \boldsymbol{b})^{\perp},$$
$$\boldsymbol{r}_n^{(k)\mathrm{CR}} \in \mathcal{K}_{n+1}(A^{(k)}, \boldsymbol{b}) \cap A^{(k)}\mathcal{K}_n(A^{(k)}, \boldsymbol{b})^{\perp}.$$

注意, 由移位不变性 (4.4), 有 $\mathcal{K}_{n+1}(A, \boldsymbol{b}) = \mathcal{K}_{n+1}(A^{(k)}, \boldsymbol{b})$,
但一般而言 $A\mathcal{K}_n(A, \boldsymbol{b}) \neq A^{(k)}\mathcal{K}_n(A^{(k)}, \boldsymbol{b})$. 因此,

$$\mathcal{K}_{n+1}(A, \boldsymbol{b}) \cap A\mathcal{K}_n(A, \boldsymbol{b})^{\perp} \neq \mathcal{K}_{n+1}(A^{(k)}, \boldsymbol{b}) \cap A^{(k)}\mathcal{K}_n(A^{(k)}, \boldsymbol{b})^{\perp}.$$

这意味着 $\boldsymbol{r}_n^{\mathrm{CR}}$ 和 $\boldsymbol{r}_n^{(k)\mathrm{CR}}$ 不是由同一个一维子空间产生的, 它们并不共线.

4.2.3 移位 MINRES 方法

移位 MINRES 方法基于 Section 3.1.3 中的 MINRES 方法, 它实现残差 2-范数的极小化:

$$\min_{\boldsymbol{x}_n^{(k)} \in \mathcal{K}_n(A^{(k)}, \boldsymbol{b}) (= \mathcal{K}_n(A, \boldsymbol{b}))} \|\boldsymbol{b} - A^{(k)} \boldsymbol{x}_n^{(k)}\|. \tag{4.20}$$

下面介绍如何高效求解上述极小化问题, 并注意, 以下思想对应于 Section 4.4.3 将介绍的移位 GMRES 方法 [43] 的特殊情形. 由 Section 1.9.4 中的 Lanczos 过程, $\boldsymbol{x}_n^{(k)} \in \mathcal{K}_n(A, \boldsymbol{b})$ 可改写为

$$\boldsymbol{x}_n^{(k)} = V_n \boldsymbol{y}_n^{(k)}, \quad k = 1, 2, \ldots, m, \tag{4.21}$$

其中 $\boldsymbol{y}_n^{(k)} \in \mathbb{C}^n$. 对于 (4.1), 相应残差向量为 $\boldsymbol{r}_n^{(k)} := \boldsymbol{b} - A^{(k)} \boldsymbol{x}_n^{(k)} = \boldsymbol{b} - (A + \sigma_k I)\boldsymbol{x}_n^{(k)}$. 由 (4.21)

和 Lanczos 过程的矩阵形式 (1.42), 得

$$\begin{aligned}\boldsymbol{r}_n^{(k)} &= \boldsymbol{b} - (A+\sigma_k I)V_n\boldsymbol{y}_n^{(k)} \\ &= V_{n+1}g_1\boldsymbol{e}_1 - (AV_n + \sigma_k V_n)\boldsymbol{y}_n^{(k)} \\ &= V_{n+1}(g_1\boldsymbol{e}_1 - T_{n+1,n}^{(k)}\boldsymbol{y}_n^{(k)}), \quad T_{n+1,n}^{(k)} := T_{n+1,n} + \sigma_k \begin{bmatrix} I_n \\ \boldsymbol{0}^\top \end{bmatrix}.\end{aligned} \tag{4.22}$$

这里 $\boldsymbol{e}_1 = (1, 0, \ldots, 0)^\top$ 为第一个单位向量, $g_1 = (\boldsymbol{b}, \boldsymbol{b})^{1/2}$. 由 (4.20) 和 (4.22), 有

$$\begin{aligned}\min_{\boldsymbol{x}_n \in \mathcal{K}_n(A,\boldsymbol{b})} \|\boldsymbol{b} - A^{(k)}\boldsymbol{x}_n\| &= \min_{\boldsymbol{y}_n^{(k)} \in \mathbb{C}^n} \|V_{n+1}(g_1\boldsymbol{e}_1 - T_{n+1,n}^{(k)}\boldsymbol{y}_n^{(k)})\| \\ &= \min_{\boldsymbol{y}_n^{(k)} \in \mathbb{C}^n} \|g_1\boldsymbol{e}_1 - T_{n+1,n}^{(k)}\boldsymbol{y}_n^{(k)}\|.\end{aligned} \tag{4.23}$$

极小化问题 (4.23) 可利用 Section 3.1.3 中介绍的 Givens 旋转求解. 移位 MINRES 方法的算法见 Algorithm 4.3.

Algorithm 4.3 移位 MINRES 方法

Input: $\boldsymbol{x}_0^{(k)} = \boldsymbol{p}_{-1}^{(k)} = \boldsymbol{p}_0^{(k)} = \boldsymbol{0},\ \boldsymbol{v}_1 = \boldsymbol{b}/(\boldsymbol{b},\boldsymbol{b})^{1/2},\ g_1^{(k)} = (\boldsymbol{b},\boldsymbol{b})^{1/2},\ \beta_{-1} = 0$

Output: $\boldsymbol{x}_n^{(k)}$ 对于 $l = 1, 2, \ldots, m$

for $n = 1, 2, \ldots$ **do**
  {Lanczos 过程}
  $\alpha_n = (\boldsymbol{v}_n, A\boldsymbol{v}_n)$
  $\widetilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \alpha_n \boldsymbol{v}_n - \beta_{n-1}\boldsymbol{v}_{n-1}$
  $\beta_n = (\widetilde{\boldsymbol{v}}_{n+1}, \widetilde{\boldsymbol{v}}_{n+1})^{1/2}$
  $\boldsymbol{v}_{n+1} = \widetilde{\boldsymbol{v}}_{n+1}/\beta_n$
  $t_{n-1,n}^{(k)} = \beta_{n-1},\ t_{n,n}^{(k)} = \alpha_n + \sigma_\ell,\ t_{n+1,n}^{(k)} = \beta_n$
  {用 Givens 旋转求解最小二乘问题}
  for $k = 1, 2, \ldots, m$ **do**
    if $|g_{n+1}^{(k)}|/\|\boldsymbol{b}\| > \epsilon$ **then**
      for $i = \max\{1, n-2\}, \ldots, n-1$ **do**

$$\begin{bmatrix} t_{i,n}^{(k)} \\ t_{i+1,n}^{(k)} \end{bmatrix} = \begin{bmatrix} c_i^{(k)} & s_i^{(k)} \\ -\bar{s}_i^{(k)} & c_i^{(k)} \end{bmatrix} \begin{bmatrix} t_{i,n}^{(k)} \\ t_{i+1,n}^{(k)} \end{bmatrix}$$

      end for

$$c_n^{(k)} = \frac{|t_{n,n}^{(k)}|}{\sqrt{|t_{n,n}^{(k)}|^2 + |t_{n+1,n}^{(k)}|^2}}$$

$$\bar{s}_n^{(k)} = \frac{t_{n+1,n}^{(k)}}{t_{n,n}^{(k)}} c_n^{(k)}$$

$$t_{n,n}^{(k)} = c_n^{(k)} t_{n,n}^{(k)} + s_n^{(k)} t_{n+1,n}^{(k)}$$

$$t_{n+1,n}^{(k)} = 0$$

$$\begin{bmatrix} g_n^{(k)} \\ g_{n+1}^{(k)} \end{bmatrix} = \begin{bmatrix} c_n^{(k)} & s_n^{(k)} \\ -\bar{s}_n^{(k)} & c_n^{(k)} \end{bmatrix} \begin{bmatrix} g_n^{(k)} \\ 0 \end{bmatrix}$$

      {更新近似解 $\boldsymbol{x}_n^{(k)}$}

$$\boldsymbol{p}_n^{(k)} = \boldsymbol{v}_n - (t_{n-2,n}^{(k)}/t_{n-2,n-2}^{(k)})\boldsymbol{p}_{n-2}^{(k)} - (t_{n-1,n}^{(k)}/t_{n-1,n-1}^{(k)})\boldsymbol{p}_{n-1}^{(k)}$$

$$\boldsymbol{x}_n^{(k)} = \boldsymbol{x}_{n-1}^{(k)} + (g_n^{(k)}/t_{n,n}^{(k)})\boldsymbol{p}_n^{(k)}$$

    end if
  end for
  if 对所有 ℓ 有 $|g_{n+1}^{(k)}|/\|\boldsymbol{b}\| \le \epsilon$, then 退出.
end for

由 Algorithm 4.3 和推导过程可见:

1. 需要计算 $A^{(s)}$ 与向量的乘积, 但不需要计算其他 k 对应的 $A^{(k)}$ 与向量的乘积, 这一性质与移位 CG 方法和移位 CR 方法相同;

2. 移位 MINRES 方法的近似解 $\boldsymbol{x}_n^{(k)}$ 与将 MINRES 方法 (以 $\boldsymbol{x}_0 = \boldsymbol{0}$) 应用于 $A^{(k)}\boldsymbol{x}^{(k)} = \boldsymbol{b}$ 得到的第 n 个近似解相同.

4.3 移位复对称线性方程组

本节始终假设 (4.1) 中的系数矩阵 $A^{(k)}$ 为复对称矩阵, 即 $A^{(k)} = (A^{(k)})^{\top} \neq (A^{(k)})^{\mathrm{H}}$.

4.3.1 移位 COCG 方法

移位 COCG 方法 [184] 的推导与 (4.6)–(4.19) 相同. 移位 CG 方法与移位 COCG 方法的唯一区别是 α_n 和 β_n. 移位 COCG 方法的算法见 Algorithm 4.4.

Algorithm 4.4 移位 COCG 方法 (注意: $(\bar{\boldsymbol{a}}, \boldsymbol{b}) = \boldsymbol{a}^{\top}\boldsymbol{b}$)

Input: 选择种子系统 $s \in S = \{1, 2, \ldots, m\}$ 并令 $\boldsymbol{r}_0^{(s)} = \boldsymbol{b}$, $\beta_{-1}^{(s)} = 0$

Input: $\boldsymbol{x}_0^{(k)} = \boldsymbol{p}_{-1}^{(k)} = \boldsymbol{0}$, $\pi_0^{(s,k)} = \pi_{-1}^{(s,k)} = 1$ 对于 $k = 1, 2, \ldots, m$

Output: $\boldsymbol{x}_n^{(k)}$ 对于 $k = 1, 2, \ldots, m$

1: **for** $n = 0, 1, \ldots,$ 直至收敛 **do**

2: $\quad \boldsymbol{p}_n^{(s)} = \boldsymbol{r}_n^{(s)} + \beta_{n-1}^{(s)} \boldsymbol{p}_{n-1}^{(s)}$

3: $\quad \alpha_n^{(s)} = \dfrac{(\bar{\boldsymbol{r}}_n^{(s)}, \boldsymbol{r}_n^{(s)})}{(\bar{\boldsymbol{p}}_n^{(s)}, A^{(s)} \boldsymbol{p}_n^{(s)})}$

4: $\quad \boldsymbol{x}_{n+1}^{(s)} = \boldsymbol{x}_n^{(s)} + \alpha_n^{(s)} \boldsymbol{p}_n^{(s)}$

5: $\quad$ {移位系统开始}

6: $\quad$ 执行移位 CG 方法 (Algorithm 4.1) 的第 6–15 行.

7: $\quad$ {移位系统结束}

8: $\quad \boldsymbol{r}_{n+1}^{(s)} = \boldsymbol{r}_n^{(s)} - \alpha_n^{(s)} A^{(s)} \boldsymbol{p}_n^{(s)}$

9: $\quad \beta_n^{(s)} = \dfrac{(\bar{\boldsymbol{r}}_{n+1}^{(s)}, \boldsymbol{r}_{n+1}^{(s)})}{(\bar{\boldsymbol{r}}_n^{(s)}, \boldsymbol{r}_n^{(s)})}$

10: **end for**

Algorithm 4.4 具有下列性质:

1. 需要计算 $A^{(s)}$ 与向量的乘积, 但不需要计算其他 k 对应的 $A^{(k)}$ 与向量的乘积;
2. 移位 COCG 方法的近似解 $\boldsymbol{x}_n^{(k)}$ 与将 COCG 方法 (以 $\boldsymbol{x}_0 = \boldsymbol{0}$) 应用于 $A^{(k)}\boldsymbol{x}^{(k)} = \boldsymbol{b}$ 得到的第 n 个近似解相同.

当对所有 k 矩阵 $A^{(k)}$ 都为实对称矩阵时, 移位 COCG 方法等价于移位 CG 方法. 文献 [190] 将移位 COCG 方法推广到求解下列线性方程组:

$$(A + \sigma_k B)\boldsymbol{x}^{(k)} = \boldsymbol{b}, \tag{4.24}$$

它们称为广义移位线性方程组. 这里 A 和 B 为实对称矩阵, σ_k 为复数, 因而系数矩阵 $A+\sigma_k B$ 为复对称矩阵. 容易看出, 若 B 为单位矩阵, 则广义移位线性方程组 (4.24) 退化为移位线性方程组. 注意, 移位 COCG 方法和其他移位 Krylov 子空间方法不能应用于 (4.24), 因为移位不变性不再成立, 即当 $i \neq j$ 时 $\mathcal{K}_n(A + \sigma_i B, \boldsymbol{b}) \neq \mathcal{K}_n(A + \sigma_j B, \boldsymbol{b})$. 一种补救方法是考虑下列移位线性方程组:

$$(B^{-1}A + \sigma_k I)\boldsymbol{x}^{(k)} = B^{-1}\boldsymbol{b},$$

其中假设 B 非奇异. 然而, $B^{-1}A+\sigma_k I$ 为非 Hermitian 矩阵, 因而可以使用非 Hermitian 线性方程组的移位 Krylov 子空间方法. 另一方面, 已有研究表明, 无需采用这种通用求解器, 就能

构造与移位 COCG 方法类似的算法, 称为广义移位 COCG 方法 [190]. 若 $B = I$, 则广义移位 COCG 方法退化为移位 COCG 方法. 如 Section 2.2.1 所指出, 若 Hamiltonian 矩阵为实对称矩阵, 则移位线性方程组的系数矩阵 $(\epsilon + i\delta)I - H$ 为复对称矩阵. 因此, 移位 COCG 方法是一种可选方法, 其在计算物理中的应用例如见 [73], [110], [111]. 文献 [100] 开发了移位 COCG 方法及相关求解器的开源库. 广义移位线性方程组也出现在下列线性方程组中:

$$(\sigma^2 A + \sigma B + C)\boldsymbol{x} = \boldsymbol{b}, \tag{4.25}$$

其中 A, B, C 为复对称矩阵. 文献 [158], [159] 考虑了这种类型的参数化线性方程组及其在结构动力学中的应用. 文献 [158], §3 中一个有趣的观察是, 可以将 (4.25) 变换为下列线性方程组:

$$\left(\begin{bmatrix} B & C \\ C^\top & O \end{bmatrix} + \sigma \begin{bmatrix} A & O \\ O & -C^\top \end{bmatrix} \right) \begin{bmatrix} \boldsymbol{y} \\ \boldsymbol{x} \end{bmatrix} = \begin{bmatrix} \boldsymbol{b} \\ \boldsymbol{0} \end{bmatrix} \tag{4.26}$$

前提是 C 非奇异. 由于 A, B, C 为复对称矩阵, (4.26) 的系数矩阵构成复对称的广义移位线性方程组. 因此, 广义移位 COCG 方法也可用于求解 (4.25).

4.3.1.1 种子切换技术

由 Algorithm 4.4 (另见 Algorithm 4.1 的第 8 行) 可知, 若 $|\pi_n^{(s,k)}| = |R_n^{(s)}(\sigma_s - \sigma_k)| \geq 1$, 则 $\|\boldsymbol{r}_n^{(k)}\| \leq \|\boldsymbol{r}_n^{(s)}\|$. 因而, 若能找到满足 $|R_n^{(s)}(\sigma_s - \sigma_k)| \geq 1$ 的种子系统, 就可求解所有移位系统. 然而, 除 [67], Corollary 1 中讨论的一些特殊情形外, 预先找到这样的系统极其困难. 种子切换技术 [166] 可以避免这一问题, 包含下列步骤:

(S1) 选择一个种子系统, 然后启动 Algorithm 4.4.

(S2) 若在第 n 次迭代已求解种子系统, 则从尚未求解的系统中选取一个新种子系统.

(S3) 利用新种子系统从第 $(n+1)$ 次迭代开始执行 Algorithm 4.4.

在 (S2) 中, 选取新种子系统 $\widetilde{s}$ 的一个准则可以是

$$\widetilde{s} = \arg\max_{i \in I} \{\|\boldsymbol{r}_n^{(i)}\|\},$$

其中 I 表示尚未求解的线性方程组的指标集. 在 (S3) 中, 需要两个步骤将旧种子系统切换为新种子系统. 首先, 计算

$$\pi_{n+1}^{(s,\widetilde{s})} = R_{n+1}^{(s)}(\sigma_s - \sigma_{\widetilde{s}}), \quad \beta_n^{(\widetilde{s})} = \left(\pi_n^{(s,\widetilde{s})} / \pi_{n+1}^{(s,\widetilde{s})}\right)^2 \beta_n$$

以得到 $\boldsymbol{r}_{n+1}^{(\widetilde{s})}$ 和 $\beta_n^{(\widetilde{s})}\boldsymbol{p}_n^{(\widetilde{s})}$. 由于从 $\boldsymbol{r}_{n+1}^{(\widetilde{s})} + \beta_n^{(\widetilde{s})}\boldsymbol{p}_n^{(\widetilde{s})}$ 可以得到 $\boldsymbol{p}_{n+1}^{(\widetilde{s})}$, 因而可以从第 $(n+1)$ 次迭代开始, 用 COCG 方法求解系统 $(A + \sigma_{\widetilde{s}} I)\boldsymbol{x}^{(\widetilde{s})} = \boldsymbol{b}$. 其次, 为利用新种子 $\widetilde{s}$ 求解其余线性方程组, 必须从新种子生成参数 $\alpha_{n+1}^{(i)}$ 和 $\beta_n^{(i)}$. 可见, 利用下列多项式很容易生成这些参数:

$$\pi_{n+1}^{(\widetilde{s},i)} = R_{n+1}^{(\widetilde{s})}(\sigma_{\widetilde{s}} - \sigma_i) \quad \text{for all } i \in I.$$

为得到上述多项式, 需要计算

$$\alpha_i^{(\widetilde{s})} = \left(\pi_i^{(s,\widetilde{s})} / \pi_{i+1}^{(s,\widetilde{s})}\right) \alpha_i, \quad \beta_j^{(\widetilde{s})} = \left(\pi_j^{(s,\widetilde{s})} / \pi_{j+1}^{(s,\widetilde{s})}\right)^2 \beta_j$$

其中 $i = 0, \ldots, n$, $j = 0, \ldots, n-1$. 因此, 切换策略只需标量运算, 而且可见, 若不发生中断, 反复执行 (S2) 至 (S3) 的过程, 就能在不损失上一次切换之前已生成的 Krylov 子空间维数的情况下继续求解这些系统. 种子切换技术使我们摆脱种子系统选择问题, 也可应用于 Section 4.2.1 中的移位 CG 方法和 Section 4.4.1 中的移位 BiCG 方法, 因为移位系统的移位 CG (BiCG) 方法的残差向量就是真正的 CG (BiCG) 残差.

4.3.2 移位 COCR 方法

移位 COCR 方法 [172] 的推导也与 (4.6)–(4.19) 相同. 移位 CR 方法与移位 COCR 方法的唯一区别是 α_n 和 β_n. 移位 COCR 方法的算法见 Algorithm 4.5. Algorithm 4.5 具有下列性质:

1. 需要计算 $A^{(s)}$ 与向量的乘积, 但不需要计算其他 k 对应的 $A^{(k)}$ 与向量的乘积, 这一性质与移位 COCG 方法相同;

2. 与移位 COCG 方法不同, 移位 COCR 方法的 $\boldsymbol{x}_n^{(k)}$ 与将 COCR 方法 (以 $\boldsymbol{x}_0 = \boldsymbol{0}$) 应用于 $A^{(k)}\boldsymbol{x}^{(k)} = \boldsymbol{b}$ 得到的第 n 个近似解不同.

Algorithm 4.5 移位 COCR 方法 (注意: $(\bar{\boldsymbol{a}}, \boldsymbol{b}) = \boldsymbol{a}^{\top}\boldsymbol{b}$)

Input: 选择种子系统 $s \in S = \{1, 2, \ldots, m\}$ 并令 $\boldsymbol{r}_0^{(s)} = \boldsymbol{b}$, $\beta_{-1}^{(s)} = 0$
Input: $\boldsymbol{x}_0^{(k)} = \boldsymbol{p}_{-1}^{(k)} = \boldsymbol{0}$, $\pi_0^{(s,k)} = \pi_{-1}^{(s,k)} = 1$ 对于 $k = 1, 2, \ldots, m$
Output: $\boldsymbol{x}_n^{(k)}$ 对于 $k = 1, 2, \ldots, m$

1: **for** $n = 0, 1, \ldots,$ 直至收敛 **do**
2: $\quad \boldsymbol{p}_n^{(s)} = \boldsymbol{r}_n^{(s)} + \beta_{n-1}^{(s)} \boldsymbol{p}_{n-1}^{(s)}$
3: $\quad (A^{(s)}\boldsymbol{p}_n^{(s)} = A^{(s)}\boldsymbol{r}_n^{(s)} + \beta_{n-1}^{(s)} A^{(s)}\boldsymbol{p}_{n-1}^{(s)})$
4: $\quad \alpha_n^{(s)} = \dfrac{(\bar{\boldsymbol{r}}_n^{(s)}, A^{(s)}\boldsymbol{r}_n^{(s)})}{\overline{(A^{(\sigma_s)}\boldsymbol{p}_n^{(s)}}, A^{(s)}\boldsymbol{p}_n^{(s)})}$
5: $\quad \boldsymbol{x}_{n+1}^{(s)} = \boldsymbol{x}_n^{(s)} + \alpha_n^{(s)} \boldsymbol{p}_n^{(s)}$
6: $\quad$ {移位系统开始}
7: $\quad$ 执行移位 CG 方法 (Algorithm 4.1) 的第 6–15 行.
8: $\quad$ {移位系统结束}
9: $\quad \boldsymbol{r}_{n+1}^{(s)} = \boldsymbol{r}_n^{(s)} - \alpha_n^{(s)} A^{(s)}\boldsymbol{p}_n^{(s)}$
10: $\quad \beta_n^{(s)} = \dfrac{(\bar{\boldsymbol{r}}_{n+1}^{(s)}, A^{(s)}\boldsymbol{r}_{n+1}^{(s)})}{(\bar{\boldsymbol{r}}_n^{(s)}, A^{(s)}\boldsymbol{r}_n^{(s)})}$
11: **end for**

当对所有 k 矩阵 $A^{(k)}$ 都为实对称矩阵时, 移位 COCR 方法等价于移位 CR 方法.

4.3.3 移位 QMR_SYM 方法

文献 [62] 提出了求解非 Hermitian 线性方程组的移位 QMR 方法. 如 [61] 所知, 移位 QMR_SYM 方法是移位 QMR 方法的简化, 且移位 QMR_SYM 的推导与移位 MINRES 方法类似. 由 Section 1.9.3 中的复对称 Lanczos 过程, $\boldsymbol{x}_n \in \mathcal{K}_n(A, \boldsymbol{b})$ 可改写为

$$\boldsymbol{x}_n^{(k)} = V_n \boldsymbol{y}_n^{(k)}, \quad k = 1, 2, \ldots, m, \tag{4.27}$$

其中 $\boldsymbol{y}_n^{(k)} \in \mathbb{C}^n$. 对于 (4.1), 相应残差向量为 $\boldsymbol{r}_n^{(k)} := \boldsymbol{b} - (A+\sigma_k I)\boldsymbol{x}_n^{(k)}$. 由 (4.27) 和复对称 Lanczos 过程的矩阵形式 (1.41), 得

$$\begin{aligned}\boldsymbol{r}_n^{(k)} &= \boldsymbol{b} - (A+\sigma_k I)V_n \boldsymbol{y}_n^{(k)} \\ &= V_{n+1} g_1 \boldsymbol{e}_1 - (AV_n + \sigma_k V_n)\boldsymbol{y}_n^{(k)} \\ &= V_{n+1}(g_1 \boldsymbol{e}_1 - T_{n+1,n}^{(k)} \boldsymbol{y}_n^{(k)}), \quad T_{n+1,n}^{(k)} := T_{n+1,n} + \sigma_k \begin{bmatrix} I_n \\ \boldsymbol{0}^\top \end{bmatrix}.\end{aligned} \tag{4.28}$$

这里 $\boldsymbol{e}_1 = (1, 0, \ldots, 0)^\top$ 为第一个单位向量, $g_1 = (\bar{\boldsymbol{b}}, \boldsymbol{b})^{1/2}$. 与 Section 3.2.3 中的 QMR_SYM 方法一样, $\boldsymbol{y}_n^{(k)}$ 通过求解下列极小化问题确定.

$$\min_{\boldsymbol{y}_n^{(k)} \in \mathbb{C}^n} \| g_1 \boldsymbol{e}_1 - T_{n+1,n}^{(k)} \boldsymbol{y}_n^{(k)} \|. \tag{4.29}$$

极小化问题 (4.29) 可利用 Section 3.1.3 中介绍的 Givens 旋转求解. 移位 QMR_SYM 方法的算法见 Algorithm 4.6. Algorithm 4.6 具有下列性质:

1. 需要计算 $A^{(s)}$ 与向量的乘积, 但不需要计算其他 k 对应的 $A^{(k)}$ 与向量的乘积, 这一性质与移位 CG 方法和移位 CR 方法相同;

2. 移位 QMR_SYM 方法的近似解 $\boldsymbol{x}_n^{(k)}$ 与将 QMR_SYM 方法 (以 $\boldsymbol{x}_0 = \boldsymbol{0}$) 应用于 $A^{(k)}\boldsymbol{x}^{(k)} = \boldsymbol{b}$ 得到的第 n 个近似解相同.

Algorithm 4.6 移位 QMR_SYM 方法 (注意: $(\bar{\boldsymbol{a}}, \boldsymbol{b}) = \boldsymbol{a}^\top \boldsymbol{b}$)

Input: $\boldsymbol{x}_0^{(k)} = \boldsymbol{p}_{-1}^{(k)} = \boldsymbol{p}_0^{(k)} = \boldsymbol{0},\ \boldsymbol{v}_1 = \boldsymbol{b}/(\bar{\boldsymbol{b}}, \boldsymbol{b})^{1/2},\ g_1^{(k)} = (\bar{\boldsymbol{b}}, \boldsymbol{b})^{1/2},\ \beta_{-1} = 0$

Output: $\boldsymbol{x}_n^{(k)}$ 对于 $l = 1, 2, \ldots, m$

1: **for** $n = 1, 2, \ldots$ **do**
2: (复对称 Lanczos 过程)
3: $\alpha_n = (\bar{\boldsymbol{v}}_n, A\boldsymbol{v}_n)$
4: $\widetilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \alpha_n \boldsymbol{v}_n - \beta_{n-1}\boldsymbol{v}_{n-1}$
5: $\beta_n = (\overline{\widetilde{\boldsymbol{v}}_{n+1}}, \widetilde{\boldsymbol{v}}_{n+1})^{1/2}$
6: $\boldsymbol{v}_{n+1} = \widetilde{\boldsymbol{v}}_{n+1}/\beta_n$
7: $t_{n-1,n}^{(\ell)} = \beta_{n-1},\ t_{n,n}^{(\ell)} = \alpha_n + \sigma_\ell,\ t_{n+1,n}^{(\ell)} = \beta_n$
8: (用 Givens 旋转求解最小二乘问题)
9: 执行移位 MINRES 方法 (Algorithm 4.3) 的第 9–24 行.
10: **end for**

文献 [60] 考虑了下列移位线性方程组:

$$(A + \sigma_k I)\boldsymbol{x}^{(k)} = \boldsymbol{b}, \tag{4.30}$$

其中 A 为 Hermitian 矩阵, σ_k 为复数. 因而系数矩阵 $(A + \sigma_k I)$ 既非 Hermitian, 也非复对称. 在这种情况下, 可以使用非 Hermitian 移位线性方程组的移位 QMR 方法. 另一方面, 文献 [60] 中有一个比移位 QMR 方法更高效的算法, 它不需要计算 $(A + \sigma_k I)^{\mathrm{H}}$ 与向量的乘积,

即每步迭代所需的矩阵向量乘法次数仅为一次. 其关键思想是用 Lanczos 过程生成 Krylov 子空间 $\mathcal{K}_n(A,\boldsymbol{b})$, 并利用其基向量求解 (4.30). 注意, 若不生成 $\mathcal{K}_n(A,\boldsymbol{b})$, 而生成 σ 为复数时的 $\mathcal{K}_n(A+\sigma I,\boldsymbol{b})$, 则需要双 Lanczos 过程; 在矩阵向量乘法最耗时的情况下, 其计算开销约为 Lanczos 过程的两倍. 文献 [168] 将移位 QMR_SYM 方法推广到求解 (复对称) 广义移位线性方程组 (4.24). 与 Section 4.3.1 所述的广义移位 COCG 方法类似, 当 $B=I$ 时, 广义移位 QMR_SYM 方法 [168] 退化为移位 QMR_SYM 方法. 此外, 文献 [168] 基于 [167] 中用于复对称移位线性方程组的移位加权 QMR_SYM 方法, 提出了求解复对称广义移位线性方程组的相应算法.

4.4 移位非 Hermitian 线性方程组

本节始终假设 (4.1) 中的系数矩阵 $A^{(k)}$ 为非 Hermitian 矩阵, 即 $A^{(k)}\neq(A^{(k)})^{\mathrm{H}}$.

4.4.1 移位 BiCG 方法

如 Section 4.2.1 所见, 种子系统 $A\boldsymbol{x}=\boldsymbol{b}$ 的 CG 方法的第 i 个残差向量 $\boldsymbol{r}_i$, 与移位系统 $(A+\sigma I)\boldsymbol{x}^{\sigma}=\boldsymbol{b}$ 的第 i 个残差向量 $\boldsymbol{r}_i^{\sigma}$ 共线. Theorem 4.1 给出了 Krylov 子空间方法的两个残差向量共线条件的推广结果.

Theorem 4.1:

[[67]] 设 $W_1\subseteq W_2\subseteq\cdots\subseteq W_k$ 为 $\mathbb{C}^N$ 的子空间序列, 当 $i=1,2,\ldots,k$ 时满足 $\dim(W_i)=i$ 和 $W_i\cap\mathcal{K}_{i+1}(A,\boldsymbol{b})^{\perp}=\{\boldsymbol{0}\}$. 设 $\boldsymbol{x}_i\in\mathcal{K}_i(A,\boldsymbol{b})^{\perp}$ 为 $A\boldsymbol{x}=\boldsymbol{b}$ 的近似解, 由残差 $\boldsymbol{r}_i=\boldsymbol{b}-A\boldsymbol{x}_i=p_i(A)\boldsymbol{b}$ 的下列 Petrov–Galerkin 条件定义:

$$\boldsymbol{r}_i\perp W_i\quad\text{for } i=1,2,\ldots,k,$$

其中 $p_i(A)=\sum_{k=0}^{i}c_kA^i$. 类似地, 设 $\boldsymbol{x}_i^{\sigma}\in\mathcal{K}_i(A+\sigma I,\boldsymbol{b})=\mathcal{K}_i(A,\boldsymbol{b})$ 为 $(A+\sigma I)\boldsymbol{x}^{\sigma}=\boldsymbol{b}$ 的解的近似, 其残差 $\boldsymbol{r}^{\sigma}=\boldsymbol{b}-(A+\sigma I)\boldsymbol{x}_i^{\sigma}=p_i^{\sigma}(A+\sigma I)\boldsymbol{b}$ 同样满足

$$\boldsymbol{r}_i^{\sigma}\perp W_i\quad\text{for } i=1,2,\ldots,k.$$

则 $\boldsymbol{r}_i$ 与 $\boldsymbol{r}_i^{\sigma}$ 共线, 即存在 $\pi_i^{\sigma}\in\mathbb{C}$, 使得 $\boldsymbol{r}_i=\pi_i^{\sigma}\boldsymbol{r}_i^{\sigma}$.

Proof: 设 U_1,U_2 为 $\mathbb{C}^N$ 的子空间. 由标准线性代数可知, $(U_1\cap U_2)^{\perp}=U_1^{\perp}+U_2^{\perp}$. 由假设 $W_i\cap\mathcal{K}_{i+1}(A,\boldsymbol{b})^{\perp}=\{\boldsymbol{0}\}$, 有

$$(W_i\cap\mathcal{K}_{i+1}(A,\boldsymbol{b})^{\perp})^{\perp}=\{\boldsymbol{0}\}^{\perp}\Leftrightarrow W_i^{\perp}+\mathcal{K}_{i+1}(A,\boldsymbol{b})=\mathbb{C}^N.\tag{4.31}$$

有 $\dim W_i^{\perp}=N-i$ 和 $\mathcal{K}_{i+1}(A,\boldsymbol{b})=i+1$, 因而 $\dim W_i^{\perp}+\dim\mathcal{K}_{i+1}(A,\boldsymbol{b})=N+1$. 另一方面, 由 (4.31), 有 $\dim(W_i^{\perp}+\mathcal{K}_{i+1}(A,\boldsymbol{b}))=\dim\mathbb{C}^N=N$. 因此

$$\dim(W_i^{\perp}\cap\mathcal{K}_{i+1}(A,\boldsymbol{b}))=1.$$

由于 $\boldsymbol{r}_i,\boldsymbol{r}_i^{\sigma}\in\mathcal{K}_{i+1}(A,\boldsymbol{b})\cap W_i^{\perp}$, 两个残差向量 $\boldsymbol{r}_i,\boldsymbol{r}_i^{\sigma}$ 位于同一个一维子空间中, 这意味着 $\boldsymbol{r}_i,\boldsymbol{r}_i^{\sigma}$ 共线.

■

Theorem 4.1 表明, 利用关系 $W_n = \mathcal{K}_n(A^{\mathrm{H}}, \boldsymbol{r}_0^*)$, $A\boldsymbol{x} = \boldsymbol{b}$ 的 BiCG 残差向量 $\boldsymbol{r}_n$ 与 $(A + \sigma_k I)\boldsymbol{x}^{(k)} = \boldsymbol{b}$ 的 BiCG 残差向量 $\boldsymbol{r}_n^{(k)}$ 共线, 即

$$\boldsymbol{r}_n = \pi_n^{(k)} \boldsymbol{r}_n^{(k)}.$$

因此, 沿用 Section 4.2.1 中移位 CG 方法的推导, 得到 Algorithm 4.7 所列的移位 BiCG 方法算法. 类似地, Section 3.3.3 中的 BiCR 方法也被发展用于求解移位线性方程组, 在 [85] 中称为移位 BiCR 方法.

4.4.2 移位 BiCGSTAB 方法

文献 [67] 提出了移位 BiCGSTAB 方法和移位 BiCGSTAB(ℓ) 方法.
由于移位 BiCGSTAB(ℓ) 方法的推导较为复杂, 这里基于 [140] 中的说明介绍移位 BiCGSTAB 方法的推导. 为推导移位 BiCGSTAB 方法, 考虑种子系统 $A\boldsymbol{x} = \boldsymbol{b}$ 和移位系统 $(A+\sigma I)\boldsymbol{x}^{\sigma} = \boldsymbol{b}$. 首先回顾 Section 3.3.8 中的 BiCGSTAB 方法. 设 $\boldsymbol{r}_{n+1}$ 为 BiCGSTAB 方法的残差向量. 则 $\boldsymbol{r}_{n+1}$ 和其他迭代量 $\boldsymbol{t}_n, \boldsymbol{p}_n, \boldsymbol{x}_n$ 表示为

Algorithm 4.7 移位 BiCG 方法

Input: 选择种子系统 $s \in S = \{1, 2, \ldots, m\}$ 并令 $\boldsymbol{r}_0^{(s)} = \boldsymbol{b}$, $\beta_{-1}^{(s)} = 0$
Input: A 和 σ_s 对于 $k = 1, 2, \ldots, m$
Input: $\boldsymbol{x}_0^{(k)} = \boldsymbol{p}_{-1}^{(k)} = \boldsymbol{p}_{-1}^{*(k)} = \boldsymbol{0}$, $\pi_0^{(s,k)} = \pi_{-1}^{(s,k)} = 1$ 对于 $k = 1, 2, \ldots, m$
Input: 选择 $\boldsymbol{r}_0^* \in \mathbb{C}^N$, 例如 $\boldsymbol{r}_0^* = \boldsymbol{r}_0$
Output: $\boldsymbol{x}_n^{(k)}$ 对于 $k = 1, 2, \ldots, m$

1: **for** $n = 0, 1, \ldots,$ 直至收敛 **do**
2: $\quad \boldsymbol{p}_n^{(s)} = \boldsymbol{r}_n^{(s)} + \beta_{n-1}^{(s)} \boldsymbol{p}_{n-1}^{(s)}, \quad \boldsymbol{p}_n^{*(s)} = \boldsymbol{r}_n^{*(s)} + \bar{\beta}_{n-1}^{(s)} \boldsymbol{p}_{n-1}^{*(s)}$
3: $\quad \alpha_n^{(s)} = \dfrac{(\boldsymbol{r}_n^{*(s)}, \boldsymbol{r}_n^{(s)})}{(\boldsymbol{p}_n^{*(s)}, A^{(s)} \boldsymbol{p}_n^{(s)})}$
4: $\quad \boldsymbol{x}_{n+1}^{(s)} = \boldsymbol{x}_n^{(s)} + \alpha_n^{(s)} \boldsymbol{p}_n^{(s)}$
5: $\quad$ {移位系统开始}
6: $\quad$ 执行移位 CG 方法 (Algorithm 4.1) 的第 6–15 行.
7: $\quad$ {移位系统结束}
8: $\quad \boldsymbol{r}_{n+1}^{(s)} = \boldsymbol{r}_n^{(s)} - \alpha_n^{(s)} A^{(s)} \boldsymbol{p}_n^{(s)}, \quad \boldsymbol{r}_{n+1}^{*(s)} = \boldsymbol{r}_n^{*(s)} - \bar{\alpha}_n A^{(s)\mathrm{H}} \boldsymbol{p}_n^{*(s)}$
9: $\quad \beta_n^{(s)} = \dfrac{(\boldsymbol{r}_{n+1}^{*(s)}, \boldsymbol{r}_{n+1}^{(s)})}{(\boldsymbol{r}_n^{*(s)}, \boldsymbol{r}_n^{(s)})}$
10: **end for**

$$\begin{aligned}
\boldsymbol{p}_n &:= Q_n(A)\boldsymbol{p}_n^{\mathrm{BiCG}} = \boldsymbol{r}_n + \beta_{n-1}(\boldsymbol{p}_{n-1} - \zeta_{n-1} A \boldsymbol{p}_{n-1}) \\
&= \boldsymbol{r}_n + \beta_{n-1}\left[\boldsymbol{p}_{n-1} + \frac{\zeta_{n-1}}{\alpha_{n-1}}(\boldsymbol{t}_{n-1} - \boldsymbol{r}_{n-1})\right], \\
\boldsymbol{t}_n &:= Q_n(A)\boldsymbol{r}_{n+1}^{\mathrm{BiCG}} = \boldsymbol{r}_n - \alpha_n A \boldsymbol{p}_n, \\
\boldsymbol{r}_{n+1} &:= Q_{n+1}(A)\boldsymbol{r}_{n+1}^{\mathrm{BiCG}} = \boldsymbol{t}_n - \zeta_n A \boldsymbol{t}_n, \\
\boldsymbol{x}_{n+1} &:= \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n + \zeta_n \boldsymbol{t}_n,
\end{aligned}$$

其中 $\boldsymbol{r}_{n+1}^{\mathrm{BiCG}}$ 和 $\boldsymbol{p}_n^{\mathrm{BiCG}}$ 分别为第 $n+1$ 次迭代的 BiCG 残差向量和第 n 次迭代的 BiCG 搜索方向, 多项式 $Q_{n+1}(z)$ 定义为

$$Q_0(\lambda) := 1, \tag{4.32}$$

$$Q_{n+1}(\lambda) := (1-\zeta_n\lambda)Q_n(\lambda), \quad n = 0, 1, \ldots \tag{4.33}$$

对于 BiCGSTAB 方法, ζ_n 取为使 $\|\boldsymbol{r}_{n+1}\|$ 极小. 回顾 Section 4.4.1 中, $(A+\sigma I)\boldsymbol{x}^\sigma = \boldsymbol{b}$ 的移位 BiCG 方法残差向量写为

$$\boldsymbol{r}_{n+1}^{\sigma,\mathrm{BiCG}} = \xi_{n+1}^\sigma \boldsymbol{r}_{n+1}^{\mathrm{BiCG}}, \quad \xi_{n+1}^\sigma \in \mathbb{C}.$$

这里, 由 (4.14), 对所有 n 将标量 ξ_{n+1}^σ 定义为 $\xi_{n+1}^\sigma := (\pi_{n+1}^\sigma)^{-1}$, 因而有

$$\begin{aligned}
\xi_{n+1}^\sigma &= \frac{1}{\left(1+\frac{\beta_{n-1}}{\alpha_{n-1}}\alpha_n + \alpha_n\sigma\right)\pi_n^\sigma - \frac{\beta_{n-1}}{\alpha_{n-1}}\alpha_n\pi_{n-1}^\sigma} \\
&= \frac{1}{\left(1+\frac{\beta_{n-1}}{\alpha_{n-1}}\alpha_n + \alpha_n\sigma\right)(\xi_n^\sigma)^{-1} - \frac{\beta_{n-1}}{\alpha_{n-1}}\alpha_n(\xi_{n-1}^\sigma)^{-1}} \\
&= \frac{\xi_n^\sigma\xi_{n-1}^\sigma\alpha_{n-1}}{(1+\alpha_n\sigma)\xi_{n-1}^\sigma\alpha_{n-1} + \alpha_n\beta_{n-1}(\xi_{n-1}^\sigma - \xi_n^\sigma)},
\end{aligned} \tag{4.34}$$

其中 $\xi_{-1}^\sigma = \xi_0^\sigma = 1$. 现在可以介绍移位 BiCGSTAB 方法. 移位 BiCGSTAB 方法的残差向量定义为

$$\boldsymbol{r}_{n+1}^{\sigma,\mathrm{STAB}} := Q_{n+1}^\sigma(A)\boldsymbol{r}_{n+1}^{\sigma,\mathrm{BiCG}},$$

其中 $Q_{n+1}^\sigma(\lambda)$ 递归定义为

$$Q_{n+1}^\sigma(\lambda) := [1-\zeta_n^\sigma(\lambda+\sigma)]Q_n^\sigma(\lambda), \quad n = 0, 1, \ldots \tag{4.35}$$

这里 $Q_0^\sigma(\lambda) := 1$. ζ_n^σ 取为使移位 BiCGSTAB 残差与 BiCGSTAB 残差共线, 即

$$\boldsymbol{r}_{n+1}^{\sigma,\mathrm{STAB}} = c_n\boldsymbol{r}_{n+1}^{\mathrm{STAB}}, \quad c_n \in \mathbb{C}.$$

为此, 确定 ζ_n^σ 使

$$Q_{n+1}^\sigma(\lambda) = \tau_{n+1}^\sigma Q_{n+1}(\lambda), \quad \tau_{n+1}^\sigma \in \mathbb{C}. \tag{4.36}$$

于是, 移位 BiCGSTAB 方法的残差向量定义并表示为

$$\begin{aligned}
\boldsymbol{r}_{n+1}^{\sigma,\mathrm{STAB}} &:= Q_{n+1}^\sigma(A)\boldsymbol{r}_{n+1}^{\sigma,\mathrm{BiCG}} \\
&= \tau_{n+1}^\sigma Q_{n+1}(A)\xi_{n+1}^\sigma\boldsymbol{r}_{n+1}^{\mathrm{BiCG}} \\
&= \tau_{n+1}^\sigma\xi_{n+1}^\sigma Q_{n+1}(A)\boldsymbol{r}_{n+1}^{\mathrm{BiCG}} \\
&= \tau_{n+1}^\sigma\xi_{n+1}^\sigma\boldsymbol{r}_{n+1}^{\mathrm{STAB}}.
\end{aligned} \tag{4.37}$$

下面确定 (4.37) 中的参数 $\tau_{n+1}^\sigma, \xi_{n+1}^\sigma$. 由 (4.33), (4.35) 和 (4.36) 得

$$\underbrace{\tau_{n+1}^\sigma(1-\zeta_n\lambda)Q_n(\lambda)}_{Q_{n+1}^\sigma(\lambda)} = [1-\zeta_n^\sigma(\lambda+\sigma)]\underbrace{\tau_n^\sigma Q_n(\lambda)}_{Q_n^\sigma(\lambda)}. \tag{4.38}$$

比较 (4.38) 两边的系数, 得

$$\zeta_n\tau_{n+1}^\sigma = \zeta_n^\sigma\tau_n^\sigma, \quad \tau_{n+1}^\sigma = (1-\zeta_n^\sigma\sigma)\tau_n^\sigma,$$

由此得到

$$\tau_{n+1}^{\sigma}=\frac{\tau_n^{\sigma}}{1+\zeta_n\sigma},\quad \zeta_n^{\sigma}=\frac{\zeta_n}{1+\zeta_n\sigma}, \tag{4.39}$$

其中由关系 (4.36) 和 $Q_0^{\sigma}(\lambda)=Q_0(\lambda)=1$ 得 $\tau_0^{\sigma}=1$. 现在, $(A+\sigma I)\boldsymbol{x}^{\sigma}=\boldsymbol{b}$ 的移位 BiCGSTAB 迭代量为

$$\begin{aligned}\boldsymbol{p}_n^{\sigma}&:=Q_n^{\sigma}(A)\boldsymbol{p}_n^{\sigma,\mathrm{BiCG}}=\boldsymbol{r}_n^{\sigma}+\beta_{n-1}^{\sigma}\left[\boldsymbol{p}_{n-1}^{\sigma}+\frac{\zeta_{n-1}^{\sigma}}{\alpha_{n-1}^{\sigma}}(\boldsymbol{t}_{n-1}^{\sigma}-\boldsymbol{r}_{n-1}^{\sigma})\right]\\&=\tau_n^{\sigma}\xi_n^{\sigma}\boldsymbol{r}_n+\beta_{n-1}^{\sigma}\left[\boldsymbol{p}_{n-1}^{\sigma}+\frac{\zeta_{n-1}^{\sigma}}{\alpha_{n-1}^{\sigma}}\tau_{n-1}^{\sigma}(\xi_n^{\sigma}\boldsymbol{t}_{n-1}-\xi_{n-1}^{\sigma}\boldsymbol{r}_{n-1})\right],\end{aligned} \tag{4.40}$$

$$\boldsymbol{t}_n^{\sigma}:=Q_n^{\sigma}(A)\boldsymbol{r}_{n+1}^{\sigma,\mathrm{BiCG}}=\tau_n^{\sigma}\xi_{n+1}^{\sigma}Q_n\boldsymbol{r}_{n+1}^{\mathrm{BiCG}}=\tau_n^{\sigma}\xi_{n+1}^{\sigma}\boldsymbol{t}_n, \tag{4.41}$$

$$\boldsymbol{r}_{n+1}^{\sigma}:=Q_{n+1}^{\sigma}(A)\boldsymbol{r}_{n+1}^{\sigma,\mathrm{BiCG}}=\tau_{n+1}^{\sigma}\xi_{n+1}^{\sigma}Q_{n+1}\boldsymbol{r}_{n+1}^{\mathrm{BiCG}}=\tau_{n+1}^{\sigma}\xi_{n+1}^{\sigma}\boldsymbol{r}_{n+1}^{\mathrm{STAB}}, \tag{4.42}$$

$$\boldsymbol{x}_{n+1}^{\sigma}:=\boldsymbol{x}_n^{\sigma}+\alpha_n^{\sigma}\boldsymbol{p}_n^{\sigma}+\zeta_n^{\sigma}\boldsymbol{t}_n^{\sigma}. \tag{4.43}$$

由 (4.34), (4.39), (4.40)–(4.43), 就得到移位 BiCGSTAB 方法, 其算法见 Algorithm 4.8.

4.4.3 移位 GMRES 方法

本小节推导移位 GMRES 方法 [43]. 为此, 考虑将 GMRES 方法应用于移位线性方程组 $(A+\sigma I)\boldsymbol{x}=\boldsymbol{b}$. 为简便起见, 令初始猜测为 $\boldsymbol{x}_0=\boldsymbol{0}$. 则 GMRES 方法在下列 Krylov 子空间上寻找近似解:

$$\boldsymbol{x}_n^{\sigma}\in\mathcal{K}_n(A+\sigma I,\boldsymbol{b}). \tag{4.44}$$

由移位不变性 (4.4), 有 $\mathcal{K}_n(A+\sigma I,\boldsymbol{b})=\mathcal{K}_n(A,\boldsymbol{b})$. 因此, 使用 Arnoldi 过程时, $\mathcal{K}_n(A+\sigma I,\boldsymbol{b})$ 的标准正交基向量等价于 $\mathcal{K}_n(A,\boldsymbol{b})$ 的标准正交基向量, 即 Section 1.9.1 中的 V_n. 于是, (4.44) 及相应残差表示为

$$\boldsymbol{x}_n^{\sigma}=V_n\boldsymbol{y}_n,\quad \boldsymbol{y}_n\in\mathbb{C}^n$$

Algorithm 4.8 移位 BiCGSTAB 方法

Input: $\boldsymbol{x}_0 = \boldsymbol{0},\ \boldsymbol{p}_{-1} = \boldsymbol{0},\ \boldsymbol{r}_0 = \boldsymbol{b},\ \beta_{-1} = 0$

Input: 选择 $\boldsymbol{r}_0^* \in \mathbb{C}^N$, 例如 $\boldsymbol{r}_0^* = \boldsymbol{r}_0$

Input: $\boldsymbol{x}_0^{(k)} = \boldsymbol{0},\ \boldsymbol{p}_0^{(k)} = \boldsymbol{b},\ \beta_{-1}^{(k)} = 0,\ \alpha_{-1}^{(k)} = \xi_{-1}^{(k)} = \xi_0^{(k)} = \tau_0^{(k)} = 1$ 对于 $k = 1, 2, \ldots, m$

Output: $\boldsymbol{x}_n^{(k)}$

for $n = 0, 1, \ldots$ **do**

$\boldsymbol{p}_n = \boldsymbol{r}_n + \beta_{n-1}(\boldsymbol{p}_{n-1} - \zeta_{n-1} A\boldsymbol{p}_{n-1})$

$\alpha_n = \dfrac{(\boldsymbol{r}_0^*, \boldsymbol{r}_n)}{(\boldsymbol{r}_0^*, A\boldsymbol{p}_n)}$

$\boldsymbol{t}_n = \boldsymbol{r}_n - \alpha_n A\boldsymbol{p}_n$

$\zeta_n = \dfrac{(A\boldsymbol{t}_n, \boldsymbol{t}_n)}{(A\boldsymbol{t}_n, A\boldsymbol{t}_n)}$

$\boldsymbol{x}_{n+1} = \boldsymbol{x}_n + \alpha_n \boldsymbol{p}_n + \zeta_n \boldsymbol{t}_n$

$\boldsymbol{r}_{n+1} = \boldsymbol{t}_n - \zeta_n A\boldsymbol{t}_n$

$\beta_n = \dfrac{\alpha_n}{\zeta_n} \times \dfrac{(\boldsymbol{r}_0^*, \boldsymbol{r}_{n+1})}{(\boldsymbol{r}_0^*, \boldsymbol{r}_n)}$

{移位系统开始}

for $k = 1, 2, \ldots, m$ **do**

$$\xi_{n+1}^{(k)} = \frac{\xi_n^{(k)} \xi_{n-1}^{(k)} \alpha_{n-1}}{(1 + \alpha_n \sigma)\xi_{n-1}^{(k)} \alpha_{n-1} + \alpha_n \beta_{n-1}(\xi_{n-1}^{(k)} - \xi_n^{(k)})}$$

$$\alpha_n^{(k)} = \frac{\xi_{n+1}^{(k)}}{\xi_n^{(k)}} \alpha_k$$

$$\zeta_n^{(k)} = \frac{\zeta_n}{1 + \zeta_n \sigma}$$

$$\boldsymbol{x}_{n+1}^{(k)} = \boldsymbol{x}_n^{(k)} + \alpha_n^{(k)} \boldsymbol{p}_n^{(k)} + \zeta_n^{(k)} \boldsymbol{t}_n^{(k)}$$

$$\tau_{n+1}^{(k)} = \frac{\tau_n^{(k)}}{1 + \zeta_n \sigma}$$

$$\beta_n^{(k)} = \left(\frac{\xi_{n+1}^{(k)}}{\xi_n^{(k)}} \right)^2 \beta_k$$

$$\begin{aligned} \boldsymbol{p}_{n+1}^{(k)} &= \tau_{n+1}^{(k)} \xi_{n+1}^{(k)} \boldsymbol{r}_{n+1} \\ &\quad + \beta_n^{(k)} \left[\boldsymbol{p}_n^{(k)} + \frac{\zeta_n^{(k)}}{\alpha_n^{(k)}} \tau_n^{(k)} (\xi_{n+1}^{(k)} \boldsymbol{t}_n^{(k)} - \xi_n^{(k)} \boldsymbol{r}_n) \right] \end{aligned}$$

end for

{移位系统结束}

end for

以及

$$\begin{aligned}
\boldsymbol{r}_n^\sigma &= \boldsymbol{b} - (A+\sigma I)\boldsymbol{x}_n^\sigma \\
&= \|\boldsymbol{b}\| V_{n+1}\boldsymbol{e}_1 - AV_n\boldsymbol{y}_n - \sigma V_n\boldsymbol{y}_n \\
&= \|\boldsymbol{b}\| V_{n+1}\boldsymbol{e}_1 - V_{n+1}H_{n+1,n}\boldsymbol{y}_n - \sigma V_{n+1}\begin{bmatrix}\boldsymbol{y}_n \\ 0\end{bmatrix} \\
&= \|\boldsymbol{b}\| V_{n+1}\boldsymbol{e}_1 - V_{n+1}H_{n+1,n}\boldsymbol{y}_n - \sigma V_{n+1}\begin{bmatrix}I_n \\ \mathbf{0}^\top\end{bmatrix}\boldsymbol{y}_n \\
&= V_{n+1}\left[\|\boldsymbol{b}\|\boldsymbol{e}_1 - \left(H_{n+1,n} + \begin{bmatrix}\sigma I_n \\ \mathbf{0}^\top\end{bmatrix}\right)\boldsymbol{y}_n\right] \\
&= V_{n+1}(\|\boldsymbol{b}\|\boldsymbol{e}_1 - H_{n+1,n}^\sigma \boldsymbol{y}_n),
\end{aligned}$$

其中 I_n 为 $n\times n$ 单位矩阵, $H_{n+1,n}^\sigma$ 为所谓的移位 Hessenberg 矩阵, 其元素在 $i\neq j$ 时满足 $(H_{n+1,n}^\sigma)_{i,j} = (H_{n+1,n})_{i,j}$, 在 $i=j$ 时满足 $(H_{n+1,n}^\sigma)_{i,j} = (H_{n+1,n})_{i,j} + \sigma$. 于是, 类似于 (3.63), 通过求解下式可使残差范数极小

$$\boldsymbol{y}_n^\sigma := \arg\min_{\boldsymbol{y}\in\mathbb{C}^n} \|\beta\boldsymbol{e}_1 - H_{n+1,n}^\sigma \boldsymbol{y}\|.$$

上述最小二乘问题可利用 Section 3.1.3 中介绍的 Givens 旋转高效求解. 利用 $\boldsymbol{y}_n^\sigma$, 近似解现为

$$\boldsymbol{x}_n^\sigma = V_n\boldsymbol{y}_n^\sigma.$$

若系数矩阵 A 为 Hermitian 矩阵且 $\sigma\in\mathbb{R}$, 则移位 GMRES 方法退化为 Section 4.2.3 中的移位 MINRES 方法. 移位 GMRES 方法的算法见 Algorithm 4.9.

Algorithm 4.9 移位 GMRES 方法

Input: σ_k $(k = 1, 2, \ldots, m)$

Output: $\boldsymbol{x}_n^{(k)}$ $(k = 1, 2, \ldots, m)$

1: $\boldsymbol{g} = (\|\boldsymbol{b}\|, 0, \ldots, 0)^\top$, $\boldsymbol{v}_1 = \boldsymbol{b}/\|\boldsymbol{b}\|$
2: **for** $n = 1, 2, \ldots$ **do**
3: (Arnoldi 过程)
4: $\boldsymbol{t} = A\boldsymbol{v}_n$
5: **for** $i = 1, 2, \ldots, n$ **do**
6: $h_{i,n} = (\boldsymbol{v}_i, \boldsymbol{t})$
7: $\boldsymbol{t} = \boldsymbol{t} - h_{i,n}\boldsymbol{v}_i$
8: **end for**
9: $h_{n+1,n} = \|\boldsymbol{t}\|$
10: $\boldsymbol{v}_{n+1} = \boldsymbol{t}/h_{n+1,n}$
11: {移位系统开始}
12: $h_{i,n}^{(k)} = h_{i,n}$ 对于 $i \neq n$
13: $h_{n,n}^{(k)} = h_{n,n} + \sigma_k$
14: **for** $k = 1, 2, \ldots, m$ **do**
15: {Givens 旋转}
16: **for** $i = 1, 2, \ldots, n-1$ **do**
17: $\begin{bmatrix} h_{i,n}^{(k)} \\ h_{i+1,n}^{(k)} \end{bmatrix} = \begin{bmatrix} c_i^{(k)} & s_i^{(k)} \\ -\bar{s}_i^{(k)} & c_i^{(k)} \end{bmatrix} \begin{bmatrix} h_{i,n}^{(k)} \\ h_{i+1,n}^{(k)} \end{bmatrix}$
18: **end for**
19: $c_n^{(k)} = \dfrac{|h_{n,n}^{(k)}|}{\sqrt{|h_{n,n}^{(k)}|^2 + |h_{n+1,n}^{(k)}|^2}}$
20: $\bar{s}_n = \dfrac{h_{n+1,n}^{(k)}}{h_{n,n}^{(k)}} c_n$
21: $h_{n,n}^{(k)} = c_n^{(k)} h_{n,n}^{(k)} + s_n^{(k)} h_{n+1,n}^{(k)}$
22: $h_{n+1,n}^{(k)} = 0$
23: $\begin{bmatrix} g_n^{(k)} \\ g_{n+1}^{(k)} \end{bmatrix} = \begin{bmatrix} c_n^{(k)} & s_n^{(k)} \\ -\bar{s}_n^{(k)} & c_n^{(k)} \end{bmatrix} \begin{bmatrix} g_n^{(k)} \\ 0 \end{bmatrix}$
24: (检查收敛性)
25: **if** $|g_{n+1}^{(k)}|/\|\boldsymbol{b}\| \leq \epsilon$, **then**
26: $\boldsymbol{x}_n^{(k)} = V_n (H_n^{(k)})^{-1} \boldsymbol{g}^{(k)}$
27: **end if**
28: **end for**
29: {移位系统结束}
30: **end for**

GMRES 方法与移位 GMRES 方法的区别在于初始猜测 $\boldsymbol{x}_0 = \boldsymbol{0}$ 和 Algorithm 4.9 中对应于生成 $H_{n+1,n}^{\sigma}$ 的第 13 行. 注意, 经过 Givens 旋转后, H_n 成为上三角矩阵, 因而第 26 行的 $\boldsymbol{c}^{(k)} := (H_n^{(k)})^{-1}\boldsymbol{g}$ 可通过求解 $H_n^{(k)}\boldsymbol{c}^{(k)} = \boldsymbol{g}$ 容易地得到. 如 Section 3.3.5 所述, 由于存储需求

和计算开销增长, 当迭代次数很多时, GMRES 方法并不实用. 相比之下, Algorithm 3.17 中介绍的重启 GMRES 方法在实践中很有用. 出于同样原因, 当迭代次数很多时, 移位 GMRES 方法也不实用. 因而自然可以考虑移位线性方程组的重启版本. 然而, 若采用 Algorithm 3.17 的思想, 就面临一个困难: 重启后初始残差向量 $\boldsymbol{r}_0^{(i)}$ 和 $\boldsymbol{r}_0^{(j)}$ 一般不再共线, 因而无法共享 Krylov 子空间的基向量来求解移位线性方程组. Frommer 和 Gräsner [68] 巧妙地绕过了这一困难. 下面介绍其思想和相应算法. 设 $\boldsymbol{r}_n$ 为 $A\boldsymbol{x}=\boldsymbol{b}$ 的 GMRES 方法残差向量. 他们考虑强制使移位系统 $(A+\sigma I)\boldsymbol{x}^\sigma=\boldsymbol{b}$ 的残差向量 $\boldsymbol{r}_n^\sigma$ 与 $\boldsymbol{r}_n$ 共线, 即寻找近似解 $\boldsymbol{x}^\sigma=\boldsymbol{x}_0^\sigma+V_n\boldsymbol{y}_n^\sigma$ 使得

$$\boldsymbol{r}_n^\sigma=\beta_n\boldsymbol{r}_n.$$

$\boldsymbol{r}_n$ 表示为

$$\boldsymbol{r}_n=\boldsymbol{b}-A\boldsymbol{x}_n=V_{n+1}\boldsymbol{z}_{n+1},$$

其中 $\boldsymbol{z}_{n+1}:=\|\boldsymbol{r}_0\|\boldsymbol{e}_1-H_{n+1,n}\boldsymbol{y}_n$. 现在有下列等式:

$$\begin{aligned}
\boldsymbol{r}_n^\sigma=\beta_n\boldsymbol{r}_n &\Leftrightarrow \boldsymbol{b}-A^\sigma\boldsymbol{x}_n^\sigma=\beta_nV_{n+1}\boldsymbol{z}_{n+1}\\
&\Leftrightarrow \boldsymbol{b}-A^\sigma(\boldsymbol{x}_0^\sigma+V_n\boldsymbol{y}_n^\sigma)=\beta_nV_{n+1}\boldsymbol{z}_{n+1}\\
&\Leftrightarrow \beta_0\boldsymbol{r}_0-A^\sigma V_n\boldsymbol{y}_n^\sigma=\beta_nV_{n+1}\boldsymbol{z}_{n+1}\\
&\Leftrightarrow \beta_0\boldsymbol{r}_0-V_{n+1}H_{n+1,n}^\sigma\boldsymbol{y}_n^\sigma=\beta_nV_{n+1}\boldsymbol{z}_{n+1}\\
&\Leftrightarrow \beta_0\boldsymbol{r}_0=V_{n+1}(\beta_n\boldsymbol{z}_{n+1}+H_{n+1,n}^\sigma\boldsymbol{y}_n^\sigma)\\
&\Leftrightarrow V_{n+1}(H_{n+1,n}^\sigma\boldsymbol{y}_n^\sigma+\beta_n\boldsymbol{z}_{n+1})=\beta_0\boldsymbol{r}_0\\
&\Leftrightarrow V_{n+1}(H_{n+1,n}^\sigma\boldsymbol{y}_n^\sigma+\beta_n\boldsymbol{z}_{n+1})=\beta_0\|\boldsymbol{r}_0\|V_{n+1}\boldsymbol{e}_1\\
&\Leftrightarrow H_{n+1,n}^\sigma\boldsymbol{y}_n^\sigma+\beta_n\boldsymbol{z}_{n+1}=\beta_0\|\boldsymbol{r}_0\|\boldsymbol{e}_1,
\end{aligned}$$

由此, 通过求解下列 $(n+1)\times(n+1)$ 线性方程组确定 β_n 和 $\boldsymbol{y}_n^\sigma$:

$$[H_{n+1,n}^\sigma\ \boldsymbol{z}_{n+1}]\begin{bmatrix}\boldsymbol{y}_n^\sigma\\ \beta_n\end{bmatrix}=\beta_0\|\boldsymbol{r}_0\|\boldsymbol{e}_1. \tag{4.45}$$

重启后, 新初始残差向量 $\boldsymbol{r}_0(=\boldsymbol{r}_n),\boldsymbol{r}_0^\sigma(=\boldsymbol{r}_n^\sigma)$ 共线. 因此, 可利用 $A\boldsymbol{x}=\boldsymbol{b}$ 的 Krylov 子空间 $\mathcal{K}_n(A,\boldsymbol{b})$ 求解移位线性方程组 $(A+\sigma I)\boldsymbol{x}^\sigma=\boldsymbol{b}^\sigma$. 重启移位 GMRES 方法的算法见 Algorithm 4.10. 注意, 由关系 $\|\boldsymbol{r}_n^{(k)}\|=\|\beta_n^{(k)}\boldsymbol{r}_n\|$, 可通过监测 $|\beta_n^{(k)}|\times\|\boldsymbol{r}_n\|$ 检查 Algorithm 4.10 中 $\|\boldsymbol{r}_n^{(k)}\|$ 的收敛性. 关于 (重启) 移位 GMRES 方法的进一步发展, 见 [55], [105], [115], [175] 及其中参考文献. 完全正交化方法 (FOM) [149] 是 CG 方法在求解非 Hermitian 线性方程组方面的推广. 与 CG 方法一样, FOM 的残差向量 $\boldsymbol{r}_n^{\mathrm{FOM}}$ 满足

$$\boldsymbol{r}_n^{\mathrm{FOM}}(\in\mathcal{K}_{n+1}(A,\boldsymbol{b}))\perp\mathcal{K}_n(A,\boldsymbol{b}).$$

Algorithm 4.10 重启移位 GMRES 方法

Input: σ_k $(k=1,2,\dots,m)$

Input: $\boldsymbol{x}_0 \in \mathbb{C}^N$

Input: 令 $\boldsymbol{x}_0^{(k)} \in \mathbb{C}^N$ 使 $\boldsymbol{r}_0^{(k)} = \beta_0^{(k)} \boldsymbol{r}_0$, 例如对所有 k 令 $\boldsymbol{x}_0 = \boldsymbol{x}_0^{(k)} = \boldsymbol{0}$.

Output: $\boldsymbol{x}_n^{(k)}$ $(k=1,2,\dots,m)$

$\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0,\ \beta = \|\boldsymbol{r}_0\|$

以 $\boldsymbol{v}_1 = \boldsymbol{r}_0/\beta$ 执行 Algorithm 1.8 中的 Arnoldi 过程.

计算 $\boldsymbol{y}_n$ 使 $\|\beta\boldsymbol{e}_1 - H_{n+1,n}\boldsymbol{y}_n\|$ 极小.

$\boldsymbol{x}_n = \boldsymbol{x}_0 + V_n\boldsymbol{y}_n,\ \boldsymbol{z}_{n+1} = \beta\boldsymbol{e}_1 - H_{n+1,n}\boldsymbol{y}_n$

{移位系统}

for $k=1,2,\dots,m$ **do**

$$H_{n+1,n}^{(k)} = H_{n+1,n} + \begin{bmatrix} \sigma_k I_n \\ \boldsymbol{0}^\top \end{bmatrix}$$

求解 $[H_{n+1,n}^{(k)}\ \boldsymbol{z}_{n+1}] \begin{bmatrix} \boldsymbol{y}_n^{(k)} \\ \beta_n^{(k)} \end{bmatrix} = \beta_0^{(k)}\beta\boldsymbol{e}_1.$

$\boldsymbol{x}_n^{(k)} = \boldsymbol{x}_0^{(k)} + V_m\boldsymbol{y}_m^{(k)}$

end for

if 尚未收敛 **then**

令 $\boldsymbol{x}_0 = \boldsymbol{x}_n,\ \boldsymbol{x}_0^{(k)} = \boldsymbol{x}_n^{(k)},\ \beta_0^{(k)} = \beta_n^{(k)}$, 并转到第 1 行.

end if

将 FOM 应用于移位线性方程组时, 有

$$\boldsymbol{r}_n^{\sigma,\mathrm{FOM}} (\in \mathcal{K}_{n+1}(A+\sigma I, \boldsymbol{b})) \perp \mathcal{K}_n(A+\sigma I, \boldsymbol{b}).$$

于是由移位不变性 $\mathcal{K}_n(A+\sigma I, \boldsymbol{b}) = \mathcal{K}_n(A, \boldsymbol{b})$, 得

$$\boldsymbol{r}_n^{\mathrm{FOM}}, \boldsymbol{r}_n^{\sigma,\mathrm{FOM}} \in \mathcal{K}_{n+1}(A, \boldsymbol{b}) \cap \mathcal{K}_n(A, \boldsymbol{b})^\perp.$$

由 (4.5), 两个残差向量 $\boldsymbol{r}_n^{\mathrm{FOM}}, \boldsymbol{r}_n^{\sigma,\mathrm{FOM}}$ 属于一维子空间. 因此, 应用于 $A\boldsymbol{x} = \boldsymbol{b}$ 和 $(A+\sigma I)\boldsymbol{x}^\sigma = \boldsymbol{b}$ 的残差向量共线. 这意味着考虑重启时, 无需采用 (4.45) 的技巧来保持共线性, 因为每步迭代的残差向量都共线. 关于重启移位 FOM 的细节, 见 [158]. 关于重启移位 FOM 的进一步发展, 见 [55], [112] 及其中参考文献.

4.4.4 移位 IDR(s) 方法

Section 3.3.9 中的 IDR(s) 方法被推广用于求解移位线性方程组,
相应算法称为移位 IDR(s) 方法 [48].
在推导移位 IDR(s) 方法之前, 先介绍 IDR 定理 (Theorem 3.12) 的一个变型.

Corollary 4.2: [[48]] 设 $\boldsymbol{r}_0^{(i)}$ $(i=1,\ldots,m)$ 彼此共线, $\mathcal{G}_0^{(i)}=\mathcal{K}_n(A+\sigma_i I,\boldsymbol{r}_0^{(i)})$, $\mathcal{S}$ 为 $\mathbb{R}^n$ 的子空间, 并如下定义子空间序列 $\mathcal{G}_j^{(i)}$

$$\mathcal{G}_j^{(i)}=[I-\omega_j^{(i)}(A+\sigma_i I)](\mathcal{G}_{j-1}^{(i)}\cap\mathcal{S}),\quad \omega_j^{(i)}\neq 0,\quad j=1,2,\ldots,$$

则有 $\mathcal{G}_j^{(1)}=\mathcal{G}_j^{(2)}=\cdots=\mathcal{G}_j^{(m)}$.

Corollary 4.2 表明, 若考虑求解下列线性方程组 (种子系统) 和移位线性方程组:

$$A\boldsymbol{x}=\boldsymbol{b},\quad (A+\sigma I)\boldsymbol{x}^{\sigma}=\boldsymbol{b},$$

则移位 BiCGSTAB 方法中所述的共线方式 $\boldsymbol{r}_i=\pi_i^{\sigma}\boldsymbol{r}_i^{\sigma}$ 很有前景, 因为 $\boldsymbol{r}_i=\boldsymbol{0}$ 和 $\pi_i^{\sigma}\neq 0$ 导出 $\boldsymbol{r}_i^{\sigma}=\boldsymbol{0}$. 这里 $\boldsymbol{r}_i$ 为原线性方程组 (种子系统) $A\boldsymbol{x}=\boldsymbol{b}$ 的第 i 个残差向量, $\boldsymbol{r}_i^{\sigma}$ 为移位线性方程组 $(A+\sigma I)\boldsymbol{x}^{\sigma}=\boldsymbol{b}$ 的第 i 个残差向量. 现在介绍当 $\boldsymbol{r}_i=\pi_i^{\sigma}\boldsymbol{r}_i^{\sigma}$ $(\pi_i^{\sigma}\in\mathbb{C},\ i=0,1,\ldots,k)$ 时, 如何从 $\boldsymbol{r}_{k+1}$ 的信息计算 $\boldsymbol{r}_{k+1}^{\sigma}$. 由 Algorithm 3.24 中 IDR(s) 方法的初始步骤, 种子系统的初始残差向量为

$$\boldsymbol{r}_{k+1}=\boldsymbol{r}_k-\omega_k A\boldsymbol{r}_k,\quad (k=0,\ldots,s-1). \tag{4.46}$$

类似于 (4.46), 移位系统的初始残差向量如下:

$$\boldsymbol{r}_{k+1}^{\sigma}=\boldsymbol{r}_k^{\sigma}-\omega_k^{\sigma}(A+\sigma I)\boldsymbol{r}_k^{\sigma}=(1-\sigma\omega_k^{\sigma})\boldsymbol{r}_k^{\sigma}-\omega_k^{\sigma}A\boldsymbol{r}_k^{\sigma}, \tag{4.47}$$

其中参数 $\omega_k^{\sigma}\in\mathbb{C}$ 未知. 将关系 $\boldsymbol{r}_i=\pi_i^{\sigma}\boldsymbol{r}_i^{\sigma}$ $(i=0,\ldots,k)$ 代入 (4.46), 得

$$\boldsymbol{r}_{k+1}^{\sigma}=\frac{\pi_k^{\sigma}}{\pi_{k+1}^{\sigma}}\boldsymbol{r}_k^{\sigma}-\omega_k\frac{\pi_k^{\sigma}}{\pi_{k+1}^{\sigma}}A\boldsymbol{r}_k^{\sigma}. \tag{4.48}$$

由关系 $\boldsymbol{r}_i=\pi_i^{\sigma}\boldsymbol{r}_i^{\sigma}$, (4.46) 和残差多项式 $R_i(0)=1$, $\boldsymbol{r}_i=R_i(A)\boldsymbol{r}_0$, 得

$$\begin{cases}\pi_k^{\sigma}=R_k(-\sigma),\\ \pi_{k+1}^{\sigma}=\pi_k^{\sigma}+\omega_k\sigma\pi_k^{\sigma}.\end{cases}$$

因此, 算出 π_k^{σ} 后就可得到 (4.48) 中的参数 π_{k+1}^{σ}. 比较 (4.47) 和 (4.48) 的系数, 得到 $\omega_k^{\sigma}=\omega_k\pi_k^{\sigma}/\pi_{k+1}^{\sigma}$. 移位系统相应的近似解表示为

$$\boldsymbol{x}_{k+1}^{\sigma}=\boldsymbol{x}_k^{\sigma}+\omega_k^{\sigma}\boldsymbol{r}_k^{\sigma}. \tag{4.49}$$

接下来介绍在 Algorithm 3.24 中 IDR(s) 方法的主步骤里利用 $\boldsymbol{r}_{k+1}$ 推导 $\boldsymbol{r}_{k+1}^{\sigma}$ 的过程. 由 (3.110) 和 (3.111), IDR(s) 方法的 $\boldsymbol{v}_k$ 和 $\boldsymbol{r}_{k+1}$ 表示为

$$\boldsymbol{v}_k=\boldsymbol{r}_k-\sum_{l=1}^{s}\gamma_l\Delta\boldsymbol{r}_{k-l}=(1-\gamma_1)\boldsymbol{r}_k+\sum_{l=1}^{s-1}(\gamma_l-\gamma_{l+1})v\boldsymbol{r}_{k-l}+\gamma_s\boldsymbol{r}_{k-s}, \tag{4.50}$$

$$\boldsymbol{r}_{k+1}=(I-\omega_j A)\boldsymbol{v}_k. \tag{4.51}$$

由 (4.50) 和 (4.51), 得

$$\boldsymbol{r}_{k+1}=(I-\omega_j A)\left((1-\gamma_1)\boldsymbol{r}_k+\sum_{l=1}^{s-1}(\gamma_l-\gamma_{l+1})\boldsymbol{r}_{k-l}+\gamma_s\boldsymbol{r}_{k-s}\right). \tag{4.52}$$

将移位系统的残差向量定义为与 (4.52) 相同的形式如下:

$$\begin{aligned}\boldsymbol{r}_{k+1}^{\sigma} &= (I-\omega_j^{\sigma}(A+\sigma I))\left((1-\gamma_1^{\sigma})\boldsymbol{r}_k^{\sigma}+\sum_{l=1}^{s-1}(\gamma_l^{\sigma}-\gamma_{l+1}^{\sigma})\boldsymbol{r}_{k-l}^{\sigma}+\gamma_s^{\sigma}\boldsymbol{r}_{k-s}^{\sigma}\right)\\ &= \left(I-\frac{\omega_j^{\sigma}}{1-\sigma\omega_j^{\sigma}}A\right)(1-\sigma\omega_j^{\sigma})\left((1-\gamma_1^{\sigma})\boldsymbol{r}_k^{\sigma}+\sum_{l=1}^{s-1}(\gamma_l^{\sigma}-\gamma_{l+1}^{\sigma})\boldsymbol{r}_{k-l}^{\sigma}+\gamma_s^{\sigma}\boldsymbol{r}_{k-s}^{\sigma}\right),\end{aligned} \tag{4.53}$$

其中参数 $\omega_j^{\sigma},\gamma_1^{\sigma},\ldots,\gamma_s^{\sigma}\in\mathbb{C}$ 未知. 将 $\boldsymbol{r}_i=\pi_i^{\sigma}\boldsymbol{r}_i^{\sigma}$ $(i=0,1,\ldots,k+1)$ 代入 (4.52), 得

$$\boldsymbol{r}_{k+1}^{\sigma}=(I-\omega_j A)\left((1-\gamma_1)\frac{\pi_k^{\sigma}}{\pi_{k+1}^{\sigma}}\boldsymbol{r}_k^{\sigma}+\sum_{l=1}^{s-1}(\gamma_l-\gamma_{l+1})\frac{\pi_{k-l}^{\sigma}}{\pi_{k+1}^{\sigma}}\boldsymbol{r}_{k-l}^{\sigma}+\gamma_s\frac{\pi_{k-s}^{\sigma}}{\pi_{k+1}^{\sigma}}\boldsymbol{r}_{k-s}^{\sigma}\right). \tag{4.54}$$

由 (4.52), $\boldsymbol{r}_i=\pi_i^{\sigma}\boldsymbol{r}_i^{\sigma}$ $(i=0,1,\ldots,k+1)$ 和 $R_i(0)=1$, 得到 π_{k+1}^{σ} 如下:

$$\pi_{k+1}^{\sigma}=(1+\omega_j\sigma)\left((1-\gamma_1)\pi_k^{\sigma}+\sum_{l=1}^{s-1}(\gamma_l-\gamma_{l+1})\pi_{k-l}^{\sigma}+\gamma_s\pi_{k-s}^{\sigma}\right). \tag{4.55}$$

比较 (4.53) 和 (4.54) 的相应系数, 得

$$\begin{cases}\dfrac{\omega_j^{\sigma}}{1-\sigma\omega_j^{\sigma}}=\omega_j,\\ 1-\gamma_1^{\sigma}=\dfrac{1-\gamma_1}{1-\sigma\omega_j^{\sigma}}\times\dfrac{\pi_k^{\sigma}}{\pi_{k+1}^{\sigma}},\\ \gamma_l^{\sigma}-\gamma_{l+1}^{\sigma}=\dfrac{\gamma_l-\gamma_{l+1}}{1-\sigma\omega_j^{\sigma}}\times\dfrac{\pi_{k-l}^{\sigma}}{\pi_{k+1}^{\sigma}},\\ \gamma_s^{\sigma}=\dfrac{\gamma_s}{1-\sigma\omega_j^{\sigma}}\times\dfrac{\pi_{k-s}^{\sigma}}{\pi_{k+1}^{\sigma}}\end{cases} \tag{4.56}$$

其中 $l=1,2,\ldots,s-1$. 由 (4.56), 参数 $\omega_k^{\sigma},\gamma_1^{\sigma},\ldots,\gamma_s^{\sigma}$ 确定为

$$\begin{cases}\omega_j^{\sigma}=\dfrac{\omega_j}{1+\sigma\omega_j},\\ \gamma_1^{\sigma}=1-(1-\gamma_1)(1+\sigma\omega_j)\dfrac{\pi_k^{\sigma}}{\pi_{k+1}^{\sigma}},\\ \gamma_{l+1}^{\sigma}=\gamma_l^{\sigma}-(\gamma_l-\gamma_{l+1})(1+\sigma\omega_j)\dfrac{\pi_{k-l}^{\sigma}}{\pi_{k+1}^{\sigma}}\end{cases} \tag{4.57}$$

其中 $l=1,2,\ldots,s-1$. 利用关系 $\boldsymbol{r}_{k+1}^{\sigma}=\boldsymbol{b}-(A+\sigma I)\boldsymbol{x}_{k+1}^{\sigma}$, 可由 (4.53) 导出近似解 $\boldsymbol{x}_{k+1}^{\sigma}$ 为

$$\boldsymbol{x}_{k+1}^{\sigma}=\boldsymbol{x}_k^{\sigma}+\omega_j^{\sigma}\boldsymbol{v}_k^{\sigma}-\sum_{l=1}^{s}\gamma_l^{\sigma}\Delta\boldsymbol{x}_{k-l}^{\sigma}, \tag{4.58}$$

其中 $\boldsymbol{v}_k^{\sigma}=\boldsymbol{r}_k^{\sigma}-\sum_{l=1}^{s}\gamma_l^{\sigma}\Delta\boldsymbol{r}_{k-l}^{\sigma}$. 实际计算时, 可以用种子系统的残差向量 $\boldsymbol{r}_{k-s},\ldots,\boldsymbol{r}_k$ 表示 $\boldsymbol{v}_k^{\sigma}$. 上述所有步骤给出 Algorithm 4.11 中的移位 IDR(s) 方法.

Algorithm 4.11 移位 IDR(s) 方法 (种子系统: $(A+\sigma_f I)\boldsymbol{x}=\boldsymbol{b}$)

Input: $\boldsymbol{x}_0^{(i)}=\boldsymbol{0},\ \boldsymbol{r}_0=\boldsymbol{b},\ P\in\mathbb{C}^{n\times s},\ \pi_0^{(f,i)}=1,\ \sigma_i$ 对于 $i=1,\ldots,m$

Output: $\boldsymbol{x}^{(i)}$ 对于 $i=1,\ldots,m$

for $k=0,1,\ldots,s-1$ **do**
  $\boldsymbol{v}=(A+\sigma_f I)\boldsymbol{r}_k,\ \omega=\boldsymbol{v}^{\mathrm{H}}\boldsymbol{r}_k/\boldsymbol{v}^{\mathrm{H}}\boldsymbol{v}$
  $\Delta X(:,k+1)=\omega\boldsymbol{r}_k,\ \Delta R(:,k+1)=-\omega\boldsymbol{v}$
  $\boldsymbol{x}_{k+1}^{(f)}=\boldsymbol{x}_k^{(f)}+\Delta X(:,k+1),\ \boldsymbol{r}_{k+1}=\boldsymbol{r}_k+\Delta R(:,k+1)$
  (移位系统的迭代)
  for $i(\neq f)=1,2,\ldots,m$ **do**
    if $\|\boldsymbol{r}_k^{(i)}\|>\varepsilon\|\boldsymbol{b}\|$ **then**
      $\pi_{k+1}^{(f,i)}=\pi_k^{(f,i)}+\omega(\sigma_i-\sigma_f)\pi_k^{(f,i)},\quad \omega^{(f,i)}=\dfrac{\omega}{1+\omega(\sigma_i-\sigma_f)}$
      $\boldsymbol{x}_{k+1}^{(i)}=\boldsymbol{x}_k^{(i)}+\dfrac{\omega^{(f,i)}}{\pi_k^{(f,i)}}\boldsymbol{r}_k$
    end if
  end for
end for
$j=1,\ k=s,\ M=P^{\mathrm{H}}\Delta R,\ \boldsymbol{h}=P^{\mathrm{H}}\boldsymbol{r}_k$
while 尚未满足停止准则 **do**
  for $l=0,1,\ldots,s$ **do**
    从 $M\boldsymbol{c}=\boldsymbol{h}$ 解出 $\boldsymbol{c}$
    $\boldsymbol{q}=-\Delta R\boldsymbol{c},\ \boldsymbol{v}=\boldsymbol{r}_k+\boldsymbol{q}$
    if $l=0$ **then**
      $\boldsymbol{t}=(A+\sigma_f I)\boldsymbol{v},\ \omega=\boldsymbol{t}^{\mathrm{H}}\boldsymbol{v}/\boldsymbol{t}^{\mathrm{H}}\boldsymbol{t}$
      $\Delta R(:,j)=\boldsymbol{q}-\omega\boldsymbol{t},\ \Delta X(:,j)=-\Delta X\boldsymbol{c}+\omega\boldsymbol{v}$
    else
      $\Delta X(:,j)=-\Delta X\boldsymbol{c}+\omega\boldsymbol{v},\ \Delta R(:,j)=-(A+\sigma_f I)\Delta X(:,j)$
    end if
    $\boldsymbol{r}_{k+1}=\boldsymbol{r}_k+\Delta R(:,j),\ \boldsymbol{x}_{k+1}^{(f)}=\boldsymbol{x}_k^{(f)}+\Delta X(:,j)$
    $\delta\boldsymbol{m}=P^{\mathrm{H}}\Delta R(:,j),\ M(:,j)=\delta\boldsymbol{m},\ \boldsymbol{h}=\boldsymbol{h}+\delta\boldsymbol{m}$

Algorithm 4.11 移位 IDR(s) 方法 (续)

(移位系统的迭代)

$\gamma_1 = c_{j-1},\ \gamma_2 = c_{j-2}, \ldots, \gamma_{j-1} = c_1$

$\gamma_j = c_s,\ \gamma_{j+2} = c_{s-1}, \ldots, \gamma_s = c_j$

for $i(\neq f) = 1, 2, \ldots, m$ **do**

  if $\|\boldsymbol{r}_k^{(i)}\| > \varepsilon\|\boldsymbol{b}\|$ **then**

    $\alpha_i = 1 + \omega(\sigma_i - \sigma_f)$

    $$\pi_{k+1}^{(f,i)} = \alpha_i \left((1-\gamma_1)\pi_k^{(f,i)} + \sum_{g=1}^{s-1} (\gamma_g - \gamma_{g+1})\pi_{k-g}^{(f,i)} + \gamma_s \pi_{k-s}^{(f,i)} \right)$$

    $$\gamma_1^{(f,i)} = 1 - \alpha_i(1-\gamma_1)\pi_k^{(f,i)}/\pi_{k+1}^{(f,i)}$$

    $$\gamma_{g+1}^{(f,i)} = \gamma_g^{(f,i)} - (\gamma_g - \gamma_{g+1})\alpha_i \pi_{k-g}^{(f,i)}/\pi_{k+1}^{(f,i)} \quad (g = 1, \ldots, s-1)$$

    $$\boldsymbol{x}_{k+1}^{(i)} = \boldsymbol{x}_k^{(i)} + \omega\boldsymbol{v}/\pi_{k+1}^{(f,i)} - \sum_{g=1}^{s} \gamma_g^{(f,i)} (\boldsymbol{x}_{k+1-g}^{(i)} - \boldsymbol{x}_{k-g}^{(i)})$$

  end if

end for

$k = k + 1,\ j = j + 1$

$j = (j-1)\%s + 1$ (%: 取模运算, 即 $a\%n = r$, 其中 $a = mn + r$.)

end for

end while

5 在矩阵函数中的应用

正数的平方根在古代就已受到关注. 事实上, 制作于公元前 1800 年至公元前 1600 年之间的 Yale Babylonian Collection YBC 7289 泥板上就有 $\sqrt{2}$ 的一个近似值.

另一方面, 矩阵平方根等矩阵函数的概念相对较新: 矩阵平方根的概念由 Cayley 于 1858 年发现 [33], 矩阵函数的一个定义则由 Sylvester 于 1883 年给出 [180].

如今, 矩阵函数出现在粒子物理学, 量子信息和控制理论等许多科学领域, 因而许多研究者开发了高效的数值算法.

本章先介绍矩阵函数的定义, 然后介绍矩阵平方根, 矩阵 p 次根, 矩阵指数, 矩阵对数和矩阵三角函数等矩阵函数的数值算法. 我们将看到, Krylov 子空间方法或移位 Krylov 子空间方法可用于计算大型矩阵函数的特定元素. 关于矩阵函数最著名的书是 Higham 的著作 [97]. 以下关于矩阵函数的说明以 [36], [97], [154] 为基础, 且矩阵 A 的大小为 n-by-n.

5.1 Jordan 标准形

在矩阵函数的若干等价定义中, 我们采用利用 Jordan 标准形的定义. 本节介绍 Jordan 标准形.

Definition 5.1 (Jordan 块): 下面的 $m \times m$ 方阵称为 Jordan 块:

$$J_k = \begin{bmatrix} \lambda_k & 1 & & \\ & \lambda_k & \ddots & \\ & & \ddots & 1 \\ & & & \lambda_k \end{bmatrix} \in \mathbb{C}^{m\times m},$$

其中 $\lambda_k \in \mathbb{C}$. 由于 J_k 是上三角矩阵, J_k 的所有特征值都与对角元素相同, 即 λ_k.

Definition 5.2 (Jordan 矩阵): Jordan 块 $J_1 \in \mathbb{C}^{m_1\times m_1}$, $J_2 \in \mathbb{C}^{m_2\times m_2}$, ..., $J_p \in \mathbb{C}^{m_p\times m_p}$ 的直和为

$$J = \operatorname{diag}(J_1, J_2, \ldots, J_p) = \begin{bmatrix} J_1 & & & \\ & J_2 & & \\ & & \ddots & \\ & & & J_p \end{bmatrix} \in \mathbb{C}^{n\times n}. \tag{5.1}$$

该矩阵称为 Jordan 矩阵. 这里, $n = m_1 + m_2 + \cdots + m_p$, 符号 $\operatorname{diag}(J_1, J_2, \ldots, J_p)$ 表示以 $J_1, J_2, \ldots, J_p$ 为对角块的矩阵.

✍ Example 5.3:

设 $p = 3$, $m_1 = 2$, $m_2 = 1$, $m_3 = 3$, $\lambda_1 = 1$, $\lambda_2 = \lambda_3 = 2$. 则 $n = m_1 + m_2 + m_3 = 6$, 并得到下面的 6×6 Jordan 矩阵:

$$J = \operatorname{diag}(J_1, J_2, J_3) = \left[\begin{array}{cc|c|ccc} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 1 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 2 & 1 & 0 \\ 0 & 0 & 0 & 0 & 2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 2 \end{array}\right].$$

Theorem 5.4 (Jordan 标准形):

方阵 $A \in \mathbb{C}^{n\times n}$ 相似于一个 Jordan 矩阵, 即对任意方阵 A, 存在非奇异矩阵 Z 使得

$$Z^{-1}AZ = J.$$

形式 $A = ZJZ^{-1}$ 称为 A 的 Jordan 标准形.

✍ **Example 5.5:**

下面的矩阵相似于 Example 5.3 中的 Jordan 矩阵 J.

$$A = \begin{bmatrix} -1 & 1 & -1 & -2 & -2 & -1 \\ 1 & 2 & 2 & 1 & 2 & 1 \\ -3 & -3 & -4 & -3 & -5 & -3 \\ 4 & 0 & 3 & 5 & 4 & 2 \\ 4 & 4 & 7 & 4 & 8 & 4 \\ -3 & -2 & -5 & -3 & -5 & -1 \end{bmatrix}.$$

事实上, 可以看到 $Z^{-1}AZ = J$, 其中

$$Z = \begin{bmatrix} 1 & 0 & 0 & -1 & 0 & 0 \\ 0 & 1 & -1 & 0 & 1 & -1 \\ 0 & -1 & 2 & 0 & -2 & 1 \\ -1 & 0 & 0 & 2 & 0 & -1 \\ 0 & 1 & -2 & 0 & 3 & -2 \\ 0 & -1 & 1 & -1 & -2 & 4 \end{bmatrix}, \quad Z^{-1} = \begin{bmatrix} 3 & 1 & 1 & 2 & 1 & 1 \\ 1 & 3 & 2 & 1 & 1 & 1 \\ 1 & 2 & 3 & 1 & 2 & 1 \\ 2 & 1 & 1 & 2 & 1 & 1 \\ 1 & 1 & 2 & 1 & 2 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

在矩阵函数的几种定义中, 下一节采用利用 Theorem 5.4 中 Jordan 标准形的定义.

5.2 矩阵函数的定义与性质

矩阵 A 的全部特征值构成的集合称为 A 的谱. 矩阵的谱上的函数定义如下.

Definition 5.6 (矩阵的谱上的函数): 设 $\lambda_1, \lambda_2, \ldots, \lambda_s$ 是矩阵 A 的互异特征值, n_i 是对应于 A 的特征值 λ_i 的一个 Jordan 块的大小. 如果存在下列值:

$$f(\lambda_i), \frac{\mathrm{d}}{\mathrm{d}x}f(\lambda_i), \ldots, \frac{\mathrm{d}^{n_i-1}}{\mathrm{d}x^{n_i-1}}f(\lambda_i) \quad (i = 1, 2, \ldots, s),$$

则称 $f(x)$ 为矩阵 A 的谱上的函数.

以下用符号 $f^{(i)}(x)$ 代替 $\dfrac{\mathrm{d}^i}{\mathrm{d}x^i}f(x)$.

✍ **Example 5.7:**

设 A 为 Example 5.3 中的矩阵 J. 由于 $\lambda_1 = \lambda_2 = 1$ 且 $\lambda_3 = 2$, 互异特征值的个数为 $s = 2$. 对应于 $\lambda_1 = \lambda_2 = 1$ 和 $\lambda_3 = 2$ 的 Jordan 块的最大大小分别为 $n_1 = 2$ 和 $n_2 = 3$. 矩阵 A 的谱为 $\{\lambda_1, \lambda_3\}$. 现在给出矩阵 A 的谱上的函数的一个例子. 设 $f(x) = x^{-1}$. 则 $f(x)$ 是矩阵 A 的谱上的函数, 因为 $f^{(1)}(x) = -x^{-2}$, $f^{(2)}(x) = 2x^{-3}$, 且存在下列谱上的函数值: $f(\lambda_1) = 1$, $f^{(1)}(\lambda_1) = -1$, $f(\lambda_3) = 1/2$, $f^{(1)}(\lambda_3) = -1/4$ 及 $f^{(2)}(\lambda_3) = 1/4$.

利用 Theorem 5.4 中的 Jordan 标准形和 Definition 5.6 中矩阵的谱上的函数, 可以如下定义 A 的矩阵函数:

Definition 5.8 (矩阵函数): 设 $f(x)$ 是矩阵 $A \in \mathbb{C}^{n\times n}$ 的谱上的函数, $A = ZJZ^{-1}$ 是 A 的 Jordan 标准形. 则矩阵函数 $f(A) = f(ZJZ^{-1})$ 定义如下:

$$f(A) = Z\,\mathrm{diag}(f(J_1), f(J_2), \ldots, f(J_p))Z^{-1}.$$

这里 $f(J_k)$ 为下面的 $m_k \times m_k$ 矩阵:

$$f(J_k) = \begin{bmatrix} f(\lambda_k) & \frac{f^{(1)}(\lambda_k)}{1!} & \cdots & \frac{f^{(m_k-1)}(\lambda_k)}{(m_k-1)!} \\ & f(\lambda_k) & \ddots & \vdots \\ & & \ddots & \frac{f^{(1)}(\lambda_k)}{1!} \\ & & & f(\lambda_k) \end{bmatrix} \in \mathbb{C}^{m_k\times m_k},$$

且 $m_k \times m_k$ 是对应于特征值 λ_k 的 Jordan 块的大小.

由 Definition 5.8 可知, 当 λ_k 是矩阵 A 的一个特征值时, 矩阵函数 $f(A)$ 的特征值为 $f(\lambda_k)$.

特别地, 当矩阵 A 可对角化时, Jordan 块为对角矩阵 D_k, $k = 1, 2, \ldots, p$. 于是由 $f(A) = Z\,\mathrm{diag}(D_1, D_2, \ldots, D_p)Z^{-1}$ 可知, $f(A)$ 与 A 具有相同的特征向量.

✍ Example 5.9:

设 A 为 Example 5.3 中的矩阵 J, 并设 $f(x) = x^{-1}$. A 的 Jordan 标准形为 $A = ZJZ^{-1}$, 其中 Z 是单位矩阵. 则 $f(A) = f(ZJZ^{-1}) = f(J)$, 因而由 $\lambda_1 = \lambda_2 = 1$, $\lambda_3 = 2$ 及 Definition 5.8, 有

$$f(A) = \left[\begin{array}{cc|c|ccc} f(\lambda_1) & \frac{f^{(1)}(\lambda_1)}{1!} & 0 & 0 & 0 & 0 \\ 0 & f(\lambda_1) & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & f(\lambda_2) & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & f(\lambda_3) & \frac{f^{(1)}(\lambda_3)}{1!} & \frac{f^{(2)}(\lambda_3)}{2!} \\ 0 & 0 & 0 & 0 & f(\lambda_3) & \frac{f^{(1)}(\lambda_3)}{1!} \\ 0 & 0 & 0 & 0 & 0 & f(\lambda_3) \end{array}\right]$$

$$= \left[\begin{array}{cc|c|ccc} 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 1 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 2^{-1} & -4^{-1} & 8^{-1} \\ 0 & 0 & 0 & 0 & 2^{-1} & -4^{-1} \\ 0 & 0 & 0 & 0 & 0 & 2^{-1} \end{array}\right].$$

容易看出 $f(A)$ 是矩阵 A 的逆, 即从矩阵函数 $f(A)$ 的定义导出了 A^{-1}.

✍ **Example 5.10:**

我们考虑 A 的矩阵函数的其他例子. 设

$$A = \begin{bmatrix} -1 & 6 & -9 \\ 4 & -4 & 12 \\ 3 & -5 & 11 \end{bmatrix}.$$

则 Jordan 标准形为

$$A = ZJZ^{-1},$$

其中

$$Z^{-1}AZ = J = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

且

$$Z = \begin{bmatrix} 3 & 0 & -1 \\ 0 & 2 & 1 \\ -1 & 1 & 1 \end{bmatrix}, \quad Z^{-1} = \begin{bmatrix} 1 & -1 & 2 \\ -1 & 2 & -3 \\ 2 & -3 & 6 \end{bmatrix}.$$

利用这些矩阵, 对应于 $f(x) = x^{1/2}$ 的 $A^{1/2}$ 为

$$A^{1/2} = Z \begin{bmatrix} \sqrt{2} & \dfrac{1}{2\sqrt{2}} & -\dfrac{1}{16\sqrt{2}} \\ 0 & \sqrt{2} & \dfrac{1}{2\sqrt{2}} \\ 0 & 0 & \sqrt{2} \end{bmatrix} Z^{-1}.$$

容易看出 $A^{1/2}A^{1/2} = A$.

最后一个例子是 A 的矩阵指数函数. 对应于 $f(x) = e^x$ 的矩阵指数函数 e^A 为

$$e^A = Z \begin{bmatrix} e^2 & e^2 & \dfrac{e^2}{2} \\ 0 & e^2 & e^2 \\ 0 & 0 & e^2 \end{bmatrix} Z^{-1}.$$

注意, Jordan 标准形的计算在数值上是不稳定的. 因此, 这些例子并不用于实际计算.

下面列举矩阵函数的一些性质. 详情可参见例如 [97].

Theorem 5.11 (矩阵函数的性质):

设 $f(x)$ 是矩阵 A 的谱上的函数. 则:

(1) $Af(A) = f(A)A$.

(2) $XA = AX \Rightarrow Xf(A) = f(A)X$.

(3) $f(A^\top) = f(A)^\top$.

(4) $f(XAX^{-1}) = Xf(A)X^{-1}$.

(5) 如果 λ_k 是 A 的一个特征值, 则 $f(\lambda_k)$ 是 $f(A)$ 的一个特征值.

设 $f(x)$ 与 $g(x)$ 是矩阵 A 的谱上的函数. 则:

(6) 如果 $(f+g)(x) := f(x) + g(x)$, 则 $(f+g)(A) = f(A) + g(A)$.

(7) 如果 $(fg)(x) := f(x)g(x)$, 则 $(fg)(A) = f(A)g(A)$.

利用 Definition 5.6, 可得

(8) $f(A) = g(A)$ 当且仅当
$f(\lambda_i) = g(\lambda_i), \ldots, f^{(n_i-1)}(\lambda_i) = g^{(n_i-1)}(\lambda_i)$
$(i = 1, 2, \ldots, s)$.

由 Definition 5.8, 可以定义与初等函数 $x^{1/n}$, e^x, $\sin x$, $\cos x$, $\log x$ 对应的 $A^{1/n}$, e^A, $\sin A$, $\cos A$, $\log A$. 然而, 由于 Jordan 标准形的数值计算相当不稳定, 不建议通过 Jordan 标准形计算矩阵函数. 因此, 许多研究者设计了数值稳定的矩阵函数算法.

以下介绍计算矩阵函数的一些数值算法. 我们将看到, Krylov 子空间的概念有助于计算其中一些矩阵函数.

5.3 矩阵平方根与矩阵 p 次根

5.3.1 矩阵平方根

实数 $a > 0$ 的平方根为 $a^{1/2}$ 与 $-a^{1/2}$, 其主平方根为 $a^{1/2}$. 主平方根的概念如下推广到复数: 设 z 是复数, 对 $z = re^{i\theta}$ $(r > 0, -\pi < \theta < \pi)$, 定义 $z^{1/2} = r^{1/2}e^{i\theta/2}$. 这称为 z 的主平方根.

现在考虑矩阵平方根的情形, 并定义 A 的主平方根. 如果 $X^2 = A$, 则称 X 为 A 的一个平方根. 矩阵的平方根与复数或实数的平方根的区别在于, 前者可能有无穷多个. 例如, 设

$$S_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad S_2 = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}, \quad S(\theta) = \begin{bmatrix} \cos\theta & \sin\theta \\ \sin\theta & -\cos\theta \end{bmatrix}.$$

则对任意 θ, 都有 $S_1^2 = S_2^2 = S(\theta)^2 = I$. 这些都是单位矩阵的平方根. 因此, 这是一个具有无穷多个平方根的例子. 虽然平方根有无穷多个, 但特征值位于复平面右半平面的平方根是唯一的. 这个唯一的平方根称为矩阵的主平方根. 在上述例子中, S_1 是单位矩阵 I 的主平方根.

下一个定理给出矩阵主平方根的概念.

Theorem 5.12 (矩阵主平方根):
设 A 是复方阵, 其特征值 ($\lambda_k \neq 0$) 不位于复平面的负实轴上. 则 A 存在唯一一个平方根, 使得该平方根的所有特征值都位于复平面的右半平面.

在 Theorem 5.12 中, 这个唯一的矩阵平方根称为 A 的矩阵主平方根, 记为 $A^{1/2}$.

以下介绍计算矩阵 (主) 平方根的数值算法, 包括直接法和迭代法.

首先介绍一种著名的基于 Schur 分解的直接法. Schur 分解是将矩阵 A 分解为 QTQ^{H}, 其中 T 是上三角矩阵, Q 是酉矩阵. 如果能够计算 $U := T^{1/2}$, 则 $(QUQ^{\mathrm{H}})^2 = QU^2Q^{\mathrm{H}} = QTQ^{\mathrm{H}} = A$. 因此, 矩阵 (主) 平方根为 $A^{1/2} = QUQ^{\mathrm{H}}$. 由于 U 也是上三对角矩阵, 可以通过依次求解 $U^2 = T$ 得到 U. 所得算法见 Algorithm 5.1. 这里, 矩阵 T (及 U) 的 (i, j) 元素记为 t_{ij} (及 u_{ij}).

Algorithm 5.1 (直接法) 求 $X = A^{1/2}$ 的 Schur 方法

1: 计算 A 的 Schur 分解, 即 $A = QTQ^{\mathrm{H}}$.
2: **for** $i = 1, 2, \ldots, n$ **do**
3: $\quad u_{ii} = t_{ii}^{1/2}$
4: **end for**
5: **for** $j = 2, 3, \ldots, n$ **do**
6: $\quad$ **for** $i = j-1, j-2, \ldots, 1$ **do**
7: $\quad\quad u_{ij} = \dfrac{1}{u_{ii} + u_{jj}}(t_{ij} - \displaystyle\sum_{k=i+1}^{j-1} u_{ik}u_{kj})$
8: $\quad$ **end for**
9: **end for**
10: $X = QUQ^{\mathrm{H}}$

假设矩阵 A 满足 Theorem 5.12 中的条件. 由 Algorithm 5.1 的第 3 行可知, $u_{ii} = t_{ii}^{1/2}$ $(i = 1, 2, \ldots, n)$, 且 u_{ii} 是复数 t_{ii} 的主平方根. 根据对矩阵 A 的假设, 所有 u_{ii} 都位于复平面的右半平面. 因此, Algorithm 5.1 第 7 行的分母 $u_{ii} + u_{jj} \neq 0$. 所以 Algorithm 5.1 不会发生中断.

下面介绍一些迭代方法. 这些迭代方法可视为求解 $X^2 = A$ 的 Newton 方法 (Newton–Raphson 方法). 现在给出矩阵平方根的 Newton 方法的一种著名推导.

设 X_k 是 $A^{1/2}$ 的近似解, E 是相应的误差矩阵, 满足 $X_k + E = A^{1/2}$. 如果能够得到 E, 使得

$$(X_k + E)^2 = (A^{1/2})^2 \Leftrightarrow X_k^2 + EX_k + X_kE + E^2 = A,$$

则有 $X_k + E = A^{1/2}$. 但由于存在附加项 $EX_k + X_kE$, 求解原问题可能变得更加困难. 另一方面, 由假设 $X_k \approx A^{1/2}$, 可以预期 $E \approx O$ (零矩阵). 因此, 不考虑上面的校正方程, 而考虑从 $X_k^2 + E_kX_k + X_kE_k = A$ 求出 E_k, 这是 (2.59) 中的 Sylvester 方程. 于是得到如下 Newton 方法:

1. 设置初始猜测 X_0,
2. 对 $k = 1, 2, \ldots$, 直到收敛,

3. 求解 $E_kX_k + X_kE_k = A - X_k^2$ 以得到 E_k,

4. $X_{k+1} = X_k + E_k$.

5. 结束

如果选择初始猜测 X_0 使得 $AX_0 = X_0A$, 则 Sylvester 方程得到简化. 在这一选择下, 可以证明 $E_kX_k = X_kE_k$. 于是由
$E_k = (1/2)X_k^{-1}(A - X_k^2) = (1/2)(X_k^{-1}A - X_k)$,
可得 $X_k + E_k = (1/2)(X_k + X_k^{-1}A)$. 现在在 Algorithm 5.2 中给出矩阵平方根的 Newton 方法.

上述推导的细节以及基于 Fréchet 导数的推导见 [96].

Algorithm 5.2 (迭代法) 求 $X = A^{1/2}$ 的 Newton 方法

1: 选择 X_0 使得 $AX_0 = X_0A$, 例如 $X_0 = I$.
2: **for** $k = 0, 1, \ldots,$ 直到收敛 **do**
3: $\quad X_{k+1} = \dfrac{1}{2}(X_k + X_k^{-1}A)$
4: **end for**

设 A 是满足 Theorem 5.12 中条件的矩阵. 如果 $A^{1/2}X_0$ 的所有特征值都位于复平面的右半平面, 则 X_k 二次收敛到矩阵平方根 $A^{1/2}$ [97, p.140].

在 Algorithm 5.2 中, 初始猜测的选择受到限制, 即 $AX_0 = X_0A$. 通常选择 $X_0 = A$ 或 $X_0 = I$. 由于矩阵多项式 $c_0I + c_1A + c_2A^2 + \cdots + c_nA^n$ 与 A 可交换, 自然可以选择
$X_0(= c_0I + c_1A + c_2A^2 + \cdots + c_nA^n)$,
使得残差的范数 $\|X_0^2 - A\|$ 最小. 文献 [132] 提出了利用 Gröbner 基和低次矩阵多项式进行近似最小化的方法.

Algorithm 5.2 的一种数值稳定变体是增量 Newton (IN) 迭代 [101], 见 Algorithm 5.3.

Algorithm 5.3 (迭代法) 求 $X = A^{1/2}$ 的 IN 方法

1: $X_0 = A$; $E_0 = \dfrac{1}{2}(I - A)$
2: **for** $k = 0, 1, \ldots,$ 直到收敛 **do**
3: $\quad X_{k+1} = X_k + E_k$
4: $\quad E_{k+1} = -\dfrac{1}{2}E_kX_{k+1}^{-1}E_k$
5: **end for**

Algorithm 5.3 的一个特点是矩阵 A 不出现在主迭代, 即第 3 行中. 虽然 Algorithm 5.3 比 Algorithm 5.2 在数值上更稳定, 但它需要额外的矩阵与矩阵乘法. 每个迭代步中, Algorithm 5.3 的计算成本约为 Algorithm 5.2 的 7/4 倍.

5.3.2 矩阵 p 次根

本节介绍求解矩阵 p 次根的直接法和迭代法.

满足 $X^p = A$ 的矩阵 X 称为矩阵 p 次根. 如 Section 5.3.1 所述, 矩阵平方根可能有无穷多个. 其中有由 Theorem 5.12 刻画的矩阵主平方根. 类似地, 矩阵主 p 次根的概念如下给出:

Theorem 5.13 (矩阵主 p 次根)**:**
设 A 是复方阵, 其特征值 ($\lambda_k \neq 0$) 不位于复平面的负实轴上. 则 A 存在唯一一个 p 次根, 使得该 p 次根的所有特征值都位于扇形区域 $\{z \in \mathbb{C} : -\pi/p < \arg(z) < \pi/p\}$.

这个唯一的矩阵 p 次根称为 A 的矩阵主 p 次根, 记为 $A^{1/p}$. 如果 $p = 2$, 则 Theorem 5.13 与 Theorem 5.12 等价.

以下介绍求 $A^{1/p}$ 的直接法. Schur 方法是一种基于 Schur 分解的直接法. 对矩阵 A 进行 Schur 分解 ($A = QTQ^{\mathrm{H}}$) 后, 有 $A^{1/p} = QT^{1/p}Q^{\mathrm{H}}$. 为求得 $T^{1/p}$, 设 $U := T^{1/p}$. 由于 U 是上三对角矩阵, U^p 也是上三对角矩阵. 于是只需求解 $U^p = T$. 所得算法汇总于 Algorithm 5.4.

Algorithm 5.4 (直接法) 求 $X = A^{1/p}$ 的 Schur 方法

计算 Schur 分解 $A = QTQ^{\mathrm{H}}$.
for $j = 1, 2, \ldots, n$ **do**
  $u_{jj} = t_{jj}^{1/p}$, $v_{jj}^{(1)} = 1$, $v_{jj}^{(k+2)} = u_{jj}^{k+1}$ $(k = 0, 1, \ldots, p-2)$
  for $i = j-1, j-2, \ldots, 1$ **do**
    for $k = 0, 1, \ldots, p-2$ **do**
      $$w_{k+2} = \sum_{\ell=i+1}^{j-1} u_{i\ell} v_{\ell j}^{(k+2)}$$
    end for
    $$u_{ij} = (t_{ij} - \sum_{k=0}^{p-2} v_{ii}^{(p-k-1)} w_{k+2}) / (\sum_{k=0}^{p-1} v_{ii}^{(p-k)} v_{jj}^{(k+1)})$$
    for $k = 0, 1, \ldots, p-2$ **do**
      $$v_{ij}^{(k+2)} = \sum_{\ell=0}^{k} v_{ii}^{(k-\ell+1)} u_{ij} v_{jj}^{(\ell+1)} + \sum_{\ell=0}^{k-1} v_{ii}^{(k-\ell)} w_{\ell+2}$$
    end for
  end for
end for
$X = QUQ^{\mathrm{H}}$

至于迭代方法, Algorithms 5.5 和 5.6 分别是 Algorithms 5.2 和 5.3 的推广.

Algorithm 5.5 (迭代法) 求 $A^{1/p}$ 的 Newton 方法

设置 p, 并选择 X_0 使得 $AX_0 = X_0A$, 例如 $X_0 = A$.
for $k = 0, 1, \ldots,$ 直到收敛 **do**
  $$X_{k+1} = \frac{1}{p}[(p-1)X_k + X_k^{1-p}A]$$
end for

Algorithm 5.6 的计算成本高于 Algorithm 5.5. 另一方面, 文献 [102] 表明 Algorithm 5.6 具有数值稳定性. Newton 方法的其他有用变体可参见例如 [102, Eqs. (3.6), (3.9)]. 文献 [187] 给出了 Algorithm 5.6 的一种具有数值稳定性且计算成本较低的变体.

Algorithm 5.6 (迭代法) 求 $A^{1/p}$ 的增量 Newton 方法

1: 设置 p, $X_0 = I$, $E_0 = \dfrac{1}{p}(A - I)$.
2: **for** $k = 0, 1, \ldots,$ 直到收敛 **do**
3: $\quad X_{k+1} = X_k + E_k$, $F_k = X_k X_{k+1}^{-1}$
4: $\quad E_{k+1} = -\dfrac{1}{p} E_k \left[X_{k+1}^{-1} I + 2X_{k+1}^{-1} F_k + \cdots + (p-1) X_{k+1}^{-1} F_k^{p-2} \right] E_k$
5: **end for**

5.4 矩阵指数函数

如 Definition 5.8 所示, 矩阵指数函数通过 Jordan 标准形定义. 另一方面, 下面的等价定义有助于计算该矩阵函数:

$$e^A := I + A + \frac{1}{2!}A^2 + \frac{1}{3!}A^3 + \cdots . \tag{5.2}$$

该级数对任意方阵 $A \in \mathbb{C}^{n\times n}$ 都收敛. 本节介绍矩阵指数函数的数值算法.

矩阵函数的基本性质汇总于 Theorem 5.14. 这些性质容易由 (5.2) 中的定义证明.

Theorem 5.14 (矩阵指数函数的性质):
对 $A, B \in \mathbb{C}^{n\times n}$:

(1) $e^O = I$, 其中 O 为零矩阵;

(2) 如果 $AB = BA$, 则 $e^A e^B = e^{A+B}$;

(3) $e^A e^{-A} = I$, $(e^A)^{-1} = e^{-A}$;

(4) $e^{XAX^{-1}} = X e^A X^{-1}$, 其中 X 为非奇异矩阵;

(5) $e^{A^{\mathrm{H}}} = (e^A)^{\mathrm{H}}$.

由 Theorem 5.14 的第 (5) 项, 如果 A 是 Hermitian 矩阵, 则矩阵指数函数 e^A 也是 Hermitian 矩阵. 由 Theorem 5.14 的第 (3) 和 (5) 项, 如果 A 是斜 Hermitian 矩阵 ($A^{\mathrm{H}} = -A$), 则矩阵指数函数 e^A 是酉矩阵, 即 $(e^A)^{\mathrm{H}} e^A = e^{-A} e^A = I$.

在 Theorem 5.14 的第 (2) 项中, 对标量 a 与 b 取 $A = aC$ 及 $B = bC$, 得到 $e^{aC} e^{bC} = e^{(a+b)C}$. 这里, 如果 $a = 1$ 且 $b = -1$, 则得到 $e^C e^{-C} = e^{0C} = e^O = I$, 这对应于 Theorem 5.14 的第 (3) 项.

5.4.1 矩阵指数函数的数值算法

近似计算矩阵指数函数的最简单方法之一是截断 (5.2) 中的级数, 即
$I + A + \dfrac{1}{2!}A^2 + \dfrac{1}{3!}A^3 + \cdots + \dfrac{1}{m!}A^m$.
如果矩阵 A 的所有特征值都接近于零, 则在 n 较小时截断误差就可能较小. 然而, 对于 A 的特征值不具有这种分布的矩阵 A, 上述方法效率不高.

计算矩阵指数函数的一种更好且简单的方法是考虑 $e^{A/s}$ 而不是 e^A. 可以看到, 对 $s \geq 1$, 有 $(e^{A/s})^s = e^A$, 且 A/s 的所有特征值都是 A 的特征值的 s 分之一. 于是截断下列级数:

$$e^{A/s} = I + \frac{1}{1!s}A + \frac{1}{2!s^2}A^2 + \frac{1}{3!s^3}A^3 + \cdots \tag{5.3}$$

得到

$$F_{r,s} = I + \frac{1}{1!s}A + \frac{1}{2!s^2}A^2 + \frac{1}{3!s^3}A^3 + \cdots + \frac{1}{r!s^r}A^r, \tag{5.4}$$

它给出近似 $(F_{r,s})^s \approx e^A$. 此时有下列性质:[1]

Theorem 5.15 ([179]):

设 $A \in \mathbb{C}^{n\times n}$, $F_{r,s}$ 是 (5.4) 中给出的截断. 则

$$\|e^A - (F_{r,s})^s\| \le \frac{\|A\|^{r+1}e^{\|A\|}}{s^r(r+1)!}, \tag{5.5}$$

其中 $\|\cdot\|$ 是任意具有次乘性的矩阵范数.[2]

根据 Theorem 5.15 的右端, 可以通过计算 $\|A\|$ 或其上界来估计截断误差. 例如, 考虑 $s = 2^m$ 的情形. 假设 $\|A\| = 10$. 那么, 若取 $r = 8$, $m = 9$, $s = 2^m$, 则 (5.5) 的右端约为 1.29×10^{-14}. 这意味着, 如果估计出 $\|A\|$, 就可以确定 s 与 r, 使截断误差小于给定容差. 当 $\|A\|$ 是 Frobenius 范数时, 容易根据 Frobenius 范数的定义计算该范数. 当 $\|A\|$ 是矩阵 2-范数时, 需要 A 的最大奇异值, 它也容易通过 Golub–Kahan 双对角化过程 (Algorithm 3.29) 估计.

一种计算成本更低的方法如下: 不计算 (5.4), 而采用 $e^{A/s}$ 的 Padé 逼近, 并计算该逼近的 s 次幂. 矩阵指数函数 e^A 的 Padé 逼近 $R_{p,q}(A)$ 如下:

$$R_{p,q}(A) = [D_{p,q}(A)]^{-1}N_{p,q}(A), \tag{5.6}$$

其中

$$N_{p,q}(A) = \sum_{j=0}^{p} n_j A^j, \quad n_j = \frac{(p+q-j)!}{(p+q)!} \times \frac{p!}{j!(p-j)!},$$

$$D_{p,q}(A) = \sum_{j=0}^{q} d_j (-A)^j, \quad d_j = \frac{(p+q-j)!}{(p+q)!} \times \frac{q!}{j!(q-j)!}.$$

近似矩阵指数时, 建议采用对角 Padé 逼近, 即 $p = q$. 由
$e^A = [e^{A/2^m}]^{2^m} \approx [R_{q,q}(A/2^m)]^{2^m}$,
需要选择两个参数 q 与 m. 如果 $\|A\|/2^m \le 1/2$, 则

$$[R_{q,q}(A/2^m)]^{2^m} = e^{A+E}, \quad \frac{\|E\|}{\|A\|} \le 8\left[\frac{\|A\|}{2^m}\right]^{2q} \frac{(q!)^2}{(2q)!(2q+1)!}.$$

详情见 [133, p.12]. 该不等式给出最优参数 q 与 m, 使得在对给定容差 ϵ 截断误差满足 $\|E\|/\|A\| \le \epsilon$ 的条件下, $q+m$ 最小. 对容差 $\epsilon = 10^{-15}$, 随 $\|A\|$ 变化的最优参数列于 Table 5.1. 相关研究见 [8], [98].

如果计算出 Schur 分解 $A = QTQ^{\mathrm{H}}$, 则由 (5.2) 可得 $e^A = e^{QTQ^{\mathrm{H}}} = Qe^TQ^{\mathrm{H}}$. 因此, 只需利用 (对角) Padé 逼近计算 e^T.

特别地, 当 A 是 Hermitian 矩阵时, T 成为对角矩阵. 设 $\lambda_1, \lambda_2, \ldots, \lambda_n$ 是 T 的对角元素.

[1] 在 [179] 中, Theorem 5.15 是在 Banach 代数上陈述的, 是一个更一般的结果.

[2] 术语 "次乘性" 见 Section 1.1.2 的 (Nm4).

则矩阵指数函数可由下式计算:

$$e^A = Q \begin{bmatrix} e^{\lambda_1} & & \\ & \ddots & \\ & & e^{\lambda_n} \end{bmatrix} Q^{\mathrm{H}}. \tag{5.7}$$

进一步, 如果 A 可对角化, 即对非奇异矩阵 X 和对角矩阵 D 有 $A = XDX^{-1}$, 则 $e^A = Xe^DX^{-1}$, 其中 e^D 与 (5.7) 中的形式相同. 另一方面, 如果 X 的条件数很大, 这种方法在数值上可能不稳定.

表 5.1: 满足误差容差 $\epsilon = 10^{-15}$ 的 $R_{q,q}(A/2^m)$ 中的最优参数 q 与 m.

$\|A\|$	10^{-3}	10^{-2}	10^{-1}	1	10	10^2	10^3	10^4	10^5	10^6
q	3	3	4	6	6	6	6	6	6	6
m	0	0	0	1	5	8	11	15	18	21

5.4.2 矩阵指数函数与向量的乘积

考虑如下线性常微分方程:

$$\frac{\mathrm{d}\boldsymbol{x}(t)}{\mathrm{d}t} = A\boldsymbol{x}(t) + \boldsymbol{f}(t), \quad \boldsymbol{x}(0) = \boldsymbol{x}_0, \tag{5.8}$$

其中 $A \in \mathbb{C}^{n\times n}$, $\boldsymbol{x}, \boldsymbol{f} \in \mathbb{C}^n$, $t \in \mathbb{R}$. 利用矩阵指数函数可如下给出其解:

$$\boldsymbol{x}(t) = e^{tA}\boldsymbol{x}_0 + \int_0^t e^{(t-\tau)A}\boldsymbol{f}(\tau)\,\mathrm{d}\tau. \tag{5.9}$$

可以看到, (5.9) 中出现了矩阵指数函数与向量的乘积. 特别地, 如果 $\boldsymbol{f}(t) = \boldsymbol{0}$, 则解可写为 $\boldsymbol{x}(t) = e^{tA}\boldsymbol{x}_0$. 解的这一显式形式意味着, 如果得到这些乘积, 就知道任意时刻 t 的解.

如果已经计算出矩阵指数函数, 则只需计算矩阵与向量的乘积, 其计算成本为 n^2 阶, 其中 n 是一个大小为 $n \times n$ 的矩阵. 另一方面, 即使矩阵 A 是稀疏的, 计算矩阵指数函数的成本也可能为 n^3 阶. 因此, 在求解 (5.8) 时计算矩阵指数函数并不高效. 注意, 该解只需要矩阵指数函数与向量相乘的结果. 在这种情况下, 利用 Krylov 子空间近似该乘积是有用的, 下面予以介绍.

由于矩阵指数函数 e^{tA} 由级数 (5.2) 定义, 自然可以利用 m 次矩阵多项式 $p_m(A)$ 近似 e^{tA}. 矩阵多项式 $p_m(A)$ 与向量相乘的结果属于 Krylov 子空间 $\mathcal{K}_m(A, \boldsymbol{x}_0)$. 因此, 最优近似由求解下列最小二乘问题给出:

$$\min_{\boldsymbol{x}_m \in \mathcal{K}_m(A,\boldsymbol{x}_0)} \|e^{tA}\boldsymbol{x}_0 - \boldsymbol{x}_m\|. \tag{5.10}$$

设 V_m 是以 $\mathcal{K}_m(A, \boldsymbol{x}_0)$ 的标准正交化基向量为列的矩阵. 则任意向量 $\boldsymbol{x} \in \mathcal{K}_m(A, \boldsymbol{x}_0)$ 均可写为 $\boldsymbol{x} = V_m\boldsymbol{y}$, 其中 $\boldsymbol{y} \in \mathbb{C}^m$. 因此 (5.10) 等价于

$$\min_{\boldsymbol{y}\in\mathbb{C}^m} \|e^{tA}\boldsymbol{x}_0 - V_m\boldsymbol{y}\|.$$

其解为 $\boldsymbol{y} = V_m^{\mathrm{H}}e^{tA}\boldsymbol{x}_0$, 因此 (5.10) 的解为

$$(\boldsymbol{x}(t) \approx)\ \boldsymbol{x}_m = V_mV_m^{\mathrm{H}}e^{tA}\boldsymbol{x}_0. \tag{5.11}$$

以下考虑利用 Section 1.9.1 中的 Arnoldi 过程计算 $\mathcal{K}_m(A, \boldsymbol{x}_0)$ 的基向量, 并给出 (5.11) 的近似解. 根据 Section 1.9.1 中的 Arnoldi 过程, 由 $\boldsymbol{x}_0 = \|\boldsymbol{x}_0\| V_m \boldsymbol{e}_1$ 可得

$\boldsymbol{x}_m = \|\boldsymbol{x}_0\| V_m (V_m^{\mathrm{H}} e^{tA} V_m) \boldsymbol{e}_1$,

其中 $\boldsymbol{e}_1 = [1, 0, \ldots, 0]^\top$. 进一步, 利用 (1.39) 中的关系 $V_m^{\mathrm{H}} A V_m = H_m$ (Hessenberg 矩阵), 考虑近似

$V_m^{\mathrm{H}} e^{tA} V_m \approx e^{t V_m^{\mathrm{H}} A V_m} = e^{tH_m}$.

于是有

$$\boldsymbol{x}_m = \|\boldsymbol{x}_0\| V_m (V_m^{\mathrm{H}} e^{tA} V_m) \boldsymbol{e}_1 \approx \|\boldsymbol{x}_0\| V_m e^{tH_m} \boldsymbol{e}_1. \tag{5.12}$$

表达式 $e^{tH_m} \boldsymbol{e}_1$ 是一个 $m \times m$ 小型矩阵指数函数与向量的乘积, 其计算成本为 $O(m^2)$. 这是因为 (e^{tH_m} 的) Padé 逼近与向量相乘需要计算 $H_m^{-1} \boldsymbol{v}$, 即求解线性方程组 $H_m \boldsymbol{z} = \boldsymbol{v}$, 其计算成本为 $O(m^2)$.

如果矩阵 A 是 Hermitian 矩阵, 则可利用 Theorem 5.16 选择适当的 m, 以满足给定的误差容差.

Theorem 5.16 ([99], Theorem 2)**:**

设 A 是 Hermitian 矩阵, 其所有特征值都位于区间 $[-4\rho, 0]$ 中. 则 (5.12) 的误差为

$$\|e^{tA}\boldsymbol{x}_0 - \|\boldsymbol{x}_0\|_2 V_m e^{tH_m} \boldsymbol{e}_1\| \le \begin{cases} c \cdot e^{-\frac{m^2}{5\rho t}} & (\sqrt{4\rho t} \le m \le 2\rho t), \\ c \cdot \dfrac{1}{\rho t e^{\rho t}} \cdot \left(\dfrac{e\rho t}{m}\right)^m & (2\rho t \le m), \end{cases}$$

其中 $c = 10\|\boldsymbol{x}_0\|$.

实际计算中, 先估计 A 的最小特征值 $\lambda_{\min}$, 并设置 $\rho = -\lambda_{\min}/4$. 然后选择 Krylov 子空间的维数 m, 使得 Theorem 5.16 中的误差上界小于给定容差, 例如 10^{-12}. 文献 [139] 展示了该误差界的有效性, 并设计了避免 Krylov 子空间基向量丧失正交性的实用计算方法和高效的并行计算方法.

5.5 矩阵三角函数

这里考虑矩阵三角函数: 矩阵正弦函数 $\sin(A)$ 和矩阵余弦函数 $\cos(A)$. $\sin(x)$ 与 $\cos(x)$ 可展开为

$$\sin(x) = x - \frac{1}{3!}x^3 + \frac{1}{5!}x^5 - \frac{1}{7!}x^7 + \cdots,$$
$$\cos(x) = 1 - \frac{1}{2!}x^2 + \frac{1}{4!}x^4 - \frac{1}{6!}x^6 + \cdots.$$

利用上述展开, 对任意 $A \in \mathbb{C}^{n\times n}$, 如下定义 $\sin(A)$ 与 $\cos(A)$:

$$\sin(A) = A - \frac{1}{3!}A^3 + \frac{1}{5!}A^5 - \frac{1}{7!}A^7 + \cdots,$$
$$\cos(A) = I - \frac{1}{2!}A^2 + \frac{1}{4!}A^4 - \frac{1}{6!}A^6 + \cdots.$$

下面给出 $\sin(A)$ 与 $\cos(A)$ 的基本性质.

Theorem 5.17 ($\sin(A)$ 与 $\cos(A)$ 的一些性质)**:**
对 $A, B \in \mathbb{C}^{n\times n}$, 下列事实成立:

(1) $e^{iA} = \cos(A) + i\sin(A)$;

(2) $\sin(A) = (e^{iA} - e^{-iA})/2i$;

(3) $\cos(A) = (e^{iA} + e^{-iA})/2$;

(4) $\sin(-A) = -\sin(A)$, $\cos(-A) = \cos(A)$;

(5) $\sin(A)^2 + \cos(A)^2 = I$;

(6) 如果 $AB = BA$, 则
$\sin(A \pm B) = \sin(A)\cos(B) \pm \cos(A)\sin(B)$;

(7) 如果 $AB = BA$, 则
$\cos(A \pm B) = \cos(A)\cos(B) \mp \sin(A)\sin(B)$.

Theorem 5.17 的第 (2) 与 (3) 项可由其第 (1) 项导出. 由第 (2) 和 (3) 项, 可通过矩阵指数函数得到 $\sin(A)$ 与 $\cos(A)$. 如果 A 是实矩阵, 则由 Theorem 5.17 的第 (1) 项可知, $\cos(A)$ 对应于 e^{iA} 的实部, $\sin(A)$ 对应于 e^{iA} 的虚部. 由此得到计算 $\sin(A)$ 与 $\cos(A)$ 的 Algorithm 5.7, 如 Algorithm 5.7 所示.

Algorithm 5.7 计算 $S = \sin(A)$ 和 $C = \cos(A)$

1: 计算 $X = e^{iA}$.
2: **if** A 是实矩阵 **then**
3: $S = \mathrm{Im}(X)$ (A 的虚部), $C = \mathrm{Re}(X)$ (A 的实部)
4: **end if**
5: **if** A 是复矩阵 **then**
6: $S = \dfrac{1}{2i}(X - X^{-1})$, $C = \dfrac{1}{2}(X + X^{-1})$
7: **end if**

5.6 矩阵对数

对 $A \in \mathbb{C}^{n\times n}$, 满足 $e^X = A$ 的矩阵 X 称为矩阵对数. 回顾 Theorem 5.12 中介绍的矩阵主平方根概念. 类似地, 矩阵主对数的概念见 Theorem 5.18.

Theorem 5.18 (矩阵主对数):
设 A 是复方阵, 其特征值 $(\lambda_k \neq 0)$ 不位于复平面的负实轴上. 则 A 存在唯一一个矩阵对数, 使得该平方根的所有特征值都位于带状区域 $\{z \in \mathbb{C} : -\pi < \mathrm{Im}(z) < \pi\}$.

Theorem 5.18 中的唯一矩阵称为 A 的矩阵主对数, 记为 $\log(A)$. 下面给出 $\log(A)$ 的一些性质.

Theorem 5.19 ($\log(A)$ 的一些性质):
设 A 是复方阵, 其特征值 $(\lambda_k \neq 0)$ 不位于复平面的负实轴上. 则对 $-1 \leq \alpha \leq 1$, 有 $\log(A^\alpha) = \alpha \log(A)$. 特别地, $\log(A^{-1}) = -\log(A)$, $\log(A^{1/2}) = (1/2)\log(A)$.

Theorem 5.19 的证明见 [97, Theorem 11.2] 的证明.

以下介绍计算 $\log(A)$ 的一些数值算法. 当矩阵 A 满足 $\rho(A-I) < 1$ 时, $\log(A)$ 可以如下展开:[3]

$$\begin{aligned} \log(A) &= \log(I + (A-I)) \\ &= (A-I) - \frac{1}{2}(A-I)^2 + \frac{1}{3}(A-I)^3 - \frac{1}{4}(A-I)^4 + \cdots, \end{aligned} \tag{5.13}$$

这就是 Neumann 级数, 另见 Section 3.5.3. 如果 A 不满足 $\rho(A-I) < 1$, 则不能利用 (5.13) 近似计算 $\log(A)$. 即使在这种情况下, 作下列修改后该展开仍可使用: 由 Theorem 5.19 可知, 对自然数 k, 有

$$\log(A) = k \log(A^{1/k}),$$

而且随着 k 增大, $A^{1/k}$ 更接近单位矩阵 I. 因此, 只需根据 (5.13) 找到 k, 使下列级数收敛:

$$\begin{aligned} \log(A^{1/k}) &= \log(I + (A^{1/k} - I)) \\ &= (A^{1/k} - I) - \frac{1}{2}(A^{1/k} - I)^2 + \frac{1}{3}(A^{1/k} - I)^3 + \cdots. \end{aligned} \tag{5.14}$$

实际计算中, 设置 $k = 2^m$, 并在 (5.14) 的右端采用 $\log(1+x)$ 的 Padé 逼近. 下面是 $\log(I+X)$ 的对角 Padé 逼近的一些例子:

$$\begin{aligned} R_{1,1}(X) &= (2I + X)^{-1}(2X), \\ R_{2,2}(X) &= (6I + 6X + X^2)^{-1}(6X + 3X^2), \\ R_{3,3}(X) &= (60I + 90X + 36X^2 + 3X^3)^{-1}(60X + 60X^2 + 11X^3). \end{aligned}$$

算法列于 Algorithm 5.8. 矩阵 2^m 次根见 Section 5.3.2. Algorithm 5.8 的改进见 [9].

Algorithm 5.8 计算 $\log(A)$

1: 设置自然数 m, 使得 $\rho(A^{1/2^m} - I) < 1$.
2: $X = A^{1/2^m} - I$
3: $Y = R_{q,q}(X)$ (对角 Padé 逼近)
4: $\log(A) \approx 2^m Y$

[3] $\rho(X) = |\lambda_{\max}|$ 是 X 的谱半径, 见 Section 1.6.4.

基于 Newton 方法的数值算法列于 Algorithms 5.9 和 5.10. 在一定条件下, Algorithm 5.9 局部二次收敛到 $\log(A)$, 而 Algorithm 5.10 局部三次收敛到 $\log(A)$.

Algorithm 5.9 求 $X = \log(A)$ 的迭代方法 1

1: 选择 X_0 使得 $AX_0 = X_0A$, 例如 $X_0 = A$.
2: **for** $k = 0, 1, \ldots,$ 直到收敛 **do**
3: $\quad X_{k+1} = X_k - I + e^{-X_k}A$
4: **end for**

Algorithm 5.10 求 $X = \log(A)$ 的迭代方法 2

1: 选择 X_0 使得 $AX_0 = X_0A$, 例如 $X_0 = A$.
2: **for** $k = 0, 1, \ldots,$ 直到收敛 **do**
3: $\quad X_{k+1} = X_k + \frac{1}{2}(e^{-X_k}A - A^{-1}e^{X_k})$
4: **end for**

在 Algorithm 5.10 中, 设 $Y = e^{-X_k}A$. 由 Theorem 5.14 的第 (3) 项, 有 $Y^{-1} = A^{-1}e^{X_k}$, 从而得到 $X_{k+1} = X_k + (Y - Y^{-1})/2$. 这意味着 Algorithm 5.10 的计算成本约等于 Algorithm 5.9 的计算成本与 Y^{-1} 的计算成本之和. 由于 Y^{-1} 的计算成本相对远小于矩阵指数函数的计算成本, Algorithm 5.9 的成本与 Algorithm 5.10 的成本近乎相等. 因此, 由于 Algorithm 5.10 具有三次收敛性, 它会比 Algorithm 5.9 更快.

当矩阵 A 是 Hermitian 正定矩阵时, A 的 Schur 分解对应于特征分解 $A = QDQ^{\mathrm{H}}$, 其中 Q 是酉矩阵, D 是以 A 的特征值 λ_i 为对角元素的对角矩阵, 因而
$e^X = A \Leftrightarrow e^X = QDQ^{\mathrm{H}} \Leftrightarrow Q^{\mathrm{H}}e^XQ = D \Leftrightarrow e^{Q^{\mathrm{H}}XQ} = D.$
设 $M = Q^{\mathrm{H}}XQ$. 则满足 $e^M = D$ 的 M 可写为
$M = \mathrm{diag}(\log(\lambda_1), \ldots, \log(\lambda_n))$.
由 M 的定义, 有 $X(= \log(A)) = QMQ^{\mathrm{H}}$, 从而得到如下计算 $\log(A)$ 的方式:

$$\log(A) = Q \begin{bmatrix} \log(\lambda_1) & & \\ & \ddots & \\ & & \log(\lambda_n) \end{bmatrix} Q^{\mathrm{H}}.$$

一般地, 如果矩阵 A 满足 Theorem 5.18 的假设且可对角化 (即 $A = VDV^{-1}$), 则可以尝试由下式计算 $\log(A)$:

$$\log(A) = V \begin{bmatrix} \log(\lambda_1) & & \\ & \ddots & \\ & & \log(\lambda_n) \end{bmatrix} V^{-1}.$$

然而, 如果 V 是病态的, 则由于计算 V^{-1} 的数值不稳定性, 不推荐使用这种方法.

如果矩阵 A 是大型稀疏矩阵, 且需要 $\log(A)$ 的特定元素, 则计算下列积分 (例如 [97, Theorem 11.1]) 可以是一种选择:

$$\log(A) = (A - I)\int_0^1 [t(A - I) + I]^{-1}\,\mathrm{d}t. \tag{5.15}$$

将第 j 个单位向量 $\boldsymbol{e}_j$ 从右侧作用于 (5.15), 得到

$$\log(A)\boldsymbol{e}_j = (A-I)\int_0^1 [t(A-I)+I]^{-1}\boldsymbol{e}_j \,\mathrm{d}t. \tag{5.16}$$

这意味着计算 (5.16) 的右端就得到 $\log(A)$ 的第 i 列向量. 现在设 $\boldsymbol{x}^{(t)} = [t(A-I)+I]^{-1}\boldsymbol{e}_j$. 则有

$$[t(A-I)+I]\boldsymbol{x}^{(t)} = \boldsymbol{e}_j,$$

这是关于 t 的 (连续) 移位线性方程组. 对于数值求积, 如果将 t 离散为 $t_1, t_2, \ldots, t_m$, 则需要求解

$$[t_i(A-I)+I]\boldsymbol{x}^{(i)} = \boldsymbol{e}_j \quad \text{for } i = 1, 2, \ldots, m. \tag{5.17}$$

对 $t_i \neq 0$, 上述方程可改写为

$$[A + (t_i^{-1}-1)I]\widetilde{\boldsymbol{x}}^{(i)} = \boldsymbol{e}_j \quad \text{for } i = 1, 2, \ldots, m, \tag{5.18}$$

其中 $\widetilde{\boldsymbol{x}}^{(i)} = t_i\boldsymbol{x}^{(i)}$. 因此, 通过求积公式计算矩阵对数的特定元素是 Chapter 4 中移位 Krylov 子空间方法的一项重要应用.

在众多数值求积方法中, 双指数 (DE) 公式 [183] 被认为是最成功的方法之一, 尤其是在被积函数具有端点 (近) 奇异性时. DE 公式的发展可参见例如 [192].

文献 [188] 首次考虑利用 DE 公式计算矩阵函数. 以下介绍用于矩阵对数的 DE 公式. 对 (5.15) 作变量变换 $u = 2t - 1$, 得到

$$\log(A) = (A-I)\int_{-1}^1 [(1+u)(A-I)+2I]^{-1} \,\mathrm{d}u. \tag{5.19}$$

然后, 利用 DE 变换 $u = \tanh(\sinh(x))$, 得到

$$\log(A) = (A-I)\int_{-\infty}^{\infty} F_{\mathrm{DE}}(x) \,\mathrm{d}x, \tag{5.20}$$

其中

$$F_{\mathrm{DE}}(x) := \cosh(x)\,\mathrm{sech}^2(\sinh(x))[(1+\tanh(\sinh(x)))(A-I)+2I]^{-1}. \tag{5.21}$$

注意, 对任意 $x \in (-\infty, \infty)$, F_{DE} 中的矩阵 $(1+\tanh(\sinh(x)))(A-I)+2I$ 均非奇异.

利用 DE 公式计算 $\log(A)$ 的算法见 Algorithm 5.11. Algorithm 5.11 的理论细节见 [188].

由 Algorithm 5.11 的第 16 行, 向量 $(A-I)T\boldsymbol{e}_j$ 是 $\log(A)$ 的第 j 列, 并且由第 3 和 15 行可知, 需要求解下列方程:

Algorithm 5.11 基于 DE 公式计算 $\log(A)$

Input: $A \in \mathbb{R}^{n\times n}$, $m \in \mathbb{N}$, $\epsilon > 0$ 为积分区间截断误差容差

Output: $X \approx \log(A)$

设置 $F_{\mathrm{DE}}(x) = \cosh(x)\operatorname{sech}^2(\sinh(x))[(1+\tanh(\sinh(x)))(A-I)+2I]^{-1}$.

计算 $\|A-I\|$, $\|A^{-1}\|$ 及 $\rho(A)$.

$\theta = |\log(\rho(A))|$

$\epsilon_{\max} = \dfrac{3}{\theta}\dfrac{\|A-I\|\|A^{-1}\|}{1+\|A^{-1}\|}$

if $\epsilon \geq \epsilon_{\max}$ **then**

    $\epsilon \leftarrow \epsilon_{\max}/2$

end if

$a = \min\left\{\dfrac{\theta\epsilon}{3\|A-I\|}, \dfrac{1}{2\|A-I\|}\right\}$

$b = \max\left\{1-\dfrac{\theta\epsilon}{3\|A-I\|\|A^{-1}\|}, \dfrac{2\|A^{-1}\|}{2\|A^{-1}\|+1}\right\}$

$l = \operatorname{arsinh}(\operatorname{artanh}(2a-1))$

$r = \operatorname{arsinh}(\operatorname{artanh}(2b-1))$

$h = (r-l)/(m-1)$

$T = \dfrac{h}{2}(F_{\mathrm{DE}}(l)+F_{\mathrm{DE}}(r)) + h\displaystyle\sum_{i=1}^{m-2} F_{\mathrm{DE}}(l+ih)$

$X = (A-I)T$

$$[(1+\tanh(\sinh(l+ih)))(A-I)+2I]\boldsymbol{x} = \boldsymbol{e}_j \quad \text{for } i = 0,1,\ldots,m-1,$$

这也可以改写为移位线性方程组. 改写方式见 (5.17) 和 (5.18). 因此, 如果矩阵 A 是大型稀疏矩阵, 则 (移位) Krylov 子空间方法很有吸引力.

5.7 矩阵分数次幂

矩阵分数次幂可利用 $\log(A)$ 如下定义:

$$A^\alpha = \exp(\alpha\log(A)), \tag{5.22}$$

其中 $0 < \alpha < 1$. 条件 $0 < \alpha < 1$ 看似限制过强, 但在实际中已经足够. 事实上, 如果想计算 $A^{2.3}$, 则 $A^{2.3}$ 可分解为 $A^{2.3} = A^2A^{0.3}$. 因此, 问题在于如何计算 $A^{0.3}$, 它对应于 (5.22) 中 $\alpha = 0.3$ 的情形.

通过 (5.22) 中的定义计算 A^α 需要两个矩阵函数: 矩阵对数和矩阵指数函数. 另一方面, 下列积分形式中不出现这两个矩阵函数:

$$A^\alpha = \frac{\sin(\alpha\pi)}{\alpha\pi}A\int_0^\infty (t^{1/\alpha}I+A)^{-1}\,\mathrm{d}t \quad (0<\alpha<1). \tag{5.23}$$

文献 [186] 考虑将 DE 公式应用于 (5.23), 如下所述:

$$A^\alpha = \int_{-\infty}^{\infty} F_{\mathrm{DE}}(x)\,\mathrm{d}x, \tag{5.24}$$

其中

$$F_{\mathrm{DE}}(x) = t'(x)F(t(x)), \quad F(t) = \frac{\sin(\alpha\pi)}{\alpha\pi}A(t^{1/\alpha}I+A)^{-1}. \tag{5.25}$$

利用 DE 公式计算 A^α 的算法见 Algorithm 5.12. Algorithm 5.12 的理论细节见 [186].

Algorithm 5.12 计算 A^α 的 m 点 DE 公式

1: **Input** $A \in \mathbb{R}^{n\times n}$, $\alpha \in (0,1)$, $\epsilon > 0$, m
2: $l, r = \texttt{GetInterval}(A, \alpha, \epsilon)$
3: 设置 $\widetilde{F}_{\mathrm{DE}}(x) := \exp(\alpha\pi\sinh(x)/2)\cosh(x)[\exp(\pi\sinh(x)/2)I + A]^{-1}$.
4: $h = (r-l)/(m-1)$
5: $T = h[\widetilde{F}_{\mathrm{DE}}(l) + \widetilde{F}_{\mathrm{DE}}(r)]/2 + h\displaystyle\sum_{k=1}^{m-2} \widetilde{F}_{\mathrm{DE}}(l+kh)$
6: **Output** $\sin(\alpha\pi)AT/2 \approx A^\alpha$
7:
8: **function** $\texttt{GetInterval}(A, \alpha, \epsilon)$
9: 计算 $\|A\|$, $\|A^{-1}\|$.
10: $a_1 = [\alpha\pi(1+\alpha)\epsilon]/[4\sin(\alpha\pi)(1+2\alpha)]$, $a_2 = (2\|A^{-1}\|)^{-\alpha}$
11: $a = \min\{a_1, a_2\}$
12: $b_1 = [\pi(1-\alpha)(2-\alpha)\epsilon]^{\alpha/(\alpha-1)}/[4\sin(\alpha\pi)(3-2\alpha)\|A\|]^{\alpha/(\alpha-1)}$, $b_2 = (2\|A\|)^\alpha$
13: $b = \max\{b_1, b_2\}$
14: $l = \operatorname{asinh}(2\log(a)/\alpha\pi)$, $r = \operatorname{asinh}(2\log(b)/\alpha\pi)$
15: **return** l, r
16: **end function**

由 Algorithm 5.12, 算得的 A^α 的第 j 列为 $(\sin(\alpha\pi)AT/2)\boldsymbol{e}_j$. 因此, 由第 5 行, 必须利用下列方程计算各 $\widetilde{F}_{\mathrm{DE}}(l+ih)\boldsymbol{e}_j$:

$$e^{\alpha\pi\sinh(l+ih)/2}\cosh(l+ih)(e^{\pi\sinh(l+ih)/2}I + A)\boldsymbol{x}^{(i)} = \boldsymbol{e}_j \quad \text{for } i = 0, 1, \ldots, m-1.$$

类似于 (5.17) 和 (5.18), 上述方程可改写为移位线性方程组. 因此, 如果矩阵 A 是大型稀疏矩阵, 使用 (移位) Krylov 子空间方法将是一种选择.

软件

本书侧重于算法设计, 省略了 Krylov 子空间方法的详细历史. 对于希望详细了解其历史的读者, 强烈推荐 Gérard Meurant 与 Jurjen Duintjer Tebbens 的著作 [129]. 该书提供了详细的历史介绍, 包含一千余条参考文献, 以及对许多用户都很实用的 Krylov 子空间方法的 Matlab/Octave 函数.

为方便 Krylov 子空间方法的潜在用户, 下面列出一些可用的软件包.

- Fortran 90 (Piotr J. Flatau 的 CCGPACK 2.0)

  https://code.google.com/archive/p/conjugate-gradient-lib/

  → downloads → `ccgpak2_0.zip`

  用户手册 https://arxiv.org/abs/1208.4869

  提供 COCR, CSYM, BiCGSTAB(ℓ), GPBiCG(m, ℓ), BiCOR 等方法.

- Fortran 90 (Kω)

  Kω: 移位 Krylov 子空间方法的开源库

  https://www.pasums.issp.u-tokyo.ac.jp/komega/en/

  提供移位 CG, 移位 COCG 和移位 BiCG 方法.

- GNU Octave (版本 6.4.0)

  https://octave.org/doc/v6.4.0/Specialized-Solvers.html#Specialized-Solvers

  提供 CG, CR, BiCG, QMR, CGS, BiCGSTAB, GMRES 等方法.

- Julia (IterativeSolvers.jl)

  https://iterativesolvers.julialinearalgebra.org/dev/

  提供 CG, MINRES, GMRES, IDR(s) 和 BiCGSTAB(ℓ) 方法.

- Python (SciPy 1.8.0)

  https://docs.scipy.org/doc/scipy/reference/sparse.linalg.html

  提供 CG, MINRES, BiCG, QMR, CGS, BiCGSTAB, GMRES 等方法.

参考文献

1. Abe, K., Fujino, S.: Converting BiCR method for linear equations with complex symmetric matrices. App. Math. Comput. 321, 564–576 (2018)

2. Abe, K., Sleijpen, G.L.G.: BiCR variants of the hybrid BiCG methods for solving linear systems with nonsymmetric matrices. J. Comput. Appl. Math. 234, 985–994 (2010)

3. Absil, P.-A., Mahony, R., Sepulchre, R.: Optimization Algorithms on Matrix Manifolds. Princeton University Press, Princeton (2008)

4. Aihara, K.: GPBi-CGstab(L): A Lanczos-type product method unifying Bi-CGstab(L) and GPBi-CG. Numer. Linear Algebra Appl. 27, e2298 (2020)

5. Aihara, K., Abe, K., Ishiwata, E.: An alternative implementation of the IDRstab method saving vector updates. JSIAM Lett. 3, 69–72 (2011)

6. Aihara, K., Abe, K., Ishiwata, E.: A quasi-minimal residual variant of IDRstab using the residual smoothing technique. Appl. Math. Comput. 236, 67–77 (2014)

7. Aihara, K., Abe, K., Ishiwata, E.: A variant of IDRstab with reliable update strategies for solving sparse linear systems. J. Comput. Appl. Math. 259, 244–258 (2014)

8. Al-Mohy, A.H., Higham, N.J.: A new scaling and squaring algorithm for the matrix exponential. SIAM J. Matrix Anal. Appl. 31, 970–989 (2009)

9. Al-Mohy, A.H., Higham, N.J.: Improved inverse scaling and squaring algorithms for the matrix logarithm. SIAM J. Sci. Comput. 34, C153–C169 (2012)

10. Axelsson, O.: Iterative Solution Methods. Cambridge University Press, Cambridge (1994)

11. Baglama, J., Reichel, L.: Augmented GMRES-type methods. Numer. Linear Algebra Appl. 14, 337–350 (2007)

12. Bai, Z., Demmel, J., Dongarra, J., Ruhe, A., van der Vorst, H.A. (eds.): Templates for the Solution of Algebraic Eigenvalue Problems. SIAM, Philadelphia (2000)

13. Baker, A.H., Dennis, J.M., Jessup, E.R.: On improving linear solver performance: a block variant of GMRES. SIAM J. Sci. Comput. 27, 1608–1626 (2006)

14. Baker, A.H., Jessup, E.R., Kolev, T.V.: A simple strategy for varying the restart parameter in GMRES(m). J. Comput. Appl. Math. 230, 751–761 (2009)

15. Baker, A.H., Jessup, E.R., Manteuffel, T.: A technique for accelerating the convergence of restarted GMRES. SIAM J. Matrix Anal. Appl. 26, 962–984 (2005)

16. Bank, R.E., Chan, T.F.: An analysis of the composite step biconjugate gradient method. Numer. Math. 66, 295–319 (1993)

17. Bank, R.E., Chan, T.F.: A composite step bi-conjugate gradient algorithm for nonsymmetric linear systems. Numer. Algorithms 7, 1–16 (1994)

18. Barrett, R., Berry, M., Chan, T.F., Demmel, J., Donato, J., Dongarra, J., Eijkhout, V., Pozo, R., Romine, C., van der Vorst, H.: Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods. SIAM, Philadelphia (1994)

19. Bayliss, A., Goldstein, C., Turkel, E.: An iterative method for the Helmholtz equation. J. Comput. Phys. 49, 443–457 (1983)

20. Benzi, M.: Preconditioning techniques for large linear systems: a survey. J. Comput. Phys. 182, 418–477 (2002)

21. Benzi, M., Cullum, J.K., Tůma, M.: Robust approximate inverse preconditioning for the conjugate gradient method. SIAM J. Sci. Comput. 22, 1318–1332 (2000)

22. Benzi, M., Meyer, C.D., Tůma, M.: A sparse approximate inverse preconditioner for the conjugate gradient method. SIAM J. Sci. Comput. 17, 1135–1149 (1996)

23. Benzi, M., Szyld, D.B., van Duin, A.: Orderings for incomplete factorization preconditioning of nonsymmetric problems. SIAM J. Sci. Comput. 20, 1652–1670 (1999)

24. Benzi, M., Tůma, M.: A sparse approximate inverse preconditioner for nonsymmetric linear systems. SIAM J. Sci. Comput. 19, 968–994 (1998)

25. Benzi, M., Tůma, M.: A robust preconditioner with low memory requirements for large sparse least squares problems. SIAM J. Sci. Comput. 25, 499–512 (2003)

26. Boyd, S., Vandenberghe, L.: Convex Optimization. Cambridge University Press, Cambridge (2004)

27. Bramble, J.H.: Multigrid Methods. Longman Scientific and Technical, Harlow (1993)

28. Briggs, W.L., Henson, V.E., McCormick, S.F.: A Multigrid Tutorial, 2nd edn. SIAM, Philadelphia (2000)

29. Bruaset, A.M.: A Survey of Preconditioned Iterative Methods. Routledge, Boca Raton (1995)

30. Bunse-Gerstner, A., Stöver, R.: On a conjugate gradient-type method for solving complex symmetric linear systems. Linear Algebra Appl. 287, 105–123 (1999)

31. Cabral, J.C., Schaerer, C.E., Bhaya, A.: Improving GMRES(m) using an adaptive switching controller. Numer. Linear Algebra Appl. 27, e2305 (2020)

32. Carpentieri, B., Jing, Y.-F., Huang, T.-Z.: The BiCOR and CORS iterative algorithms for solving nonsymmetric linear systems. SIAM J. Sci. Comput. 33, 3020–3036 (2011)

33. Cayley, A.: A memoir on the theory of matrices. Philos. Trans. R. Soc. 148, 17–37 (1858)

34. Chan, T.F., Gallopoulos, E., Simoncini, V., Szeto, T., Tong, C.H.: A quasi-minimal residual variant of the Bi-CGSTAB algorithm for nonsymmetric systems. SIAM J. Sci. Comput. 15, 338–347 (1994)

35. Chan, T.F., Szeto, T.: Composite step product methods for solving nonsymmetric linear systems. SIAM J. Sci. Comput. 17, 1491–1508 (1996)

36. Chiba, K.: Gyouretsu no Kansu to Jordan Hyoujun Kei (in Japanese). Scientist Press, Tokyo (2010)

37. Choi, S.-C.: Minimal residual methods for complex symmetric, skew symmetric, and skew Hermitian systems. Tech. rep., Comput. Inst., Univ. Chicago, Chicago, IL, USA (2013)

38. Chronopoulos, A.T., Ma, S.: On squaring Krylov subspace iterative methods for non-symmetric linear systems. Tech. Rep. TR 89-67 (1989)

39. Cipra, B.A.: The best of the 20th century: Editors name top 10 algorithms. SIAM News 33, 1–2 (2000)

40. Clemens, M., Weiland, T., Rienen, U.V.: Comparison of Krylov-type methods for complex linear systems applied to high-voltage problems. IEEE Trans. Magn. 34, 3335–3338 (1998)

41. Craig, E.J.: The N -step iteration procedures. J. Math. Phys. 34, 64–73 (1955)

42. Cuthill, E., McKee, J.: Reducing the bandwidth of sparse symmetric matrices. In: Proceedings of the 1969 24th National Conference, pp. 157–172. Association for Computing Machinery, New York, NY, USA (1969)

43. Datta, B.N., Saad, Y.: Arnoldi methods for large Sylvester-like observer matrix equations, and an associated algorithm for partial spectrum assignment. Linear Algebra Appl. 154–156, 225–244 (1991)

44. Davis, T.A.: Direct Methods for Sparse Linear Systems. SIAM, Philadelphia (2006)

45. Demmel, J.W.: Applied Numerical Linear Algebra. SIAM, Philadelphia (1997)

46. Du, L., Sogabe, T., Yu, B., Yamamoto, Y., Zhang, S.-L.: A block IDR(s) method for nonsymmetric linear systems with multiple right-hand sides. J. Comput. Appl. Math. 235, 4095–4106 (2011)

47. Du, L., Sogabe, T., Zhang, S.-L.: A variant of the IDR(s) method with the quasi-minimal residual strategy. J. Comput. Appl. Math. 236, 621–630 (2011)

48. Du, L., Sogabe, T., Zhang, S.-L.: IDR(s) for solving shifted nonsymmetric linear systems. J. Comput. Appl. Math. 274, 35–43 (2015)

49. Du, Y.-S., Hayami, K., Zheng, N., Morikuni, K., Yin, J.-F.: Kaczmarz-type inner-iteration preconditioned flexible GMRES methods for consistent linear systems. SIAM J. Sci. Comput. 43, S345–S366 (2021)

50. Dubois, P.F., Greenbaum, A., Rodrigue, G.H.: Approximating the inverse of a matrix for use in iterative algorithms on vector processors. Computing 22, 257–268 (1979)

51. Duff, I.S., Erisman, A.M., Reid, J.K.: Direct Methods for Sparse Matrices. Oxford University Press, Oxford (1986)

52. Duff, I.S., Meurant, G.A.: The effect of ordering on preconditioned conjugate gradients. BIT Numer. Math. 29, 635–657 (1989)

53. Eisenstat, S.C.: Efficient implementation of a class of preconditioned conjugate gradient methods. SIAM J. Sci. Stat. Comput. 2, 1–4 (1981)

54. Eisenstat, S.C., Elman, H.C., Schultz, M.H.: Variational iterative methods for nonsymmetric systems of linear equations. SIAM J. Numer. Anal. 20, 345–357 (1983)

55. Elbouyahyaoui, L., Heyouni, M., Tajaddini, A., Saberi-Movahed, F.: On restarted and deflated block FOM and GMRES methods for sequences of shifted linear systems. Numer. Algorithms 87, 1257–1299 (2021)

56. Eldén, L., Savas, B.: A Newton-Grassmann method for computing the best multilinear rank-(r_1, r_2, r_3) approximation of a tensor. SIAM J. Matrix Anal. Appl. 31, 248–271 (2009)

57. Faber, V., Manteuffel, T.: Necessary and sufficient conditions for the existence of a conjugate gradient method. SIAM J. Numer. Anal. 21, 352–362 (1984)

58. Fletcher, R.: Conjugate gradient methods for indefinite systems. In: Watson, G.A. (ed.) Numerical Analysis, pp. 73–89. Springer, Berlin, Heidelberg (1976)

59. Forsythe, G.E., Straus, E.G.: On best conditioned matrices. Proc. Amer. Math. Soc. 6, 340–345 (1955)

60. Freund, R.W.: On conjugate gradient type methods and polynomial preconditioners for a class of complex non-Hermitian matrices. Numer. Math. 57, 285–312 (1990)

61. Freund, R.W.: Conjugate gradient-type methods for linear systems with complex symmetric coefficient matrices. SIAM J. Sci. Stat. Comput. 13, 425–448 (1992)

62. Freund, R.W.: Solution of shifted linear systems by quasi-minimal residual iterations. In: Reichel, L., Ruttan, A., Varga, R.S. (eds.) Numerical Linear Algebra and Scientific Computation, pp. 101–122. De Gruyter (1993)

63. Freund, R.W.: A transpose-free quasi-minimal residual algorithm for non-Hermitian linear systems. SIAM J. Sci. Comput. 14, 470–482 (1993)

64. Freund, R.W., Golub, G.H., Nachtigal, N.M.: Iterative solution of linear systems. Acta Numer. 1, 57–100 (1992)

65. Freund, R.W., Malhotra, M.: A block QMR algorithm for non-Hermitian linear systems with multiple right-hand sides. Linear Algebra Appl. 254, 119–157 (1997)

66. Freund, R.W., Nachtigal, N.M.: QMR: a quasi-minimal residual method for non-Hermitian linear systems. Numer. Math. 60, 315–339 (1991)

67. Frommer, A.: BiCGStab(ℓ) for families of shifted linear systems. Computing 70, 87–109 (2003)

68. Frommer, A., Glässner, U.: Restarted GMRES for shifted linear systems. SIAM J. Sci. Comput. 19, 15–26 (1998)

69. Frommer, A., Lippert, T., Medeke, B., Schilling, K. (eds.): Numerical challenges in lattice quantum chromodynamics: joint interdisciplinary workshop of John von Neumann institute for computing, Jülich, and Institute of Applied Computer Science, Wuppertal University, August 1999. Springer-Verlag, Berlin Heidelberg (2000)

70. Frommer, A., Maass, P.: Fast CG-based methods for Tikhonov-Phillips regularization. SIAM J. Sci. Comput. 20, 1831–1850 (1999)

71. Fujino, S.: GPBiCG(m, l): A hybrid of BiCGSTAB and GPBiCG methods with efficiency and robustness. Appl. Numer. Math. 41, 107–117 (2002)

72. Fujino, S., Zhang, S.-L.: Hanpukuhou no Suri (in Japanese). Asakura Shoten, Tokyo (1996)

73. Fujiwara, T., Hoshi, T., Yamamoto, S., Sogabe, T., Zhang, S.-L.: Novel algorithm of large-scale simultaneous linear equations. J. Phys.: Condens. Matter 22, 074206 (2010)

74. Gaul, A., Gutknecht, M.H., Liesen, J., Nabben, R.: A framework for deflated and augmented Krylov subspace methods. SIAM J. Matrix Anal. Appl. 34, 495–518 (2013)

75. George, A.: Computer implementation of the finite element method. Tech. Rep. STAN-CS-208, Stanford University, Stanford, CA, USA (1971)

76. George, A.: Nested dissection of a regular finite element mesh. SIAM J. Numer. Anal. 10, 345–363 (1973)

77. George, A., Liu, J.W.: The evolution of the minimum degree ordering algorithm. SIAM Rev. 31, 1–19 (1989)

78. Golub, G., Kahan, W.: Calculating the singular values and pseudo-inverse of a matrix. J. Soc. Ind. Appl. Math. Ser. B Numer. Anal. 2, 205–224 (1965)

79. Golub, G.H., Loan, C.F.V.: Matrix Computations, 4th edn. Johns Hopkins University Press, Baltimore (2013)

80. Golub, G.H., O' Leary, D.P.: Some history of the conjugate gradient and Lanczos algorithms: 1948–1976. SIAM Rev. 31, 50–102 (1989)

81. Greenbaum, A.: Iterative Methods for Solving Linear Systems. SIAM, Philadelphia (1997)

82. Grote, M.J., Huckle, T.: Parallel preconditioning with sparse approximate inverses. SIAM J. Sci. Comput. 18, 838–853 (1997)

83. Gu, X.-M., Clemens, M., Huang, T.-Z., Li, L.: The SCBiCG class of algorithms for complex symmetric linear systems with applications in several electromagnetic model problems. Comput. Phys. Commun. 191, 52–64 (2015)

84. Gu, X.-M., Huang, T.-Z., Li, L., Li, H., Sogabe, T., Clemens, M.: Quasi-minimal residual variants of the COCG and COCR methods for complex symmetric linear systems in electromagnetic simulations. IEEE Trans. Microw. Theory Tech. 62, 2859–2867 (2014)

85. Gu, X.-M., Huang, T.-Z., Meng, J., Sogabe, T., Li, H.B., Li, L.: BiCR-type methods for families of shifted linear systems. Comput. Math. Appl. 68, 746–758 (2014)

86. Guennouni, A.E., Jbilou, K., Sadok, H.: A block version of BiCGSTAB for linear systems with multiple right-hand sides. Electron. Trans. Numer. Anal. 16, 129–142 (2003)

87. Gutknecht, M.H.: Variants of BiCGSTAB for matrices with complex spectrum. SIAM J. Sci. Comput. 14, 1020–1033 (1993)

88. Gutknecht, M.H.: IDR explained. Electron. Trans. Numer. Anal. 36, 126–148 (2009)

89. Gutknecht, M.H.: Spectral deflation in Krylov solvers: a theory of coordinate space based methods. Electron. Trans. Numer. Anal. 39, 156–185 (2012)

90. Gutknecht, M.H.: Deflated and augmented Krylov subspace methods: a framework for deflated BiCG and related solvers. SIAM J. Matrix Anal. Appl. 35, 1444–1466 (2014)

91. Hackbusch, W.: Multi-Grid Methods and Applications. Springer-Verlag, Berlin Heidelberg (1985)

92. Hackbusch, W.: Iterative Solution of Large Sparse Systems of Equations, Applied Mathematical Sciences, vol. 95. Springer-Verlag, New York (1994)

93. Hadjidimos, A.: Successive overrelaxation (SOR) and related methods. J. Comput. Appl. Math. 123, 177–199 (2000)

94. Hayami, K., Yin, J.-F., Ito, T.: GMRES methods for least squares problems. SIAM J. Matrix Anal. Appl. 31, 2400–2430 (2010)

95. Hestenes, M.R., Stiefel, E.: Methods of conjugate gradients for solving linear systems. J. Res. Natl. Bur. Stand. 49, 409–436 (1952)

96. Higham, N.J.: Newton' s method for the matrix square root. Math. Comp. 46, 537–549 (1986)

97. Higham, N.J.: Functions of Matrices. SIAM, Philadelphia (2008)

98. Higham, N.J.: The scaling and squaring method for the matrix exponential revisited. SIAM Rev. 51, 747–764 (2009)

99. Hochbruck, M., Lubich, C.: On Krylov subspace approximations to the matrix exponential operator. SIAM J. Numer. Anal. 34, 1911–1925 (1997)

100. Hoshi, T., Kawamura, M., Yoshimi, K., Motoyama, Y., Misawa, T., Yamaji, Y., Todo, S., Kawashima, N., Sogabe, T.: Kω - Open-source library for the shifted Krylov subspace method of the form $(zI - H)\boldsymbol{x} = \boldsymbol{b}$. Comput. Phys. Commun. 258, 107536 (2021)

101. Iannazzo, B.: A note on computing the matrix square root. Calcolo 40, 273–283 (2003)

102. Iannazzo, B.: On the Newton method for the matrix pth root. SIAM J. Matrix Anal. Appl. 28, 503–523 (2006)

103. Imakura, A., Li, R.-C., Zhang, S.-L.: Locally optimal and heavy ball GMRES methods. Japan J. Indust. Appl. Math. 33, 471–499 (2016)

104. Imakura, A., Sogabe, T., Zhang, S.-L.: An efficient variant of the GMRES(m) method based on the error equations. E. Asian J. App. Math. 2, 19–32 (2012)

105. Imakura, A., Sogabe, T., Zhang, S.-L.: An efficient variant of the restarted shifted GMRES method for solving shifted linear systems. J. Math. Res. Appl. 33, 127–141 (2013)

106. Imakura, A., Sogabe, T., Zhang, S.-L.: A look-back-type restart for the restarted Krylov subspace methods for solving non-Hermitian linear systems. Japan J. Indust. Appl. Math. 35, 835–859 (2018)

107. Ipsen, I.C.F.: Numerical Matrix Analysis: Linear Systems and Least Squares. SIAM, Philadelphia (2009)

108. Ishikawa, K.-I., Sogabe, T.: A thick-restart Lanczos type method for Hermitian J-symmetric eigenvalue problems. Japan J. Indust. Appl. Math. 38, 233–256 (2021)

109. Itoh, S.: Improvement of preconditioned bi-Lanczos-type algorithms with residual norm minimization for the stable solution of systems of linear equations. Japan J. Indust. Appl. Math. 39, 17–74 (2022)

110. Iwase, S., Hoshi, T., Ono, T.: Numerical solver for first-principles transport calculation based on real-space finite-difference method. Phys. Rev. E 91, 063305 (2015)

111. Jin, S., Bulgac, A., Roche, K., Wlazłowski, G.: Coordinate-space solver for superfluid many-fermion systems with the shifted conjugate-orthogonal conjugate-gradient method. Phys. Rev. C 95, 044302 (2017)

112. Jing, Y., Huang, T.: Restarted weighted full orthogonalization method for shifted linear systems. Comput. Math. Appl. 57, 1583–1591 (2009)

113. Jing, Y., Huang, T., Carpentieri, B., Duan, Y.: Exploiting the composite step strategy to the biconjugate A-orthogonal residual method for non-Hermitian linear systems. J. Appl. Math. 2013, 16pp. (2013)

114. Jing, Y., Huang, T.-Z., Zhang, Y., Li, L., Cheng, G.-H., Ren, Z.-G., Duan, Y., Sogabe, T., Carpentieri, B.: Lanczos-type variants of the COCR method for complex nonsymmetric linear systems. J. Comput. Phys. 228, 6376–6394 (2009)

115. Jing, Y., Yuan, P., Huang, T.: A simpler GMRES and its adaptive variant for shifted linear systems. Numer. Linear Algebra Appl. 24, e2076 (2017)

116. Johnson, O.G., Micchelli, C.A., Paul, G.: Polynomial preconditioners for conjugate gradient calculations. SIAM J. Numer. Anal. 20, 362–376 (1983)

117. Joubert, W.: On the convergence behavior of the restarted GMRES algorithm for solving nonsymmetric linear systems. Numer. Linear Algebra Appl. 1, 427–447 (1994)

118. Kikuchi, F.: Yugen Yosoho Gaisetsu (in Japanese). Saiensu-Sha, Tokyo (1980)

119. Lanczos, C.: Solution of systems of linear equations by minimized iterations. J. Res. Natl. Bur. Stand. 49, 33 (1952)

120. Lee, D., Hoshi, T., Sogabe, T., Miyatake, Y., Zhang, S.-L.: Solution of the k-th eigenvalue problem in large-scale electronic structure calculations. J. Comput. Phys. 371, 618–632 (2018)

121. Liesen, J., Strakoš, Z.: On optimal short recurrences for generating orthogonal Krylov subspace bases. SIAM Rev. 50, 485–503 (2008)

122. Liesen, J., Strakoš, Z.: Krylov Subspace Methods: Principles and Analysis. Oxford University Press, Oxford (2012)

123. Liu, J.W.H.: Modification of the minimum-degree algorithm by multiple elimination. ACM Trans. Math. Softw. 11, 141–153 (1985)

124. Liu, L., Sekiya, K., Ogino, M., Masui, K.: A COMINRES-QLP method for solving complex symmetric linear systems. Electr. Eng. Jpn. 214, e23325 (2021)

125. Lottes, J.: Towards Robust Algebraic Multigrid Methods for Nonsymmetric Problems. Springer, Cham (2017)

126. Luenberger, D.: Hyperbolic pairs in the method of conjugate gradients. SIAM J. Appl. Math. 6, 1263–1267 (1969)

127. Meijerink, J.A., van der Vorst, H.A.: An iterative solution method for linear systems of which the coefficient matrix is a symmetric M-matrix. Math. Comp. 31, 148 – 162 (1977)

128. Meng, G.-Y.: A practical asymptotical optimal SOR method. Appl. Math. Comput. 242, 707–715 (2014)

129. Meurant, G., Tebbens, J.D.: Krylov Methods for Nonsymmetric Linear Systems: From Theory to Computations, Springer Series in Computational Mathematics, vol. 57. Springer Nature, Switzerland (2020)

130. Miyatake, Y., Sogabe, T., Zhang, S.-L.: On the equivalence between SOR-type methods for linear systems and the discrete gradient methods for gradient systems. J. Comput. Appl. Math. 342, 58–69 (2018)

131. Miyatake, Y., Sogabe, T., Zhang, S.-L.: Adaptive SOR methods based on the Wolfe conditions. Numer. Algorithms 84, 117–132 (2020)

132. Mizuno, S., Moriizumi, Y., Usuda, T.S., Sogabe, T.: An initial guess of Newton' s method for the matrix square root based on a sphere constrained optimization problem. JSIAM Lett. 8, 17–20 (2016)

133. Moler, C., Van Loan, C.: Nineteen dubious ways to compute the exponential of a matrix, twenty-five years later. SIAM Rev. 45, 3–49 (2003)

134. Morgan, R.B.: A restarted GMRES method augmented with eigenvectors. SIAM J. Matrix Anal. Appl. 16, 1154–1171 (1995)

135. Morgan, R.B.: GMRES with deflated restarting. SIAM J. Sci. Comput. 24, 20 – 37 (2002)

136. Morikuni, K., Hayami, K.: Inner-iteration Krylov subspace methods for least squares problems. SIAM J. Matrix Anal. Appl. 34, 1–22 (2013)

137. Moriya, K., Nodera, T.: The DEFLATED-GMRES(m, k) method with switching the restart frequency dynamically. Numer. Linear Algebra Appl. 7, 569–584 (2000)

138. Nachtigal, N.M., Reddy, S.C., Trefethen, L.N.: How fast are nonsymmetric matrix iterations? SIAM J. Matrix Anal. Appl. 13, 778–795 (1992)

139. Noritake, S., Imakura, A., Yamamoto, Y., Zhang, S.-L.: A large-grained parallel solver for linear simultaneous ordinary differential equations based on matrix exponential and its evaluation (in Japanese). Trans. JSIAM 19, 293–312 (2009)

140. Ogasawara, M., Tadano, H., Sakurai, T., Itoh, S.: A Krylov subspace method for shifted linear systems and its application to eigenvalue problems (in Japanese). Trans. JSIAM 14, 193–205 (2004)

141. Ogino, M., Takei, A., Sugimoto, S., Yoshimura, S.: A numerical study of iterative substructuring method for finite element analysis of high frequency electromagnetic fields. Comput. Math. Appl. 72, 2020–2027 (2016)

142. O' Leary, D.P.: The block conjugate gradient algorithm and related methods. Linear Algebra Appl. 29, 293–322 (1980)

143. Paige, C.C., Saunders, M.A.: Solution of sparse indefinite systems of linear equations. SIAM J. Numer. Anal. 12, 617–629 (1975)

144. Paige, C.C., Saunders, M.A.: LSQR: An algorithm for sparse linear equations and sparse least squares. ACM Trans. Math. Softw. 8, 43–71 (1982)

145. Parlett, B.N., Taylor, D.R., Liu, Z.A.: A look-ahead Lanczos algorithm for unsymmetric matrices. Math. Comp. 44, 105–124 (1985)

146. Pearson, J.W., Pestana, J.: Preconditioners for Krylov subspace methods: An overview. GAMM-Mitteilungen 43, e202000015 (2020)

147. Pommerell, C.: Solution of Large Unsymmetric Systems of Linear Equations. Ph.D. thesis, ETH Zürich, Switzerland (1992)

148. Quinn, J.A., Sugiyama, M.: A least-squares approach to anomaly detection in static and sequential data. Pattern Recognit. Lett. 40, 36–40 (2014)

149. Saad, Y.: Krylov subspace methods for solving large unsymmetric linear systems. Math. Comp. 37, 105–126 (1981)

150. Saad, Y.: ILUT: A dual threshold incomplete LU factorization. Numer. Linear Algebra Appl. 1, 387–402 (1994)

151. Saad, Y.: Iterative Methods for Sparse Linear Systems, 2nd edn. SIAM, Philadelphia (2003)

152. Saad, Y., Schultz, M.H.: GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. SIAM J. Sci. Stat. Comput. 7, 856–869 (1986)

153. Saito, S., Tadano, H., Imakura, A.: Development of the block BiCGSTAB(ℓ) method for solving linear systems with multiple right hand sides. JSIAM Lett. 6, 65–68 (2014)

154. Sakurai, T., Matsuo, T., Katagiri, T. (eds.): Numerical Linear Algebra: Theory and HPC (in Japanese). Kyoritsu Shuppan, Tokyo (2018)

155. Sato, H., Aihara, K.: Riemannian Newton' s method on the Grassmann manifold exploiting the quotient structure (in Japanese). Trans. JSIAM 28, 205–241 (2018)

156. Sattler, K.D. (ed.): 21st Century Nanoscience—A Handbook: Nanophysics Source Book (1st ed.). CRS Press, Boca Raton (2020)

157. Simoncini, V.: A stabilized QMR version of block BiCG. SIAM J. Matrix Anal. Appl. 18, 419–434 (1997)

158. Simoncini, V.: Restarted full orthogonalization method for shifted linear systems. BIT Numer. Math. 43, 459–466 (2003)

159. Simoncini, V., Perotti, F.: On the numerical solution of $(\lambda^2 A + \lambda B + C)\boldsymbol{x} = \boldsymbol{b}$ and application to structural dynamics. SIAM J. Sci. Comput. 23, 1875–1897 (2002)

160. Simoncini, V., Szyld, D.B.: Recent computational developments in Krylov subspace methods for linear systems. Numer. Linear Algebra Appl. 14, 1–59 (2007)

161. Simoncini, V., Szyld, D.B.: Interpreting IDR as a Petrov-Galerkin method. SIAM J. Sci. Comput. 32, 1898–1912 (2010)

162. Sleijpen, G.L.G., Fokkema, D.R.: BiCGstab(ℓ) for linear equations involving unsymmetric matrices with complex spectrum. Electron. Trans. Numer. Anal. 1, 11 – 32 (1993)

163. Sleijpen, G.L.G., Sonneveld, P., van Gijzen, M.B.: Bi-CGSTAB as an induced dimension reduction method. Appl. Numer. Math. 60, 1100–1114 (2010)

164. Sleijpen, G.L.G., van Gijzen, M.B.: Exploiting BiCGstab(ℓ) strategies to induce dimension reduction. SIAM J. Sci. Comput. 32, 2687–2709 (2010)

165. Sogabe, T.: Extensions of the Conjugate Residual Method. Ph.D. thesis, The University of Tokyo, Tokyo, Japan (2006)

166. Sogabe, T., Hoshi, T., Zhang, S.-L., Fujiwara, T.: A numerical method for calculating the Green' s function arising from electronic structure theory. In: Kaneda, Y., Kawamura, H., Sasai, M. (eds.) Frontiers of Computational Science, pp. 189–195. Springer, Berlin, Heidelberg (2007)

167. Sogabe, T., Hoshi, T., Zhang, S.-L., Fujiwara, T.: On a weighted quasi-residual minimization strategy for solving complex symmetric shifted linear systems. Electron. Trans. Numer. Anal. 31, 126–140 (2008)

168. Sogabe, T., Hoshi, T., Zhang, S.-L., Fujiwara, T.: Solution of generalized shifted linear systems with complex symmetric matrices. J. Comput. Phys. 231, 5669–5684 (2012)

169. Sogabe, T., Sugihara, M., Zhang, S.-L.: An extension of the conjugate residual method to nonsymmetric linear systems. J. Comput. Appl. Math. 226, 103–113 (2009)

170. Sogabe, T., Yamamoto, Y.: Basic Mathematical Algorithm for Computational Science (in Japanese). Zhang, S.-L. (ed.), Kaneda, Y., (ed. in chief), Sasai, M (ed. in chief), Kyoritsu Shuppan, Tokyo (2019)

171. Sogabe, T., Zhang, S.-L.: A COCR method for solving complex symmetric linear systems. J. Comput. Appl. Math. 199, 297–303 (2007)

172. Sogabe, T., Zhang, S.-L.: An extension of the COCR method to solving shifted linear systems with complex symmetric matrices. E. Asian J. Appl. Math. 1, 97–107 (2011)

173. Sonneveld, P.: CGS, a fast Lanczos-type solver for nonsymmetric linear systems. SIAM J. Sci. Stat. Comput. 10, 36–52 (1989)

174. Sonneveld, P., van Gijzen, M.B.: IDR(s): A family of simple and fast algorithms for solving large nonsymmetric systems of linear equations. SIAM J. Sci. Comput. 31, 1035–1062 (2008)

175. Soodhalter, K.M., Szyld, D.B., Xue, F.: Krylov subspace recycling for sequences of shifted linear systems. Appl. Numer. Math. 81, 105–118 (2014)

176. Sosonkina, M., Watson, L.T., Kapania, R.K., Walker, H.F.: A new adaptive GMRES algorithm for achieving high accuracy. Numer. Linear Algebra Appl. 5, 275–297 (1998)

177. Stiefel, E.: Relaxationsmethoden bester strategie zur lösung linearer gleichungssysteme. Comment. Math. Helv. 29, 157–179 (1955)

178. Sugiyama, M.: Superfast-trainable multi-class probabilistic classifier by least-squares posterior fitting. IEICE Trans. Inform. Syst. E93-D, 2690–2701 (2010)

179. Suzuki, M.: Generalized Trotter' s formula and systematic approximants of exponential operators and inner derivations with applications to many-body problems. Communs Math. Phys. 51, 183–190 (1976)

180. Sylvester, J.J.: On the equation to the secular inequalities in the planetary theory. Philos. Mag. 16, 267–269 (1883)

181. Tadano, H.: Development of the Block BiCGGR^2 method for linear systems with multiple right-hand sides. Japan J. Indust. Appl. Math. 36, 563–577 (2019)

182. Tadano, H., Sakurai, T., Kuramashi, Y.: Block BiCGGR: A new block Krylov subspace method for computing high accuracy solutions. JSIAM Lett. 1, 44–47 (2009)

183. Takahasi, H., Mori, M.: Double exponential formulas for numerical integration. Publ. Res. Inst. Math. Sci. 9, 721–741 (1973)

184. Takayama, R., Hoshi, T., Sogabe, T., Zhang, S.-L., Fujiwara, T.: Linear algebraic calculation of the Green' s function for large-scale electronic structure theory. Phys. Rev. B 73, 165108 (2006)

185. Tanio, M., Sugihara, M.: GBi-CGSTAB(s, L): IDR(s) with higher-order stabilization polynomials. J. Comput. Appl. Math. 235, 765–784 (2010)

186. Tatsuoka, F., Sogabe, T., Miyatake, Y., Kemmochi, T., Zhang, S.-L.: Computing the matrix fractional power with the double exponential formula. Electron. Trans. Numer. Anal. 54, 558–580 (2021)

187. Tatsuoka, F., Sogabe, T., Miyatake, Y., Zhang, S.-L.: A cost-efficient variant of the incremental newton iteration for the matrix pth root. J. Math. Res. Appl. 37, 97–106 (2017)

188. Tatsuoka, F., Sogabe, T., Miyatake, Y., Zhang, S.-L.: Algorithms for the computation of the matrix logarithm based on the double exponential formula. J. Comput. Appl. Math. 373, 112396 (2020)

189. Taylor, D.: Analysis of the Look Ahead Lanczos Algorithm. Ph.D. thesis, University of California, Berkley (1982)

190. Teng, H., Fujiwara, T., Hoshi, T., Sogabe, T., Zhang, S.-L., Yamamoto, S.: Efficient and accurate linear algebraic methods for large-scale electronic structure calculations with nonorthogonal atomic orbitals. Phys. Rev. B 83, 165103 (2011)

191. Trefethen, L.N., Bau, D.: Numerical Linear Algebra: Twenty-Fifth. Anniversary SIAM, Philadelphia (2022)

192. Trefethen, L.N., Weideman, J.A.C.: The exponentially convergent trapezoidal rule. SIAM Rev. 56, 385–458 (2014)

193. Trottenberg, U., Oosterlee, C.W., Schüller, A.: Multigrid. Academic Press, NewYork (2001)

194. van der Sluis, A.: Condition numbers and equilibration of matrices. Numer. Math. 14, 14–23 (1969)

195. van der Vorst, H.A.: Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems. SIAM J. Sci. Stat. Comput. 13, 631–644 (1992)

196. van der Vorst, H.A.: Iterative Krylov Methods for Large Linear Systems. Cambridge University Press, Cambridge (2003)

197. van der Vorst, H.A., Melissen, J.B.M.: A Petrov-Galerkin type method for solving $A\boldsymbol{x} = \boldsymbol{b}$, where A is symmetric complex. IEEE Trans. Magn. 26, 706–708 (1990)

198. van Gijzen, M.B.: A polynomial preconditioner for the GMRES algorithm. J. Comput. Appl. Math. 59, 91–107 (1995)

199. van Gijzen, M.B., Sonneveld, P.: Algorithm 913: An elegant IDR(s) variant that efficiently exploits biorthogonality properties. ACM Trans. Math. Softw. 38, 5:1–5:19 (2011)

200. Varga, R.S.: Matrix Iterative Analysis, 2nd edn. Springer-Verlag, Berlin Heidelberg (2000)

201. Vinsome, P.K.W.: Orthomin, an iterative method for solving sparse sets of simultaneous linear equations. In: Fourth Symposium on Reservoir Simulation, pp. 149–159. Society of Petroleum Engineers of AIME (1976)

202. Vital, B.: Etude de Quelques Méthodes de Résolution de Problèmes Linéaires de Grande Taille sur Multiprocesseur. Ph.D. thesis, Univ. de Rennes I, Rennes, France (1990)

203. Weiss, R.: Parameter-Free Iterative Linear Solvers, 1st edn. Akademie Verlag, Berlin (1996)

204. Wesseling, P.: An Introduction to Multigrid Methods. Wiley, Chichester (1991)

205. Wesseling, P., Sonneveld, P.: Numerical experiments with a multiple grid and a preconditioned Lanczos type method. In: Rautmann, R. (ed.) Approximation Methods for Navier-Stokes Problems, pp. 543–562. Springer, Berlin, Heidelberg (1980)

206. Yin, J.-F., Hayami, K.: Preconditioned GMRES methods with incomplete Givens orthogonalization method for large sparse least-squares problems. J. Comput. Appl. Math. 226, 177–186 (2009)

207. Young, D.M.: The search for high level parallelism for the iterative solution of large sparse linear systems. In: Carey, G.F. (ed.) Parallel Supercomputing: Methods, Algorithms and Applications, pp. 89–105. Wiley, Chichester, UK (1989)

208. Young, D.M., Vona, B.R.: On the use of rational iterative methods for solving large sparse linear systems. Appl. Numer. Math. 10, 261–278 (1992)

209. Zemke, J.-P.M.: Variants of IDR with partial orthonormalization. Electron. Trans. Numer. Anal. 46, 245–272 (2017)

210. Zhang, J.H., Zhao, J.: A novel class of block methods based on the block $AA^\top$-Lanczos bi-orthogonalization process for matrix equations. Int. J. Comput. Math. 90, 341–359 (2013)

211. Zhang, L., Nodera, T.: A new adaptive restart for GMRES(m) method. ANZIAM J. 46, C409–C425 (2004)

212. Zhang, S.-L.: GPBi-CG: Generalized product-type methods based on Bi-CG for solving non-symmetric linear systems. SIAM J. Sci. Comput. 18, 537–551 (1997)

尚未翻译部分的源文定位

以下为已核实的源文引用目标. 这里只提供定位, 不是尚未完成章节的译文.